# 1

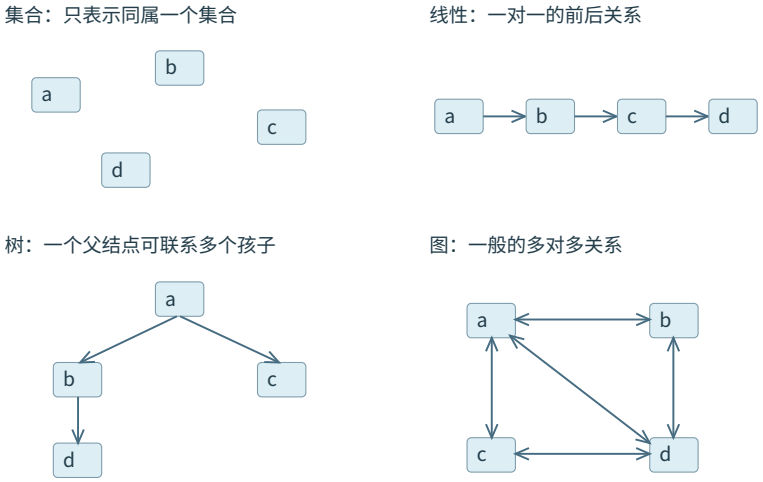



Figure 1.1: 四类逻辑结构

## 1.1 数据结构与抽象数据类型

| 概念 | 定义与判断依据 |
|---|---|
| 数据元素 | 数据处理的基本单位；由若干数据项组成 |
| 数据项 | 构成数据元素的不可分割的最小单位 |
| 数据对象 | 性质相同的数据元素的集合 |
| 逻辑结构 | 元素间的关系：集合、线性、树形、图状 |
| 存储结构 | 元素及其关系的机器表示：顺序、链式、索引、散列 |
| 数据运算 | 运算的抽象定义及其在具体存储结构上的实现 |
| 数据类型 | 值的集合及定义在该集合上的操作 |
| 抽象数据类型（ADT） | 数据对象、关系和操作的抽象描述，隐藏具体实现 |

数据结构的三要素为**逻辑结构、存储结构、数据的运算**。同一逻辑结构可采用不同存储结构；链式存储可表示线性表、树或图。栈、队列属于线性结构，允许的操作位置各有约束。顺序存储以连续位置体现关系，链式存储以链接体现关系。

**例 1-1**

在存储数据时，通常不仅要存储各数据元素的值，还要存储（　）。

A. 数据的操作方法　B. 数据元素的类型　C. 数据元素之间的关系
D. 数据的存取方法

## 1.2 算法特性与正确性

算法是求解一类问题的有限操作序列，具有**有穷性、确定性、可行性、输入、输出**五项特性：对每次合法输入，有限步结束；每步含义明确且能在有限时间内执行；输入可为零个或多个，输出至少一个。返回值或修改后的状态均可作为输出。

算法设计还评价正确性、可读性、健壮性及时间、空间效率。可行性只要求操作可执行；时空效率取决于操作次数和所需资源。

循环正确性按**初始化—保持—终止**证明循环不变式，并说明循环确实接近终止边界。最大值扫描要求非空数组；若接受空数组，应返回失败。

**精确次数与增长阶**

普通 for 循环执行 $n$ 轮时，条件一般检查 $n+1$ 次。选基本操作的粒度会改变常数，紧增长阶保持相同。

```cpp
bool maximum(int a[], int n, int &answer) {
    if (n <= 0) return false;
    int best = a[0];
    for (int i = 1; i < n; ++i)
        if (a[i] > best) best = a[i];
    answer = best;
    return true;
}
```

第 $i$ 轮前，`best` 为 `a[0..i-1]`的最大值；更新后扩展到 `a[0..i]`。终止时覆盖全部元素。合法输入做 $n-1$ 次元素比较，时间 $\Theta(n)$、辅助空间 $\Theta(1)$。比较记录中的关键字时，移动操作须携带整条记录的各数据项。

## 1.3 渐近时间复杂度

选择基本操作，计算其执行次数 $T(n)$，再提取增长阶。单位成本模型将定长比较、赋值、数组访问看作常数时间；大整数、外存访问等需另定成本模型。

- $O(f(n))$：渐近上界；$\Omega(f(n))$：渐近下界；$\Theta(f(n))$：同阶的上下界。
- 顺序执行：相加后取主阶；重复或嵌套：按实际每轮次数求和。
- 常数因子与低次项不改变增长阶；固定底数的对数只相差常数因子。

按渐近增长率排序，$c>1$ 为常数，对数底数为固定大于 1 的常数：

$$1 < \log n < n < n\log n < n^2 < n^3 < c^n < n! < n^n.$$

| 代码特征 | 次数关系 | 紧数量级 |
|---|---|---|
| `i++` 到 $n$ | $n$ | $\Theta(n)$ |
| `i*=2` 或整数 `i/=2` | 次数与 $\log_2 n$ 同阶 | $\Theta(\log n)$ |
| 第 $i$ 轮执行 $i$ 次，$i=1..n$ | $\sum_{i=1}^{n} i = n(n+1)/2$ | $\Theta(n^2)$ |
| 第 $i$ 轮执行 $2^i$ 次，共至 $\log_2 n$ 轮 | 等比和 | $\Theta(n)$ |
| $T(n)=T(n-1)+\Theta(1)$ | 线性层数 | $\Theta(n)$ |
| $T(n)=2T(n/2)+\Theta(n)$ | 每层线性，约 $\log_2 n$ 层 | $\Theta(n\log n)$ |

**例 1-2**

下列函数的时间复杂度是（ ）。

```
int func(int n) {
    int i=0, sum=0;
    while (sum<n) sum += ++i;
    return i;
}
```

A. $O(\log n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n\log n)$

**例 1-3**

下列程序段的时间复杂度是（ ）。

```
int sum=0;
for (int i=1; i<n; i*=2)
    for (int j=0; j<i; j++)
        sum++;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$

**例 1-4**

设 n 是描述问题规模的非负整数，程序段 `x=0; while(n>=(x+1)*(x+1)) x=x+1;`的时间复杂度是（ ）。

A. $O(\log n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n^2)$

**最好、最坏、平均**均针对相同规模的输入：分别取最少、最多、按给定概率分布加权的工作量。平均值需要概率模型。等概率且保证成功的顺序查找平均比较 $(n+1)/2$ 次；包含失败时另加失败概率与代价。$O/\Omega/\Theta$ 与最好/最坏属于两套不同分类。

## 1.4 辅助空间与递归栈

辅助空间统计同一时刻存活的额外状态，通常排除输入，输出区按题意处理。顺序复用的空间取峰值，同时保留的空间相加。原地算法通常要求常数辅助空间。

| 同时活跃状态 | 峰值辅助空间 |
|---|---|
| 常数个变量反复执行 $n$ 轮 | $\Theta(1)$ |
| 每轮复用长度 $n$ 的数组，共 $n$ 轮 | $\Theta(n)$ |
| 递归深度 $n$，每层常数空间 | $\Theta(n)$ |
| 一条调用链各层保留 $n, n-1, \ldots, 1$ 个单元 | $\Theta(n^2)$ |
| 一条调用链各层保留 $n, n/2, n/4, \ldots$ 个单元 | $\Theta(n)$ |
| 二叉树递归遍历，树高 $h$ | $O(h)$；总时间 $\Theta(n)$ |

递归树的调用总数决定总工作量，最长未返回调用链决定栈深。快速排序的普通递归实现在平均情况下需 $O(\log n)$ 栈空间，最坏需 $O(n)$。

# 2 线性表

## 2.1 线性表、位序与表示

线性表是 $n$ 个相同数据类型元素的有限序列；$n=0$ 为空表。非空表首元素无直接前驱，末元素无直接后继，其余各有一个直接前驱和直接后继。元素值可重复。

已有元素位序为 $1..n$；插入位序为 $1..n+1$。数组 `data[0..n-1]`中，位序 $i$ 对应下标 $i-1$。本章带头单链表中，`L` 为头指针，指向不保存有效数据的头结点，首元结点为 `L->next`。

## 2.2 顺序表：寻址、插入与删除

首元素地址 $b$、每元素占 $w$ 个地址单位时，第 $i$ 个元素地址为 $b+(i-1)w$，按位访问 $O(1)$。无序按值查找最坏 $O(n)$。

```c
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
```

```c
void InitList(SqList &L) { L.length = 0; }
ElemType GetElem(SqList L, int i) {
    return L.data[i - 1];  // 前提: 1 <= i <= L.length
}
int LocateElem(SqList L, ElemType e) {
    for (int i = 0; i < L.length; ++i)
        if (L.data[i] == e) return i + 1;
    return 0;
}
```

`GetElem` 要求位序合法；`LocateElem` 成功返回首次相等元素的位序，失败返回 0。若元素为结构体，按题意定义比较规则。

```cpp
bool ListInsert(SqList &L, int i, ElemType e) {
    if (i < 1 || i > L.length + 1) return false;
    if (L.length >= MaxSize) return false;
    for (int j = L.length; j >= i; --j)
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = e;
    ++L.length;
    return true;
}
bool ListDelete(SqList &L, int i, ElemType &e) {
    if (i < 1 || i > L.length) return false;
    e = L.data[i - 1];
    for (int j = i; j < L.length; ++j)
        L.data[j - 1] = L.data[j];
    --L.length;
    return true;
}
```

| 操作 | 搬移方向 | 位序 $i$ 的移动数 | 等概率平均移动数 |
|---|---|---|---|
| 插入 | 后向前 | $n-i+1$ | $n/2$，共 $n+1$ 个位置 |
| 删除 | 前向后 | $n-i$ | $(n-1)/2$，共 $n$ 个位置 |

追加、删末元素可为 $O(1)$；一般插删最坏 $O(n)$。失败操作保持表长，输出参数只在删除成功后有效。

**例 2-1**

在线性表 $a_0, a_1, \ldots, a_{100}$ 中，删除元素 $a_{50}$ 需要移动（　）个元素。

A. 0　B. 50　C. 51　D. 0 或 50

## 动态顺序表

动态顺序表仍用连续数组；结构记录 `data`、`length`、`MaxSize`。容量不足时申请更大的连续区，复制 $n$ 个有效元素并释放旧区，单次扩容时间 $O(n)$，新旧区共存时额外空间 $O(n)$。静态容量固定时，表长仍可变化。

以下动态类型与前面的静态宏 `MaxSize` 示例独立使用；`ElemType` 取整型，需包含 `<stdlib.h>`。接口以申请成功、`len>0` 且容量运算不溢出为前提。

```
#define InitSize 100
typedef struct {
    ElemType *data;
    int MaxSize;
    int length;
} SeqList;
void InitList(SeqList &L) {
    L.data = (ElemType *)malloc(InitSize * sizeof(ElemType));
    L.length = 0;
    L.MaxSize = InitSize;
}
void IncreaseSize(SeqList &L, int len) {
    ElemType *p = L.data;
    L.data = (ElemType *)malloc(
        (L.MaxSize + len) * sizeof(ElemType));
    for (int i = 0; i < L.length; ++i)
        L.data[i] = p[i];
    L.MaxSize += len;
    free(p);
}
```

工程实现先用临时指针接收申请结果，成功后替换 `data`。清空只令表长为 0，销毁才释放连续区。

## 2.3 单链表：定位与链接修改

```
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
bool InitList(LinkList &L) {
    L = (LNode *)malloc(sizeof(LNode));
    if (L == NULL) return false;
    L->next = NULL;
    return true;
}
```

结点间无需连续；按位定位沿 `next` 逐个前进，最坏 $O(n)$。`malloc/free` 成对使用。

```
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    int j = 0;
    LNode *p = L;
    while (p != NULL && j < i) {
        p = p->next;
        ++j;
    }
    return p;
}
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) p = p->next;
    return p;
}
```

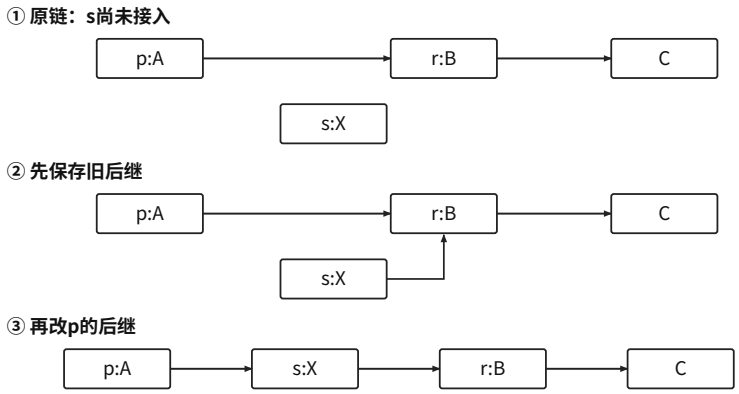



Figure 2.1: 单链表后插的链接顺序

带头结点时 `GetElem(L,0)`返回头结点，超过表长返回空。按值查找从首元结点开始。

```
bool InsertNextNode(LNode *p, ElemType e) {
    if (p == NULL) return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) return false;
    s->data = e;
    s->next = p->next;
    p->next = s;
    return true;
}
bool ListInsert(LinkList &L, int i, ElemType e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    return InsertNextNode(p, e);
}
bool ListDelete(LinkList &L, int i, ElemType &e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    if (p == NULL || p->next == NULL) return false;
    LNode *q = p->next;
    e = q->data;
    p->next = q->next;
    free(q);
    return true;
}
```

插入先令新结点指向旧后继，再修改前驱链接；删除先保存被删结点和后继，再摘除、释放。已知前驱的局部修改为 $O(1)$；按位序插删还需 $O(n)$ 定位。

仅给结点 $p$，在允许改变数据与结点身份时，可通过复制数据实现前插或删自身；删除自身的复制后继法要求 $p$ 有后继，尾结点不适用。共享后缀结点需统一管理释放。

不带头结点时空表为 `L==NULL`。位置 1 插入执行 `s->next=L; L=s;`；删除首元执行 `p=L; L=L->next; free(p)`。修改头指针的接口须用引用或二级指针。

### 建表与原地逆置

| 方法 | 每轮接入 | 输入次序 | 建 $n$ 结点时间 |
|---|---|---|---|
| 头插 | `s->next=L->next; L->next=s` | 逆序 | $O(n)$ |
| 尾插 | `r->next=s; r=s`，结束置 `r->next=NULL` | 保序 | $O(n)$，维护尾指针 |

头插前令 `L->next=NULL`；尾插前令 `r=L`。每轮重新寻找尾部的尾插建表需 $O(n^2)$。教材交互式建表以 9999 为结束标志时，9999 不能同时作为普通数据。

**递归逆置的空间**

递归逆置沿链保留尚未返回的调用，辅助栈为 $O(n)$。迭代版的常数空间来自只同时保存固定个数的结点指针。

```
void reverseList(LNode *L) {       // 带头结点
    LNode *prev=NULL, *cur=L->next;
    while (cur!=NULL) {
        LNode *nxt=cur->next;
        cur->next=prev;
        prev=cur;
        cur=nxt;
    }
    L->next=prev;
}
```

逆置维护已逆置前缀与未处理后缀，改 `cur->next` 前保存后继。时间 $\Theta(n)$、辅助空间 $\Theta(1)$，空表与单元素表均适用。

**例 2-2**

已知一个带有表头结点的单链表，结点结构为 `data | next`，只给出了头指针 `list`。在不改变链表的前提下，设计尽可能高效的算法，查找倒数第 $k$ 个位置上的结点（$k$ 为正整数）。若成功，输出该结点的 `data` 值并返回 1；否则只返回 0。要求说明设计思想、实现步骤，并用 C、C++ 或 Java 描述算法，在关键处加注释。

## 2.4 双链表、循环链表与静态链表

双链表增加 `prior` 域。已知有效结点后可直接取得前驱；按位查找仍需遍历。非循环链的首尾边界要检查空指针。

```c
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinklist;
bool InsertNextDNode(DNode *p, DNode *s) {
    if (p == NULL || s == NULL) return false;
    s->next = p->next;
    if (p->next != NULL) p->next->prior = s;
    s->prior = p;
    p->next = s;
    return true;
}
bool DeleteNextDNode(DNode *p) {
    if (p == NULL || p->next == NULL) return false;
    DNode *q = p->next;
    p->next = q->next;
    if (q->next != NULL) q->next->prior = p;
    free(q);
    return true;
}
```

**例 2-3**

在双链表中向 p 所指的结点之前插入结点 q，正确操作是（　）。

```
A. p->prior=q; q->next=p;
   p->prior->next=q; q->prior=p->prior;
B. q->prior=p->prior; p->prior->next=q;
   q->next=p; p->prior=q->next;
C. q->next=p; p->next=q;
   q->prior->next=q; q->next=p;
D. p->prior->next=q; q->next=p;
   q->prior=p->prior; p->prior=q;
```

| 表示 | 空状态 | 遍历终止或关键条件 |
|---|---|---|
| 带头普通单链表 | `L->next==NULL` | 当前指针为 NULL |
| 带头循环单链表 | `L->next==L` | 重新遇到头结点；尾满足 `p->next==L` |
| 带头循环双链表 | `L->next==L && L->prior==L` | 正反向均以头结点为边界 |
| 无头循环单链表，仅尾指针 `rear` | 空状态另行约定 | 首元为 `rear->next` |

两个非空、无头循环单链表给定尾指针时，交换两个尾部的后继即可 $O(1)$ 拼接；只给头指针通常须先找尾。带头结点版本还要删除或绕过多余头结点。

**静态链表**用数组保存结点、游标保存后继，逻辑顺序由游标决定；按逻辑位序访问仍需遍历。须管理空闲位置。例如底稿表示以 0 号为头结点、`next=-1` 为链尾、`next=-2` 为空闲，三个状态各有用途。物理地址可按数组下标寻址，不能将该下标直接作为逻辑位序。

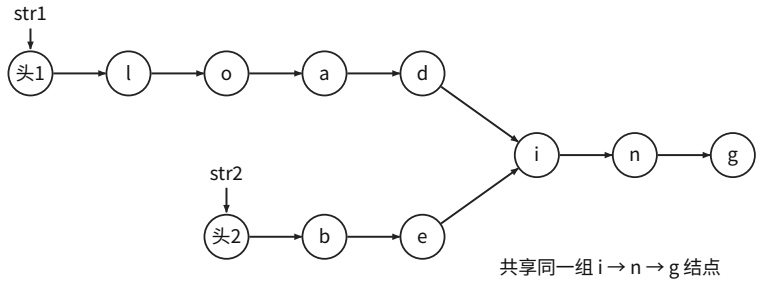



Figure 2.2: 两个前缀分别连接到同一个 i 结点，随后共用 n、g 结点

## 2.5 线性扫描、双指针与归并

**连续删除**：指针 $p$ 始终指向已处理区的最后一个保留结点；删除 $p$ 后继后继续检查新后继，保留后继时才推进 $p$。

```
void eraseAll(LNode* L, int x) {
    LNode* p = L;
    while (p->next != NULL) {
        if (p->next->data == x) {
            LNode* q = p->next;
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

**有序数组去重**：读指针扫描，写指针维护互异前缀；仅当当前值不同于最后保留值时写入。时间 $O(n)$、空间 $O(1)$。无序输入仅消除相邻重复。

**有序归并**：每次输出两表当前首元素中较小者并推进对应指针，最后接余段。长度 $m, n$ 的两表需 $O(m+n)$ 时间。数组常需输出区；允许破坏原链表时可重接结点。稳定合并相等时先取左表；集合并集相等时只留一次并同时推进。

**例 2-4**

将 $n$ $(n > 1)$ 个整数存放在一维数组 $R$ 中。设计一个时间、空间都尽可能高效的算法，将序列循环左移 $p$ $(0 < p < n)$ 个位置，即把 $(X_0, \ldots, X_{n-1})$ 变为 $(X_p, \ldots, X_{n-1}, X_0, \ldots, X_{p-1})$。给出思想、C/C++ 或 Java 实现，并说明时间与空间复杂度。

**例 2-5**

采用带头结点的单链表保存单词，两个单词可共享相同后缀的存储空间。例如 `loading` 与 `being` 共享从字符 `i` 开始的结点。`str1`、`str2` 指向两个头结点，结点为 `data | next`。设计时间上尽可能高效的算法，找到共同后缀的起始结点，给出思想、程序及时间复杂度。

**例 2-6**

整数序列 $A=(a_0,\dots,a_{n-1})$ 满足 $0\le a_i<n$。某值出现次数超过 $n/2$ 时，称它为主元素。例如 $(0,5,5,3,5,7,5,5)$ 的主元素为 5，而 $(0,5,5,3,5,1,5,7)$ 没有主元素。设计尽可能高效的算法，存在主元素时输出该值，否则输出 $-1$；给出思想、程序及时间、空间复杂度。

## 2.6 顺序表与链表的操作代价

| 操作或资源 | 顺序表 | 带头单链表 |
|---|---|---|
| 按位访问 | $O(1)$ | $O(n)$ |
| 无序按值查找 | $O(n)$ | $O(n)$ |
| 按位插入、删除 | 最坏 $O(n)$ | 定位最坏 $O(n)$ |
| 已知前驱后插、删后继 | 仍可能搬移后缀 | $O(1)$ |
| 额外空间 | 未用容量、扩容临时区 | 每结点链接域与分配开销 |

顺序表连续访问具有局部性；链表适合已定位后的频繁局部修改。空间比较还要计容量利用率、链接宽度和分配开销。

# 3 栈、队列和数组

## 3.1 栈与合法出栈序列

栈只允许在栈顶插入、删除，遵循后进先出。空栈不能出栈，固定容量满栈不能入栈。

检验输出序列：按目标输出依次处理；当前栈顶不符时继续按输入次序入栈，到目标位于栈顶时弹出。输入耗尽且栈顶仍不符即失败。模拟期间最大栈深为实现该输出所需的最小容量。

从空栈出发并回到空栈的合法入出操作序列满足：入栈与出栈总数相同，每个前缀中出栈数不超过入栈数。$n$ 个不同元素的合法出栈序列数为

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

**例 3-1**

栈 $S$ 和队列 $Q$ 初始均为空，元素 `a,b,c,d,e,f,g` 依次入栈 $S$。每个元素出栈后立即进入队列 $Q$，7 个元素出队顺序为 `b,d,c,f,e,a,g`，则栈容量至少是（　）。

A. 1　　B. 2　　C. 3　　D. 4

### 顺序栈与共享栈

顺序栈下标从 0 开始，`top` 指当前栈顶，空时为 $-1$。

```
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
void InitStack(SqStack &S) { S.top = -1; }
bool StackEmpty(SqStack S) { return S.top == -1; }
bool Push(SqStack &S, ElemType x) {
    if (S.top == MaxSize - 1) return false;
    S.data[++S.top] = x;
    return true;
}
bool Pop(SqStack &S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top--];
    return true;
}
bool GetTop(SqStack S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top];
    return true;
}
```

若 `top` 改指下一空位，须同时改为空时 0、入栈 `data[top++]=x`、出栈 `x=data[--top]`。

共享栈从同一数组两端向中间增长。教材 `ShStack` 采用如下初值：

```
typedef struct {
    ElemType data[MaxSize];
    int top0, top1;
} ShStack;
void InitStack(ShStack &S) {
    S.top0 = -1;
    S.top1 = MaxSize;
}
```

**模差保留的信息**

$(rear - front + M) \bmod M$ 把距离 0 与距离 M 都映射为 0。全部 M 个单元可用时，必须增加长度或操作标志来保存空满差别。

| 操作 | 0 号栈 | 1 号栈 |
|---|---|---|
| 判空 | `top0==-1` | `top1==MaxSize` |
| 入栈 | `data[++top0]=x` | `data[--top1]=x` |
| 出栈 | `x=data[top0--]` | `x=data[top1++]` |

两栈共用的满条件为 `top1-top0==1`。入栈先判满，出栈先判各自是否为空。

### 链栈

链栈通常无头结点，以链头为栈顶，初始化 `S=NULL`。头插、头删均为 $O(1)$；入栈可能因申请失败而失败。

```
typedef struct Linknode {
    ElemType data;
    struct Linknode *next;
} *LiStack;
bool Push(LiStack &S, ElemType x) {
    LiStack p = (LiStack)malloc(sizeof(struct Linknode));
    if (p == NULL) return false;
    p->data = x;
    p->next = S;
    S = p;
    return true;
}
bool Pop(LiStack &S, ElemType &x) {
    if (S == NULL) return false;
    LiStack p = S;
    x = p->data;
    S = p->next;
    free(p);
    return true;
}
```

## 3.2 队列、双端队列与循环表示

队列在队尾插入、队头删除，先进先出。双端队列允许两端插入和删除；输入受限版本只允许一端输入，输出受限版本只允许一端输出。两种受限模型须分别模拟。

按 1、2、3、4 顺序输入，输入受限双端队列有 22 种输出，不能得到 `4,2,1,3` 及 `4,2,3,1`；输出受限双端队列有 22 种输出，不能得到 `4,1,`

`3,2` 及 `4,2,3,1`。普通栈有 14 种输出。

循环队列采用长度 $M$ 的数组，`front` 指队头，`rear` 指下一入队位置。入队后推进 `rear`，出队后推进 `front`，均对 $M$ 取模。

| 区分空满的方法 | 空 | 满 | 最大元素数 |
|---|---|---|---|
| 牺牲一单元 | `front==rear` | `(rear+1)%M==front` | $M-1$ |
| 增加 `size` | `size==0` | `size==M` | $M$ |
| 最近操作 `tag` | 指针相同且 tag 为出队 | 指针相同且 tag 为入队 | $M$ |

`tag` 初始为出队态，成功操作后才更新。牺牲单元方案长度为

$$len = (rear - front + M) \bmod M.$$

```
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
void InitQueue(SqQueue &Q) { Q.front = Q.rear = 0; }
bool QueueEmpty(SqQueue Q) { return Q.front == Q.rear; }
bool EnQueue(SqQueue &Q, ElemType x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}
bool DeQueue(SqQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    return true;
}
bool GetHead(SqQueue Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    return true;
}
```

**例 3-2**

循环队列存于数组 `A[21]`，`front` 指向队首元素的前一个位置，`rear` 指向队尾元素。当前 `front=8`，`rear=3`，队列长度为（ ）。

A. 5　B. 6　C. 16　D. 17

若 `front` 指队头、`rear` 指当前队尾，可令空状态 `front=0,rear=M-1`；入队先推进 `rear` 再写，出队先取后推进 `front`。牺牲一单元时空条件 `(rear+1)%M==front`、满条件 `(rear+2)%M==front`，长度 `(rear-front+1+M)%M`。数组 `A[0..n]`共有 $n+1$ 单元，模数须取 $n+1$。

## 链队列

单链队列以链头为队头、链尾为队尾，保存 `front`、`rear`。带头结点时 `front` 始终指向头结点；初始化和入队的 `void` 接口以下均假设申请成功。

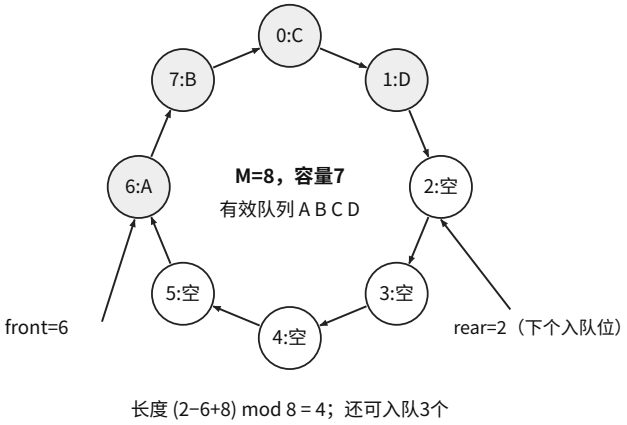



Figure 3.1: 循环队列的环绕与空满状态

```c
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode;
typedef struct {
    LinkNode *front, *rear;
} LinkQueue;
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (LinkNode *)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
bool IsEmpty(LinkQueue Q) { return Q.front == Q.rear; }
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;
    Q.rear = s;
}
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    LinkNode *p = Q.front->next;
    x = p->data;
    Q.front->next = p->next;
    if (Q.rear == p) Q.rear = Q.front;
    free(p);
    return true;
}
```

删除最后一个数据结点后必须令 `rear=front`。不带头结点时空状态为两个指针均为空；第一次入队同时设置二者，最后一次出队同时置空。普通单链表尾删需寻找前驱，时间 $O(n)$；循环双链表可通过队头的 `prior` 访问尾部。

## 3.3 括号、表达式与递归

括号匹配：左括号入栈；右括号必须与栈顶同型并弹出；扫描结束栈必须为空。以下代码复用顺序栈，`ElemType` 改为 `char` 或能保存字符编

码的类型。

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S);
    for (int i = 0; i < length; ++i) {
        char c = str[i];
        if (c == '(' || c == '[' || c == '{') {
            if (!Push(S, c)) return false;
        } else if (c == ')' || c == ']' || c == '}') {
            ElemType x;
            if (!Pop(S, x)) return false;
            if ((c == ')' && x != '(') ||
                (c == ']' && x != '[') ||
                (c == '}' && x != '{')) return false;
        }
    }
    return StackEmpty(S);
}
```

时间 $O(n)$，最坏辅助空间 $O(n)$；固定栈容量应不小于所需最大深度。`([)]`在第三个字符首次失败。

| 表达式形式 | 扫描方向 | 遇二元运算符的弹出顺序 |
|---|---|---|
| 后缀 | 左至右 | 先右操作数，再左操作数 |
| 前缀 | 右至左 | 先左操作数，再右操作数 |

后缀求值中，操作数入栈；运算符弹出两个值，按左右角色计算后压回。须检查操作数不足、除零与末尾恰留一个值。

**例 3-5**

栈 $S_1$ 保存整数，$S_2$ 保存运算符。函数 F() 依次：(1) 从 $S_1$ 弹出操作数 a、b；(2) 从 $S_2$ 弹出运算符 op；(3) 执行 b op a；(4) 把结果压入 $S_1$。初始 $S_1$ 自底向顶为 5、8、3、2，$S_2$ 自底向顶为 *、−、+。调用 3 次 F() 后，$S_1$ 栈顶值是（　）。

A. −15　　B. 15　　C. −20　　D. 20

**中缀转后缀**：操作数直接输出；左括号入栈；右括号触发弹出直到匹配左括号；遇运算符先输出优先级更高的栈顶运算符。左结合运算符同级也弹出，右结合运算符另按结合规则处理；最后清空运算符栈。多位数、一元符号须先识别为词法单元。

例如 `A+B-C*D/E+F` 转换为 `A B + C D * E / - F +`。`x+y*(z-u)/v` 的前缀为 `+ x / * y - z u v`，后缀为 `x y z u - * v / +`。

**双栈直接求中缀值**：操作数立即压入值栈；运算符栈按上述转换规则管理。每逢应输出运算符时，改为弹出左右值、计算并压回。左括号只作边界。

数值式 $((15/(7-(1+1)))\times 3)-(2+(1+1))$ 的后缀为 `15 7 1 1 + - / 3 * 2 1 1 + + -`，结果为 5，操作数栈峰值为 4。

## 调用次数与调用栈

递归栈保存尚未返回的参数、局部变量及返回位置。调用总数与最长活跃调用链分别控制时间、栈空间。

```c
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    else return n * factorial(n - 1);
}
int Fib(int n) {
    if (n == 0) return 0;
    else if (n == 1) return 1;
    else return Fib(n - 1) + Fib(n - 2);
}
```

输入为非负整数, 结果须能由 `int` 表示。阶乘时间 $O(n)$、栈空间 $O(n)$。朴素 Fibonacci 满足 $T(n)=T(n-1)+T(n-2)+O(1)$，时间呈指数增长，栈空间 $O(n)$。`Fib(4)`返回 3，共 9 次调用，最长活跃链 4 层。

若 `F(n)`在 $n\le 3$ 时返回 1，否则返回 `F(n-2)+F(n-4)+1`，调用次数满足 $C(n)=1+C(n-2)+C(n-4)$、基例 $C(n)=1$；因此 $C(4)=3, C(6)=5, C(8)=9$。递推中的 1 统计当前调用。C/C++ 加号两侧调用的先后不保证固定；求“第几次调用”时须检查是否依赖该顺序。嵌套调用须先求实参：基例返回 2、递归返回 `func(x-2)+func(x-4)` 时，`func(func(5))`先由 3 次调用求出内层结果 4，第 4 次才调用外层 `func(4)`。

按层处理用队列；树层序遍历和图 BFS 均遵循先发现先扩展，图还需防止重复入队。

## 3.4 多维数组寻址

二维数组行范围 $l_1..u_1$、列范围 $l_2..u_2$，行数 $r=u_1-l_1+1$、列数 $c=u_2-l_2+1$；首元素地址 $b$，元素宽度 $w$ 个地址单位：

$$\begin{aligned}\text{行优先:}\quad & Addr(A[i,j])=b+[(i-l_1)c+(j-l_2)]w,\\ \text{列优先:}\quad & Addr(A[i,j])=b+[(j-l_2)r+(i-l_1)]w.\end{aligned}$$

三维行优先的相对元素位置为 $((i-l_1)n_2+(j-l_2))n_3+(k-l_3)$。先算元素偏移，再乘元素宽度；按字编址与按字节编址需统一单位。

## 3.5 特殊矩阵的压缩存储

对称矩阵满足 $a_{ij}=a_{ji}$，先将目标换到保存半边。三角矩阵另一半是同一公共常量，常量可非零。三对角矩阵仅 $|i-j|\le 1$ 的位置可能非零。下表矩阵坐标从 1 开始，压缩数组下标 $k$ 从 0 开始，$n$ 为阶数：

| 保存方式 | 有效区域 | 下标 $k$ |
|---|---|---|
| 下三角行优先 | $i\ge j$ | $i(i-1)/2+j-1$ |
| 上三角列优先 | $i\le j$ | $j(j-1)/2+i-1$ |
| 上三角行优先 | $i\le j$ | $(i-1)(2n-i+2)/2+j-i$ |
| 下三角列优先 | $i\ge j$ | $(j-1)(2n-j+2)/2+i-j$ |
| 三对角行优先 | $\lvert i-j\rvert\le 1$ | $2i+j-3$ |

对称矩阵保存 $n(n+1)/2$ 个值；三角矩阵再加 1 个公共常量单元；三对角矩阵 $n\ge 2$ 时共 $3n-2$ 个值，$n=1$ 时直接保存一个值。

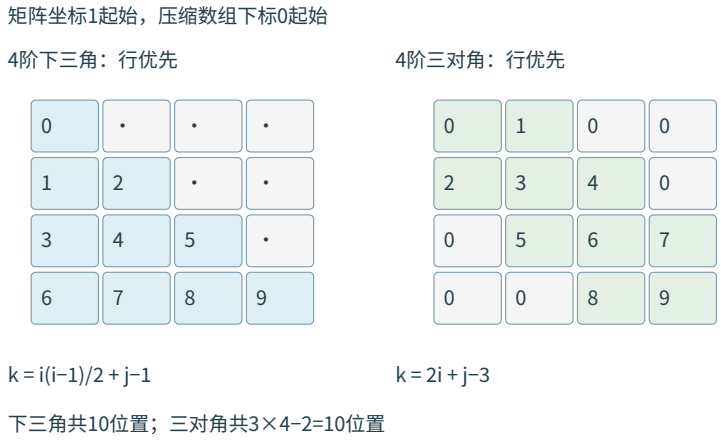



Figure 3.2: 压缩后的连续下标

**例 3-3**

100 阶三对角矩阵 $M$ 的元素 $m_{i,j}$ $(1 \le i, j \le 100)$ 按行优先依次压缩存入下标从 0 开始的一维数组 $N$。$m_{30,30}$ 在 $N$ 中的下标为（　）。

A. 86　　B. 87　　C. 88　　D. 89

**例 3-4**

10×10 对称矩阵 $M$ 的上三角元素 $m_{i,j}$ $(1 \le i \le j \le 10)$ 按列优先存入 C 语言一维数组 $N$。$m_{7,2}$ 在 $N$ 中的下标为（　）。

A. 15　　B. 16　　C. 22　　D. 23

反向定位：若矩阵、数组均 0 起始且下三角行优先，令 $T_i = i(i+1)/2$，找 $T_i \le k < T_{i+1}$，再取 $j = k - T_i$；等价地

$$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, \qquad j = k - i(i+1)/2.$$

浮点实现需用整数边界复核。三对角坐标 1 起始、数组 0 起始且 $0 \le k < 3n-2, n \ge 2$ 时：

$$i = \left\lceil \frac{k+2}{3} \right\rceil, \qquad j = k - 2i + 3.$$

**稀疏矩阵**非零位置无固定规律，三元组保存 **(行,列,值)**并另记总行数、总列数、非零元数；全零行也计入行数。十字链表让每个非零结点同时进入行链、列链，修改须维护两组链接。坐标到压缩记录的定位依赖表示，不能直接套普通二维数组公式。

# 4 串

## 4.1 串、子串与基本运算

串是零个或多个字符组成的有限序列，长度为字符数。空串长度 0，空格串含实际空格字符。子串由主串中连续字符构成，位置取其首字符位置；相同子串可有多个出现位置。

两串相等要求长度相同且对应字符相同。字典序比较第一对不同字符；已比较部分相同且一方先结束时，较短串较小。

| 操作 | 结果与条件 |
|---|---|
| 赋值、复制 | 容量足够；复制地址与独立复制字符的语义须明确 |
| 判空、求长 | 依赖长度成员或结束标志 |
| 清空、销毁 | 清空保留存储；销毁回收动态存储 |
| 连接 | 总长度可容纳 |
| 取子串 | 检查起点、长度和末位置 |
| 比较 | 返回正、零、负值表示大小 |
| 定位 | 本章首次成功返回 1 起始位置，失败返回 0 |

允许空子串时, 长 $n$ 的串须满足 $1 \le pos \le n+1$、$len \ge 0$、$pos+len-1 \le n$； `pos=n+1` 只允许 `len=0`。模式匹配约定空模式返回 1。

## 4.2 存储表示与基本操作代码

本章顺序串的字符从 1 开始，`ch[0]`弃用，长度另存。`ch[MAXLEN]`的有效字符容量为 `MAXLEN-1`，保持 $0 \le length < MAXLEN$。

```c
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN];
    int length;
} SString;
```

显式长度表示无需`'\0'` 结束符；转用 C 字符串库前须转换布局。其他布局如首单元保存长度、0 起始另存长度、结束符表示等，均有各自容量与求长规则。

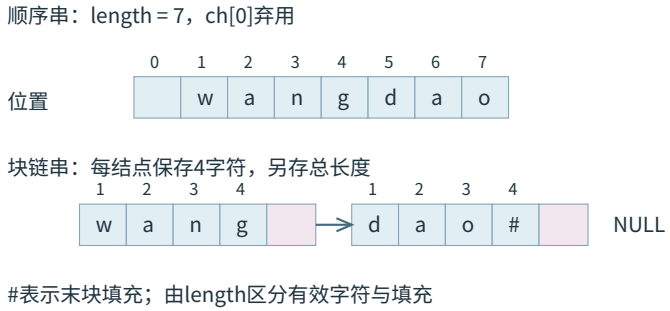



Figure 4.1: 顺序串与块链串

```
bool SubString(SString &Sub, SString S, int pos, int len) {
    if (pos < 1 || pos > S.length + 1 || len < 0 ||
        len > S.length - pos + 1)
        return false;
    for (int i = pos; i < pos + len; ++i)
        Sub.ch[i - pos + 1] = S.ch[i];
    Sub.length = len;
    return true;
}
int StrCompare(SString S, SString T) {
    for (int i = 1; i <= S.length && i <= T.length; ++i)
        if (S.ch[i] != T.ch[i])
            return (unsigned char)S.ch[i] -
                   (unsigned char)T.ch[i];
    return S.length - T.length;
}
```

```
bool Concat(SString &T, SString S1, SString S2) {
    if (S1.length + S2.length >= MAXLEN) return false;
    for (int i = 1; i <= S1.length; ++i)
        T.ch[i] = S1.ch[i];
    for (int i = 1; i <= S2.length; ++i)
        T.ch[S1.length + i] = S2.ch[i];
    T.length = S1.length + S2.length;
    return true;
}
```

`SubString` 时间 $O(len)$；比较最坏 $O(\min(n,m))$；连接时间 $O(n+m)$。上述定长串按值传入副本，输出与输入同名时仍能读取原值。`unsigned char` 使字符编码按无符号值比较。

堆串采用 `char *ch; int length;`管理动态连续区，需处理分配、复制、释放。链串可每结点保存一字符或多字符形成块，块尾填充须与真实字符区分。仅计字符与指针时，每结点 $q$ 个 1 字节字符、指针 $p$ 字节的存储密度为 $q/(q+p)$；结构体对齐另计。

## 4.3 朴素模式匹配

主串长 $n$、模式长 $m$，字符位置均从 1 开始。失配时本轮起点为 $i-j+1$，改从下一起点比较：`i=i-j+2; j=1`。

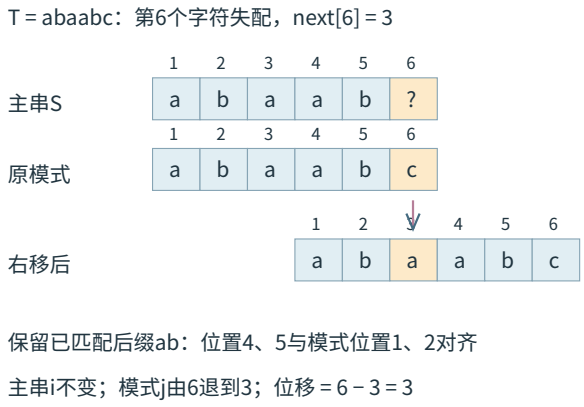



Figure 4.2: 失配后保留已匹配段的边界

```
int Index(SString S, SString T) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) {
            ++i; ++j;
        } else {
            i = i - j + 2;
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

成功时 $j = m + 1$，返回 $i - m$；主串先耗尽则失败。非空模式最多有 $n - m + 1$ 个候选起点，最坏时间 $O((n - m + 1)m)$，常记 $O(nm)$；辅助工作空间 $O(1)$。首起点成功时比较 $m$ 次。

## 4.4 KMP 与 next 的 1 起始约定

在 $(i, j)$ 失配前，$S[i - j + 1..i - 1] = T[1..j - 1]$。模式移位后仍保留的重合部分，必须是该已匹配前缀的相等真前后缀。

设 $\ell$ 为串 $T[1..j - 1]$ 的最长相等真前后缀长度，则

$$next[1] = 0, \qquad next[j] = \ell + 1 \quad (j > 1).$$

`next[j]`是下一次比较的模式位置。失配令 `j=next[j]`，主串位置保持；`j==0` 时下一轮同时推进 `i`、`j`，恢复模式首字符。

```
int Index_KMP(SString S, SString T, int next[]) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) {
            ++i;
            ++j;
        } else {
            j = next[j];
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

**前缀函数换算**

本章 next 值对应当前位置之前的前缀。若另有 0 起始前缀函数 $\pi$，则 $j>1$ 时 $next[j]=\pi[j-2]+1$；换定义时匹配代码也须同步。

短路求值保证 `j==0` 时不读取 `T.ch[0]`。主串位置单调递增，同一个主串字符可能与多个模式位置比较。

**例 4-1**

主串 `S=abaabaabcabaabc`，模式 `T=abaabc`。分别用朴素匹配和普通 next 表的 KMP 查找首次出现位置，计算实际字符比较次数。字符位置与 next 均从 1 开始，`next[1]=0`。

## 线性时间建立 next

已知 `next[i]=j`：若 `j==0` 或 `T.ch[i]==T.ch[j]`，则下一前缀的边界可扩展，写入 `next[++i]=++j`；失配时沿 `next[j]`尝试更短边界。

```
void get_next(SString T, int next[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}
```

| 模式 | 普通 next |
|---|---|
| `google` | `0,1,1,1,2,1` |
| `abaabc` | `0,1,1,2,2,3` |
| `ababaa` | `0,1,1,2,3,4` |
| `aaaab` | `0,1,2,3,4` |

`google` 在位置 5 之前的 `goog` 有长度 1 的边界 `g`，故 `next[5]=2`。主串 `googlogoogoogle` 首次匹配 `google` 的位置为 10。

## 4.5 nextval 优化

已知 `S.ch[i]`与 `T.ch[j]`失配, 若 `T.ch[next[j]]==T.ch[j]`, 退到 `next[j]`后仍必败。令 `nextval[1]=0`, $j \geq 2$ 时:

$$nextval[j] = \begin{cases} next[j], & T[j] \neq T[next[j]], \\ nextval[next[j]], & T[j] = T[next[j]]. \end{cases}$$

```
void get_nextval(SString T, int nextval[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            if (T.ch[i] != T.ch[j])
                nextval[i] = j;
            else
                nextval[i] = nextval[j];
        } else {
            j = nextval[j];
        }
    }
}
```

将优化表作为 `Index_KMP` 的第三个实参，匹配循环保持相同。

| 模式 | nextval |
|---|---|
| abaabc | 0,1,0,2,1,3 |
| ababaa | 0,1,0,1,0,4 |
| aaaab | 0,0,0,0,4 |

**例 4-2**

主串 `aaacaaaabdaaaab`，模式 `aaaab`。比较普通 next 和优化 nextval 的匹配过程，求首次成功位置与两者字符比较次数。采用 1 起始位置和 `next[1]=nextval[1]=0`。

## 4.6 KMP 的代价与边界

建表中 $i$ 最多推进 $m-1$ 次, $j$ 的回退总量受此前增量限制, 建表 $O(m)$; 匹配中 $i$ 最多推进 $n$ 次，累积回退同样受推进约束，扫描 $O(n)$。总时间 $O(n+m)$，表空间 $O(m)$；已给表时单独扫描记为 $O(n)$。

每次跳过的对齐都由已匹配前缀的边界关系排除，故不遗漏合法起点。统计字符比较只计实际字符判等，`j==0` 检查单独计控制操作。查找全部出现时还需约定是否允许重叠，以及成功后的继续位置。

固定数组按值传参的复制成本在教材分析中按预设容量处理；可增长规模的实现应把实际复制成本计入。

# 5 树与二叉树

## 5.1 树的定义、度与计数

非空树有唯一根，除根外每结点恰有一个双亲，各子树互不相交，根到任意结点有唯一路径。结点的度为孩子数，树的度为结点度最大值，度 0 为叶。兄弟具有同一双亲；有序树区分各子树先后，无序树无此要求。

森林为零棵或多棵不相交的树；删去非空树根得到子树森林，给森林加统一根得到一棵树。

本章**根为第 1 层，叶高度 1，空树高度 0**；路径长按边数计，根到第 $i$ 层的路径长为 $i-1$。树高取最大层号，树的路径长度取根到所有结点路径长之和。

设 $n_i$ 为度 $i$ 结点数，$n$ 为总结点数：

$$n=\sum_{i} n_i, \qquad n-1=\sum_{i} i n_i, \qquad n_0=1+\sum_{i\ge 2}(i-1)n_i.$$

度 1 结点可增加树高而不改变叶数，叶数与各高阶度数未必确定总结点数。

**例 5-1**

一棵度为 4 的树，含 20 个度为 4 的结点、10 个度为 3 的结点、1 个度为 2 的结点、10 个度为 1 的结点。叶结点数为（　）。

A. 41　　B. 82　　C. 113　　D. 122

| $m>1$ 时的条件 | 结论 |
|---|---|
| $m$ 叉树，第 $i$ 层 | 至多 $m^{i-1}$ 个结点 |
| 高 $h$ 的 $m$ 叉树 | $h \le n \le (m^h-1)/(m-1)$ |
| 高 $h$、树的度恰为 $m$ | 最少 $h+m-1$ 个结点，须 $h\ge 2$ |
| $n$ 结点 $m$ 叉树 | 最小高度 $\lceil \log_m(n(m-1)+1)\rceil$，最大 $n$ |

$m$ 叉树允许每结点少于 $m$ 个孩子；度恰为 $m$ 要求至少一结点有 $m$ 个孩子。$m=1$ 时为链，$n=h$；单结点树的度为 0。

## 5.2 二叉树、完全性与存储

二叉树每结点至多两个孩子，左右位置有别。非空二叉树满足

$$n_0=n_2+1, \qquad h\le n\le 2^h-1, \qquad h_{\min}=\lceil \log_2(n+1)\rceil.$$

满二叉树高 $h$ 时每层均满，共 $2^h-1$ 个结点；完全二叉树的已有结点恰占满二叉树从上至下、从左至右编号的前 $n$ 个位置。完全二叉树高 $\lfloor \log_2 n\rfloor+1$，非叶 $\lfloor n/2\rfloor$ 个、叶 $\lceil n/2\rceil$ 个，度 1 结点至多一个且只能有左孩子。

| 编号起点 | 双亲 | 左孩子 | 右孩子 |
|---|---|---|---|
| 1 | $\lfloor i/2\rfloor$，$i>1$ | $2i$ | $2i+1$ |

| 编号起点 | 双亲 | 左孩子 | 右孩子 |
|---|---|---|---|
| 0 | $\lfloor (i-1)/2 \rfloor$，$i>0$ | $2i+1$ | $2i+2$ |

完全树的孩子编号不超过末编号才存在；普通树顺序存储还须检查该位置是否为空。高 $h$ 的全右链末编号为 $2^h-1$，顺序存储可能浪费大量空位。

**例 5-2**

对于任意一棵高度为 5、有 10 个结点的二叉树，采用顺序存储结构保存，每个结点占 1 个存储单元，只存放结点数据信息。存放该二叉树需要的存储单元数量至少为（　）。

A. 31　　B. 16　　C. 15　　D. 10

```c
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

`BiTree` 为根指针，空树 `T==NULL`。$n$ 个结点的二叉链表有 $2n$ 个孩子域、$n-1$ 个非空域、$n+1$ 个空域；再加双亲域时，共有 $n+2$ 个空域。

## 5.3 遍历、非递归状态与还原

先序为根—左—右，中序为左—根—右，后序为左—右—根；层序以队列按层从左到右处理。

```c
void PreOrder(BiTree T) {
    if (T!=NULL) {
        visit(T);
        PreOrder(T->lchild);
        PreOrder(T->rchild);
    }
}
void InOrder(BiTree T) {
    if (T!=NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
void PostOrder(BiTree T) {
    if (T!=NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

递归遍历时间 $\Theta(n)$、栈空间 $O(h)$。表达式树先序、后序分别给出前缀、后缀形式；中序输出须按结构添加必要括号。后序适合在孩子结果齐备后计算高度、结点数或释放结点。

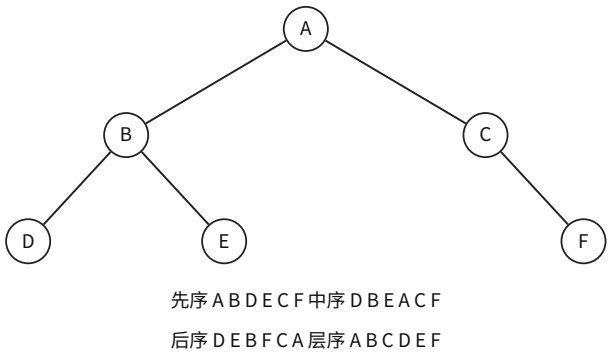



Figure 5.1: 同一二叉树的四种遍历

```c
int TreeDepth(BiTree T) {
    if (T==NULL) return 0;
    int hl=TreeDepth(T->lchild), hr=TreeDepth(T->rchild);
    return 1+(hl>hr ? hl : hr);
}
```

高度代码按空树 0、叶 1 返回；时间 $O(n)$、栈空间 $O(h)$。交换每个结点的左右孩子会反向原中序次序，保留结点身份、叶数和树高。

非递归中序沿左链压栈，遇空弹栈访问并进入右子树；后序须保存最近访问结点 `last` 判断右子树是否已完成：

```
inorder(root):
    p=root; 建空栈 S
    while p 非空 or S 非空:
        while p 非空:
            push(S,p); p=p->lchild
        p=pop(S); visit(p)
        p=p->rchild
```

```
postorder(root):
    p=root; last=NULL; 建空栈 S
    while p 非空 or S 非空:
        if p 非空:
            push(S,p); p=p->lchild
        else:
            q=top(S)
            if q->rchild 非空 and last!=q->rchild:
                p=q->rchild
            else:
                visit(q); last=pop(S)
```

每结点入栈、出栈各一次，时间 $O(n)$、栈空间 $O(h)$。层序根先入队，出队访问后依次加入非空左右孩子，时间 $O(n)$、队列最坏 $O(n)$。判完全二叉树可按层检查孩子位置：遇首个空位置后，所有后续位置均须为空。

### 由遍历序列还原

结点标识互异时：

| 给定序列 | 唯一性与步骤 |
|---|---|
| 先序＋中序 | 先序首项定根，中序分左右；按子树长度切先序 |
| 后序＋中序 | 后序末项定根，中序分左右；按长度切后序 |
| 层序＋中序 | 层序首项定根，中序分左右；稳定筛分余下层序 |
| 先序＋后序 | 一般不唯一，单孩子左右无法区分 |
| 先序＋层序、后序＋层序 | 一般不唯一，两结点单左与单右树即可说明 |

双亲信息的作

后序中，X叶若是双亲右孩子，其后继为双亲；有右兄弟的左孩子，后继为右兄弟子树的首个后序结点。兄弟位置决定返回路

只给 $n$ 个互异标识的先序序列，二叉树形态数为卡特兰数 $C_n$。重复值、显式空标记或规定满树等条件需另行分析。

例如层序 `A B C D E`、中序 `A C B E D` 还原为 `A(空,B(C,D(E,空)))`，后序为 `C E D B A`。

**例 5-3**

p、q、v 都是二叉树 T 中的结点，v 有两个孩子，T 的中序遍历序列形如 …,p,v,q,…。正确的是（　）。

A. p 没有右孩子，q 没有左孩子　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　D. p 有右孩子，q 有左孩子

## 5.4 线索二叉树

原空左域保存指定遍历的前驱，原空右域保存后继；`ltag`、`rtag` 以 0 表示孩子、1 表示线索。$n+1$ 个原空孩子域包括首前驱、末后继边界；若只统计指向普通结点的线索，边界另算。

中序线索化用 `pre` 保存刚访问结点；原链表标志须全部为 0，以下采用无头结点、首尾线索指空的表示。

```c
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;  // 0: 孩子; 1: 线索
} ThreadNode, *ThreadTree;

void InThread(ThreadTree p, ThreadTree &pre) {
    if (p!=NULL) {
        InThread(p->lchild, pre);
        if (p->lchild==NULL) {
            p->lchild=pre;
            p->ltag=1;
        }
        if (pre!=NULL && pre->rchild==NULL) {
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
        InThread(p->rchild, pre);
    }
}
void CreateInThread(ThreadTree T) {
```

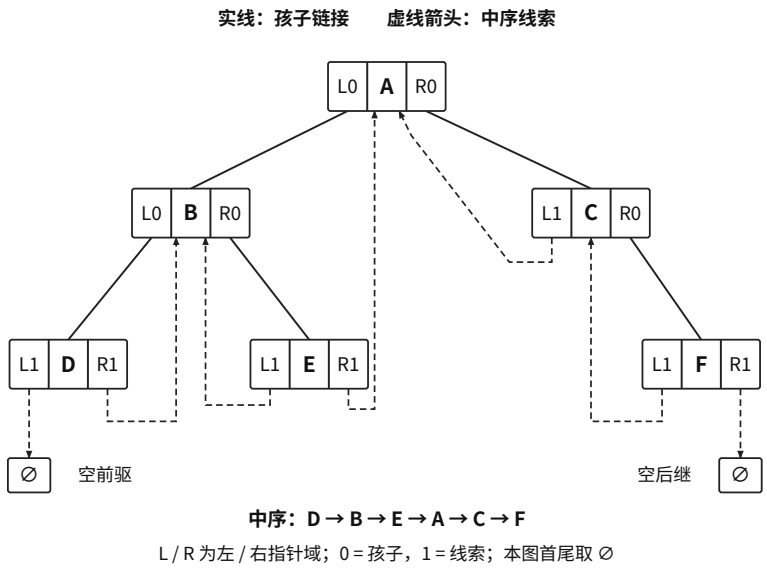



Figure 5.2: 孩子链接与中序线索

```
    ThreadTree pre=NULL;
    if (T!=NULL) {        InThread(T, pre);
        pre->rchild=NULL;
        pre->rtag=1;
    }
}
```

```
ThreadNode *Firstnode(ThreadNode *p) {
    while (p->ltag==0) p=p->lchild;
    return p;
}
ThreadNode *Nextnode(ThreadNode *p) {
    if (p->rtag==0) return Firstnode(p->rchild);
    return p->rchild;
}
void InOrderThread(ThreadNode *T) {
    if (T==NULL) return;
    for (ThreadNode *p=Firstnode(T); p!=NULL; p=Nextnode(p))
        visit(p);
}
```

`Firstnode` 要求非空参数且线索标志完整；整趟中序访问时间 $O(n)$、工作空间 $O(1)$。已线索化的树递归时须按标志只走真实孩子，避免沿线索返回祖先。

| 线索次序 | 前驱 | 后继 |
|---|---|---|
| 中序 | 左线索直接取；否则左子树最右结点 | 右线索直接取；否则右子树最左结点 |
| 先序 | 左线索直接取；有左孩子时通常需双亲信息 | 有左孩子取左孩子，否则取右域 |
| 后序 | 有右孩子取右孩子，否则取左域 | 右线索直接取；有右孩子时通常需双亲信息 |

有双亲域时：先序左孩子的前驱为双亲；右孩子有左兄弟时为左兄弟子树最后一个先序结点，否则为双亲。后序右孩子的后继为双亲；左孩子

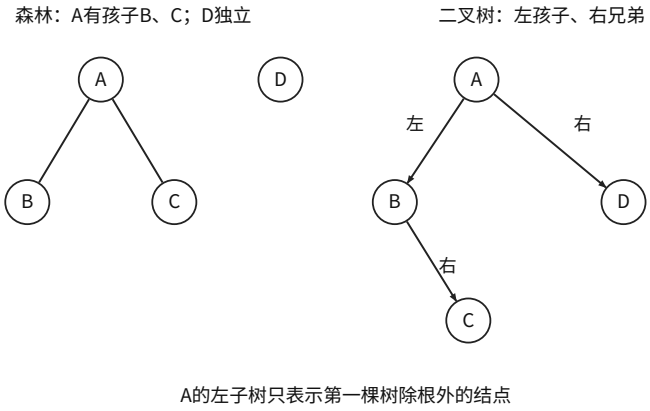



Figure 5.3: 树、森林与二叉树转换

有右兄弟时为右兄弟子树第一个后序结点，否则为双亲。

先序线索化访问根后新增的左线索可能回指祖先，递归左侧须查 `ltag`。后序末结点为根，根可能有右孩子，收尾只处理原空右域。

## 5.5 普通树、森林的存储与转换

| 表示 | 保存内容 | 直接支持的方向 |
|---|---|---|
| 双亲表示 | 每结点保存双亲下标，根为 −1 | 查双亲 $O(1)$，找全部孩子常需全扫描 |
| 孩子表示 | 顶点表保存各自孩子链 | 枚举孩子，反查双亲需搜索或附加域 |
| 孩子兄弟表示 | `firstchild`、`nextsibling` | 首孩子与下一个兄弟 |

```c
#define MAX_TREE_SIZE 100
typedef struct {
    ElemType data;
    int parent;
} PTNode;
typedef struct {
    PTNode nodes[MAX_TREE_SIZE];
    int n;
} PTree;

typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

森林双亲数组可有多个根标志，孩子兄弟表示把各根沿兄弟链接相连。转换规则为**左孩子、右兄弟**：普通树首孩子成为左孩子，后续兄弟沿右链；单棵树根无右孩子，森林根沿右链。

逆转换时，某结点的左孩子及其连续右链是原树全部孩子。二叉树根的左子树只含森林第一棵树除根外的结点，其余树在根的右子树中。

**例 5-4**

将森林转换成对应二叉树。若二叉树中 u 是 v 的父结点的父结点，则原森林中 u 与 v 可能的关系是（ ）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　B. I 和 II　C. I 和 III　D. I、II 和 III

| 原结构遍历 | 对应二叉树遍历 |
|---|---|
| 树先根、森林先序 | 先序 |
| 树后根、森林中序 | 中序 |

普通树层序遍历用兄弟链枚举出队结点的全部孩子并入队。

## 5.6 哈夫曼树与前缀编码

给定叶权 $w_i$、根到叶的边数 $d_i$：

$$WPL=\sum_i w_i d_i = \text{所有合并生成的新权值之和}, \qquad \bar{L}=\frac{WPL}{\sum_i w_i}.$$

每轮合并当前最小的两个权根，生成权值为和的新根并放回候选，直到一棵树。$n$ 个叶需 $n-1$ 轮，有 $n-1$ 个内部结点、$2n-1$ 个总结点，无度 1 结点；高最多 $n$。优先队列实现时间 $O(n\log n)$、空间 $O(n)$。

```
Q=全部带权叶构成的小根优先队列
while size(Q)>1:
    a=extractMin(Q); b=extractMin(Q)
    insert(Q, new Node(a.weight+b.weight, a, b))
返回唯一根；空输入返回空
```

最优性依据：最优树中可令两个最小权占最深兄弟叶而不增大 WPL，收缩后仍为同类最优子问题。字符置于叶，根到叶的 0、1 路径给出前缀编码；相同权选取及左右分支互换可使形态、码字不唯一，最优 WPL 相同。单符号码长另作约定。

**例 5-5**

6 个字符出现频次分别为 3、4、5、6、8、10，为它们构造哈夫曼编码，加权平均长度为（ ）。

A. 2.4　B. 2.5　C. 2.67　D. 2.75

频次 C、A、B、D 为 80、10、8、2 时，合并权为 10、20、100，WPL=130 bit，平均 1.3 bit/字符。相比固定 2 位的 200 bit 节省 35%，暂未计码表传递开销。可取 C=0、A=10、D=110、B=111。

**例 5-6**

哈夫曼编码长度不超过 4，已有两个字符编码为 1 和 01，还最多能对几个字符编码?

A. 2　B. 3　C. 4　D. 5

$m$ 叉最优合并每轮取最小 $m$ 个权；需使 $(n'-1)\bmod(m-1)=0$，缺位补最少零权虚叶，内部结点为 $(n'-1)/(m-1)=\lceil (n-1)/(m-1)\rceil$。

## 5.7 并查集

双亲数组表示集合森林。根存负的集合规模，非根存双亲下标；`Find` 返回根代表，合并参数应先求根。按规模将小树接到大树；路径压缩把查找路径直接接根，不改变集合划分。

```
void Initial(int S[], int n) {
    for (int i=0; i<n; ++i) S[i]=-1;
}
```

```
void UnionBySize(int S[], int Root1, int Root2) {
    if (Root1==Root2) return;
    if (S[Root2]>S[Root1]) {
        S[Root1]+=S[Root2];
        S[Root2]=Root1;
    } else {
        S[Root2]+=S[Root1];
        S[Root1]=Root2;
    }
}
int FindCompress(int S[], int x) {
    int root=x;
    while (S[root]>=0) root=S[root];
    while (x!=root) {
        int t=S[x];
        S[x]=root;
        x=t;
    }
    return root;
}
```

该优化组始终维护负规模；基础 `Union` 只写根标志的版本不能交叉调用。单独按规模合并时高不超过 $\lfloor \log_2 n \rfloor + 1$；同时路径压缩时 $m$ 次操作的总开销为 $O(m\alpha(n))$，为摊还界。

Kruskal 中两个端点代表相同表示已选边中连通，加边会成环；代表不同才选边并合并。

## 5.8 堆与优先队列

二叉堆同时满足完全形状与堆序：大根堆双亲不小于孩子，小根堆双亲不大于孩子。根为全局最值，兄弟间无序。

1 起始数组中叶下标 $\lfloor n/2 \rfloor + 1..n$。小根堆最大值可在叶中取得；互异关键字大根堆的第二大值位于根的孩子中。

插入先放数组末尾再上滤；删根先以末元素填根、缩小范围，再与较优孩子比较下滤。两者最坏 $O(\log n)$。下滤要求左右子树已成堆：

```
void HeadAdjust(int A[], int k, int len) {
    A[0]=A[k];
    for (int i=2*k; i<=len; i*=2) {
        if (i<len && A[i]<A[i+1]) ++i;
        if (A[0]>=A[i]) break;
        A[k]=A[i]; k=i;
    }
    A[k]=A[0];
}
```

从 $\lfloor n/2 \rfloor$ 向前逐个下滤建堆，按各高度工作量求和为 $O(n)$。逐个插入建堆的最坏上界为 $O(n \log n)$。堆只约束祖先后代间的局部序，BST 另按左右子树关键字区间有序。

# 6 图

## 6.1 图、度、路径与连通性

图 $G=(V,E)$ 由顶点集和边集组成。本章默认无自环、无平行边的简单图，$n=|V|>0$。无向边为无序对，有向弧为有序对；带权长度为沿途权值和，无权长度为边数。简单路径无重复顶点，简单回路仅首尾相同。

| 结论 | 条件 |
|---|---|
| 最大边数 $n(n-1)/2$ | 无向简单图 |
| 最大弧数 $n(n-1)$ | 有向简单图 |
| $\sum_v TD(v)=2\|E\|$，奇度顶点数为偶数 | 无向图 |
| $\sum_v ID(v)=\sum_v OD(v)=\|E\|$ | 有向图 |
| 生成树有 $n-1$ 条边 | 无向连通且无环，覆盖全部顶点 |
| 生成森林有 $n-c$ 条树边 | 无向图有 $c$ 个连通分量 |

无向连通要求任意两顶点存在路径；强连通要求有向图任意两顶点相互可达。连通/强连通分量均为相应性质的极大子图。生成树为极小连通子图，分量保留该块原有全部边。

无向图若 $|E|<n-1$ 必不连通；$|E|=n-1$ 还需连通或无环条件才能判为树；$|E|\geq n$ 必有环。保证连通需 $|E|>(n-1)(n-2)/2$。有 $c$ 个分量时 $|E|\geq n-c$。

**例 6-1**

无向图有 7 个顶点，要保证图在任何情况下都连通，需要的边数最少是（　）。

A. 6　　B. 15　　C. 16　　D. 21

$n>1$ 的强连通有向图至少 $n$ 条弧，有向环达到下界；单顶点零弧图按零长度路径也可强连通。有向树须唯一入度 0 的根、其余点入度 1，根可达所有顶点且无环。距离指最短路径长，不可达记 $\infty$。

## 6.2 存储结构与基本操作

| 表示 | 空间 | 判断给定边 | 枚举出邻点 |
|---|---|---|---|
| 邻接矩阵 | $\Theta(n^2)$ | $O(1)$ | $O(n)$ |
| 邻接表 | $\Theta(n+\|E\|)$ | 扫该顶点边链 | $O(OD(v))$ |

无向矩阵对称；有向矩阵按行计出度、按列计入度。无向邻接表每边有两个表结点，有向出边表每弧一个，`arcnum` 始终记实际边/弧数。带权无边用 $\infty$，零权边与无边分开；最短路初始化对角线为 0。

```c
#define MaxVertexNum 100
typedef struct {
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;

typedef struct ArcNode {
    int adjvex;
    struct ArcNode *next;
    // 带权图在此补充权值域
} ArcNode;
typedef struct VNode {
    VertexType data;
    ArcNode *first;
} VNode, AdjList[MaxVertexNum];
typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

顶点编号为 `0..vexnum-1`；`VertexType`、`EdgeType` 按应用定义。图遍历以下使用 `Graph` 抽象类型，已实现 `FirstNeighbor`、`NextNeighbor`，无邻点返回 $-1$；有向邻点按出弧方向。其他基本操作包括判边、枚举邻边、增删顶点/边、读写边权。

出边邻接表求全部入度可扫描所有边，时间 $O(n+|E|)$；求某点入度通常同样需扫描全表。删除顶点须删除全部入边和出边。无向矩阵可保存一侧三角，连对角线需 $n(n+1)/2$ 单元，省去已知零对角线需 $n(n-1)/2$ 单元，空间阶仍为 $O(n^2)$。

**十字链表**：每条有向弧一个结点，保存弧头、弧尾、同头链接、同尾链接；每顶点有入弧、出弧两个链头。**邻接多重表**：每条无向边一个结点，保存两端点及对应两条边链接；沿某顶点枚举须判断它位于哪一端。

0/1 邻接矩阵按普通算术相乘时，$(A^k)_{ij}$ 为 $i$ 到 $j$ 恰走 $k$ 条边的游走数，允许重复顶点和边；布尔乘法只表示存在性。

## 6.3 DFS、BFS 与搜索森林

DFS 在当前顶点无未访问邻点时回退；每次进入新顶点即标记，外层遍历全部未访问顶点。

```c
bool visited[MaxVertexNum];
void DFS(Graph G, int v) {
    visit(v);
    visited[v]=true;
    for (int w=FirstNeighbor(G,v); w>=0;
         w=NextNeighbor(G,v,w))
        if (!visited[w]) DFS(G,w);
}
void DFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) DFS(G,v);
}
```

**例 6-2**

无向图顶点为 {a,b,c,d,e,f}, 边为 {ab,ae,ac,be,cf,fd,ed}。进行深度优先遍历，不能得到的序列为（ ）。

A. acfdeb　B. aebdfc　C. aedfcb　D. abecdf

BFS 入队时标记，以防多个前驱重复入队。队列保存已发现但未扩展顶点。

**同权边的唯一性检查**

对某棵 MST 加入一条非树边会形成唯一环。若该边权等于环上最大树边权，可交换得到另一棵同权生成树。

```
void BFS(Graph G, int v) {
    SqQueue Q;
    InitQueue(Q);
    visit(v);
    visited[v]=true;
    EnQueue(Q,v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q,v);
        for (int w=FirstNeighbor(G,v); w>=0;
             w=NextNeighbor(G,v,w)) {
            if (!visited[w]) {
                visit(w);
                visited[w]=true;
                EnQueue(Q,w);
            }
        }
    }
}
void BFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) BFS(G,v);
}
```

队列应能容纳全部可能同时等待的顶点，牺牲单元的循环队列须额外一个物理单元。完整遍历时间：邻接表 $O(n+|E|)$，矩阵 $O(n^2)$；访问标志、DFS 栈或 BFS 队列最坏均 $O(n)$。

遍历序列由起点、外层扫描次序、邻点枚举次序共同决定。首次发现边组成生成森林；无向图树数等于连通分量数，有向图启动次数受扫描顺序影响，不能直接据此计强连通分量。

### BFS 求同权最短路

初始化 `d[*]=INF,path[*]=-1`、`d[s]=0`，发现未访问邻点 $w$ 时令 `d[w]=d[v]+1; path[w]=v`。队列按距离分层扩展，首次发现即取得最少边数；适用于有向或无向同权边图。

图中顶点 1 至 8 的距离依次为 `1,0,2,3,2,1,2,3`，前驱为 `2,-1,6,3,1,2,6,7`。目标 8 反向追踪得到 $8 \leftarrow 7 \leftarrow 6 \leftarrow 2$，反转为最短路 $2 \rightarrow 6 \rightarrow 7 \rightarrow 8$。

## 6.4 最小生成树

MST 在带权无向连通图中，最小化生成树的总边权；不连通时求最小生成森林。路径到某源点的距离不属于该目标。

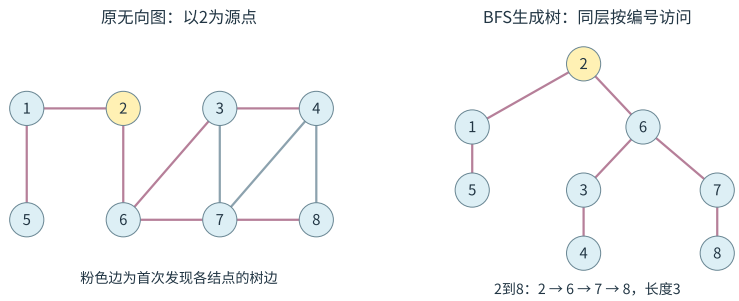



Figure 6.1: 以 2 为源点的 BFS 生成树

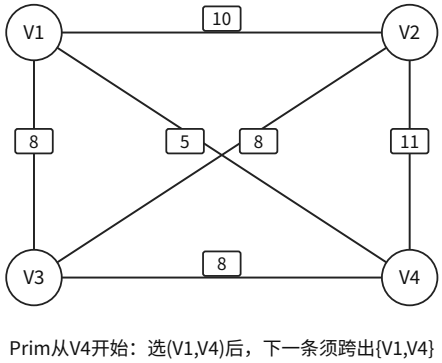



Figure 6.2: 四顶点带权无向图；三条权为 8 的边形成选择分支

```
Prim(s):
    U={s}; lowCost[v]=w(s,v)，无边记无穷
    重复直到选入全部顶点:
        u=U外lowCost最小者
        若lowCost[u]=无穷: 原图不连通，停止
        选入parent[u]到u的边; U加入u
        对每个U外邻点v:
            若w(u,v)<lowCost[v]: 更新lowCost[v]及parent[v]
```

```
Kruskal(G):
    将边按权非降排序；每个顶点初始化为单独集合
    对每条边(u,v,w):
        若find(u)!=find(v):
            选入此边; unite(u,v)
    若选入边数<n-1: 得到最小生成森林而非生成树
```

Prim 的 `lowCost[v]`保存当前集合到 $v$ 的最轻跨割边；Kruskal 按非降权处理边，端点属于不同当前分量时选边并合并。割的轻边可用交换论证安全加入：加入形成环后删去不更轻的跨割边，连通与最优性保持。

**例 6-3**

求下图最小生成树，可能为 Kruskal 第 2 次选中、但不可能为从 $V_4$ 开始的 Prim 第 2 次选中的边是（　）。

A. $(V_1, V_3)$　　B. $(V_1, V_4)$　　C. $(V_2, V_3)$　　D. $(V_3, V_4)$

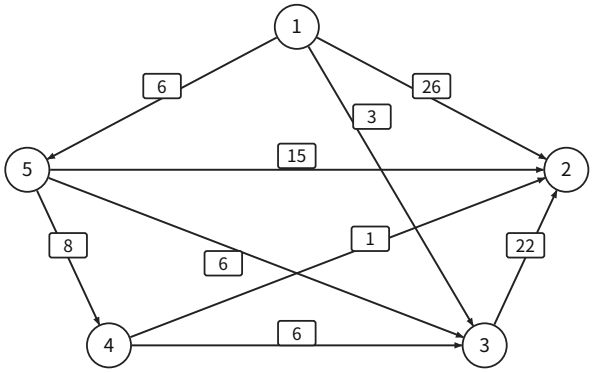



Figure 6.3: 原题有向边与权值；箭头方向参与松弛条件

**路径恢复的终止状态**

前驱链的源点标记 –1 只是回溯终止；距离为无穷才表示不可达。Floyd 中转矩阵则需按中转点递归拆分路径。

矩阵朴素 Prim 时间 $O(n^2)$；Kruskal 主要为排序 $O(|E|\log|E|)$，分量用并查集。边权互异足以保证 MST 唯一；有同权边时仍可能唯一。

## 6.5 Dijkstra 与 Floyd

Dijkstra 求非负权单源最短路。`dist` 为已知路径上界，每轮永久确定集合外 `dist` 最小顶点，再按 `dist[u]+w(u,v)` 松弛。出现负边时标准算法失去通用保证；所有剩余距离为无穷时停止。

```
Dijkstra(s):
    dist[s]=0；其余为无穷；S为空
    重复选择S外dist最小的u:
        若dist[u]=无穷：停止
        S加入u
        对每条u到v，若v未确定且dist[u]+w(u,v)<dist[v]:
            更新dist[v]；parent[v]=u
```

**例 6-4**

使用 Dijkstra 求下图从顶点 1 到其余顶点的最短路径，`dist` 依次保存到 2、3、4、5 的距离。求出第二条最短路径后，`dist` 内容为（　）。

A. 26、3、14、6　　B. 25、3、14、6　　C. 21、3、14、6　　D. 15、3、14、6

Floyd 求任意点对最短路，允许负边，要求相关路径无负环。允许前 $k$ 个中转点的阶段递推为

$$D^{(k)}[i,j]=\min\{D^{(k-1)}[i,j],\,D^{(k-1)}[i,k]+D^{(k-1)}[k,j]\}.$$

```
Floyd(D):
    D[i][i]=0；其他位置按直接边初始化
    for k=0..n-1:
        for i=0..n-1:
            for j=0..n-1:
                若D[i][k]与D[k][j]均有限:
                    D[i][j]=min(D[i][j],D[i][k]+D[k][j])
```

中转点 $k$ 必须在最外层；加法前检查两段距离有限。最终 $D[v][v]<0$ 表明负环，受其影响的点对无有限最短值。无向负边可往返形成负权闭合游走。

矩阵 Dijkstra 单源 $O(n^2)$，全源重跑 $O(n^3)$；Floyd 时间 $O(n^3)$、距离矩阵 $O(n^2)$。BFS 只最小化同权边数。

例如直接矩阵

$$D=\begin{pmatrix}0 & 6 & 13\\ 10 & 0 & 4\\ 5 & \infty & 0\end{pmatrix}$$

依次允许 0、1、2 中转，分别更新 $D[2,1]=11$、$D[0,2]=10$、$D[1,0]=9$。Floyd 路径矩阵记录更新所用中转点，递归拆分两段；Dijkstra/BFS 的 `path[v]`记录直接前驱，两者解释不同。

## 6.6 DAG、拓扑排序与有向环

AOV 网以顶点为活动、弧为先后依赖。拓扑序须使每条 $u\to v$ 的 $u$ 在 $v$ 前；存在拓扑序当且仅当图为 DAG。

```
topological(G):
    统计所有顶点的入度；把入度0者入队Q
    count=0
    while Q非空:
        u=出队；输出u；count++
        对每条u到v: indeg[v]--
            若indeg[v]==0: v入队
    返回count==n
```

邻接表时间 $O(n+|E|)$，矩阵 $O(n^2)$。每轮零入度候选恰有一个且处理完全部顶点，拓扑序唯一。顶点按拓扑顺序编号时，邻接矩阵呈严格上三角；任意编号的 DAG 未必如此。

**例 6-5**

载人航天工程包含众多子工程，为保证有序开展，需要明确前导子工程。这可抽象为有向图的拓扑序列问题。已知图 G 采用邻接矩阵存储：

```
typedef struct {
    int numVertices,numEdges;
    char VerticesList[MAXV];
    int Edge[MAXV][MAXV];
} MGraph;
```

设计 `int uniquely(MGraph G)` 判定 G 是否存在唯一拓扑序列，是则返回 1，否则返回 0。给出设计思想及带关键注释的 C 或 C++ 实现。

逆拓扑可从出度 0 开始删顶点及入弧，借助逆邻接表。DAG 的 DFS 完成顺序为逆拓扑，反转即拓扑。DFS 判有向环用三态：未访问、栈中、已完成；指向栈中顶点的边表明有环，指向已完成顶点无此保证。无向图检查回边须排除通往双亲的树边。

DAG 可共享相同子表达式：操作符和有序输入均相同才合并。例 $(a*b)*(a*b)*(a*b)*c$ 可共享一次 $a*b$；多个输入位置可指同一结

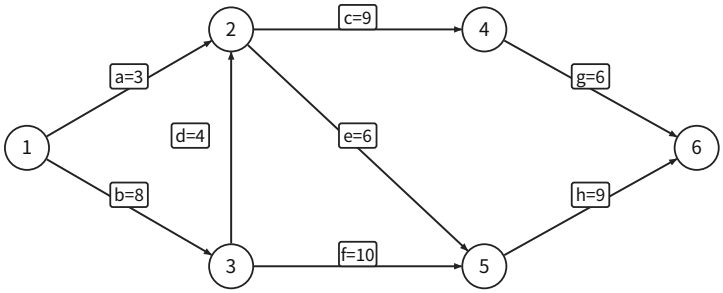



Figure 6.4: 箭头旁给出活动名和持续时间；d 的方向为 3→2

点。含副作用表达式须遵守原求值语义。

## 6.7 AOE 网与关键路径

AOE 网以顶点为事件、边为活动、非负权为持续时间，须无环。多源、多汇可用零时长虚边统一。事件等全部入活动完成才能发生。

$$
\begin{aligned}
ve[v] &= \max_{u \to v}\{ve[u] + w(u,v)\},\\
vl[u] &= \min_{u \to v}\{vl[v] - w(u,v)\},\\
e(u,v) &= ve[u], \quad l(u,v) = vl[v] - w(u,v).
\end{aligned}
$$

`ve` 按拓扑序取最大，`vl` 从汇点工期按逆拓扑取最小。活动余量 $l - e$ 为 0 时为关键活动；源至汇最长带权路径为关键路径，其长度为最短总工期。

```
criticalPath(G):
    取得拓扑序T；若不存在则模型非法
    ve[*]=0
    按T正序，对每条u到v: ve[v]=max(ve[v],ve[u]+w(u,v))
    D=ve[统一汇点]; vl[*]=D
    按T逆序，对每条u到v: vl[u]=min(vl[u],vl[v]-w(u,v))
    对每条活动u到v:
        e=ve[u]; l=vl[v]-w(u,v)
        若e==l: 该边为关键活动
```

**例 6-6**

下图 AOE 网包含 8 个活动。通过同时加快若干活动，可缩短整个工程工期。下列选项中，加快所列活动进度就可缩短工期的是（　）。

A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

增加关键活动时长会延长工期；缩短工期须使所有当前关键路径均缩短，持续压缩还可能产生新关键路径。

# 7 查找

## 7.1 关键字与平均查找长度

查找表为同类型记录的集合。主关键字唯一标识记录，次关键字可对应多条记录；静态表仅查找，动态表还支持插入、删除。存储为数组或链表不能单独决定静态、动态属性。

$$ASL = \sum_i p_i C_i.$$

先确定事件集合、概率 $p_i$ 与代价 $C_i$。成功事件为已有记录，失败事件可按关键字区间或初始散列地址分布；失败 ASL 分母未必等于表长或记录数。树结点访问次数、关键字比较次数、外存读盘次数分别统计。

## 7.2 顺序查找与分块查找

无序表逐项比较，等概率成功平均 $(n+1)/2$ 次，失败比较 $n$ 个实际记录。有序递增表从头查找时，当前值等于目标即成功，大于目标才可提前失败。

```c
typedef struct {
    ElemType *elem;
    int TableLen;
} SSTable;
int Search_Seq(SSTable ST, ElemType key) {
    ST.elem[0]=key;
    int i;
    for (i=ST.TableLen; ST.elem[i]!=key; --i) ;
    return i;
}
```

有效记录为 `elem[1..TableLen]`，0 号为可写哨兵，空表也需此位置。失败包括哨兵比较时为 $n+1$ 次；返回 0 表示失败。结构体按值传入仍共享指针所指数组，写哨兵会改变原数组 0 号单元。

静态顺序查找按访问概率非增序排列可减小成功 ASL。底稿六记录概率 `0.15,0.05,0.10,0.40,0.28,0.02` 原序 ASL 为 3.67，按概率排序后为 2.18。

**分块查找**要求块间有序、块内可无序，索引记录每块最大关键字及块起始位置。定位第一个最大关键字不小于目标的块，再块内顺序查找。索引折半未遇相等值时，结束的 `low` 可指目标块，超过末索引才直接失败。动态更新须维护块范围和索引最大值。

**例 7-1**

查找表有 400 个元素，查找每个元素的概率相同，采用均匀分块。先顺序查找确定所在块，块内也顺序查找。为使查找效率最高，每块包含元素个数应为（　）。

A. 8　　B. 10　　C. 20　　D. 25

## 7.3 折半查找与判定树

折半查找要求**关键字有序且能随机定位中点**。以下有效范围为 `elem[0..TableLen-1]`，采用闭区间及向下取整中点：

```
int Binary_Search(SSTable L, ElemType key) {
    int low=0, high=L.TableLen-1, mid;
    while (low<=high) {
        mid=(low+high)/2;
        if (L.elem[mid]==key) return mid;
        else if (L.elem[mid]>key) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
```

循环开始时若目标存在，必在 `[low,high]`；排除相等后用 `mid±1` 去掉已比较项，`low>high` 表示失败。大下标可用 `low+(high-low)/2` 防止求和溢出。

判定树成功次数等于内部结点层次，失败次数等于沿途内部结点数。$n \geq 1$ 时最大比较次数为 $\lfloor \log_2 n \rfloor + 1$。偶数区间取下中点时右子树多一项，树高平衡但未必完全；改取上中点会改变路径。

**例 7-2**

长度为 16 的顺序表 L 按关键字有序，采用折半查找，查找一个不存在的元素，关键字比较次数最多为（　）。

A. 4　　B. 5　　C. 6　　D. 7

长度 11 的表 `7,10,13,16,19,29,32,33,37,41,43` 中，查 33 比较 `29,37,32,33`；查 12 比较 `29,13,7,10` 后失败。成功层次分布 1、2、4、4 个，等概率成功 ASL 为 3；12 个失败区间中 4 个需 3 次、8 个需 4 次，失败 ASL 为 $11/3$。

有序链表缺少常数时间中点访问，折半式比较的对数比较次数无法直接保证对数总访问时间。

## 7.4 二叉搜索树

互异关键字 BST 的左子树全部值小于根、右子树全部值大于根，各子树递归满足。等价判据为中序严格递增。验证可传递全局允许区间；合法值包含整数极值时，用显式边界标志或更宽类型。

```
typedef struct BSTNode {
    int key;
    struct BSTNode *lchild, *rchild;
} BSTNode, *BSTree;
BSTNode *BST_Search(BSTree T, int key) {
    while (T!=NULL && key!=T->key) {
        if (key<T->key) T=T->lchild;
        else T=T->rchild;
    }
    return T;
}
int BST_Insert(BSTree &T, int k) {
    if (T==NULL) {
        T=(BSTree)malloc(sizeof(BSTNode));
        T->key=k;
        T->lchild=T->rchild=NULL;
        return 1;
    } else if (k==T->key) return 0;
    else if (k<T->key) return BST_Insert(T->lchild,k);
    else return BST_Insert(T->rchild,k);
}
void Create_BST(BSTree &T, int str[], int n) {
    T=NULL;
    for (int i=0; i<n; ++i) BST_Insert(T,str[i]);
}
```

插入默认申请成功，重复关键字返回 0；先搜索到空子树才建叶。操作为 $O(h)$，普通 BST 最坏可退化到 $O(n)$；迭代查找工作空间 $O(1)$。

删除分三类：叶直接断开；单孩子由孩子接替；双孩子用中序前驱或后继替换，再删除原前驱/后继。前驱无右孩子、后继无左孩子，因此归结到前两类。

```
erase(t,key):
    if t为空: return 空
    if key<t.key: t.lchild=erase(t.lchild,key)
    else if key>t.key: t.rchild=erase(t.rchild,key)
    else:
        if t.lchild为空: 保存t.rchild，释放t，返回保存值
        if t.rchild为空: 保存t.lchild，释放t，返回保存值
        q=t.rchild；沿q.lchild找到右子树最小结点
        t.key=q.key（记录有附加字段时一并复制）
        t.rchild=erase(t.rchild,q.key)
    return t
```

递归返回删除后的子树根，栈空间 $O(h)$。复制须带整条记录；要求保留结点身份时采用移接。

## 7.5 AVL 插入与删除

AVL 为任意结点左右高差至多 1 的 BST，平衡因子 $BF = h_L - h_R$。插入后从新叶向上，找最近失衡祖先 $A$：

| 失衡路径 | 调整 |
|---|---|
| LL | 对 A 右旋 |
| RR | 对 A 左旋 |

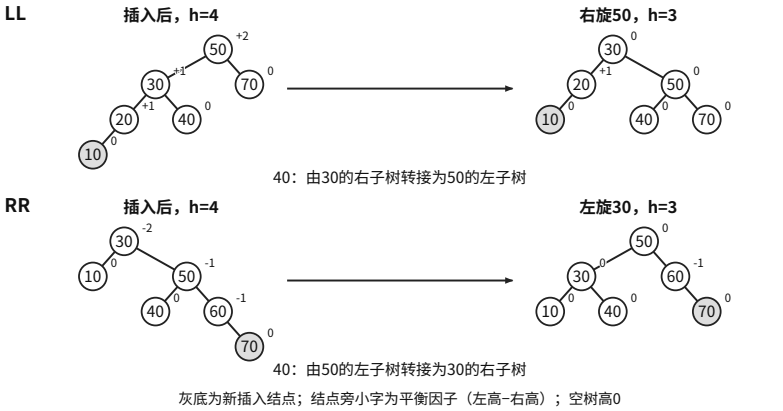



Figure 7.1: 单旋与中间子树

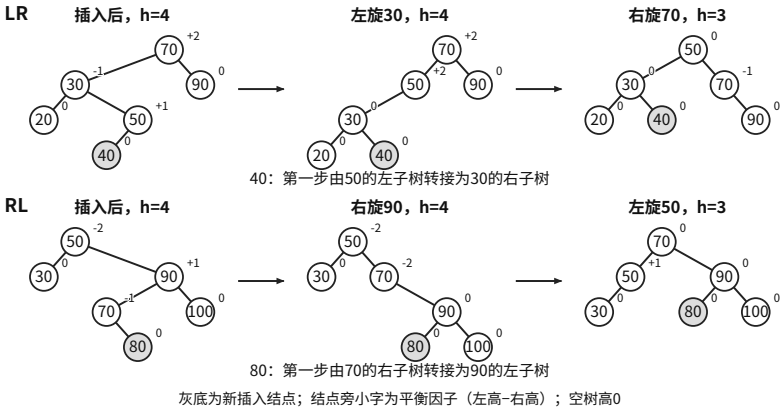



Figure 7.2: 双旋的完整两步

| 失衡路径 | 调整 |
|---|---|
| LR | 左旋 A 的左孩子，再右旋 A |
| RL | 右旋 A 的右孩子，再左旋 A |

```
void R_Rotate(BSTree &p) {
    BSTNode *lc=p->lchild;
    p->lchild=lc->rchild;
    lc->rchild=p;
    p=lc;
}
void L_Rotate(BSTree &p) {
    BSTNode *rc=p->rchild;
    p->rchild=rc->lchild;
    rc->lchild=p;
    p=rc;
}
```

引用参数接回新局部根。中间子树按关键字区间转接；若保存高度，先更新下降结点，再更新提升结点。只交换两个根值无法代替链接旋转。

**例 7-3**

把 34、23、15、98、115、28、107 依次插入初始为空的平衡二叉搜索树，要求每次插入后保持 AVL 性质，给出每步结果。

插入修复最小失衡子树后恢复原高度，可停止向上传播。最少结点数满足 $N(0)=0, N(1)=1$、$N(h)=1+N(h-1)+N(h-2)$，故高 $O(\log n)$。

**删除**先按 BST 删除，再沿实际被移除结点的祖先向上检查；修复后高度仍可能减 1，需继续传播。

| A 的 BF | 较高孩子的 BF | 调整 |
|---|---|---|
| $+2$ | $+1$ 或 0 | 右单旋 A |
| $+2$ | $-1$ | 先左旋孩子，再右旋 A |
| $-2$ | $-1$ 或 0 | 左单旋 A |
| $-2$ | $+1$ | 先右旋孩子，再左旋 A |

孩子 BF 为 0 时，单旋后高度可保持删除前高度，上层停止受影响。例如 `4(2(1,3),5)`删 5 后右旋为 `2(1,4(3,空))`，高仍为 3。查找、插删最坏均 $O(\log n)$。

固定高度形态数满足 $C(0)=C(1)=1$、$C(h)=C(h-1)^2+2C(h-1)C(h-2)$，故 $C(2)=3, C(3)=15$；该量与固定结点数的卡特兰计数不同。

**例 7-4**

一棵无重复关键字的 AVL 树，中序遍历得到降序序列。正确叙述是（　）。

A. 根的度一定为 2　　B. 最小元素一定是叶结点

C. 最后插入元素一定是叶结点　　D. 最大元素一定没有左子树

## 7.6 红黑树

红黑树为 BST 并满足：每结点红或黑；根黑；空孩子为黑 NIL；红结点的孩子黑；任一结点至后代 NIL 的路径黑结点数相同。

黑高 $b$ 不计当前结点、计终点 NIL，NIL 自身黑高 0。根黑的非空树内部结点数满足

$$2^b-1 \le n \le 2^{2b}-1.$$

相邻红结点受限，最长路径至多约为最短路径两倍，高度 $O(\log n)$。

**插入**先按 BST 插入红叶：父黑直接结束；父红时看叔。红叔则父叔黑、祖父红并上移；黑叔含 NIL，内侧折线先旋父转外侧，再旋祖父并交换提升结点与祖父颜色。最后根黑。旋转次数为常数，变色可能传播到根，完整插入仍 $O(\log n)$。

**删除**先化为至多一非空孩子：删红无黑高损失；删黑有红孩子接替时染黑；黑/NIL 接替造成缺黑。下表缺黑位置 $x$ 为父 $p$ 左孩子，兄弟 $w$ 在右，近侧为 $w$ 左孩子、远侧为 $w$ 右孩子：

| 条件 | 修复 |
|---|---|
| w 红 | w 黑、p 红，左旋 p，重新取兄弟 |
| w 黑、双孩子黑 | w 红；p 红则改黑结束，否则缺黑上移到 p |

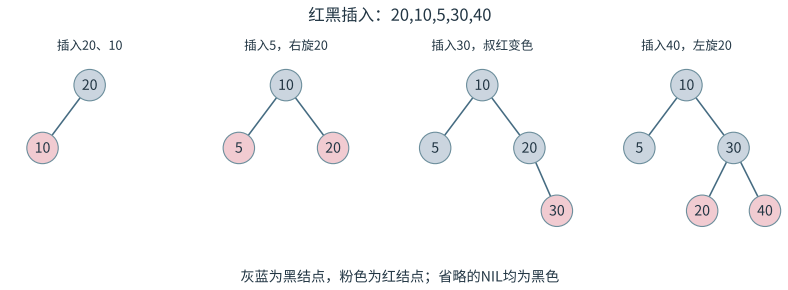



Figure 7.3: 红黑插入的黑叔旋转与红叔变色

**B 树根的特例**

非空根始终至少 1 键；非叶根可只有 2 个孩子，低于一般非根的最少孩子数。删除使根空且只余一子树时，应由子树接根。

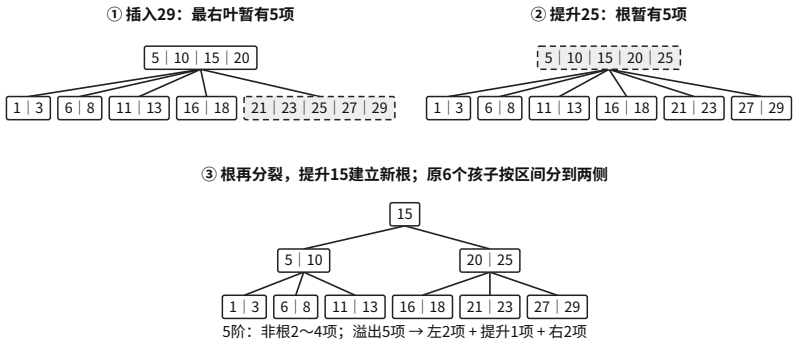



Figure 7.4: 5 阶 B 树叶分裂与根分裂

| 条件 | 修复 |
|---|---|
| w 黑、远黑近红 | 近侧黑、w 红，右旋 w，转为下一行 |
| w 黑、远红 | w 取 p 原色、p 与远侧黑，左旋 p，结束 |

缺黑在右时左右对换；缺黑到根可结束。性质检查同时核对 BST、中序、根/NIL、红红相邻及各路径黑高。

## 7.7 B 树的容量、插入与删除

$m$ 阶 B 树每数据结点至多 $m-1$ 键、$m$ 孩子；除根外至少 $t-1$ 键，$t=\lceil m/2\rceil$。非底层数据结点 $k$ 键对应 $k+1$ 个孩子区间，非叶根至少 1 键、2 孩子；底层数据结点等深。以下 $m\ge 3$，高度 $h$ 只计数据层，排除失败空叶。

$$2t^{h-1}-1\le n\le m^h-1.$$

求最大高用最少容量，求最小高用最大容量。外存一结点一页时，查找最多读 $h$ 个索引页，结点内部还需关键字比较。

**插入**沿路径到最底层数据结点插入，溢出则提升中间键、左右分裂，父溢出继续向上。内部分裂同步分配孩子，只有根分裂使高度增加。

**删除**内部键先以前驱或后继替换，再到底层删除。下溢时优先向有富余的相邻兄弟借：父分隔键下移、兄弟近侧键上移，必要时转接孩子。兄弟无富余则将下溢结点、父分隔键和兄弟合并，父可能继续下溢。空根仅剩一孩子时降层。

5 阶每非根 2 至 4 键：两个合法 2 键兄弟加父键共 5 键，超过上限；应

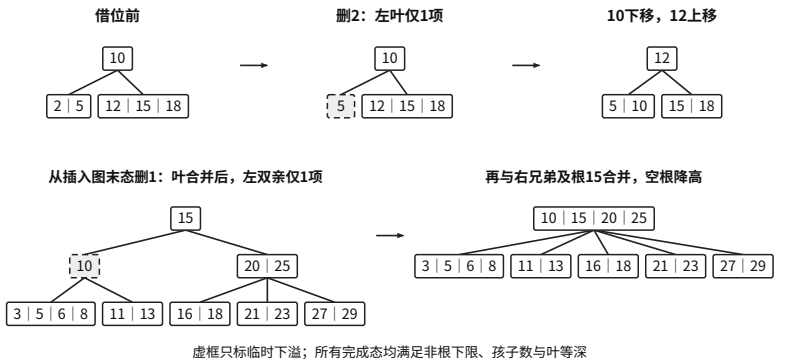



Figure 7.5: 经父分隔键借位与逐层合并

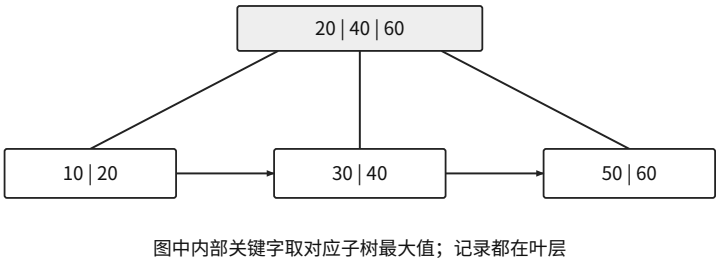



Figure 7.6: B+ 树内部索引与有序叶链

在删除造成一侧 1 键后按 $1+1+2=4$ 合并。不能直接照搬其他阶数的提前合并规则。完成态须满足键有序、子树区间、容量、孩子数及底层等深。

**例 7-5**

一棵 3 阶 B 树共有 2047 个关键字，最大高度为（ ），最小高度为（ ）。

A. 11　B. 10　C. 8　D. 7

**例 7-6**

给定 7 个不同关键字，能构造的不同 4 阶 B 树的个数最多为（ ）。

A. 7　B. 8　C. 9　D. 10

## 7.8 B+ 树

记录或记录指针集中于叶层，内部层只存索引；叶含全体关键字并按序链接。点查沿根到叶，范围查先定位起点叶，再沿叶链扫描。

本章采用王道“每个内部索引键为一棵子树最大键”约定，内部键数等于孩子数：最多 $m$，非根分支至少 $\lceil m/2 \rceil$，非叶根至少 2。B 树内部命中可结束，B+ 树记录查询仍降至叶。

每页不携整条记录通常可提高内部扇出，减少外存层数；若叶仅存记录指针，读取叶外记录的 I/O 另计。

## 7.9 散列函数与冲突处理

同一散列函数值的不同关键字为同义词。函数须确定、尽量均匀；装填因子 $\alpha = n/m$，其中 $m$ 为表长。

| 构造方法 | 条件与参数 |
|---|---|
| 直接定址 $akey + b$ | 连续且范围可控的关键字 |
| 除留余数 $key \bmod p$ | $p$ 通常取不大于表长的适当素数 |
| 数字分析 | 按已知分布选择均匀数位 |
| 平方取中 | 让多个数位共同影响结果 |

开放定址统一按 $H_i = (H(key) + d_i) \bmod m$ 计算，每次增量加在初始地址上；初始散列模数 $p$ 与探测模数 $m$ 可不同。

| 探测方法 | 增量 | 覆盖条件 |
|---|---|---|
| 线性 | $d_i = i$ | 连续 $m$ 次遍历全表 |
| 正平方 | $d_i = i^2$ | 未必覆盖全表 |
| 正负平方 | $0, 1, -1, 4, -4, \ldots$ | $m = 4j + 3$ 为素数且增量充分时可覆盖 |
| 双散列 | $i\,hash_2(key)$ | 步长与 $m$ 互质 |
| 伪随机 | 可复现的给定增量 | 检查模 $m$ 的剩余类覆盖 |

双散列步长必须模 $m$ 非零；素数表长可用 `1+(key%(m-1))`生成合法步长，负数取模另约定。线性探测易聚集；平方探测即使表未满也可能到不了空位。

### 槽状态与删除

```
hashFind(key):
    按约定的探查序列访问地址
    若槽为有效且关键字相同：成功
    若槽从未使用：失败
    若槽已删除或关键字不同：继续探查
    若可探查位置已耗尽：失败
```

插入可记住首个删除槽；若禁止重复，仍须沿后续路径排除已存在同键。表满或可达探测位置耗尽时结束，防止无限循环。链地址法真实断链删除，无需墓碑。

**例 7-7**

长度为 5、初始为空的散列表 HT，$H(k) = (k + 4) \bmod 5$，线性探查处理冲突。依次插入 2022、12、25，再删除 25，查找失败的平均查找长度为（　）。

A. 1　　B. 1.6　　C. 1.8　　D. 2.2

**例 7-8**

将 20、3、11、18、9、14、7 依次插入初始为空、长度 11 的 HT。$H(key) = (key \times 3) \bmod 11$，初始地址为 $H_0$，冲突时 $H_k = (H_0 + k^2) \bmod 11$ $(k = 1, 2, \ldots)$。

（1）画出 HT 并计算装填因子。（2）给出查找 14 时的关键字比较序列。（3）查找 8，确认失败时的散列地址是多少?

### 散列 ASL 的事件与分母

链地址成功代价为桶内比较位置，失败代价为桶长；空桶失败的关键字比较为 0，空指针判断另计。若 $m$ 个桶等概率失败，ASL 为 $n/m$；桶内有序链可提前失败，须重算概率与路径。

底稿序列 `19,14,23,1,68,20,84,27,55,11,10,79` 以 $key$ mod 13 头插入桶，非空链为 `1:79→27→1→14`、`3:55→68`、`6:84→19`、`7:20`、`10:10→23`、`11:11`。成功 ASL 为 $21/12$，13 桶等概率失败 ASL 为 $12/13$。

同序列放表长 16、初始 $key$ mod 13、线性探测模 16：0 为空，1 至 12 依次为 `14,1,68,27,55,19,20,84,79,23,11,10`，13 至 15 为空。成功 ASL 为 $30/12 = 2.5$。失败初始地址只有 0 至 12，探测长度 `1,13,12,11,10,9,8,7,6,5,4,3,2`，故失败 ASL 为 $91/13 = 7$。删除 20 留标记后，成功 ASL 为 $29/11$，失败 ASL 仍为 7。

## 7.10 按操作约束选择查找结构

| 结构 | 查找代价 | 适用条件与维护 |
|---|---|---|
| 顺序查找 | 最坏 $O(n)$ | 可无序，无有序维护 |
| 折半 | $O(\log n)$ | 有序顺序表，插删移动最坏 $O(n)$ |
| 普通 BST | $O(h)$，最坏 $O(n)$ | 动态、有序，可能退化 |
| AVL、红黑树 | 最坏 $O(\log n)$ | 动态更新并维护平衡性质 |
| B、B+ 树 | 至多 $h$ 次索引页读取 | 多路外存索引；B+ 适合叶链范围访问 |
| 散列 | 合适函数与负载下平均 $O(1)$ | 等值检索；链法最坏 $O(n)$，开放定址最坏 $O(m)$ |

有序范围检索需要关键字次序；散列扩容、树旋转、索引更新均属于维护代价。

# 8 排序

## 8.1 稳定性、内外排序与计数口径

排序移动整条记录，以关键字确定次序。稳定排序保证相等关键字记录的原相对次序；用 $2_a$、$2_b$ 追踪身份。稳定性为对全部合法输入的保证，单份无重复数据无法检验。

内部排序在处理期间全部记录驻内存；外部排序需分批交换内外存，主要计块 I/O。比较次数、移动次数、趟数和辅助空间分别统计。

本章直接插入一趟处理一个新记录，希尔一趟处理一个增量的全部组，归并一趟处理全部相邻段。快排过程题按同一深度全部未定子区间各划分一次为一趟；程序若按单次划分计趟，以具体定义为准。

**逆序数与相邻移动**

直接插入中，每向右移动一条严格较大记录就消除一个逆序；起泡每次相邻逆序交换也恰消除一个逆序。两者的数据移动量均可由初始逆序数解释。

## 8.2 直接插入与折半插入

直接插入保持有序前缀，将新值前所有严格大于它的值后移，插在旧相等值后，因而稳定。以下采用 `A[1..n]`、`A[0]`暂存兼哨兵，数组需 $n+1$ 单元。

```
void InsertSort(int A[], int n) {
    int i,j;
    for (i=2; i<=n; ++i)
        if (A[i]<A[i-1]) {
            A[0]=A[i];
            for (j=i-1; A[0]<A[j]; --j)
                A[j+1]=A[j];
            A[j+1]=A[0];
        }
}
```

有序输入每趟只比较一次，最好 $\Theta(n)$；逆序输入的有效记录后移数为 $n(n-1)/2$（暂存与写回另计），平均、最坏 $\Theta(n^2)$，空间 $O(1)$。

折半插入在有序前缀中找第一个大于新值的位置，减少比较，搬移仍可能二次：

```
void BinaryInsertSort(int A[], int n) {
    int i,j,low,high,mid;
    for (i=2; i<=n; ++i) {
        A[0]=A[i];
        low=1; high=i-1;
        while (low<=high) {
            mid=(low+high)/2;
            if (A[mid]>A[0]) high=mid-1;
            else low=mid+1;
        }
        for (j=i-1; j>=high+1; --j) A[j+1]=A[j];
        A[high+1]=A[0];
    }
}
```

每趟均完整折半定位时，总比较 $\Theta(n\log n)$，最好总时间亦 $\Theta(n\log n)$；平均、最坏移动与总时间 $\Theta(n^2)$。若先判断新值已不小于前缀末项并跳过定位，有序输入可线性，须按该实现重算。

## 8.3 起泡与简单选择

起泡仅交换相邻逆序，稳定；教材从后向前扫描，每趟把剩余最小记录送到前端。无交换即停，最好 $\Theta(n)$，平均、最坏 $\Theta(n^2)$，空间 $O(1)$。

```
void BubbleSort(int A[], int n) {
    for (int i=0; i<n-1; ++i) {
        bool flag=false;
        for (int j=n-1; j>i; --j)
            if (A[j-1]>A[j]) {
                int temp=A[j-1];
                A[j-1]=A[j]; A[j]=temp;
                flag=true;
            }
        if (flag==false) return;
    }
}
```

简单选择每趟找未排序区最小记录，与首位交换。比较总数恒为 $n(n-1)/2$，至多 $n-1$ 次交换，所有情况时间 $\Theta(n^2)$、空间 $O(1)$。远距离交换可破坏稳定性：$2_a, 2_b, 1 \to 1, 2_b, 2_a$。

```
void SelectSort(int A[], int n) {
    for (int i=0; i<n-1; ++i) {
        int min=i;
        for (int j=i+1; j<n; ++j)
            if (A[j]<A[min]) min=j;
        if (min!=i) {
            int temp=A[i]; A[i]=A[min]; A[min]=temp;
        }
    }
}
```

## 8.4 希尔排序

每个正增量 $d$ 将下标同余的记录作直接插入，完成后仅保证 $d$ 间隔有序。增量逐步缩小，末增量必须 1。希尔不稳定，复杂度依赖增量序列，空间通常 $O(1)$。

```
void ShellSort(int A[], int n) {
    int d,i,j;
    for (d=n/2; d>=1; d=d/2)
        for (i=d+1; i<=n; ++i)
            if (A[i]<A[i-d]) {
                A[0]=A[i];
                for (j=i-d; j>0 && A[0]<A[j]; j-=d)
                    A[j+d]=A[j];
                A[j+d]=A[0];
            }
}
```

有效范围 `A[1..n]`，0 号只暂存；并非每组均能经过 0 号，因此保留 `j>0` 边界。

**例 8-1**

采用希尔排序算法，输入序列为 `13,24,7,1,8,9,11,56,34,51,2,77,5`，增量依次为 5、3、1。两趟排序后为（　）。

A. 1,7,8,9,13,24,11,34,51,2,5,56,77

B. 1,7,5,2,8,9,24,11,34,51,13,77,56

C. 2,11,5,1,8,9,24,7,34,51,13,77,56

D. 2,5,11,1,8,9,7,24,34,13,51,77,56

## 8.5 快速排序与部分选择

以下首记录为枢轴、双向挖坑划分，采用 0 起始或给定闭区间：

```
int Partition(int A[], int low, int high) {
    int pivot=A[low];
    while (low<high) {
        while (low<high && A[high]>=pivot) --high;
        A[low]=A[high];
        while (low<high && A[low]<=pivot) ++low;
        A[high]=A[low];
    }
    A[low]=pivot;
    return low;
}
```

```
void QuickSort(int A[], int low, int high) {
    if (low<high) {
        int pivotpos=Partition(A,low,high);
        QuickSort(A,low,pivotpos-1);
        QuickSort(A,pivotpos+1,high);
    }
}
```

划分后枢轴就位，左区不大于枢轴、右区不小于枢轴，两区内部继续递归。单次划分 $\Theta$(区间长)；均衡划分完整时间 $\Theta(n\log n)$，连续端点划分为 $\Theta(n^2)$。首枢轴版本的有序输入为最坏情形。

随机排列或随机枢轴模型下平均 $\Theta(n\log n)$；栈平均 $O(\log n)$、最坏 $O(n)$，算法不稳定。相等值多时，划分平衡性依扫描规则而变；三路划分须独立说明状态。

例如 `49_a,38,65,97,76,13,27,49_b` 首次划分为 `27,38,13,49_a,76,97,65,49_b`，枢轴位于下标 3。

**例 8-2**

对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是（ ）。

A. 5,2,16,12,28,60,32,72

B. 2,16,5,28,12,60,32,72

C. 2,12,16,5,28,32,72,60

D. 5,2,12,28,16,32,72,60

**例 8-9**

n（n>100000）个数保存在一维数组 M 中，需要查找最小的 10 个数。

(1) 设计算法，要求平均情况下比较次数尽可能少，简述思想，不需要程序实现。

(2) 说明平均时间复杂度和空间复杂度。

## 8.6 堆排序

升序排序建大根堆，交换堆顶与当前末元素，缩小堆范围后下滤；有序后缀不再参加调整。小根堆按相同步骤将最小值放末尾，最终为降序。

```
void HeadAdjust(int A[], int k, int len) {
    A[0]=A[k];
    for (int i=2*k; i<=len; i*=2) {
        if (i<len && A[i]<A[i+1]) ++i;
        if (A[0]>=A[i]) break;
        A[k]=A[i];
        k=i;
    }
    A[k]=A[0];
}
void BuildMaxHeap(int A[], int len) {
    for (int i=len/2; i>0; --i) HeadAdjust(A,i,len);
}
void HeapSort(int A[], int len) {
    BuildMaxHeap(A,len);
    for (int i=len; i>1; --i) {
        int temp=A[i]; A[i]=A[1]; A[1]=temp;
        HeadAdjust(A,1,i-1);
    }
}
```

有效元素 `A[1..len]`，0 号暂存。下滤要求左右子树已经成堆，每层先选较大孩子再比较候选。自底向上建堆 $O(n)$，完整排序最坏 $\Theta(n\log n)$，迭代辅助空间 $O(1)$，不稳定。

**例 8-3**

小根堆为 8,15,10,21,34,16,12，删除关键字 8 之后重建堆，此过程中关键字之间的比较次数是（ ）。

A. 1 B. 2 C. 3 D. 4

53,17,78,9,45,65,87,32 从下标 4、3、2、1 依次调整，建成 87,45,78,32,17,65,53,9；第一轮排序后为 78,45,65,32,17,9,53 | 87。

## 8.7 二路归并

比较两个有序段当前首记录，相等先取左段，一段耗尽直接复制余段，保持稳定。长度 $a,b$ 的两段最坏比较 $a+b-1$ 次，具体过程可能更少。

```
// B在排序开始前分配，容量至少等于A的有效数组长度。
int *B;
void Merge(int A[], int low, int mid, int high) {
    int i,j,k;
    for (k=low; k<=high; ++k) B[k]=A[k];
    for (i=low,j=mid+1,k=i; i<=mid && j<=high; ++k)
        if (B[i]<=B[j]) A[k]=B[i++];
        else A[k]=B[j++];
    while (i<=mid) A[k++]=B[i++];
    while (j<=high) A[k++]=B[j++];
}
void MergeSort(int A[], int low, int high) {
    if (low<high) {
        int mid=(low+high)/2;
        MergeSort(A,low,mid);
        MergeSort(A,mid+1,high);
        Merge(A,low,mid,high);
    }
}
```

B 在排序前分配，容量覆盖有效下标范围。每层处理 $\Theta(n)$ 记录，共 $\Theta(\log n)$ 层；常规数组版最好、平均、最坏均 $\Theta(n\log n)$，辅助区 $\Theta(n)$，递归栈 $O(\log n)$。

自底向上把单元素视为段，段长按 1、2、4、8 增长；最后一组允许不足两个完整段。

**例 8-4**

三个有序序列 (3,5)、(7,9)、(6)，按从左到右次序选择序列进行二路归并，关键字总比较次数为（ ）。

A. 3 B. 4 C. 5 D. 6

## 8.8 基数与计数排序

LSD 从低位到高位稳定排序，每轮同桶按原顺序追加、收集，保留已处理低位关系。MSD 先按高位分桶，再递归处理各桶。负数、可变长串和空缺位需额外约定次序。

```
LSD(records,d,r):
    从最低位到最高位执行d轮：
        清空r个桶
        按当前序列从前向后，把记录追加到对应桶尾
        按桶号递增、桶内先入先出收集
```

$d$ 位、基数 $r$ 时，时间 $\Theta(d(n+r))$。已有链接域的链式记录可只用 $O(r)$ 桶头、桶尾；数组分配需 $O(n+r)$ 空间。链式分配须先保存原后继，再摘下并接桶尾；收集连接各桶首尾。递减输出按桶号逆序收集，桶内仍 FIFO。

**例 8-5**

对 `05,46,13,55,94,17,42` 进行基数排序，一趟后的结果为（　）。

A. 05,46,13,55,94,17,42

B. 05,13,17,42,46,55,94

C. 42,13,94,05,55,46,17

D. 05,13,46,55,17,42,94

### 稳定计数排序

整数范围 `min..max`，令 $k = max - min + 1$。频次累加成前缀和，再从后向前按剩余位置输出整条记录，保持稳定。

```c
void CountSort(int A[], int B[], int n, int k) {
    int i, C[k];
    for (i=0; i<k; ++i) C[i]=0;
    for (i=0; i<n; ++i) ++C[A[i]];
    for (i=1; i<k; ++i) C[i]+=C[i-1];
    for (i=n-1; i>=0; --i) {
        --C[A[i]];
        B[C[A[i]]]=A[i];
    }
}
```

该 C99 变长数组版本要求 $k > 0, n \geq 0, 0 \leq A[i] < k$，`B` 容量至少 $n$，输出在 B。一般整数用 `key-min` 作计数下标；检查范围差是否溢出和数组申请是否成功。时间、空间均 $\Theta(n+k)$，适合较小值域。

例如 $2_a, 4, 3_a, 0, 2_b, 3_b$ 的频次为 `1,0,2,2,1`，前缀和为 `1,1,3,5,6`，稳定结果为 $0, 2_a, 2_b, 3_a, 3_b, 4$。

## 8.9 外部归并：段数、缓冲与 I/O

先把内存可容纳的一批记录排成初始段，再多路归并。内存 $B$ 块，每路至少一输入缓冲、另留一输出缓冲，简单方案最多 $B-1$ 路；双缓冲等结构需另占空间。

$r \geq 1$ 个初始段、固定 $k \geq 2$ 路平衡归并：

$$s = \lceil \log_k r \rceil, \qquad \text{归并 I/O} = 2Ns, \qquad \text{含初始段生成} = 2N(1+s).$$

$N$ 为文件块数，前提为每趟全文件读写、最后输出写盘，未计寻道和缓存节省。每趟段数变为 $\lceil r/k \rceil$，末组可不足 $k$ 路。

```
mergeKRuns(runs):
    每路读入首缓冲块，另留输出块
    重复选择各非空路当前最小首记录，写入输出缓冲
    选中路的块用尽时读它的下一块
    输出块满时写外存；最后写出不足一块的剩余输出
```

16 块文件、每批装 2 块生成 8 段，生成读写 32 块；二路 $8 \to 4 \to 2 \to 1$ 共 3 趟，合计 128 块；四路 $8 \to 2 \to 1$ 共 2 趟，需 4 输入加 1 输出缓冲，合计 96 块。4500 记录、每块 75 条、内存 450 条时有 60 块、10 初始段；5 路归并两趟，含生成共 360 块 I/O。

## 置换—选择

工作区 $m$ 条，输出当前可用最小值 $x$ 后读入 $y$；$y \geq x$ 可留本段，$y < x$ 冻结至下段。可用记录耗尽才结束本段、解冻全部记录。

```
replacementSelection(input,m):
    工作区读入至多m条，标记为当前段可用
    while 工作区非空:
        若无可用记录：结束当前段，全部冻结项解冻
        输出可用记录中最小的x
        若还有输入，读入y占用x释放的位置:
            若y<x则冻结，留下一段；否则本段可用
        否则删除该工作区位置
```

输出段长度可超过工作区容量；随机输入常见平均段长约 $2m$ 的结论依赖随机模型，具体输入按实际状态计算。

**例 8-6**

对含 $n > 0$ 个记录的文件外部排序，以置换—选择生成初始归并段，工作区能保存 m 个记录。

（1）文件 19 个关键字依次为 `51,94,37,92,14,63,15,99,48,56,23,60,31,17,43,8,90,166,100`，m=4 时生成几个初始段？各是什么？

（2）对任意 m（$n \gg m > 0$），第一个初始段长度最大、最小值分别是多少？

## 败者树与最佳归并树

$k$ 路归并的败者树以各路为叶，内部保存比较败者，另存冠军。冠军输出、该路换入新首记录后，只沿原比赛路径重比，每层一次，更新 $O(\log k)$。

不等长段的最佳归并以段长为叶权，最小化归并树 WPL。二路每次合并当前两个最短段；$k$ 路每次合并当前 $k$ 个最短段，先补最少零权叶，使 $(r' - 1) \bmod (k - 1) = 0$。补虚段用于最优树的完整分支模型。

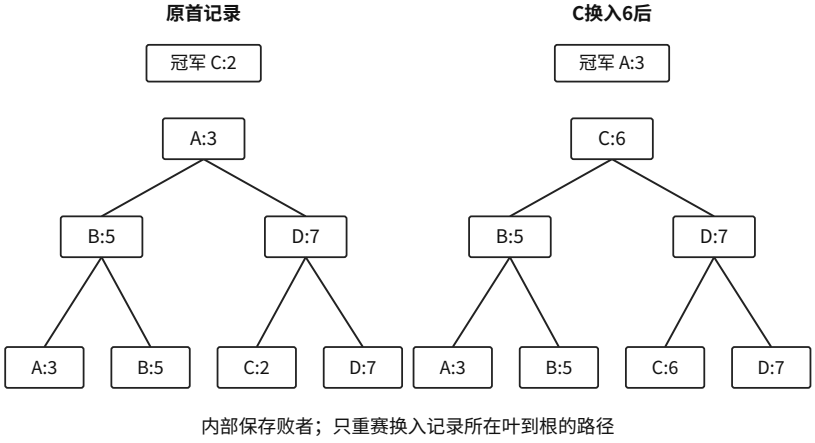



Figure 8.1: 内部保存败者，冠军单独记录

**例 8-7**

6 个升序表 A、B、C、D、E、F，分别含 10、35、40、50、60、200 个元素。通过 5 次两两合并成为一个升序表，要求最坏情况下比较总次数最小。

（1）给出完整合并过程及最坏比较总次数。（2）描述 N 个不等长升序表的合并策略并说明理由。

**例 8-8**

12 个初始段长分别为 30、44、8、6、3、20、60、18、9、62、68、85。进行 4 路外部归并，画出最佳归并树并计算 WPL。

若段长以块计，每次归并完整读写，归并阶段 I/O 为 $2WPL$，生成初始段另计；该口径按不等长段的各内部结点累计。

## 8.10 排序性能与选择

下表为完整排序总时间。平均列比较排序采用互异关键字等概率排列；折半插入每趟完整定位，起泡有早停，归并无跳过有序段优化。

| 算法 | 最好 | 平均 | 最坏 |
|---|---|---|---|
| 直接插入 | $\Theta(n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 折半插入 | $\Theta(n\log n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 起泡 | $\Theta(n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 简单选择 | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 希尔 | 依增量 | 依增量 | 依增量 |
| 快速 | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n^2)$ |
| 堆 | $O(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ |
| 二路归并 | $\Theta(n\log n)$ | $\Theta(n\log n)$ | $\Theta(n\log n)$ |
| LSD | $\Theta(d(n+r))$ | 同左 | 同左 |
| 计数 | $\Theta(n+k)$ | 同左 | 同左 |

| 算法 | 辅助空间 | 稳定性条件 |
|---|---|---|
| 直接/折半插入 | $O(1)$ | 相等新项放旧项后，稳定 |
| 起泡 | $O(1)$ | 仅严格逆序相邻交换，稳定 |

| 算法 | 辅助空间 | 稳定性条件 |
|---|---|---|
| 简单选择、希尔、堆 | $O(1)$ | 不稳定 |
| 快速 | 栈平均 $O(\log n)$、最坏 $O(n)$ | 不稳定 |
| 归并 | $\Theta(n)$ | 相等先取左段，稳定 |
| LSD | 链式 $O(r)$；数组 $O(n+r)$ | 同桶保序，稳定 |
| 计数 | $\Theta(n+k)$ | 逆向按前缀和输出整记录，稳定 |

堆排序教材常在互异关键字模型把最好列为对数线性阶；允许全相等且下滤相等即停时，本章代码的建堆与输出可总计 $\Theta(n)$。起泡无早停时，有序输入仍二次；归并若另加已有序段跳过判断，最好情况随之变化。

规模小或近有序可选直接插入；需最坏对数线性且常数空间可选堆；需稳定且可用线性辅助区可选归并；少量固定位、较小整数值域分别比较基数、计数代价。快排子区间、归并同层各段可独立处理；LSD 各位有依赖，每位内部仍可设计并行分配。

仅靠两两比较区分 $n$ 个互异关键字全部排列时，决策树至少有 $n!$ 叶，最坏比较次数至少 $\lceil \log_2(n!) \rceil = \Omega(n \log n)$。计数、基数利用关键字值域或数位结构，适用另一成本模型。

# 1 计算机系统与程序执行的层次

## 1.1 系统组成与五大功能部件

计算机系统由硬件和软件组成。主机通常包含 CPU 与主存；CPU 包含运算器、控制器和寄存器。辅存、输入设备、输出设备合称外部设备。功能部件与物理芯片的划分可不同，一个 CPU 芯片可集成多个核、Cache 和存储控制器。

| 功能部件 | 基本任务 | 典型状态或部件 |
|---|---|---|
| 运算器 | 算术、逻辑与移位运算 | ALU、操作数与结果寄存器 |
| 控制器 | 取指、译码、产生控制信号、决定后继指令 | PC、IR、控制逻辑 |
| 存储器 | 按地址保存指令与数据 | 主存保存执行所需内容，辅存提供大容量非易失空间 |
| 输入设备 | 将外部信息转换成机器能处理的形式 | 经接口输入程序和数据 |
| 输出设备 | 将内部结果转换成外部所需形式 | 经接口输出数据 |

CPU 数据通路包括寄存器、运算部件及其连接；控制器决定数据流向和操作时刻。早期冯·诺依曼机以运算器为中心，现代组织通常以存储器为中心，外设可经接口、总线和 DMA 与主存交换数据。

## 1.2 存储程序与指令执行

存储程序：执行前将指令和数据存入存储器，运行时自动取指、译码、执行。指令与数据均为二进制位串，其用途由当前执行阶段、地址和指令语义确定。冯·诺依曼机由指令控制执行次序，属于控制流驱动。

| 执行阶段 | 地址或数据来源 | 结果 |
|---|---|---|
| 取指 | PC 给出指令地址 | 指令经存储器读入 IR |
| 译码与取操作数 | IR 操作码、地址字段及寻址规则 | 确定操作、操作数和有效地址 |
| 执行与写回 | 数据通路完成运算、访存或转移 | 更新寄存器、存储器和标志 |
| 后继地址形成 | 顺序地址或转移、调用、异常入口 | 更新 PC，继续取指 |

PC 的顺序增量由地址单位和指令长度决定：按字节编址、定长 4 B 指令时通常加 4。流水线可使多条指令的各阶段重叠；程序可见效果仍须符合 ISA。

MAR 保存本次主存访问地址，MDR 保存本次交换的位串；二者可在不同阶段分别传送指令地址、数据地址以及指令、数据。

### 程序 $y = ab + c$ 的寄存器状态

按字编址，地址 0～4 存取数、乘法、加法、存数和停机指令，地址 5～8 存 $a = 2, b = 3, c = 1, y = 0$。

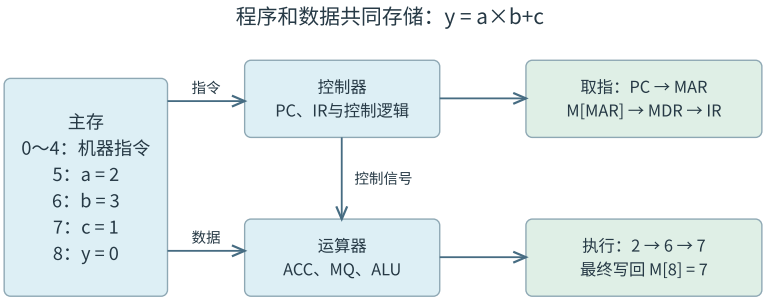



Figure 1.1: 主存按地址保存程序与数据，控制器组织取指与执行

| 指令地址 | 数据操作 | 完成状态 |
|---|---|---|
| 0 | $M[5] \rightarrow \text{ACC}$ | ACC=2，后继地址 1 |
| 1 | $\text{ACC} \times M[6] \rightarrow \text{ACC}$ | ACC=6，后继地址 2 |
| 2 | $\text{ACC} + M[7] \rightarrow \text{ACC}$ | ACC=7，后继地址 3 |
| 3 | $\text{ACC} \rightarrow M[8]$ | $M[8] = 7$，后继地址 4 |
| 4 | 停机 | 计算结束 |

示意机器中 ACC 暂存操作数或结果，MQ 辅助保存乘数或乘积低位，X 暂存另一个输入。具体寄存器配置以题设数据通路为准。

## 1.3 ISA、系统层次与软件

| 对象 | 规定或提供的内容 |
|---|---|
| ISA（指令集体系结构） | 软件可见的指令功能与编码、寄存器、数据类型、寻址方式和存储访问规则 |
| 微体系结构 | ISA 的内部实现：流水级、转发、Cache 组织、加法器结构等 |
| 操作系统 | 资源管理、进程运行环境和服务接口 |
| 系统软件 | 操作系统、语言处理程序、数据库管理系统、相关工具和库 |
| 应用软件 | 面向用户具体任务的程序 |

相同 ISA 可由不同微体系结构实现，具有不同 CPI。机器码执行还受操作系统、ABI、库及可执行文件格式约束。软硬件逻辑功能等价表示同一功能可由软件算法或专用硬件实现，选择取决于时间、空间、成本、功耗与可修改性。

常用抽象层次自下而上为微程序机器级、传统机器语言级、操作系统级、汇编语言级、高级语言级。微程序级适用于采用微程序控制的机器。“透明” 表示上层使用者无需处理下层实现细节。

### 程序翻译与装入

$$\text{源程序} \xrightarrow{\text{预处理}} \text{展开后的源程序} \xrightarrow{\text{编译}} \text{汇编程序} \xrightarrow{\text{汇编}} \text{可重定位目标文件} \xrightarrow{\text{链接}} \text{可执行文件}.$$

预处理展开头文件、宏等；链接合并目标与库，完成符号解析和重定位；装入建立进程地址空间并准备执行入口。宏和伪指令可展开为零条、一条或多条机器指令，一条高级语言语句与机器指令条数无固定对应关系。

解释器在运行时读取源程序或中间表示并完成操作，解释器自身由机器指令运行。语言实现可组合解释、预编译和即时编译。

**例 1-1**

计算机硬件能够直接执行的是（ ）。

I. 机器语言程序　II. 汇编语言程序　III. 硬件描述语言程序

A. 仅 I　B. 仅 I、II　C. 仅 I、III　D. I、II、III

## 1.4 字长、编址与容量

| 名称 | 含义 |
|---|---|
| 机器字长 | CPU 一次整数运算通常能处理的基本位宽 |
| 存储字长 | 一个存储单元保存的位数 |
| 指令字长 | 一条指令编码所占位数，可定长或变长 |
| MAR 位数 | 可编码的主存地址位数 |
| MDR 位数 | 一次主存数据交换的位宽 |

若 $m$ 位完整地址全部可用，每个地址对应 $w$ 位，则容量为 $2^m w$ bit。按字节编址时为 $2^m$ B。已安装物理容量可以小于可编址空间。

- 20 位字节地址：1 MiB；20 位、16 位字地址：2 MiB。
- $2^{10}$ 个 16 位单元：2 KiB；按字编址需 10 位，改为按字节编址需 11 位。
- 4 KiB 主存、存储字长 32 位：按字编址需 10 位，按字节编址需 12 位。

容量常用 $1\text{KiB} = 2^{10}\text{B}$、$1\text{MiB} = 2^{20}\text{B}$；频率、速率的 k、M、G 通常按 $10^3, 10^6, 10^9$。带宽为每次传送字节数乘每秒传送次数，单位进制服从题设。

## 1.5 CPU 执行时间与性能指标

响应时间为从请求到得到结果的总历时；吞吐量为单位时间完成的任务量。程序总时间可包含 CPU 工作、I/O 等待和调度等。CPU 时间按实际动态执行的指令计算。

设第 $i$ 类指令执行 $n_i$ 条、平均占 $c_i$ 个周期，主频为 $f$：

$$N = \sum_i n_i, \qquad C = \sum_i n_i c_i, \qquad \overline{\text{CPI}} = \frac{C}{N} = \sum_i \frac{n_i}{N} c_i,$$

$$T_{\text{CPU}} = \frac{N\overline{\text{CPI}}}{f} = N\overline{\text{CPI}} T_{\text{clk}}, \qquad T_{\text{clk}} = \frac{1}{f}.$$

平均 CPI 以指令条数为权重；Cache 缺失、流水停顿等也计入统计范围内的周期。多发射处理器的平均 CPI 可以小于 1，单条指令仍可能经历多个周期的延迟。

| 指标 | 定义及计算条件 |
|---|---|
| IPC | 每周期平均完成指令数；同一统计区间下 IPC $= 1/\overline{\text{CPI}}$ |
| MIPS | 每秒百万条指令，$f/(\overline{\text{CPI}} \times 10^6)$ |
| MFLOPS ／ GFLOPS ／ TFLOPS | 每秒浮点运算数分别除以 $10^6/10^9/10^{12}$ |
| 同一任务性能比 | $P_A/P_B = T_B/T_A$ |

不同 ISA、编译器和优化方案可改变动态指令数与指令组成。比较同一任务优先使用实际执行时间，主频、CPI、MIPS 和字长各自只描述部分条件。

**例 1-2**

某计算机主频为 1 GHz，程序 P 运行过程中共执行 10000 条指令，其中 80% 的指令执行平均需 1 个时钟周期，20% 的指令执行平均需 10 个时钟周期。程序 P 的平均 CPI 和 CPU 执行时间分别是（ ）。

A. 2.8，28 μs　B. 28，28 μs　C. 2.8，28 ms　D. 28，28 ms

### 性能变化的时间比

同一任务，前后两方案满足同一 CPU 时间模型时：

$$\frac{T'}{T} = \frac{N'}{N}\,\frac{\text{CPI}'}{\text{CPI}}\,\frac{f}{f'}.$$

编译优化可同时改变 $N$ 与 CPI；Cache 改进通常减少停顿；升频还可能伴随 CPI 变化。若 20 s 程序指令数变为 70%、CPI 变为 1.2 倍且主频不变，时间为 $20 \times 0.7 \times 1.2 = 16.8$ s。

## 1.6 局部加速与 Amdahl 定律

原时间中比例 $p$ 的部分加快为原速度的 $s$ 倍，其他部分不变，各部分时间可相加且无新增开销时：

$$T' = T[(1-p) + p/s], \qquad S = \frac{T}{T'} = \frac{1}{(1-p) + p/s}.$$

当 $s \to \infty$ 且 $p < 1$，加速比上限为 $1/(1-p)$。“速度提高 50%” 对应 $s = 1.5$。计算与 I/O 重叠、通信或同步开销变化时，先按实际时间关系重新建模。

**例 1-3**

假定基准程序 A 在某计算机上的运行时间为 100 s，其中 90 s 为 CPU 时间，其余为 I/O 时间。若 CPU 速度提高 50%，I/O 速度不变，则运行基准程序 A 所耗费的时间是（ ）。

A. 55 s　B. 60 s　C. 65 s　D. 70 s

增加处理器可让独立任务并发，提高吞吐量。两个各需单核 2 s 的任务在双核上同时开始，全部完成历时可由 4 s 变为 2 s；每个无排队任务的执行时间仍为 2 s。

## 1.7 器件发展阶段

电子管 → 晶体管 → 中小规模集成电路 → 大规模与超大规模集成电路。集成度提高推动可靠性、容量、速度和微型化、多核与并行计算发展。摩尔定律描述晶体管数量增长的历史经验趋势，程序加速仍由具体执行时间模型判断。

# 2 数据表示、位级运算与浮点数

## 2.1 进位计数制与位串解释

位串的真值由位宽、编码和小数点位置共同确定。1 B=8 bit，B 表示字节，b 表示位。

$$(a_k \cdots a_0.a_{-1} \cdots)_r = \sum_i a_i r^i, \qquad 0 \le a_i < r.$$

| 转换 | 规则 |
|---|---|
| 十进制整数转 $r$ 进制 | 除 $r$ 取余，余数倒序排列 |
| 十进制小数转 $r$ 进制 | 乘 $r$ 取整，整数顺序排列 |
| 二进制与八进制 | 从小数点向两侧每 3 位一组；整数端左补 0，小数端右补 0 |
| 二进制与十六进制 | 从小数点向两侧每 4 位一组，补 0 规则同上 |

有限十进制小数可有无限二进制展开，如 0.1；有限位存储时须按规定截断或舍入。

**例 2-1**

若十进制数为 137.5，则其八进制数为（　）。

A. 89.8　　B. 211.4　　C. 211.5　　D. 1011111.101

## 2.2 定点数编码与范围

真值是数学数值，机器数是约定编码下的位串。定点表示约定固定小数点位置。

| $n$ 位整数编码 | 数值规则 | 范围与零 |
|---|---|---|
| 无符号 | 全部位为正权 | $0 \sim 2^n - 1$ |
| 原码 | 最高位符号，其余为绝对值 | $-(2^{n-1}-1) \sim 2^{n-1}-1$，两个零 |
| 反码 | 正数同原码，负数数值位逐位取反 | 同原码范围，两个零 |
| 补码 | 最高位权 $-2^{n-1}$，其余正权 | $-2^{n-1} \sim 2^{n-1}-1$，一个零 |

补码值：

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i = \begin{cases} u, & u < 2^{n-1}; \\ u - 2^n, & u \ge 2^{n-1}, \end{cases}$$

其中 $u$ 为该位串的无符号值。负数补码可由同宽绝对值逐位取反加 1 得到。8 位 `FDH` 表示 $-3$，`80H` 表示 $-128$。最小负数的数学相反数超出同宽补码范围，同宽取反加 1 会保留原位串。

原码的 $-0$ 为 `1000⋯0`，反码的 $-0$ 为 `1111⋯1`。一位符号、$n-1$ 位小数时，原码、反码范围为 $[-(1-2^{-(n-1)}), 1-2^{-(n-1)}]$；补码范围为 $[-1, 1-2^{-(n-1)}]$。

整数移码采用偏置 $2^{n-1}$ 时，$E = x + 2^{n-1}$，其编码由同宽补码翻转最高位得到。IEEE 阶码采用另一偏置，按字段定义计算。

## 2.3 截断、扩展与 C 类型转换

| 变换 | 位级规则与条件 |
|---|---|
| 缩短位宽 | 题设截断模型保留低位；保值需被删除高位满足原值扩展关系 |
| 无符号扩展 | 高位补 0 |
| 补码扩展 | 高位复制原符号位 |
| 转 $n$ 位无符号数 | 按模 $2^n$ 的值转换 |
| 补码扩大后保值 | 新增高位均为原符号位 |

源数的解释决定扩展方式。16 位无符号 65530 扩为 32 位是 `0000 FFFAH`；16 位补码 $-32767$ 转 32 位无符号是 $2^{32}-32767$，位串 `FFFF 8001H`。

**例 2-2**

假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```c
unsigned short x = 65530;
unsigned int y = x;
```

得到 y 的机器数为（　）。

A. 0000 7FFAH　　B. 0000 FFFAH　　C. FFFF 7FFAH　　D. FFFF FFFAH

**例 2-3**

在 32 位计算机上执行下列 C 语言代码段后，ui 的值是（　）。

```c
short si = -32767;
unsigned int ui = si;
```

A. $2^{15}-1$　　B. $2^{15}+1$　　C. $2^{32}-2^{15}-1$　　D. $2^{32}-2^{15}+1$

C 表达式先作整数提升和通常算术转换，再确定比较与运算类型。32 位 `int i=-1; unsigned u=1;` 的常见模型中，`i<u` 为假，因 i 转换为 $2^{32}-1$。

本章 C11/C17 口径下：无符号运算按模 $2^n$；有符号算术溢出属未定义行为；超范围转有符号类型涉及实现规定。题设机器的截断、绕回模型应与语言保证分别使用。

### 逻辑移位与算术移位

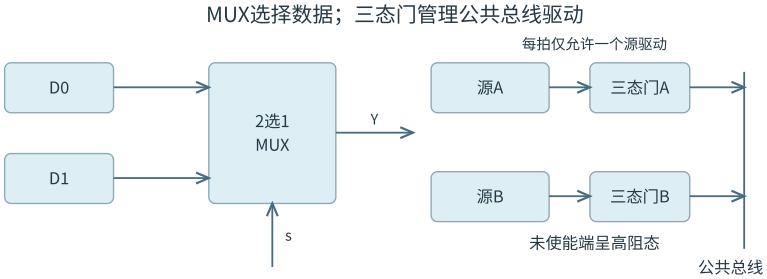



Figure 2.1: 选择器选择数据源，三态门控制公共线驱动

| 操作 | 补入位与保值条件 |
|---|---|
| 逻辑右移 | 高位补 0，无符号值除 $2^k$ 向下取整 |
| 补码算术右移 | 高位补符号位，值除 $2^k$ 向负无穷取整 |
| 左移 | 低位补 0；高位丢失可能导致溢出 |

8 位补码 $-3$ 算术右移 1 位为 $-2$；C 整数 $-3/2$ 向零截断为 $-1$。C11/C17 负有符号数右移涉及实现规定，使用题设算术右移模型时须明确该条件。左移按位结果与 C 有符号左移的合法性分别检查。

## 2.4 逻辑门、选择器与三态门

| $A, B$ | 与 $AB$ | 或 $A+B$ | 异或 $A \oplus B$ |
|---|---|---|---|
| 0,0 | 0 | 0 | 0 |
| 0,1 | 0 | 1 | 1 |
| 1,0 | 0 | 1 | 1 |
| 1,1 | 1 | 1 | 0 |

逻辑非、与、或的通常优先级依次降低；括号及非号覆盖范围优先。德摩根律：$\overline{A+B}=\bar{A}\bar{B}$，$\overline{AB}=\bar{A}+\bar{B}$。多个位异或为 1，当且仅当其中 1 的个数为奇数；可用于本位和与奇偶校验。

$k$ 选 1 MUX 至少需 $\lceil \log_2 k \rceil$ 位选择编码；2 选 1：$Y=\bar{s}D_0+sD_1$。32 位 MUX 同时选择 32 条对应位。

三态门关闭时输出高阻态，开启时驱动 0 或 1。高阻表示不主动驱动公共线。共享总线同一时刻须满足驱动源控制约束，多个源输出相反电平会争用。

## 2.5 加减运算器与进位逻辑

规定 $C_i$ 为第 $i$ 位输入进位，$C_{i+1}$ 为输出进位：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad G_i = A_i B_i, \quad P_i = A_i \oplus B_i,$$

$$C_{i+1} = G_i + P_i C_i.$$

例如 $C_2 = G_1 + P_1G_0 + P_1P_0C_0$。串行进位逐级传播；先行进位用组合逻辑展开进位，增加电路复杂度以缩短关键路径。“并行输入各位” 与 “并行生成进位” 是两项实现性质。

加减控制位 Sub 令第二输入为 $B \oplus \text{Sub}$、最低进位 $C_0 = \text{Sub}$：

$$\text{Sub} = 0:\ A + B; \qquad \text{Sub} = 1:\ A + \overline{B} + 1 = A - B \pmod{2^n}.$$

ALU 按操作控制选择算术、与、或、异或等功能，移位器位置以具体通路为准。数据位宽与操作选择编码位宽分别确定。

**例 2-4**

某计算机字长为 8 位，CPU 中有一个 8 位加法器。已知无符号数 $x = 69, y = 38$，若在该加法器中计算 $x - y$，则加法器的两个输入端信息和输入的低位进位信息分别为（　）。

A. 0100 0101、0010 0110、0
B. 0100 0101、1101 1001、1
C. 0100 0101、1101 1010、0
D. 0100 0101、1101 1010、1

## 2.6 运算标志与条件转移

本章采用 CF=1 表示减法借位的约定。设 $n$ 位加减器最高输出进位为 $C_n$：

$$\text{CF} = C_n \oplus \text{Sub}, \quad \text{OF} = C_{n-1} \oplus C_n, \quad \text{SF} = S_{n-1}, \quad \text{ZF} = [S = 0].$$

CF 判断无符号溢出或借位；OF 判断补码有符号溢出。同号相加结果异号时加法溢出；异号相减结果与被减数异号时减法溢出。采用 “无借位” 进位约定的 ISA 应按其定义改判 CF。

| 8 位运算 | 低 8 位 | CF | OF | SF | ZF |
|---|---|---|---|---|---|
| `7FH+01H` | `80H` | 0 | 1 | 1 | 0 |
| `FFH+01H` | `00H` | 1 | 0 | 0 | 1 |
| `80H-01H` | `7FH` | 0 | 1 | 0 | 0 |
| `00H-01H` | `FFH` | 1 | 0 | 1 | 0 |

计算 $a - b$ 后的比较条件：

| 条件 | 标志表达式 |
|---|---|
| 相等 | ZF=1 |
| 无符号 $a < b$ | CF=1 |
| 无符号 $a > b$ | CF=0 且 ZF=0 |
| 有符号 $a < b$ | SF ≠ OF |
| 有符号 $a > b$ | SF=OF 且 ZF=0 |

**例 2-5**

假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x−y 的机器数分别是（　）。

A. x=−65，y=41，x−y 的机器数溢出
B. x=−33，y=65，x−y 的机器数为 FFFF FF9DH
C. x=−33，y=65，x−y 的机器数为 FFFF FF9EH
D. x=−65，y=41，x−y 的机器数为 FFFF FF96H

**例 2-6**

已知 x、y 为 int 类型，当 x=100、y=200 时，执行 “x 减 y” 指令得到的溢出标志 OF 和借位标志 CF 分别为 0、1。那么当 x=10、y=−20 时，执行该指令得到的 OF 和 CF 分别是（ ）。

A. OF=0，CF=0　B. OF=0，CF=1　C. OF=1，CF=0　D. OF=1，CF=1

## 2.7 无符号乘法与乘法器组织

两个 $n$ 位数完整乘积最多需 $2n$ 位。无符号顺序乘法初态 $P=0$，$Y$ 装乘数，$X$ 装被乘数。每轮：检查 $Y_0 \to$ 若为 1 则 $P \leftarrow P+X$ 并保留进位 $\to (C,P,Y)$ 整体逻辑右移；共 $n$ 轮。

$X=1101, Y=1011$ 时：

| 轮次 | $Y_0$；加法后 $C,P$ | 右移后 $P,Y$ |
|---|---|---|
| 初态 | — | 0000,1011 |
| 1 | 1；0,1101 | 0110,1101 |
| 2 | 1；1,0011 | 1001,1110 |
| 3 | 0；0,1001 | 0100,1111 |
| 4 | 1；1,0001 | 1000,1111 |

结果 `10001111`=143。若只保存低 $n$ 位，高 $n$ 位非零表示无符号乘积溢出。

| 乘法器组织 | 特征 |
|---|---|
| 顺序乘法器 | 复用加法器与移位器，跨多周期完成 |
| 阵列乘法器 | 并行生成并压缩部分积，延迟由组合路径决定 |
| 流水线乘法器 | 多级寄存器分隔组合路径，多组运算可重叠，提高吞吐率 |

常数乘法可在语义与位宽允许时化为移位加减，如 $13x=(x\ll 3)+(x\ll 2)+x$。

### Booth 补码一位乘法

乘数后附 $q_{-1}=0$，每轮观察 $(Y_0,q_{-1})$：00、11 加 0，01 加 $X$，10 减 $X$；随后 $(P,Y,q_{-1})$ 整体算术右移。符号位参加运算，跨寄存器边界传位。

$X=1101=-3, Y=0110=6$：

| 轮次 | 观察位与操作 | 右移后 $P,Y,q_{-1}$ |
|---|---|---|
| 初态 | — | 0000,0110,0 |
| 1 | 00，加 0 | 0000,0011,0 |
| 2 | 10，减 X | 0001,1001,1 |
| 3 | 11，加 0 | 0000,1100,1 |
| 4 | 01，加 X | 1110,1110,0 |

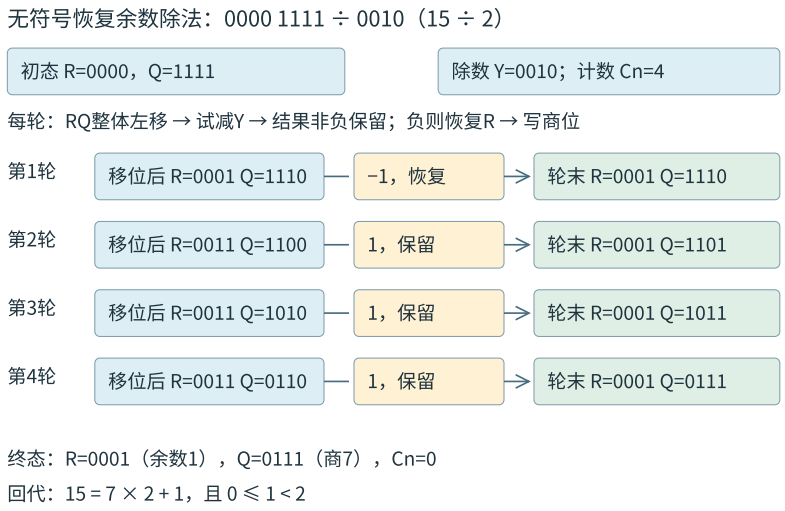



Figure 2.2: 15 除以 2 的试减、恢复和商位写入

完整乘积 `11101110`=−18。保留低 $n$ 位补码时，未溢出条件为完整积的高 $n$ 位均为低半部符号位的扩展，即完整积最高 $n+1$ 位全部相同。最小负数的取负及中间操作须配置足够保护位。

## 2.8 除法、商溢出与余数校正

无符号双精度被除数以 $(R, Q)$ 保存，各 $n$ 位，除数 $Y$ 为 $n$ 位。单精度被除数先零扩展。除数 $Y > 0$ 时，初始高半部 $R \geq Y$ 意味着商至少为 $2^n$，产生商溢出。4 位商中，255/15=17 溢出，200/15=13 余 5 可表示。

| 算法 | 每轮更新与结束条件 |
|---|---|
| 恢复余数法 | 移入下一被除数位，试减除数；非负保留差、商位 1；负则加回除数、商位 0 |
| 不恢复余数法 | 根据上一轮余数符号决定本轮减或加除数；按新余数生成商位；最终负余数须按规则校正 |

无符号最终结果必须满足 $N = QY + R$ 且 $0 \leq R < Y$。试减的暂时负值需符号或借位状态支持。

### 恢复余数法的寄存器轨迹

15/2 初态 $R = 0000, Q = 1111, Y = 0010$，每轮先整体左移 $RQ$，再试减并记录商位，共 4 轮，得到 $Q = 0111, R = 0001$。

不恢复余数法的 7/3，取 4 位有符号 $R$、3 位 $Q$：

| 轮次 | 整体左移后 R | 本轮操作 | 操作后 R | 记商后 Q |
|---|---|---|---|---|
| 初态 | — | — | 0 | 111 |
| 1 | 1 | 减 3 | −2 | 110 |
| 2 | −3 | 加 3 | 0 | 101 |
| 3 | 1 | 减 3 | −2 | 010 |

末轮 $R < 0$，再加 3 校正为 1；商 2、余 1。

### 有符号除法与异常

按向零截断约定，商符号由两操作数符号异或确定，非零余数与被除数同号，$X = QY + R$。例如 $-15/2$ 的商 $-7$、余 $-1$；$15/(-2)$ 的商 $-7$、余 1。

课件 4 位补码 $-7/3$：被除数符号扩展为 $R = 1111, Q = 1001, Y = 0011$。本例负余数试加正除数；试加改变原负号时恢复，否则保留并生成商的绝对值位。

| 轮次 | 左移后 R、Q | 试加 R | 轮末 R、Q |
|---|---|---|---|
| 1 | 1111、0010 | 0010，恢复 | 1111、0010 |
| 2 | 1110、0100 | 0001，恢复 | 1110、0100 |
| 3 | 1100、1000 | 1111，保留 | 1111、1001 |
| 4 | 1111、0010 | 0010，恢复 | 1111、0010 |

最后商绝对值 2，异号取补为 1110=$-2$；余数 1111=$-1$。整除时需单独正确接受试算零。也可在足够宽的内部表示中先求绝对值，完成无符号除法后恢复符号。

除数 0 须异常处理；$-2^{n-1}/(-1)$ 超出 $n$ 位补码商范围。固定轮数除法器按位宽迭代；提前退出须由题设明确给出。

## 2.9 IEEE 754 字段与特殊值

设尾数字段整数值为 $F$、位数为 $p$、阶码为 $E$、偏置为 $B$，符号为 $s$。

| 参数 | binary32 | binary64 |
|---|---|---|
| 符号／阶码／尾数字段位数 | 1 ／ 8 ／ 23 | 1 ／ 11 ／ 52 |
| 偏置 $B$ | 127 | 1023 |
| 规格化有效精度 | 24 位 | 53 位 |
| 规格化真阶码 | –126～127 | –1022～1023 |
| 最小正规格化数 | $2^{-126}$ | $2^{-1022}$ |
| 最小正次正规数 | $2^{-149}$ | $2^{-1074}$ |
| 最大有限正数 | $(2-2^{-23})2^{127}$ | $(2-2^{-52})2^{1023}$ |

| 字段条件 | 数值／含义 |
|---|---|
| $E=0, F=0$ | 正零或负零，由 $s$ 区分 |
| $E=0, F\neq 0$ | 次正规数（非规格化数），$(-1)^s(F/2^p)2^{1-B}$，隐含前导 0 |
| $E$ 既非全 0 也非全 1 | 规格化数，$(-1)^s(1+F/2^p)2^{E-B}$，隐含前导 1 |
| $E$ 全 1，$F=0$ | 正无穷或负无穷 |
| $E$ 全 1，$F\neq 0$ | NaN |

binary32 位串 `00000001H` 为 $2^{-149}$，`00800000H` 为 $2^{-126}$。编码前先规格化绝对值，最后独立写符号位；负浮点数按符号、指数、尾数解读。比较有限同号负数时，绝对值较大者数值较小。

**例 2-7**

$-0.4375$ 的 IEEE 754 单精度浮点数表示为（　）。

A. BEE0 0000H　B. BF60 0000H　C. BF70 0000H　D. C0E0 0000H

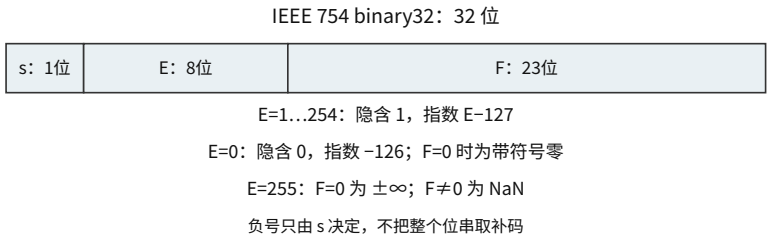



Figure 2.3: 单精度字段与指数编码

**例 2-8**

float 型数据常用 IEEE 754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若 `(f1)=CC90 0000H`、`(f2)=B0C0 0000H`，则 x 和 y 之间的关系为（　）。

A. x<y 且符号相同　B. x<y 且符号不同　C. x>y 且符号相同　D. x>y 且符号不同

## 2.10 浮点加减与舍入

顺序为：对阶 → 尾数加减 → 规格化 → 舍入 → 检查规格化与阶码范围。

| 步骤 | 规则与边界 |
|---|---|
| 对阶 | 小阶向大阶对齐，小阶尾数右移；最终阶码为原合法大阶，可能损失低位 |
| 右规 | 尾数出现 $10.xxx_2$ 时右移 1 位、阶码加 1，可能上溢 |
| 左规 | 抵消后左移尾数、阶码减少，可能触及次正规区或下溢边界；精确零单独处理 |
| 舍入 | 可能使尾数进位，再右规并检查上溢 |
| 渐进下溢 | 指数过小时可能形成次正规数，接近 0 后才舍入为零 |

最近偶数舍入：被舍部分小于半个间隔取低端，大于半个间隔取高端，恰好居中取保留有效数最低位为 0 的候选。保护位 G、舍入位 R、粘滞位 S 保留舍入依据，S 为更低被舍位的逻辑或；最近偶数下，$G=1$ 且 $(R \vee S \vee \text{LSB})=1$ 时向保留部分进 1。

**例 2-9**

下列有关浮点数加减运算的叙述中，正确的是（　）。

I. 对阶操作不会引起阶码上溢或下溢
II. 右规和尾数舍入都可能引起阶码上溢
III. 左规时可能引起阶码下溢
IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III　B. 仅 I、II、IV　C. 仅 I、III、IV　D. I、II、III、IV

浮点加法可能不满足结合律。binary32 取 $a=2^{25}, b=-2^{25}, c=1$，则 $(a+b)+c=1$，而 $b+c$ 先舍入回 $b$，得到 $a+(b+c)=0$。

### 浮点精度与数值范围

binary32 在 $[2^e, 2^{e+1})$ 中的相邻正规格化数间隔为 $2^{e-23}$。所有 $[-2^{24}, 2^{24}]$ 内整数可精确表示，界外部分整数仍可精确表示；在 $[2^{24}, 2^{25})$ 中间隔

2，在 $[2^{25}, 2^{26})$ 中间隔 4。

| 精确数学结果 | binary32 最近偶数结果 |
|---|---|
| $2^{24}+0.5$ | 16777216 |
| $2^{24}+1$ | 16777216 |
| $2^{24}+1.5$ | 16777218 |
| $(2^{24}+2)+1$ | 16777220 |

精确性由有效位与舍入决定，有限范围由指数和尾数上界共同决定。32 位补码在其范围内表示连续整数；binary32 的数值范围较大、有效精度为 24 位。

**例 2-10**

已知 $f(n)=\sum_{i=0}^{n}2^i=2^{n+1}-1=\underbrace{11\cdots1}_{n+1位}{}_2$，计算 $f(n)$ 的 C 语言函数 f1 如下。

```c
int f1(unsigned n) {
    int sum = 1, power = 1;
    for (unsigned i = 0; i <= n - 1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float，可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位，float 采用 IEEE 754 单精度标准。请回答下列问题。

(1) 当 n=0 时，f1 会出现死循环，为什么？若将 f1 中的变量 i 和 n 都定义为 int 型，则 f1 是否还会出现死循环？为什么？

(2) f1(23) 和 f2(23) 的返回值是否相等？机器数各是什么（用十六进制表示）？

(3) f1(24) 和 f2(24) 的返回值分别为 33 554 431 和 33 554 432.0，为什么不相等？

(4) $f(31)=2^{32}-1$，而 f1(31) 的返回值却为 $-1$，为什么？若使 f1(n) 的返回值与 f(n) 相等，则最大的 n 是多少？

(5) f2(127) 的机器数为 7F80 0000H，对应的值是什么？若使 f2(n) 的结果不溢出，则最大的 n 是多少？若使 f2(n) 的结果精确（无舍入），则最大的 n 是多少？

32 位 int 转 binary64 可精确，转 binary32 可能丢低位。binary32 有限值扩为 binary64 可精确保留，已经丢失的信息无法恢复。浮点转整数向零截断且须检查目标范围；NaN 与任何值含自身的相等比较为假。

## 2.11 字节序、边界对齐与数组地址

| 存放规则 | 最低地址保存的字节 |
|---|---|
| 大端 | 最高有效字节；12345678H 依次 12、34、56、78 |
| 小端 | 最低有效字节；12345678H 依次 78、56、34、12 |

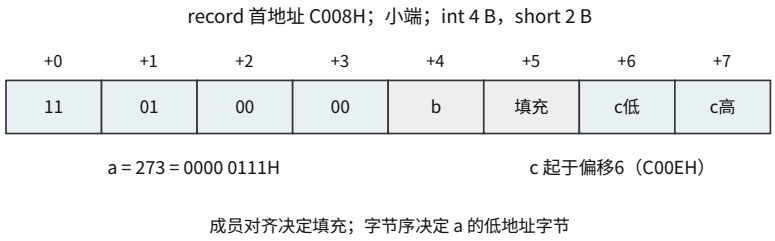



Figure 2.4: 结构体成员与填充位置

字节序只改变一个多字节对象内部各字节的位置，字节内位序、数组元素先后次序保持各自规定。–6 的 16 位补码 FFFAH 按小端存为 FA、FF。

自然对齐模型下，2 B 对象起点为 2 的倍数，4 B 对象起点为 4 的倍数。成员前填充满足各成员对齐，结构末尾填充满足相邻数组元素对齐，具体规则由题设或 ABI 给出。

成员偏移可按 $\lceil d/a \rceil a$ 对齐，其中 $d$ 为前一成员结束后的偏移，$a$ 为当前成员对齐要求。结构总长按结构对齐要求取整。

**例 2-11**

某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中的内容及 record.c 的地址分别为（　）。

A. 0x00，0xC00D　　B. 0x00，0xC00E
C. 0x11，0xC00D　　D. 0x11，0xC00E

按字节编址、元素宽 $w$ B、行优先二维数组列数 $C$：

$$\text{addr}(a[i][j]) = \text{base}(a) + (iC + j)w.$$

`struct { int id; char name[10]; int salary; }` 在 int 为 4 B 且自然对齐时，id 偏移 0，name 偏移 4，salary 偏移 16，结构长 20 B。数组首址 A0B0H，第二元素 id=12345678H 时，小端字节 56H 位于 A0C5H。

机器字长、存储字长与指令字长分别确定。按字节编址的 32 位整数占 4 个地址，按 32 位字编址则占 1 个地址；容量与跨界访问均先统一地址单位。

# 3 存储层次、地址与 Cache

## 3.1 存储层次与性能指标

寄存器 → Cache → 主存 → 外存，通常速度逐级降低、容量逐级增大、每位成本逐级降低。Cache—主存层次利用局部性缓解 CPU 与主存速度差；主存—辅存层次由硬件与操作系统配合实现虚拟存储。

| 存取方式 | 定位规则 | 典型对象 |
|---|---|---|
| 随机存取 RAM | 按地址访问任意单元，寻址步骤基本与位置无关 | SRAM、DRAM |
| 顺序存取 SAM | 按记录次序定位 | 磁带 |
| 直接存取 DAM | 先选区域，再在局部定位 | 磁盘 |
| 相联存取 CAM | 按内容或标记比较 | TLB 相联查找 |

| 指标 | 定义与条件 |
|---|---|
| 容量 | 单元数 × 每单元位数；$2^{20}$ 个 8 位单元为 1 MiB |
| 存取时间 $t_a$ | 启动一次读写到完成该操作的延迟 |
| 存取周期 $T$ | 连续两次独立访问的最短启动间隔，可含恢复时间 |
| 带宽 | 单位时间最大传送信息量；简单单端口模型为每次传送位数／存取周期 |
| 单位成本 | 总成本／总位数 |

容量中的 KiB、MiB 按 $2^{10}, 2^{20}$ B；频率和速率中的 M、G 通常按 $10^6, 10^9$。题源采用 KB、MB 时先依据题设确认进制。

## 3.2 SRAM、DRAM 与 ROM

| 器件 | 保存机制 | 读出与保持 | 常见用途 |
|---|---|---|---|
| SRAM | 双稳态电路 | 非破坏性读出，供电期间无需刷新；易失 | Cache |
| DRAM | 电容电荷 | 读后再生，周期刷新；易失，集成密度高 | 主存 |
| Flash | 非易失性闪存单元 | 按规定粒度编程与擦除，寿命有限 | SSD 等 |

随机存取、是否易失、是否破坏性读出是不同分类维度。主存与 Cache 是系统层次角色，SRAM、DRAM 是器件技术。

| ROM 类别 | 写入与修改方式 |
|---|---|
| 掩膜 ROM | 制造时写入，使用阶段固定 |
| PROM | 用户一次编程 |
| EPROM | 紫外线擦除后重写，通常整片擦除 |
| EEPROM | 电擦除、电改写，速度和次数受器件限制 |
| Flash | 通常按块擦除，读、编程、擦除粒度分别规定 |

### DRAM 刷新与地址复用

读后再生恢复刚读单元的电荷；定时刷新保持长期未访问单元的信息。刷新通常按行进行，保持时间内所有行均须刷新。

| 刷新方式 | 时间安排 |
|---|---|
| 集中刷新 | 固定时段连续刷新全部行，形成集中死时间 |
| 分散刷新 | 正常存取与刷新交替，扩大存取周期 |
| 异步刷新 | 将各行刷新均匀分散在保持时间内，定期插入短刷新周期 |

有 $R$ 行、保持时间 $T_r$、每行刷新占 $t_f$ 时，理想均匀刷新间隔为 $T_r/R$，总占用比例为 $Rt_f/T_r$。128 行、2 ms、每行 0.5 μs：间隔 15.625 μs，占比 3.2%；4096 行、64 ms、每行 100 ns：间隔 15.625 μs，占比 0.64%。命令粒度和内部刷新行为服从题设。

$2^k \times w$ 芯片含 $2^k$ 个 $w$ 位单元。普通非复用 SRAM 通常需 $k$ 根地址线、$w$ 根数据线。DRAM 将行、列地址分时送到复用地址引脚；若行列各 $k/2$ 位，则需 $k/2$ 根地址引脚，另配行列选通及其他控制线。

**例 3-1**

某容量为 256 MB 的存储器由若干 $4\text{M} \times 8$ 位的 DRAM 芯片构成，该 DRAM 芯片的地址引脚和数据引脚总数是（　）。

A. 19　B. 22　C. 30　D. 36

双端口 RAM 有两套地址、数据和控制线，可并行访问不同单元。同一单元的读写、写写冲突按忙信号、仲裁和芯片规定处理。

## 3.3 主存地址空间与 MAR

MAR 位数由可编码主存地址空间与编址单位决定，MDR 位数反映一次交换数据的宽度。已安装容量可以小于可编址空间。

**例 3-2**

某计算机存储器按字节编址，主存地址空间大小为 64 MB，现用 $4\text{M} \times 8$ 位的 RAM 芯片组成 32 MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是（　）。

A. 22 位　B. 23 位　C. 25 位　D. 26 位

## 3.4 单体多字与多体交叉存储

单体多字通过扩宽一次读写数据量，提高顺序访问的带宽；多体存储包含可相对独立工作的模块。高位交叉以高位选模块，同一连续区间落在同体；低位交叉以低位选模块，连续字轮流分布到不同体。

$m$ 体低位交叉、按字编址：

$$\text{模块号} = A \bmod m, \qquad \text{模块内地址} = \lfloor A/m \rfloor .$$

若给字节地址、每字 $w$ B，先用 $\lfloor A/w \rfloor$ 求字地址。模块存取周期 $T$、连续启动间隔 $\tau$，连续访问无其他冲突且 $m\tau \geq T$ 时：

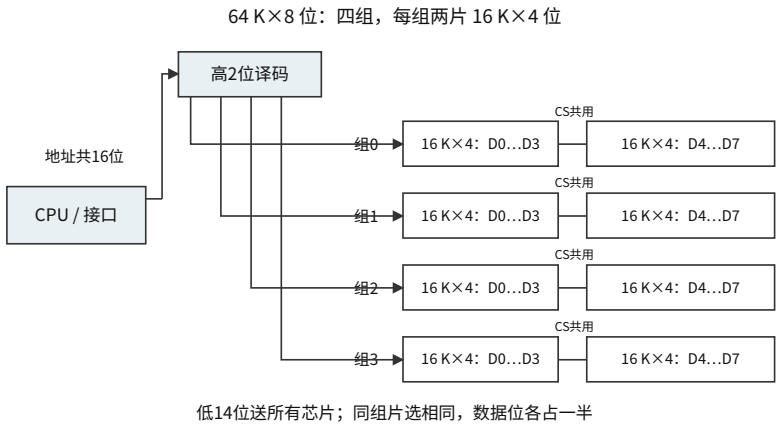



Figure 3.1: 同组并联扩位，四组译码扩字

$$t_n = T + (n-1)\tau, \qquad m \geq \lceil T/\tau \rceil.$$

$T = 200$ ns、$\tau = 50$ ns 至少需 4 体，连续 8 字为 550 ns。4 体步长 4 访问始终选同体；再次请求到达时该体尚忙才冲突。低位交叉提高连续吞吐率，第一字延迟仍由 $T$ 决定。

**例 3-3**

某计算机使用 4 体交叉编址存储器，假定在存储器总线上出现的主存地址（十进制）序列为 8005、8006、8007、8008、8001、8002、8003、8004、8000，则可能发生访存冲突的地址对是（　）。

A. 8004 和 8008　　B. 8002 和 8007　　C. 8001 和 8008　　D. 8000 和 8004

## 3.5 主存芯片字位扩展

以 $n \times b$ 芯片组成 $N \times W$ 存储器，且两维均为整数倍时：

$$\text{并行片数} = W/b, \quad \text{地址组数} = N/n, \quad \text{总片数} = \frac{N}{n}\frac{W}{b}.$$

| 扩展 | 地址与数据连接 | 片选 |
|---|---|---|
| 位扩展 | 同组地址并联，各片负责不同数据位 | 同组选中信号相同 |
| 字扩展 | 低位送片内地址，高位选择地址区间 | 各组片选互斥 |
| 字位同时扩展 | 先按目标字长组成并行组，再增加地址组 | 组内共同选中，组间译码选择 |

两片 16K $\times$ 4 构成一组 16K $\times$ 8，四组共 8 片构成 64K $\times$ 8。低 14 位选片内单元，高 2 位作 2–4 译码。

**例 3-4**

某计算机主存容量为 64 KB，其中 ROM 区为 4 KB，其余为 RAM 区，按字节编址。现要用 2K $\times$ 8 位的 ROM 芯片和 4K $\times$ 4 位的 RAM 芯片

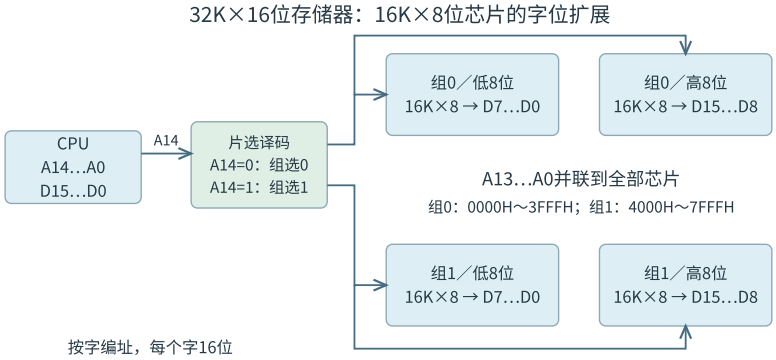



Figure 3.2: 两片作位扩展，A14 选择两组

来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是（ ）。

A. 1、15　B. 2、15　C. 1、30　D. 2、30

### 片选译码与地址范围

| 译码方式 | 特征 |
|---|---|
| 全译码 | 使用全部需要判别的高位地址，地址映射唯一 |
| 部分译码 | 省略部分高位，可能多个地址选中同一物理单元，形成地址重叠 |
| 线选法 | 直接用地址线选片，连线简单，可用组合和连续性受限制 |

片选表达式同时考虑地址译码、访存使能及有效电平。地址线高低位由题干定义，通常 A0 最低，也可有 A0 最高的题设。

首地址由片选编码接片内全 0 组成，末地址接片内全 1 组成。按字编址的 32K × 16 芯片需 15 位片内地址，高 3 位片选 111 时，范围 38000H～3FFFFH；改按字节编址后，每字 2 B，须增加字节选择与字节使能信息并重算范围。

16K × 8 组成 32K × 16：4 片，2 片一组、2 组。$A_{13} \sim A_0$ 接所有芯片，$A_{14}$ 选组；按字编址组范围 0000H～3FFFH、4000H～7FFFH。

## 3.6 磁盘结构、容量与访问时间

盘面包含磁道，磁道划分为扇区；不同盘面同半径的磁道组成柱面。典型定位字段为柱面号、磁头号、扇区号，磁头号确定盘面。

$$\text{容量} = \text{有效盘面数} \times \text{每面磁道数} \times \text{每道扇区数} \times \text{每扇区字节数}.$$

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

转速 $r$ 转/分，一圈 $60/r$ s；随机到达、角位置均匀时，平均旋转等待为 $30/r$ s。每道 $S$ 扇区、连续传 $k$ 扇区且忽略其他开销时，传输时间为 $(k/S)(60/r)$。控制器与排队开销按题设增加。

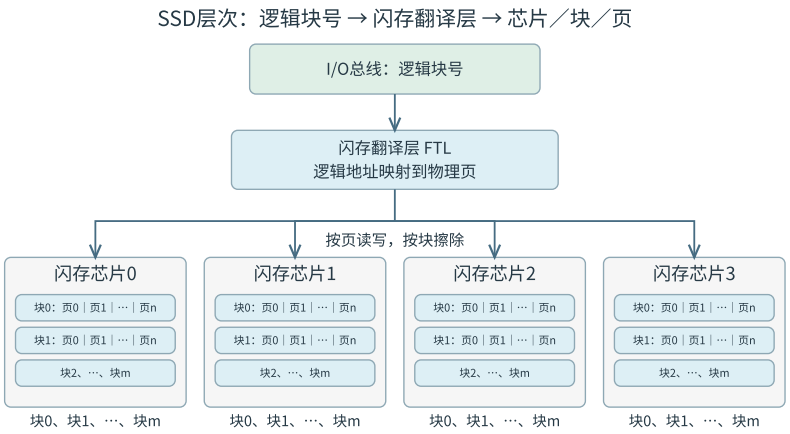



Figure 3.3: 逻辑块经 FTL 映射到闪存芯片、块与页

**例 3-5**

若磁盘转速为 7200 转/分，平均寻道时间为 8 ms，每个磁道包含 1000 个扇区，则访问一个扇区的平均存取时间大约是（　）。

A. 8.1 ms　　B. 12.2 ms　　C. 16.3 ms　　D. 20.5 ms

### RAID 组织

| 级别 | 数据与冗余组织 | 典型性质 |
|---|---|---|
| RAID0 | 条带化，无冗余 | 提高并行带宽，单盘故障可使阵列数据丢失 |
| RAID1 | 镜像 | 常见双副本有效容量约为总容量一半 |
| RAID3 | 细粒度条带，专用校验盘 | 数据盘协调并行传输 |
| RAID4 | 块级条带，专用校验盘 | 校验盘可能成为写热点 |
| RAID5 | 块级条带，分布式校验 | 分散校验写入，通常容忍单盘故障 |

容错范围由具体级别和故障分布决定，更新数据还可能引入校验一致性开销；冗余副本会同步误写或误删，备份承担历史恢复功能。

## 3.7 SSD 的页、块与磨损均衡

NAND Flash 通常按页读取或编程、按块擦除。FTL 将主机逻辑块地址映射为闪存物理页地址。具体页、块尺寸依器件而定，示例可取页 512 B～4 KiB、块 16～512 KiB。

| 管理动作 | 作用 |
|---|---|
| 异地更新 | 新数据写入空闲已擦除位置，更新映射，旧页失效 |
| 垃圾回收 | 搬走块中有效页，擦除旧块，回收空间 |
| 写放大 | 实际闪存写入量因有效页搬移等大于主机写入量 |
| 动态磨损均衡 | 新写入优先使用擦除次数较少的空闲块 |
| 静态磨损均衡 | 进一步搬移冷数据，让长期少用的块参加新写入 |

SSD 无机械寻道和旋转等待，仍有闪存、控制器、接口传输和写入管理延迟；擦写次数、写放大和管理开销须按具体模型考虑。

## 3.8 Cache 局部性与平均访问时间

时间局部性：近期访问过的同一信息可能很快再访问；空间局部性：相邻位置可能很快被访问。Cache 按块调入利用空间局部性。

$$h = \frac{N_{\text{hit}}}{N_{\text{hit}} + N_{\text{miss}}}.$$

| 时间定义／访问模型 | 平均访问时间 |
|---|---|
| 先查 Cache，缺失后额外耗时 $p$ | $t_c + (1-h)p$ |
| 完整缺失访问耗时 $t_{\text{miss}}$，已含查 Cache | $ht_c + (1-h)t_{\text{miss}}$ |
| Cache 与主存同时启动，命中取 Cache 结果 | $ht_c + (1-h)t_m$ |
| 先 Cache、缺失后启动主存，主存耗时 $t_m$ | $t_c + (1-h)t_m$ |

$t_m = 5t_c, h = 0.95$ 时，并行模型为 $1.2t_c$、加速约 4.17 倍；串行为 $1.25t_c$、加速 4 倍。$t_c = 2$ ns、缺失额外 80 ns、$h = 0.95$ 时平均 6 ns；若 80 ns 已含查 Cache，则平均 5.9 ns。

两级 Cache 的简单串行模型：AMAT $= t_1 + m_1(t_2 + m_2 p_2)$，其中 $m_2$ 为进入二级后的局部缺失率，二级全局缺失率为 $m_1 m_2$。回写、调块、重访等开销是否计入缺失损失服从题设。

**例 3-6**

有如下 C 语言程序段：

```
for (k = 0; k < 1000; k++)
    a[k] = a[k] + 32;
```

若数组 a 和变量 k 均为 int 型，int 型数据占 4 B，数据 Cache 采用直接映射方式，数据区大小为 1 KB、块大小为 16 B，该程序段执行前 Cache 为空，则该程序段执行过程中访问数组 a 的 Cache 缺失率约为（　）。

A. 1.25%　　B. 2.5%　　C. 12.5%　　D. 25%

### 块传送与突发访问

取 64 B 块、每次传 8 B，若每次均需地址 1 周期、准备 8 周期、传输 1 周期，独立访问需 $8(1+8+1) = 80$ 周期。支持一次地址、一次首数据等待、连续 8 次传输的突发模型时，共 $1+8+8 = 17$ 周期。

## 3.9 Cache 映射与地址字段

设数据区容量 $C$ B、块大小 $B$ B、$r$ 路、$m$ 位字节地址。先求行数 $L = C/B$、组数 $G = L/r$。$B, G$ 均为 2 的幂时：

$$b = \log_2 B, \quad g = \log_2 G, \quad t = m - b - g.$$

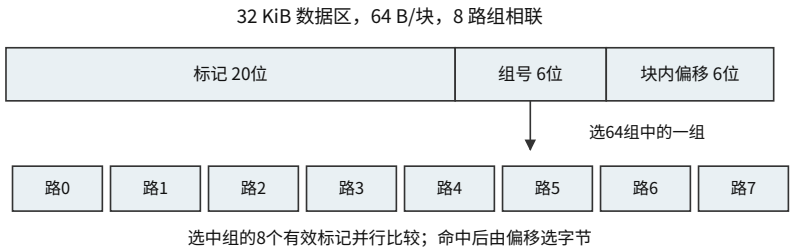



Figure 3.4: 块内偏移、组号与标记

$$q = \lfloor A/B \rfloor, \quad d = A \bmod B, \quad s = q \bmod G, \quad \text{tag} = \lfloor q/G \rfloor,$$

$$A = (\text{tag}\, G + s)B + d.$$

| 映射 | 可放位置 | 组选位数 | 比较范围 | 替换范围 |
|---|---|---|---|---|
| 直接映射 | 唯一一行 | $\log_2(C/B)$ | 该行 1 个标记 | 唯一位置 |
| $r$ 路组相联 | 唯一一组内 $r$ 行 | $\log_2(C/(Br))$ | 该组 $r$ 个有效标记 | 组内 |
| 全相联 | 任一行 | 0 | 全部有效标记 | 全 Cache |

命中须同时满足有效位为 1、标记匹配。组号用于选组，块内偏移用于选块中字节，二者均无需保存到每行标记。

固定地址位数、数据容量和块大小时，相联度增大使组号变短、标记变长，可能减少冲突缺失。强制缺失与容量限制仍存在。比较器数、布线、扇入和编码共同影响实际延迟，具体快慢需依据电路模型。

**例 3-7**

若计算机主存地址为 32 位，按字节编址，某 Cache 的数据区容量为 32 KB，主存块大小为 64 B，采用 8 路组相联映射方式，该 Cache 中比较器的个数和位数分别为（ ）。

A. 8，20　B. 8，23　C. 64，20　D. 64，23

## Cache 总容量

$$\text{总位数} = L(8B + t + V + D + \cdots) + \text{替换状态位}.$$

$V$ 为有效位，回写通常需脏位 $D$；替换位、校验位等按题设计入。给出的数据容量 $C$ 与含标记和状态位的总容量分别使用。

32 位字节地址、4 KiB 数据区、16 B 块、2 路：偏移 4 位、组号 7 位、标记 21 位；含有效和脏位且暂不计替换位，每行 151 位。直接映射无需行间替换状态。

**例 3-8**

假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写方式，则能存放 4 K 字数据的 Cache 的总容量的位数至少是（ ）。

A. 146 K　B. 147 K　C. 148 K　D. 158 K

## 3.10 Cache 替换与写策略

优先使用目标组空行；满组后在本组选择牺牲行。

| 策略 | 状态依据 | 命中更新 |
|---|---|---|
| FIFO | 装入先后 | 保持装入次序 |
| LRU | 最近使用次序 | 置为最近使用 |
| LFU | 累计使用频次 | 更新计数，初值及并列规则须给定 |
| 随机 | 随机选择 | 无需维护精确最近次序 |

两路一组访问 A、B、A、C、B，LRU 状态从最近到最久依次为 [A]、[B,A]、[A,B]、[C,A]、[B,C]，仅第 3 次命中。

4 行全相联、初始空，访问 $1,2,3,4,1,2,5,1,2,3,4,5$：FIFO 命中第 5、6 次；LRU 命中第 5、6、8、9 次。访问块 5 前，FIFO 淘汰 1，LRU 淘汰 3。

### 写命中、写缺失与脏块

| 事件与策略 | 动作 |
|---|---|
| 写命中，直写 | 同时更新 Cache 和下层，可用写缓冲；缓冲满时可能阻塞 |
| 写命中，回写 | 只更新 Cache 并置脏位，淘汰脏块时写回 |
| 写缺失，写分配 | 调块进入 Cache 再写 |
| 写缺失，非写分配 | 绕过当前 Cache，直接写下层 |

回写常配写分配、直写常配非写分配，具体组合按题设执行。干净块可直接淘汰；脏块需按回写规则保存。多核副本及 DMA 访问还需相应一致性机制。

## 3.11 虚拟存储、页表与 TLB

虚拟存储将进程虚拟地址映射为物理地址，支持部分内容驻留外存、隔离与保护。操作系统建立并维护映射、分配和换入换出；MMU 执行运行时地址转换及权限检查。TLB 命中路径可由硬件完成。

页大小 $P = 2^p$ B、虚址 $v$ 位、物理地址 $a$ 位：

$$\mathrm{VA} = \mathrm{VPN} \times P + d, \quad \mathrm{PA} = \mathrm{PPN} \times P + d,$$

VPN 为 $v - p$ 位，PPN 为 $a - p$ 位，页内偏移 $d$ 原样保留。

TLB 是页表项的高速缓存。若 TLB 共 $K$ 项、$r$ 路、低位选组，组数 $G_T = K/r$，组号 $g_T = \log_2 G_T$ 位，TLB 标记为 $v - p - g_T$ 位；物理页框号位数由物理地址决定。

| 查找对象 | 保存内容 | 未命中或异常处理 |
|---|---|---|
| TLB | 页映射、权限等 | 未命中转页表，页面仍可能已驻留 |
| 页表 | 页框号、存在位、权限、管理状态 | 合法且未驻留时缺页；非法映射、保护失败分别处理 |
| 物理地址 Cache | 主存数据块或指令块副本 | 未命中访问下一层，按策略调块 |

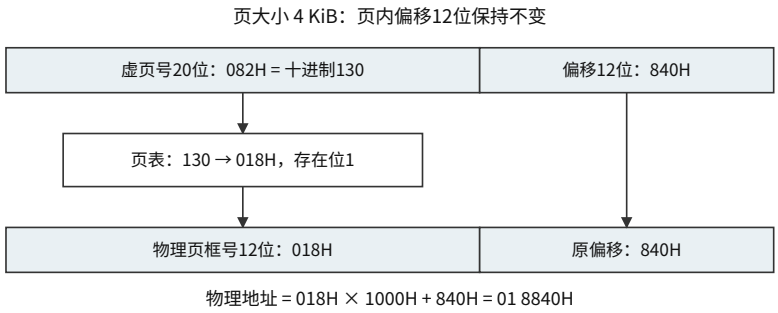



Figure 3.5: 虚页号查映射，页框号与原偏移形成物理地址

TLB 命中与 Cache 命中分别判断；页表存在位为 1 说明页面驻留，数据 Cache 仍可缺失。Cache 标记与页框号按各自字段求取。

**例 3-9**

某计算机主存地址为 24 位，采用分页虚拟存储管理方式，虚拟地址空间大小为 4 GB，页大小为 4 KB，按字节编址。某进程的页表部分内容如下。

| 虚页号 | 实页号（页框号） | 存在位 |
|---|---|---|
| 82 | 024H | 0 |
| … | … | … |
| 129 | 180H | 1 |
| 130 | 018H | 1 |

当 CPU 访问虚拟地址 0008 2840H 时，虚—实地址转换的结果是（　）。

A. 得到主存地址 02 4840H　　B. 得到主存地址 18 0840H
C. 得到主存地址 01 8840H　　D. 检测到缺页异常

**例 3-10**

某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1 KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则 TLB 表项中标记字段的位数至少是（　）。

A. 17　　B. 18　　C. 19　　D. 20

## 地址转换路径与平均时间

先转换、后查物理 Cache 的串行模型：拆 VPN、页内偏移 → 查 TLB → 未命中时查各级页表 → 检查存在位与权限 → 形成物理地址 → 查 Cache → 缺失时访问下层。

缺页时先处理并重试当前指令，普通数据 Cache 查找须等待可用映射。页表访问是否经过 Cache、TLB 与 Cache 能否并行均由题设决定。

单级页表、不计缺页、TLB 和主存串行且主存数据无 Cache 时，TLB 命中率 $h_T$、时间 $t_T$、每次主存访问 $t_M$：

$$\bar{t} = h_T(t_T + t_M) + (1 - h_T)(t_T + 2t_M).$$

多级页表增加相应查表访问；缺页需另加外存与操作系统处理时间。MMU 在地址转换中检测 TLB 缺失、缺页及权限问题，数据 Cache 缺失由 Cache 查找确定。

## 3.12 段式与段页式存储

段式地址含段号和段内偏移。段表保存基址、段长和权限；段长 $L$ 表示字节数时，合法条件为 $0 \le d < L$，物理地址为 base+$d$，末字节地址为 base+$L-1$。

段页式先由段号定位该段页表，再由段内页号查页框，最后拼接页内偏移。段界限仍须检查，最后一页的填充空间可能超出段长。

段基址 10000 B、长 3000 B，偏移 2999 对应 12999 B；偏移 3000 越界。1 KiB 页的段页式模型中，页号 2、偏移 952 恰为段内 3000，也越界。

## 3.13 数组访问次序与 TLB、Cache 综合推演

行优先数组、每元素 $w$ B、每行 $C$ 个元素：$\mathrm{addr}(a[i][j]) = A_0 + (iC + j)w$。先分别求触及页面、Cache 块和每组工作集，再计算缺页与命中数。

首地址偏移 $d$、连续区域长 $n$ B、页或块大小 $P$ 时，非空区域覆盖数为 $\lceil (d+n)/P \rceil$。每组工作集超过路数才存在相应组容量压力，总容量可容纳仍可能发生组冲突。

若 Cache 组选位全部落在页内偏移中，虚实转换保留这些位，可直接从虚址提取本题组号。超出页内偏移时需按指定地址模型取得相关物理位。

**例 3-11**

已知计算机 M 字长为 32 位，按字节编址，采用请求调页策略的虚拟存储管理方式，虚拟地址 32 位，页面大小为 4 KB；数据 Cache 采用 4 路组相联映射方式，数据区大小为 8 KB，主存块大小为 32 B。现有 C 语言程序段如下：

```
int a[24][64];
/* ... */
for (i = 0; i < 24; i++)
    for (j = 0; j < 64; j++) a[i][j] = 10;
```

已知二维数组 a 按行优先存放，在虚拟地址空间中分配的起始地址为 0042 2000H，`sizeof(int)=4`。假定在 M 上执行上述程序段之前数组 a 不在主存，且在该程序执行过程中不会发生页面置换。请回答下列问题。

(1) 数组分布在几个页面中？对于数组 a 的访问，会发生几次缺页异常？页故障地址各是什么？

(2) 不考虑变量 i 和 j，该程序段的数据访问是否具有时间局部性？为什么？

(3) 计算机 M 的虚拟地址（A31～A0）中哪几位用作块内地址？哪几位用作 Cache 组号？a[1][0] 的虚拟地址是多少？其所在主存块对应的 Cache 组号是多少？

(4) 数组 a 占用多少主存块？假设上述程序段执行过程中数组 a 的访问不会和其他数据发生 Cache 访问冲突，则数组 a 的 Cache 命中率是多少？若将循环中 i 和 j 的次序按如下方式调换，则数组 a 的 Cache 命中率又是多少？

```
for (j = 0; j < 64; j++)
    for (i = 0; i < 24; i++) a[i][j] = 10;
```

**例 3-12**

假设计算机 M 的主存地址为 24 位，按字节编址；采用分页存储管理方式，虚拟地址为 30 位，页大小为 4 KB；TLB 采用 2 路组相联方式和 LRU 替换策略，共 8 组。请回答下列问题。

(1) 虚拟地址中哪几位表示虚页号？哪几位表示页内地址？

(2) 已知访问 TLB 时虚页号高位部分用作 TLB 标记，低位部分用作 TLB 组号，M 的虚拟地址中哪几位是 TLB 标记？哪几位是 TLB 组号？

(3) 假设 TLB 初始时为空，访问的虚页号依次为 10、12、16、7、26、4、12 和 20，在此过程中，哪一个虚页号对应的 TLB 表项被替换？说明理由。

(4) 若将 M 中的虚拟地址位数增加到 32 位，则 TLB 表项的位数增加几位？

**例 3-13**

某系统按字节编址，虚拟地址为 14 位，物理地址为 12 位，页大小为 64 B。TLB 共 16 项，采用 4 路组相联；数据 Cache 使用物理地址，采用直接映射，共 16 行，每行 4 B。

按“先完成地址转换，再查数据 Cache”的串行模型分析。所有待访问的虚拟地址均合法，且允许读；页表存在位 $P=1$ 表示页面已在主存，$P=0$ 表示合法但尚未驻留，需要处理缺页。读 Cache 未命中时，从主存调入对应的 4 B 数据块。

以下给出本题所需的初始状态。表中页号、页框号、标记、Cache 行号和字节数据均用十六进制，后缀 H 表示十六进制；$V$ 为有效位，“—”表示没有可用信息。TLB 的组号和路号均在 0 至 3 之间。

**表 A　TLB 初始状态**

| 组号 | 路号 | 标记 | 有效位 V | 页框号 |
|---|---|---|---|---|
| 0 | 0 | 03H | 0 | — |
| 0 | 1 | 00H | 0 | — |
| 0 | 2 | 09H | 1 | 0DH |
| 0 | 3 | 07H | 1 | 02H |
| 3 | 0 | 07H | 0 | — |
| 3 | 1 | 0AH | 1 | 34H |
| 3 | 2 | 03H | 1 | 0DH |
| 3 | 3 | 02H | 0 | — |

**表 B　页表中与本题有关的表项**

| 虚页号 | 存在位 $P$ | 页框号 |
|---|---|---|
| 03H | 1 | 02H |
| 08H | 0 | — |

**表 C　Cache 初始状态，数据按块内偏移排列**

| 行号 | 标记 | V | 偏移 0 | 偏移 1 | 偏移 2 | 偏移 3 |
|---|---|---|---|---|---|---|
| 5H | 0DH | 1 | 36H | 72H | F0H | 1DH |
| CH | 02H | 0 | — | — | — | — |

（1）写出查 TLB 时虚拟地址的字段划分，以及查 Cache 时物理地址的字段划分，并求各字段位数。

（2）分别读取虚拟地址 03D4H、00F1H、0229H 处的一个字节，依次说明 TLB 查找、必要的页表检查和 Cache 查找，指出最终从哪里取数，或在哪一步转入异常处理。能确定字节值时给出结果。

**三次访问相互独立，均从上述初始状态开始**。某次访问引起的回填不影响另两次；不推演缺页后的页框分配与替换，也不计算访问时间。

# 4 指令系统与 C 程序的机器级表示

## 4.1 指令格式、地址字段与操作码扩展

指令由操作码、操作数/地址及寻址方式等字段组成。指令字长为编码位数，机器字长为整数处理的基本位宽，存储字长为一个存储单元的位数；三者分别确定。一个地址字段可表示寄存器编号、立即数、形式地址或位移。

| 格式 | 典型运算语义 | 隐含位置 |
|---|---|---|
| 三地址 | $M[A_3] \leftarrow M[A_1] + M[A_2]$ | 由指令定义 |
| 二地址 | $M[A_1] \leftarrow M[A_1] + M[A_2]$ | 一个源兼目的 |
| 一地址 | ACC $\leftarrow$ ACC $+ M[A]$ | 累加器 |
| 零地址 | 弹出栈顶两项运算，再压入结果 | 栈顶、SP |

地址字段数只计显式编码；实际操作数还包括隐含操作数。三地址运算若取指一次、源与目的均在主存，需取指 1 次、读源 2 次、写结果 1 次，共 4 次访存。多字指令、寄存器源、间接寻址分别改变对应次数。

固定操作码 $k$ 位至多编码 $2^k$ 种操作。扩展操作码逐层保留前缀：本层剩余 $r$ 个前缀，下一段扩展 $a$ 位，得到 $r2^a$ 个可分配编码；扣除已用编码后继续扩展。短操作码与更长操作码的前缀必须互斥。

16 位指令、地址字段 4 位：分配 15 条三地址后剩 1 个 4 位前缀；扩为 16 个 8 位码，分配 12 条二地址后剩 4 个；扩为 64 个 12 位码，分配 62 条一地址后剩 2 个；最终可分配 32 条零地址。指令总长始终 16 位。

**例 4-1**

设计某指令系统时，假设采用 16 位定长指令字格式，操作码使用扩展编码方式，地址码为 6 位，包含零地址、一地址和二地址 3 种格式的指令。若二地址指令有 12 条，一地址指令有 254 条，则零地址指令的条数最多为（ ）。

A. 0　B. 2　C. 64　D. 128

## 4.2 数据寻址、相对地址与访存次数

记形式地址 $A$、寄存器内容 $(R)$、存储单元内容 $M[x]$。表中读主存次数仅统计取得一个源操作数，取指与写回另计。

| 寻址方式 | 操作数或有效地址 EA | 读主存次数 |
|---|---|---|
| 立即 | 操作数为指令内的立即数字段 | 0 |
| 直接 | EA $= A$，操作数 $M[A]$ | 1 |
| 一次主存间接 | EA $= M[A]$，操作数 $M[\text{EA}]$ | 2 |
| 寄存器 | 操作数 $(R)$ | 0 |
| 寄存器间接 | EA $= (R)$ | 1 |
| 基址 | EA $=$ (BR) $+$ 位移 | 主存源 1 次 |
| 变址 | EA $= A +$ (IX) | 主存源 1 次 |
| 相对 | 目标 $=$ PC$_{base}$ $+$ sext(位移) $\times$ 缩放因子 | 按指令语义确定 |

间接寻址可利用比指令地址字段更宽的中间地址扩大寻址范围。基址寄存器通常存程序/数据段起点，支持重定位；变址寄存器通常存变化偏移，支持数组访问。位移的符号扩展和缩放按 ISA 规定。

按字节编址、int 占 4 B 时，addr$(a[i]) = \text{base} + 4i$。有比例变址可直接形成偏移；其他机器需先计算 $4i$。相对转移常取后继指令地址为基准，具体以题设 PC 更新时机为准。

**例 4-2**

某机器字长为 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一个字节为操作码字段，第二个字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

A. 2006H　B. 2007H　C. 2008H　D. 2009H

无 Cache、无预取、指令与操作数各占一个存储字时，读取一个直接寻址源、一次主存间接源、寄存器间接源分别至少访存 2、3、2 次，均已计一次取指。32 位指令在 16 位存储字中跨两个字，取指所需访问次数按存储组织确定。

堆栈寻址隐含使用 SP。向低地址增长、SP 指向栈顶、每槽 4 B 的模型：push 为先 SP 减 4 再写，pop 为先读再 SP 加 4。初值 1000H，两次 push、一次 pop 后 SP=0FFCH。硬堆栈使用寄存器组，软堆栈使用主存，访存开销分别计算。

## 4.3 RISC、CISC 与指令副作用

| 维度 | RISC 典型设计 | CISC 典型设计 |
|---|---|---|
| 功能 | 常用简单操作，复杂操作由指令组合 | 丰富指令，可表达较复杂操作 |
| 编码与寻址 | 格式规整，常用定长，寻址方式较少 | 格式及寻址较多，常见变长 |
| 数据访存 | Load/Store 访问数据，ALU 主要操作寄存器 | 多类指令可带主存操作数 |
| 寄存器与传参 | 通用寄存器较多，常用寄存器传参 | 传统设计寄存器较少，操作数来源丰富 |
| 控制与流水 | 常用硬布线，便于流水 | 复杂指令常用微程序，也可流水 |
| 编译 | 重视选指令、分配寄存器和调度 | 同样需要编译优化 |

表中为常见倾向。指令长度、寄存器数、控制器实现和调用约定须以具体 ISA、微架构和 ABI 为准；执行时间按指令数、CPI 及周期共同计算。

数据传送复制值；算术逻辑、移位指令产生新值并可能改标志；控制转移改变 PC；I/O 及系统控制可能涉及权限。每条指令的访存、改标志、特权等副作用依给定语义判断。

**例 4-3**

下列关于 RISC 的叙述中，错误的是（ ）。

A. 多采用硬连线方式实现控制器
B. 通常采用 Load/Store 型指令设计风格
C. 难以采用流水线数据通路实现微架构
D. 多采用寄存器传递过程调用时的参数

## 4.4 编译工具链与重定位

| 阶段 | 输入与主要结果 |
|---|---|
| 预处理 | 源文件 → 展开宏、包含文件等后的源程序 |
| 编译 | 高级语言 → 汇编表示 |
| 汇编 | 汇编表示 → 机器码目标文件，保留符号与重定位信息 |
| 链接 | 合并节、解析跨模块符号、修改地址引用 → 可执行文件 |

动态链接可将部分链接工作推迟至装入或运行时。目标/可执行文件可含符号表、调试、重定位和装入组织信息。优化后的变量可在寄存器中、成为常量或被消除。

**例 4-4**

将高级语言源程序转换为可执行目标文件的主要过程是（　）。

A. 预处理 → 编译 → 汇编 → 链接
B. 预处理 → 汇编 → 编译 → 链接
C. 预处理 → 编译 → 链接 → 汇编
D. 预处理 → 汇编 → 链接 → 编译

## 4.5 x86 操作数、有效地址与条件转移

32 位示例中 EAX、EBX、ECX、EDX 为通用寄存器，ESP 为栈顶、EBP 常作帧基址。AX 为 EAX 低 16 位，AL 与 AH 为最低、次低 8 位。

| 含义 | Intel 格式：目的在前 | AT&T 格式：源在前 |
|---|---|---|
| 立即数 5→EAX | `mov eax, 5` | `movl $5, %eax` |
| EBX→EAX | `mov eax, ebx` | `movl %ebx, %eax` |
| 读取 32 位主存数 | `mov eax, dword ptr [ebx+ecx*4+8]` | `movl 8(%ebx,%ecx,4), %eax` |

AT&T 的 b、w、l 常表示 8、16、32 位。x86 比例因子可取 1、2、4、8。`lea` 仅计算有效地址表达式；`mov` 传数据；`cmp` 形成减法标志；`test` 形成按位与标志；后两者均不保存运算结果。`shl/sal` 左移，`shr` 逻辑右移，`sar` 符号扩展右移。

以 Intel 的 `cmp eax, ebx` 为例，标志来自 EAX−EBX：

| 关系 | 有符号条件 | 无符号条件 |
|---|---|---|
| 等于 | ZF=1 | ZF=1 |
| 小于 | SF ≠ OF | CF=1 |
| 小于等于 | ZF = 1 ∨ SF ≠ OF | CF=1 或 ZF=1 |
| 大于 | ZF = 0 ∧ SF = OF | CF=0 且 ZF=0 |
| 大于等于 | SF=OF | CF=0 |

`loop` 在 32 位计数约定下先 ECX 减 1，再按 ECX 是否非零决定转移；其标志副作用须依指令定义。

## 4.6 分支、循环与动态执行次数

还原控制流时标出初始化、条件、循环体、更新和回边。入口测试型 n 次循环通常测试 n+1 次；后测循环从循环体起步时通常至少执行一次。signed 与 unsigned 选择不同分支条件。

```
// R0=数组基址，R3=元素个数，元素4 B
R1 <- 0; R2 <- 0
TEST:
    if R1 >= R3 jump EXIT
    R4 <- R1 << 2
    R5 <- M[R0 + R4]
    R2 <- R2 + R5
    R1 <- R1 + 1
    jump TEST
EXIT:
```

R3=0 时不访问数组；R3=3 时访问偏移 0、4、8。指针遍历每次加 4，终止指针为 base+4R3。动态指令数、取指次数与数据访问次数按实际路径累计。

## 4.7 调用约定、栈帧和递归状态

调用需传参、保存返回位置、安排局部区、传回返回值。ISA 提供 call/ret 或链接寄存器等机制，ABI 规定参数位置、栈对齐、返回值及寄存器保存责任。

| 控制方式 | 后继地址 | 额外状态 |
|---|---|---|
| 顺序 | 当前地址 + 指令长度 | 按指令副作用更新 |
| 条件分支 | 条件成立取目标，否则取顺序后继 | 条件及目标 |
| 调用 | 被调入口 | 保存调用后的后继地址 |
| 返回 | 已保存的返回地址 | 恢复约定现场与栈 |

链接寄存器保存返回地址时，叶子函数可无需写主存；嵌套调用/递归在再次覆盖它前保存旧返回地址。调用者保存寄存器由调用者在调用前保存仍需使用的值；被调用者保存寄存器由被调过程修改前保存、返回前恢复。

向低地址增长、SP 指栈顶、槽 4 B、call 压入返回地址的模型：SP=1000H，返回地址 2004H；call 后 SP=0FFCH，M[0FFCH]=2004H；保存旧 FP 后 SP=0FF8H；分配 8 B 局部区后 SP=0FF0H；按逆序撤销后 SP 恢复 1000H。

32 位 x86 标准建帧示例：`push ebp; mov ebp, esp`，再减 ESP 分配局部区；返回前 `mov esp, ebp; pop ebp; ret`。整数返回值常在 EAX。

| 地址 | 标准建帧后的内容 |
|---|---|
| `[ebp]` | 旧 EBP |
| `[ebp+4]` | 返回地址 |
| `[ebp+8]`、`[ebp+12]` | 前两个栈参数 |
| `[ebp-4]` 等 | 局部变量 |

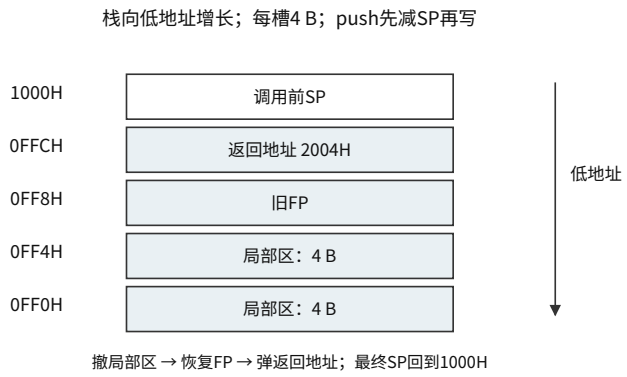



Figure 4.1: 调用、建立栈帧和返回时 SP 移动

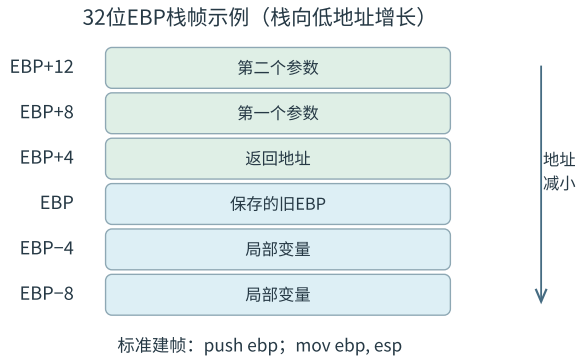



Figure 4.2: 32 位示例栈帧布局

具体布局以调用约定为准。递归每层保存独立参数、返回位置和局部状态；全局变量共享。递归空间按最大同时存活层数计算。

**例 4-5**

已知 $f(n) = n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$，计算 $f(n)$ 的 C 语言函数 f1 的源程序及其在 32 位计算机 M 上的部分机器级代码如下。

```c
int f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    else
        return 1;
}
```

以下按“行号／虚拟地址／机器指令（字节按地址递增排列）／汇编指令”四列给出，每行是一条代码记录；省略行仍保留四列。

```
 1  00401000  55              push ebp
 …  …         …               …
11  00401018  83 7D 08 01     cmp dword ptr [ebp+8], 1
12  0040101C  7E 17           jle f1+35h (00401035)
13  0040101E  8B 45 08        mov eax, dword ptr [ebp+8]
14  00401021  83 E8 01        sub eax, 1
15  00401024  50              push eax
16  00401025  E8 D6 FF FF FF  call f1 (00401000)
 …  …         …               …
19  00401030  0F AF C1        imul eax, ecx
20  00401033  EB 05           jmp f1+3Ah (0040103A)
21  00401035  B8 01 00 00 00  mov eax, 1
 …  …         …               …
26  00401040  3B EC           cmp ebp, esp
 …  …         …               …
30  0040104A  C3              ret
```

其中，机器级代码行包括行号、虚拟地址、机器指令和汇编指令，计算机 M 按字节编址，int 型数据占 32 位。请回答下列问题。

（1）计算 f(10) 需要调用函数 f1 多少次？执行哪条指令会递归调用 f1？

（2）上述代码中，哪条指令是条件转移指令？哪几条指令一定会使程序跳转执行？

（3）根据第 16 行的 call 指令，第 17 行指令的虚拟地址应是多少？已知第 16 行的 call 指令采用相对寻址方式，该指令中的偏移量应是多少（给出计算过程）？已知第 16 行的 call 指令的后 4 字节为偏移量，M 采用大端方式还是小端方式？

（4）$f(13) = 6\,227\,020\,800$，但 f1(13) 的返回值为 1 932 053 504，为什么两者不相等？要使 f1(13) 能返回正确的结果，应如何修改 f1 的源程序？

（5）第 19 行的 imul 指令（带符号整数乘）的功能是 R[eax] $\leftarrow$ R[eax] $\times$ R[ecx]，当乘法器输出的高、低 32 位乘积之间满足什么条件时，溢出标志 OF=1？要使 CPU 在发生溢出时转异常处理，编译器应在 imul 指令后加一条什么指令？

# 5 中央处理器

## 5.1 CPU 功能、寄存器与状态元件

CPU 完成指令控制、操作控制、时间控制、数据加工及规定的异常/中断处理。数据通路完成数据传送和运算，控制部件 CU 生成选择、运算、写使能与存储器读写等控制信号。

| 部件 | 内容或作用 | 组织层次 |
|---|---|---|
| ALU、MUX、移位器 | 按当前输入形成运算或选择结果 | 组合逻辑操作元件 |
| GPRs、ACC、临时寄存器 | 操作数、地址、中间结果 | 状态元件 |
| PC | 程序执行位置，更新规则依 ISA 与时序 | 程序级状态 |
| IR | 当前指令编码 | 通常属实现内部 |
| MAR | 本次主存访问地址 | 通常属实现内部 |
| MDR | 本次与主存交换的指令或数据 | 通常属实现内部 |
| PSW/FR | 条件码及规定的特权、中断等状态 | 可见性与写权限依 ISA |
| CMAR、CMDR | 微地址、当前微指令 | 微程序控制器内部 |

GPRs、PC 及规定的状态属于程序员可见模型；可见性与用户指令能否直接读写分别由 ISA 及权限定义。MAR 位数按可寻址单元数确定，MDR 位数按一次数据交换位宽确定。例如 26 位字节地址、32 位数据通路，可有 26 位 MAR 与 32 位 MDR。

**例 5-1**

数据通路由组合逻辑元件（操作元件）和时序逻辑元件（状态元件）组成。以下给出的元件中，属于操作元件的是（ ）。

I. 算术逻辑部件（ALU）　II. 程序计数器（PC）
III. 通用寄存器组（GPRs）　IV. 多路选择器（MUX）

A. 仅 I、II　B. 仅 I、IV　C. 仅 II、III　D. 仅 I、II、IV

## 5.2 指令周期与三种执行组织

指令周期覆盖一条指令从开始到完成；机器周期为某实现划分的访存等基本操作周期；时钟周期为同步状态推进的时间单位。三者倍数由具体组织给出。

| 指令阶段 | 动作 | 进入条件 |
|---|---|---|
| 取指 | PC 给地址、读指令入 IR、更新 PC | 每条指令 |
| 间址 | 取得有效地址 | 需要主存间接寻址等 |
| 执行 | 完成指令语义 | 每条指令，动作不同 |
| 中断 | 保存返回状态、进入处理入口 | 有请求且满足响应条件 |

| 组织 | 时钟约束 | 同一时刻与资源复用 |
|---|---|---|
| 单周期 | 最慢完整指令路径 | 一条指令占一长拍；同拍资源需求须同时满足 |
| 多周期 | 最慢基本步骤 | 一条指令分多拍；资源可跨拍复用 |
| 简单标量流水 | 最慢段 + 段寄存器开销 | 多条指令不同段重叠；同拍资源争用须消除 |

单周期 CPI=1、理想标量流水稳态 CPI 约 1，周期长度可不同。性能比较用执行时间 $N \times \text{CPI} \times \tau$。

## 5.3 单总线数据通路与微操作时序

微操作是部件可执行的基本动作；微命令为驱动该动作的控制命令。同一内部总线每拍通常只允许一个驱动源，多个目的在有实际通路和独立写使能时可同时接收同一值。

ALU 一端由 Y 保存第一个操作数，另一端接总线；结果先存 Z，后续再上总线写回。例 $R_1 \leftarrow R_1 + R_2$：

| 拍 | 微操作 |
|---|---|
| 1 | $R_1 \rightarrow Y$ |
| 2 | $R_2 \rightarrow \text{BUS}$，$Y + \text{BUS} \rightarrow Z$ |
| 3 | $Z \rightarrow R_1$ |

`R1→Y` 与 `R2→MAR` 同拍争用总线；`R1→Y` 与 `R1→MAR` 可同时接收。结果尚未产生、访存尚未完成时，目的寄存器应等待。

取指依赖：PC → MAR，主存读入 MDR，MDR → IR；指令地址字段需间接求址时，再送 MAR 并读出有效地址。PC 增量能否与访存并行取决于独立增量器、总线占用与图示通路。

**例 5-2**

某 CPU 中部分数据通路如下图所示，其中，GPRs 为通用寄存器组；FR 为标志寄存器，用于存放 ALU 产生的标志信息；带箭头虚线表示控制信号，如控制信号 Read、Write 分别表示主存读、主存写，MDRin 表示内部总线上数据写入 MDR，MDRout 表示 MDR 的内容送内部总线。

（1）设 ALU 的输入端 A、B 及输出端 F 的最高位分别为 A15、B15 及 F15，FR 中的符号标志和溢出标志分别为 SF 和 OF，则 SF 的逻辑表达式是什么？A 加 B、A 减 B 时 OF 的逻辑表达式分别是什么？要求逻辑表达式的输入变量为 A15、B15 及 F15。

（2）为什么要设置暂存器 Y 和 Z?

（3）若 GPRs 的输入端 rs、rd 分别为所读、写的通用寄存器的编号，则 GPRs 中最多有多少个通用寄存器？rs 和 rd 来自图中的哪个寄存器？已知 GPRs 内部有一个地址译码器和一个多路选择器，rd 应连接地址译码器还是多路选择器?

（4）取指令阶段（不考虑 PC 增量操作）的控制信号序列是什么？若从发出主存读命令到主存读出数据并传送到 MDR 共需 5 个时钟周期，则取指令阶段至少需要几个时钟周期?

（5）图中控制信号由什么部件产生？图中哪些寄存器的输出信号会连到该部件的输入端?

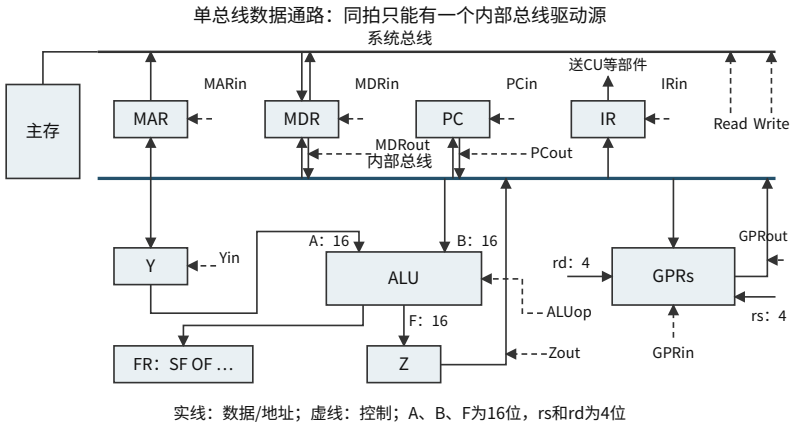



Figure 5.1: 单条内部总线连接 Y、ALU 与 Z；rs、rd 为 4 位寄存器编号

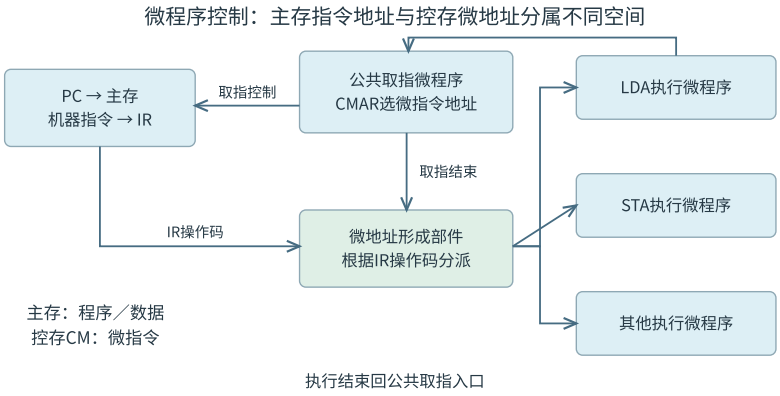



Figure 5.2: 取指入口、操作码映射和执行微程序

**例 5-3**

某指令功能为 $\mathrm{R}[r2] \leftarrow \mathrm{R}[r1] + \mathrm{M}[\mathrm{R}[r0]]$，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（　）。

I. 通用寄存器组（GPRs）　II. 算术逻辑单元（ALU）
III. 存储器（Memory）　IV. 指令译码器（ID）

A. 仅 I、II　B. 仅 I、II、III　C. 仅 II、III、IV　D. 仅 I、III、IV

## 5.4 硬布线与微程序控制器

| 维度 | 硬布线控制 | 微程序控制 |
|---|---|---|
| 控制信号来源 | 操作码、节拍、周期状态及标志经逻辑电路形成 | 控存微指令的控制字段直接或译码形成 |
| 常见特点 | 控制延迟较短；修改复杂功能需改逻辑 | 便于组织复杂控制；受控存与微周期开销影响 |
| 设计步骤 | 列微操作 → 排节拍 → 合成控制逻辑 | 列微操作/节拍 → 选编码与下址方式 → 分配控存 → 编微码 |

硬布线中某信号在指令 A 的 $T_1$ 或指令 B 的 $T_2$ 发生，可写 $C = A \cdot T_1 + B \cdot T_2$；加、乘分别为逻辑或、与，条件标志乘入相应项。

| 微程序对象 | 含义 |
|---|---|
| 机器指令 | ISA 规定的一次软件可见操作 |
| 微程序 | 实现一条机器指令等功能的微指令序列 |
| 微指令 | 一个微周期的操作控制和顺序控制信息 |

| 控制字段编码 | 位数与并行性 |
|---|---|
| 字段直接编码 | 互斥微命令同组，各组可并行；含 $k$ 条命令及空操作需 $\lceil \log_2(k+1) \rceil$ 位 |
| 字段间接编码 | 利用字段间依赖解释；进一步压缩并增加译码层次 |

水平型通常控制字较宽、并行微操作较多；垂直型编码较强、字短，可能需更多微指令；混合型折中。下微地址可来自下地址字段、顺序增量、操作码映射、条件测试或硬件强制入口。控存含 $M$ 个可区分地址时，地址至少 $\lceil \log_2 M \rceil$ 位。

静态微程序通常固化于 ROM；动态微程序使用可改写控存；毫微程序在微指令之下再设解释层。修改能力取决于硬件支持。

**例 5-4**

某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控制字段至少有（ ）。

A. 5 位　B. 6 位　C. 15 位　D. 33 位

**例 5-5**

某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是（ ）。

A. 5　B. 6　C. 8　D. 9

## 5.5 异常、中断与返回断点

| 事件 | 检测与处理 | 常见返回位置 |
|---|---|---|
| 故障 fault，如缺页 | 当前指令未完成；修复后重试 | 当前指令起点 |
| 陷阱 trap，如系统调用 | 执行有意通知/服务请求 | 通常为后继位置 |
| 终止 abort | 严重异常，通常无法可靠恢复 | 依体系结构 |
| 外部中断 | 设备、定时器等；通常在指令边界响应 | 下一条尚未执行指令 |

具体分类、保存 PC 及可恢复规则由 ISA 确定。硬件响应保存规定的断点与处理器状态、切换状态并进入入口；软件继续保存所需通用寄存器现场。“中断隐指令” 为硬件响应动作的教材称呼。中断服务程序为软件。

进入内核态后可返回原进程，也可由调度器切换进程；模式切换与上下文切换分开判断。

**例 5-6**

异常事件在当前指令执行过程中进行检测，中断请求则在当前指令执行后进行检测。下列事件中，相应处理程序执行后，必须回到当前指令重新执行的是（ ）。

A. 系统调用　B. 页缺失　C. DMA 传送结束　D. 打印机缺纸

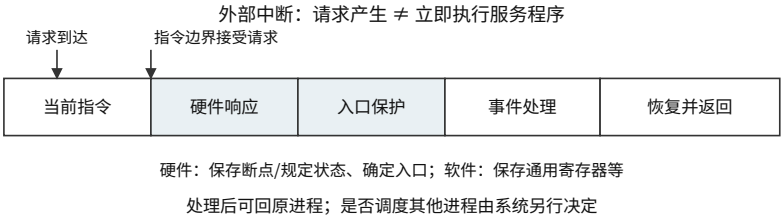



Figure 5.3: 外部请求、硬件入口与软件处理

## 5.6 流水线阶段、时钟与性能公式

| 阶段 | 典型工作 |
|---|---|
| IF | 取指 |
| ID | 译码、读源寄存器 |
| EX | ALU 运算、计算有效地址 |
| MEM | LOAD 读数据、STORE 写数据 |
| WB | 写回寄存器 |

ALU 结果在 EX 末形成，LOAD 数据通常在 MEM 末形成；STORE 通常无寄存器写回。固定五段题可要求所有指令流出五段后才计完成。阶段寄存器须携带数据、立即数、目的寄存器号、控制位、指令地址和必要异常信息。

冻结保持原有状态；气泡是不会改变有效状态的空操作。二者须配合写使能，避免重复写入或错误访存。

等时 $k$ 段、时钟 $\tau$、每拍启动一条、无冒险时：

$$T=(k+n-1)\tau,\quad \mathrm{TP}=\frac{n}{T},\quad S=\frac{nk}{k+n-1},\quad E=\frac{n}{k+n-1}.$$

$S$ 以非流水每条耗 $k\tau$ 为基线；$E$ 假定每条有效使用全部 $k$ 段。实际有用面积、原机器周期不同须重新计算。整体延长 $s$ 拍时，$T=(k+n-1+s)\tau$。$\tau \geq \max t_i + t_r$，$t_r$ 为段寄存器开销；单条延迟约 $k\tau$，稳态完成间隔为 $\tau$。

**例 5-7**

某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90 ns、80 ns、70 ns 和 60 ns，则该计算机的 CPU 时钟周期至少是（　）。

A. 90 ns　　B. 80 ns　　C. 70 ns　　D. 60 ns

按层次分部件级、处理机级、处理机间流水；按功能分单/多功能；按同一时刻功能配置分静态/动态；按有无反馈分线性/非线性。动态流水线的功能分类与指令动态调度分别判断。

## 5.7 结构、数据与控制冒险

| 类别 | 冲突及处理 |
|---|---|
| 结构冒险 | 同拍争用不足的资源；停顿、加端口、分离指令/数据存储器 |
| RAW 写后读 | 消费者需前指令新值；等待、转发、合法指令调度 |
| WAR 读后写 | 后写覆盖尚未读取的旧值；重命名可消除名字冲突 |
| WAW 写后写 | 两次写完成次序改变最终值；重命名、按序提交 |
| 控制冒险 | 条件或后继目标未定；阻塞、预测、合法调度 |

固定五段、顺序发射、ID 读且 WB 顺序写的简单模型通常只需处理 RAW；乱序或不同延迟写回可能产生 WAR/WAW。无条件转移在目标尚未获得时也可能造成控制冒险。

| 时序/旁路条件 | 消费者最早可用时刻 |
|---|---|
| 无转发，同寄存器不能同拍读写 | ID 晚于生产者 WB 至少 1 拍 |
| 无转发，同拍先写后读 | ID 可与生产者 WB 同拍 |
| EX 结果向 EX 转发 | 生产者 EX 末产生，下一拍消费者 EX 可用 |
| LOAD 的 MEM 结果向 EX 转发 | 消费者 EX 须晚于生产者 MEM；紧邻 load-use 常停 1 拍 |

转发仅能传已有结果；STORE 数据端、分支比较端是否有旁路须按图确认。硬件阻塞让前级保持、后级插气泡；编译调度只移动保持语义和副作用的指令。

**例 5-8**

下列指令序列中，指令 I1 和 I3、I2 和 I3 之间发生数据相关。假定采用“取指、译码/取数、执行、访存、写回” 五段流水线方式，那么在采用转发技术时，需要在指令 I3 之前加入（ ）条空操作指令才能使这段程序不发生数据冒险。

```
I1: add  r1, r0, 1    // R[r1] <- R[r0] + 1
I2: load r3, 12(r2)   // R[r3] <- M[R[r2] + 12]
I3: add  r5, r3, r1   // R[r5] <- R[r3] + R[r1]
```

A. 3　B. 2　C. 0　D. 1

**例 5-9**

在采用 “取指、译码/取数、执行、访存、写回”5 段流水线的 RISC 处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1: add  s2, s1, s0  // R[s2] <- R[s1] + R[s0]
I2: load s3, 0(s2)   // R[s3] <- M[R[s2] + 0]
I3: beq  t2, s3, L1  // if R[t2] = R[s3] jump to L1
I4: addi t2, t2, 20  // R[t2] <- R[t2] + 20
I5: L1: ...
```

若采用转发（旁路）技术处理数据冒险，采用硬件阻塞方式处理控制冒险，则在指令 I1～I4 的执行过程中，发生流水线阻塞的指令有（ ）。

A. 仅 I3　B. 仅 I2、I4　C. 仅 I3、I4　D. I2、I3、I4

分支预测错误时清除错误路径结果并重新取指。若分支比例为 $f_b$、误判率 $p_m$、每次损失 $d$ 拍，且无重复计费，则 $\mathrm{CPI} = \mathrm{CPI}_0 + f_b p_m d$。

## 5.8 多发射、动态调度与精确异常

| 技术 | 并行方式与约束 |
|---|---|
| 超标量 | 每周期可发射多条；实际吞吐受依赖、资源、分支及带宽约束 |
| 超流水 | 细分阶段以缩短周期；寄存器及冒险开销限制收益 |
| VLIW | 编译器将可并行操作打包到长指令；需要相应硬件支持 |
| 动态调度 | 后续就绪指令可先执行，减少长延迟造成的空闲 |
| 寄存器重命名 | 新逻辑写分配不同物理寄存器，消除 WAR/WAW 假相关；RAW 保留 |
| 按序提交 | 保持软件可见结果符合程序顺序，支持精确异常 |

最大发射宽度 $w$ 给出理想每拍最多完成 $w$ 条的上限，程序速度仍须按实际 CPI 与周期计算。

## 5.9 Flynn 分类、硬件线程与共享内存

| 分类 | 指令流/数据流 | 典型组织 |
|---|---|---|
| SISD | 单/单 | 传统单指令流执行 |
| SIMD | 单/多 | 向量处理器，同类操作处理多个数据 |
| MISD | 多/单 | 按定义识别 |
| MIMD | 多/多 | 多核、多处理器、多计算机 |

向量执行有启动开销，长向量利于摊薄；分支发散可能降低利用率。SMP 中处理器地位对等，共享单一物理地址空间，各自寄存器及部分 Cache 可私有。多计算机各有私有主存，通过消息通信。

| 硬件多线程 | 线程选择粒度 |
|---|---|
| 粗粒度 | 长延迟事件时切换，可付清空与重填代价 |
| 细粒度 | 按周期等较细粒度交替，每拍选择一个线程，可从该线程发多条 |
| SMT | 同一拍从多个线程选择就绪指令，共享执行槽 |

硬件线程需分别保存体系结构状态，可用于单核；多核为单芯片集成多个执行核。单线程加速、功耗和线程数均依实现及负载确定。共享物理地址空间还需要原子操作、同步、Cache 一致性和存储顺序约束。

**例 5-10**

下列关于并行处理技术的叙述中，不正确的是（　）。

A. 多核处理器属于 MIMD 结构
B. 向量处理器属于 SIMD 结构
C. 硬件多线程技术只可用于多核处理器
D. SMP 中所有处理器共享单一物理地址空间

**例 5-11**

某 16 位计算机中，带符号整数用补码表示，数据 Cache 和指令 Cache 分离。下面给出了指令系统中部分指令的汇编格式及对应功能，其中 Rs 和 Rd 表示寄存器，mem 表示存储单元地址，(x) 表示寄存器 x 或存储单元 x 的内容。

```
ADD   Rs, Rd
SHL   Rd
SHR   Rd
LOAD  Rd, mem
STORE Rs, mem
```

上面每行给出一种汇编格式；下表逐项给出各助记符对应的名称和功能，不改变源、目的操作数的次序。

| 名称 | 助记符 | 指令功能 |
|---|---|---|
| 加法 | ADD | (Rs) + (Rd) → Rd |
| 算术/逻辑左移 | SHL | 2(Rd) → Rd |
| 算术右移 | SHR | (Rd)/2 → Rd |
| 取数 | LOAD | (mem) → Rd |
| 存数 | STORE | (Rs) → mem |

该计算机采用 5 段流水方式执行指令，各流水段分别是取指 IF、译码/读寄存器 ID、执行/计算有效地址 EX、访问存储器 M 和结果写回寄存器 WB。流水线采用 “按序发射、按序完成” 方式，没有采用转发技术处理数据相关，并且同一个寄存器的读和写操作不能在同一个时钟周期内进行。请回答下列问题。

(1) 若 int 型变量 x 的值为 −513，存放在寄存器 R1 中，则执行指令 SHR R1 后，R1 的内容是多少（用十六进制表示)?

(2) 若某个时间段中有连续 4 条指令进入流水线，在其执行过程中没有发生任何阻塞，则执行这 4 条指令需要多少个时钟周期?

(3) 若赋值语句为 x=a+b，x、a、b 均为 int 型变量，其存储单元地址分别为 [x]、[a]、[b]。对应指令序列及流水过程如下，I3 的 ID 段和 I4 的 IF 段被阻塞的原因各是什么?

```
I1  LOAD  R1, [a]
I2  LOAD  R2, [b]
I3  ADD   R1, R2
I4  STORE R2, [x]
```

| 指令 | 成功推进的阶段与周期 | 因依赖或前级冻结而等待 |
|---|---|---|
| I1 | IF1，ID2，EX3，M4，WB5 | 无 |
| I2 | IF2，ID3，EX4，M5，WB6 | 无 |
| I3 | IF3，ID7，EX8，M9，WB10 | 第 4～6 拍在 ID 端等待 |
| I4 | IF7，ID11，EX12，M13，WB14 | 第 4～6 拍未获准取指；第 8～10 拍在 ID 端等待 |

上表将原图时空格紧凑写成 “阶段名 + 周期数”，另列等待状态：I3 第 4～6 拍在 ID 端等待，I4 第 4～6 拍尚未获准取指，第 8～10 拍又在 ID

端等待；这些等待状态保留了原图中的阻塞信息；其他未列时刻表示该指令尚未进入或已经离开流水线。

（4）若赋值语句为 `x=x*2+a`，x 和 a 均为 unsigned int 型变量，其存储单元地址分别为 [x]、[a]，执行这条语句至少需要多少个时钟周期？要求模仿上图，画出对应指令序列及流水线执行过程示意图。

# 6 总线

## 6.1 总线组成、特性与连接结构

总线是多个部件分时共享的信息传输线路及配套协议。每次事务由主设备发起，从设备按地址响应；CPU、取得使用权的 DMA 控制器均可作主设备。读事务的数据由从设备给出，写事务的数据由主设备给出。

| 信息组 | 作用与传送内容 |
|---|---|
| 地址总线 | 指定存储单元或 I/O 端口，宽度影响地址组合数 |
| 数据总线 | 传指令、操作数、状态字、命令字、中断类型号等编码值 |
| 控制总线 | 规定读写、有效时刻、请求、应答及状态 |

写控制端口时，命令字可经数据线传输，控制线说明读写性质。地址的驱动者由当前主设备决定。可寻址容量还需结合编址单位计算；数据线数决定一次并行数据量。

| 总线特性 | 规范内容 |
|---|---|
| 机械 | 连接器、尺寸、形状、引脚布局 |
| 电气 | 电平、电压、负载等 |
| 功能 | 每根信号线的意义 |
| 时间 | 建立、采样、保持与撤销关系 |

| 分类维度 | 类别及特点 |
|---|---|
| 位置/用途 | 片内总线；连接 CPU、主存、I/O 接口的系统总线；系统/设备间通信总线 |
| 数据传送 | 串行逐位发送；并行多线同拍传多位 |
| 连接结构 | 单总线共享，结构简单、争用集中；双/三/多总线分离主存及 I/O 等流量 |
| 桥接 | 缓冲数据、转换协议、协调控制与速率 |

多总线可减少争用，并增加连接与控制成本；跨桥及主存带宽仍可成为瓶颈。并行的线间偏斜、串扰及接插件复杂度限制频率和距离；高速串行可用多条 lane 并行传包。具体总线时钟及速度分别确定。

## 6.2 位宽、传送率、峰值与有效带宽

设每次有效数据 $W$ 位，每秒 $R$ 次传送，时钟频率 $f$、每周期传 $a$ 次：

$$B_{\text{peak}} = R\frac{W}{8} = af\frac{W}{8} \quad (\text{B/s}).$$

MHz 为每秒时钟次数，MT/s 为每秒数据传送次数；已给 MT/s 时直接作 $R$。总线周期为一次规定操作的时间，可能占多拍。若每事务有效载荷 $Q$ B、实际耗时 $T$，则 $B_{\text{eff}} = Q/T$，分母应含相应地址、等待、仲裁等开销。

| 计算对象 | 分子/分母 |
|---|---|
| 峰值带宽 | 数据节拍的有效载荷/数据节拍时间 |
| 事务有效带宽 | 一次事务有效载荷/事务总历时 |
| 设备速率 | 设备产生或消耗的数据/时间 |
| CPU 占用 | CPU 执行查询、服务、设置收尾的时间/总时间 |

校验位、包头等按题设从载荷中扣除；多通道仅在可并行且负载允许时相加；全双工分别计算单方向与双向合计。端到端持续吞吐受最慢环节限制。

**例 6-1**

假设某系统总线在一个总线周期中并行传输 4 B 信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10 MHz，则总线带宽是（ ）。

A. 10 MB/s　B. 20 MB/s　C. 40 MB/s　D. 80 MB/s

**例 6-2**

某 64 位并行总线每时钟周期传 4 次数据，实际数据传送率已明确给为 1333 MT/s，求单方向理想峰值带宽。若改给的是 333.25 MHz 时钟频率，结果是否相同?

32 位、100 MHz、每拍两次时，峰值 $4 \times 2 \times 100 \times 10^6 = 800$ MB/s。32 位、每秒 $33 \times 10^6$ 次有效传送时为 132 MB/s。

## 6.3 总线事务与突发传送时间

一次事务通常经过申请/分配、寻址、传输、结束。读操作：主设备发地址和读命令 → 从设备译码与准备 → 从设备给出有效数据 → 主设备采样 → 结束。写操作由主设备提供写数据。

地址/数据复用线路须分时使用；独立地址、数据线允许的重叠依协议确定。突发传送给一个首地址后连续传多个相邻数据，摊薄地址与启动开销；非突发按每个数据分别计地址、准备及传送。

| 模型 | $n$ 个数据的总时间（无额外重叠） |
|---|---|
| 非突发，每数据均付启动 | $n(t_a + t_p + t_d)$ |
| 突发，只付一次启动 | $t_a + t_p + n t_d$ |

主存准备是否每字重复、是否可流水/重叠须按题设确认。

**例 6-3**

某存储器总线宽度为 64 位，总线时钟频率为 1 GHz，在总线上传输一个数据或地址需要一个时钟周期，不支持突发传送方式。若通过该总线连接 CPU 和主存，主存每次准备一个 64 位数据需要 6 ns，主存块大小为 32 B，则读取一个主存块所需的时间是（ ）。

A. 8 ns　B. 11 ns　C. 26 ns　D. 32 ns

**例 6-4**

某存储器总线的时钟频率为 420 MHz，总线宽度为 64 位，每个时钟周期传送 2 次数据；其总线事务支持突发传送方式，最多传送 8 次数据，第 1 个时钟周期传送地址和读/写命令，从第 4 个至第 7 个时钟周期连续传送 8 次数据。该总线的总线带宽（最大数据传输率）为（ ）。

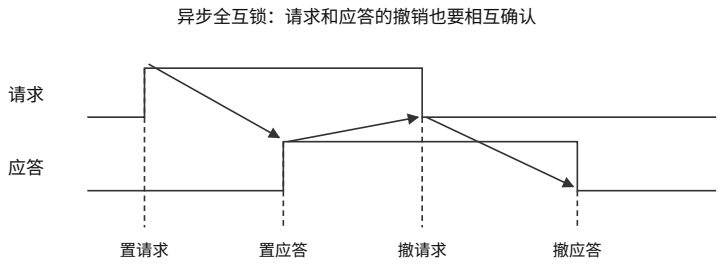



Figure 6.1: 全互锁握手的建立与撤销顺序

A. 3.84 GB/s　B. 6.72 GB/s　C. 30.72 GB/s　D. 53.76 GB/s

50 MHz 同步总线：传 8 个 4 B 字，1 拍地址 +3 拍等待 +8 拍数据，共 12 拍 =240 ns，有效速率约 133.3 MB/s。每个字独立占 1+3+1 拍时，共 40 拍 =800 ns，有效速率 40 MB/s。

等待期间若保持总线占有权，等待计入总线资源占用；若释放总线，总历时与占用时间分别累计。DMA 的总线占用与 CPU 开销分别计算。

## 6.4 同步、异步与半同步定时

| 定时 | 推进依据 | 时序条件 |
|---|---|---|
| 同步 | 共同的时钟边沿 | 周期容纳传播、译码、准备及建立时间；慢设备可插等待 |
| 异步 | 请求/应答等握手 | 各步按规定建立和撤销信号，适应速度差异 |
| 半同步 | 时钟加等待/应答 | 按规定边沿采样，慢设备请求延长准备阶段 |

异步握手按撤销信号是否等待对方确认分类：

| 类型 | 主设备撤请求 | 从设备撤应答 |
|---|---|---|
| 不互锁 | 发送后按预定时间撤销 | 按预定时间撤销 |
| 半互锁 | 等应答出现后撤销 | 按预定时间撤销 |
| 全互锁 | 等应答出现后撤销 | 等请求撤销后撤销 |

全互锁四阶段：请求置位 → 应答置位 → 请求撤销 → 应答撤销。建立和释放均相互确认，增加相应握手等待。

读时序图须标明：有效电平、地址稳定、读写命令有效、数据有效、采样点及事务起止。低有效信号以低电平为有效，重叠动作按协议判断。

串行异步字符帧需计起始、数据、校验、停止位。例如 7 数据 +1 校验 +1 起始 +1 停止 =10 位/字符；480 字符/s 对应线路 4800 bit/s、有效数据 3360 bit/s。

## 6.5 仲裁优先级与分离事务

仲裁决定多个主设备中谁获使用权；定时决定信号何时有效和事务何时推进。优先级给出先后关系，公平性衡量低优先级请求能否长期得到服务。

| 集中式仲裁 | 选择机制 | 特点 |
|---|---|---|
| 链式查询 | 允许信号沿链传播 | 控制线少；靠近仲裁器者通常优先，链路故障敏感，低位可能长期等待 |
| 计数器定时查询 | 输出设备编号，请求者比较匹配 | 起始编号、循环规则可改变优先级与公平性 |
| 独立请求 | 各设备独立请求/允许 | 响应快、优先级灵活；线路和逻辑开销较大 |

按公共请求、忙、允许的教材计线口径：$n$ 设备链式 3 条；计数器式 $\lceil \log_2 n \rceil + 2$ 条；独立请求式每设备请求/允许加公共忙，$2n+1$ 条。8 设备独立请求为 17 条。计算先确认公共信号是否计入。

分布式仲裁由各参与者按共同规则竞争或判定使用权。分离事务将请求和响应分两次占用总线：发请求后释放，准备好再申请返回结果；需用标识与排队机制匹配请求/响应。它可与同步/异步定时组合，总历时可大于总线实际占用时长。

## 6.6 常见总线和接口标准

| 标准 | 组织与主要对象 |
|---|---|
| ISA/EISA | 早期 PC 并行扩展总线 |
| VESA 局部总线 | 早期显示等高速局部外设 |
| PCI | 共享并行局部总线，支持总线主控、突发和配置 |
| AGP | 历史图形加速接口 |
| PCI Express | 点到点串行、多 lane、全双工 |
| USB | 主机与外设串行连接，支持集线器、即插即用、热插拔 |
| SATA | 串行存储接口 |
| SCSI | 设备寻址、命令与控制的接口体系 |
| RS-232 | 串行通信连接、电平与信号规范 |
| PCMCIA/CardBus | 历史便携设备卡式扩展 |

代际、lane 数、编码效率、包头与方向共同决定载荷带宽；具体版本标称值须按题给条件使用。DDR 每周期在两个边沿传数据，已给传送率时直接按该传送率计算。

# 7 输入/输出系统

## 7.1 外部设备、显示存储与缓冲

I/O 硬件包括设备、接口/控制器及连接通路；软件包括驱动与上层管理程序。输入设备将外界信息采样编码，输出设备将编码转换成显示、声音或纸面记录，外存提供持久保存。

| 设备 | 基本机制或参数 |
|---|---|
| 键盘、鼠标 | 按键扫描编码，位移和按键形成报告 |
| 扫描仪、摄像设备 | 光学信息采样编码 |
| 显示器 | 屏幕尺寸、分辨率、每像素位数、刷新频率 |
| 针式打印机 | 撞针经色带成像，可用于多联复写 |
| 喷墨打印机 | 喷墨滴成像 |
| 激光打印机 | 感光鼓成像、转印、定影 |

分辨率 $W \times H$、每像素 $b$ 位、刷新率 $f$ 时，无压缩有效像素量：

$$Q_{\text{frame}} = \frac{WHb}{8}\ \text{B}, \qquad R_{\text{pixel}} = \frac{WHbf}{8}\ \text{B/s}.$$

双缓冲容量为两帧，实际输出率按刷新频率计算。字模库存每种字符点阵，帧缓冲存当前画面像素；消隐、编码、包头与压缩按题设计入。1920×1080、24 位、60 帧/s：单帧 6220800 B，双缓冲 12441600 B，有效速率 373248000 B/s。

接口缓冲吸收瞬时速率差；持续输入高于长期消耗能力时，有限缓冲终会填满。

## 7.2 I/O 接口、端口、编址与通道

接口连接系统总线与设备，承担设备选择、地址识别、命令译码、数据缓冲、状态记录、串并转换、速度匹配，以及中断/DMA 请求。端口是 CPU 可访问的接口寄存器或地址位置，一个接口通常包含多个端口。

| 端口 | 内容与操作 |
|---|---|
| 数据 | 读已接收数据、写待发送数据 |
| 状态 | 读就绪、忙、错误等状态 |
| 控制 | 写启动、复位、工作方式等命令 |

状态读与控制写可共享地址并由读写方向区分。端口地址定位接口位置，扇区地址定位设备内部数据。命令字、状态字、中断类型号均可经数据线传送；读写等控制线规定事务性质。

**例 7-1**

下列选项中，在 I/O 总线的数据线上传输的信息包括（　）。

I. I/O 接口中的命令字　II. I/O 接口中的状态字　III. 中断类型号

A. 仅 I、II　B. 仅 I、III　C. 仅 II、III　D. I、II、III

| 编址 | 地址空间 | 访问方式 |
|---|---|---|
| 统一编址/存储器映射 I/O | 与主存共用；译码须区分内存和设备区域 | 普通访存指令 |
| 独立编址 | 单独 I/O 空间；同数值可各对应内存和端口 | 通常用专门 I/O 指令 |

接口与设备交换数据、命令和状态；接口向 CPU 提出中断请求。通道为执行通道程序的专用 I/O 处理部件，CPU 启动后由其组织系列设备操作；通道指令由通道执行，I/O 指令由 CPU 执行。DMA 主要搬运设定数据块，通道可承担更完整控制组织。

## 7.3 程序查询与服务期限

查询流程：设置命令、地址与数量 → 检查状态 → 就绪后 CPU 读/写数据端口 → 更新地址和计数 → 完成或继续。忙等查询持续检查，等待及搬运均占 CPU；定时查询在两次检查间可运行其他程序。

设设备速率 $R$ B/s、缓冲/每次交换 $q$ B，持续到达且每次取走一缓冲时：

$$\Delta t_{\text{buffer}} = \frac{q}{R}, \qquad \lambda = \frac{R}{q}\ \text{次/s}, \qquad U_{\text{CPU}} = \lambda t_{\text{service}}.$$

查询与取数必须在缓冲覆盖前完成；单缓冲的服务间隔至多为 $q/R$，还须将服务时长、相位及最长延迟纳入期限。已给每次服务 $c$ 个周期、主频 $f$ 时，$U = \lambda c/f$；已给 $I$ 条指令时，$c = I \times \text{CPI}$。

**例 7-2**

某计算机处理器主频为 50 MHz，采用定时查询方式控制设备 A 的 I/O，查询程序运行一次所用的时钟周期数至少为 500。在设备 A 工作期间，为保证数据不丢失，每秒需对其查询至少 200 次，则 CPU 用于设备 A 的 I/O 的时间占整个 CPU 时间的百分比至少是（　）。

A. 0.02%　　B. 0.05%　　C. 0.20%　　D. 0.50%

50 MHz、每次查询 100 周期：鼠标每秒 30 次占 0.006%；磁盘 2 MiB/s、每次 4 B 需 $2^{19}$ 次/s，占约 104.86%，说明该组织服务能力不足。查询开销已包含搬运时，搬运费用只计一次。

## 7.4 中断响应、现场与返回

中断 I/O 流程：CPU 启动设备 → 执行其他程序 → 设备/接口发请求并用触发器保留 →CPU 在允许边界判定未屏蔽请求 → 硬件响应 → 服务程序搬运与处理 → 恢复返回。请求产生、允许响应、开始处理及完成分别计时。

| 环节 | 典型动作 |
|---|---|
| 硬件响应 | 暂时禁止相应中断、保存断点和规定状态、定位入口 |
| 软件入口 | 保存将修改且须恢复的通用寄存器及旧屏蔽字 |
| 服务主体 | 读写数据端口、处理事件、设置控制信息 |
| 软件退出/返回 | 关中断、恢复现场与屏蔽字、按 ISA 规则恢复断点和状态 |

中断向量为定位入口的信息，向量地址为保存向量的位置，处理程序入口为代码地址，具体映射由实现规定。单重中断在处理期间关闭相应可屏蔽中断。多重中断在现场保护和屏蔽设置完成后开放允许的嵌套。

教材单重序列常写“保护现场 → 事件处理 → 恢复现场 → 开中断 → 中断返回”；实际 ISA 也可由返回指令原子恢复状态与允许位。非屏蔽中断的响应时点及保存规则仍由体系结构规定。

## 7.5 中断判优、屏蔽字与嵌套

响应优先级由硬件在当前未屏蔽请求中选谁先响应；处理优先级规定服务程序间谁可打断谁，通过屏蔽字组织。屏蔽改变可参与响应的集合，保留已经发生的请求记录。

按屏蔽位 1= 禁止、0= 允许的约定，服务程序通常屏蔽本级及不允许打断它的源，只开放更高处理优先级。每层进入时保存旧屏蔽字，退出时恢复上一层状态。

**例 7-3**

有 A、B、C 三个请求源，硬件响应优先级 A>B>C。屏蔽字按 A、B、C 排列，1 表示屏蔽。要求处理优先级 C>B>A，服务程序均在保存现场并设置屏蔽字后开中断。给出三者的屏蔽字，并追踪三请求同时待处理时的一次嵌套过程。

请求的到达时间、屏蔽状态、嵌套关系和服务时长共同决定开始及完成顺序。初始响应按硬件优先级，嵌套返回按栈顺序。

**例 7-4**

若某设备中断请求的响应和处理时间为 100 ns，每 400 ns 发出一次中断请求，中断响应所允许的最长延迟时间为 50 ns，则在该设备持续工作过程中，CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是（　）。

A. 12.5%　　B. 25%　　C. 37.5%　　D. 50%

允许响应的最长延迟为期限；CPU 占用只计实际响应/服务执行时间。每 400 ns 请求一次、服务 100 ns，CPU 占 25%；允许延迟 50 ns 另作实时性约束。

## 7.6 DMA 控制器、传送阶段与访存方式

DMA 由控制器在设备接口与主存之间直接传数据。典型组成：主存地址寄存器/计数器、传送长度计数器、数据缓冲寄存器、设备地址、请求触发器、控制/状态逻辑及完成中断机构。

| 阶段 | 执行主体与动作 |
|---|---|
| 预处理 | CPU 驱动设置方向、主存起址、设备、长度等参数并启动 |
| 数据传送 | 设备准备 → 请求 DMA→DMA 申请总线 → 授权后传送 → 更新地址和计数 |
| 后处理 | 块结束/出错通知 CPU，软件检查结果、处理错误、唤醒后续工作 |

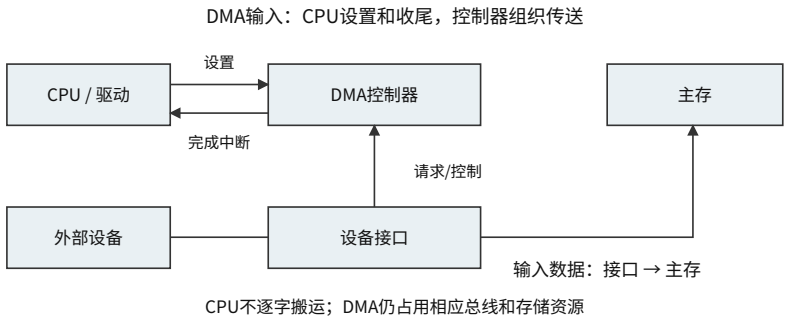



Figure 7.1: CPU 设置收尾，DMA 在接口与主存间搬运

| DMA 访存方式 | 占用与 CPU 行为 |
|---|---|
| 停止 CPU 访存/成组传送 | 集中占用相关总线，期间 CPU 所需同资源访存受阻 |
| 周期挪用 | 准备好一个/若干单位后借用存储周期，通常让已开始 CPU 访存先完成 |
| 交替访存 | CPU 与 DMA 按固定时隙分配，减少重复仲裁，需组织时序 |

DMA 占用共享资源时，CPU 可在可用寄存器、Cache 或其他独立资源上继续；停顿由实际资源需求决定。设备向 DMA 请求服务、DMA 向系统申请总线为两层请求；DMA 申请与请求 CPU 运行服务程序的中断分别判断。

**例 7-5**

若设备采用周期挪用 DMA 方式进行输入和输出，每次 DMA 传送的数据块大小为 512 字节，相应的 I/O 接口中有一个 32 位数据缓冲寄存器。对于数据输入过程，下列叙述中，错误的是（ ）。

A. 每准备好 32 位数据，DMA 控制器就发出一次总线请求
B. 相对于 CPU，DMA 控制器的总线使用权的优先级更高
C. 在整个数据块的传送过程中，CPU 不可以访问主存储器
D. 数据块传送结束时，会产生 “DMA 传送结束” 中断请求

**例 7-6**

下列关于 DMA 方式的叙述中，正确的是（ ）。

I. DMA 传送前由设备驱动程序设置传送参数
II. 数据传送前由 DMA 控制器请求总线使用权
III. 数据传送由 DMA 控制器直接控制总线完成
IV. DMA 传送结束后的处理由中断服务程序完成

A. 仅 I、II　B. 仅 I、III、IV　C. 仅 II、III、IV　D. I、II、III、IV

## 7.7 查询、中断和 DMA 的开销预算

设持续设备速率 $R$ B/s、每次数据交换 $q$ B、每块 $Q$ B、每次 CPU 服务 $t$ s、每块预后处理 $u$ s、每数据单位占总线 $b$ s：

| 量 | 计算式 | 条件 |
|---|---|---|
| 查询/中断事件率 | $R/q$ | 每次交换 $q$ B |
| 查询/中断 CPU 比例 | $(R/q)t$ | 服务开销均已计入 |

| 量 | 计算式 | 条件 |
|---|---|---|
| DMA 完成事件率 | $R/Q$ | 每块正常完成通知一次 |
| DMA 软件 CPU 比例 | $(R/Q)u$ | 预处理及后处理按块计 |
| 数据总线占用比例 | $(R/q)b$ | 单位传输不重叠，无额外未计开销 |

CPU 预算与总线预算分别核算。比例大于 1 表示处理能力不足；平均 CPU 开销低于 100% 后，还需核最长响应时间及缓冲期限。多级缓冲、批量中断、突发合并等条件改变事件粒度时，重新计数。

| 方式 | CPU 介入 | 搬运主体 |
|---|---|---|
| 忙等查询 | 连续检查、逐单位搬运 | CPU 指令 |
| 定时查询 | 定期查询并搬运 | CPU 指令 |
| 中断 I/O | 每请求响应后运行服务程序 | CPU 指令 |
| DMA | 每块设置、完成后收尾 | DMA 控制器 |

字符等低速设备常用查询/中断，大批量块传送常用 DMA；选择依速率、粒度、缓冲和实现。512 B 块、4 B 缓冲为 128 次数据单位传送，正常完成通知一次。1024 B 块则为 256 次 4 B 单位；总线事务数还取决于突发合并。

**例 7-7**

假定计算机的主频为 500 MHz，CPI 为 4。现有设备 A 和 B，其数据传输率分别为 2 MB/s 和 40 MB/s，对应 I/O 接口中各有一个 32 位数据缓冲寄存器。请回答下列问题，要求给出计算过程。

（1）若设备 A 采用定时查询 I/O 方式，每次输入/输出都至少执行 10 条指令。设备 A 最多间隔多长时间查询一次才能不丢失数据？CPU 用于设备 A 输入/输出的时间占 CPU 总时间的百分比至少是多少？

（2）在中断 I/O 方式下，若每次中断响应和中断处理的总时钟周期数至少为 400，则设备 B 能否采用中断 I/O 方式？为什么？

（3）若设备 B 采用 DMA 方式，每次 DMA 传送的数据块大小为 1000 B，CPU 用于 DMA 预处理和后处理的总时钟周期数为 500，则 CPU 用于设备 B 输入/输出的时间占 CPU 总时间的百分比最多是多少？

1 MB/s、每中断 4 B、CPU 开销 2 μs，需 50% CPU；改为 1024 B 块、每块 CPU 开销 20 μs，约 1.953%。两值均仅计软件执行，另核总线占用和服务期限。

## 7.8 设备准备与服务执行的重叠时间

计算总历时先标出 “启动下一单位” 的时刻，CPU 占用累计实际响应和服务指令。CPU 50 MHz、CPI=4；每字符设备准备 0.5 ms=25000 周期；每字符 10 线路位（1 起始、7 数据、1 校验、1 停止），持续设备最多 2000 字符/s。

若每中断响应 10 周期，服务 20 条指令，第 15 条启动下一字符，则后 5 条服务指令可与下一次准备重叠。稳定重复间隔为 $25000+10+15\times4=25070$ 周期；1000 字符约 25070000 周期，CPU 实际用 $1000(10+20\times4)=90000$ 周期。

若从首次启动准备计到第 1000 次服务完全结束，最后 5 条无后续准备可重叠，精确总历时为 $1000\times25070+5\times4=25070020$ 周期。

# 1 计算机系统概述

## 1.1 操作系统的功能与接口

操作系统是控制和管理计算机硬件、软件资源，组织程序运行，并向用户和应用提供接口的系统软件；把裸机扩充为便于使用的机器。

| 管理对象 | 基本职责 |
| --- | --- |
| 处理机 | 进程、线程、调度、同步与通信 |
| 内存 | 分配回收、地址变换、保护共享与扩充 |
| 文件 | 名称目录、存储组织、访问与保护 |
| 设备 | 分配回收、驱动、缓冲与 I/O 协调 |

编译器属于系统软件，提供高级语言编译器不属于操作系统的基本管理职责。图形界面依实现配置。

| 接口 | 使用方式 |
| --- | --- |
| 联机命令接口 | 用户交互式逐条输入命令，系统解释执行 |
| 脱机命令接口 | 预先提交作业控制命令，系统成批执行 |
| 程序接口 | 应用通过系统调用请求内核服务 |
| 图形用户界面 | 用户操作窗口等对象，由应用及系统服务完成请求 |

联机、脱机描述命令提交与交互方式，与是否联网分别判断。

## 1.2 并发、共享、虚拟与异步

| 特征 | 结论与边界 |
| --- | --- |
| 并发 | 多个活动在同一时间间隔内推进；单核交替执行即可成立 |
| 并行 | 多个活动在同一时刻执行；单 CPU 也可与独立 I/O 设备并行 |
| 共享 | 多个并发活动共同使用资源，分为互斥共享、同时共享 |
| 虚拟 | 物理资源被抽象为逻辑资源；时分复用形成虚拟处理机，空分复用支持虚拟存储 |
| 异步 | 进程以不可预知的速度推进；自身指令依赖仍须满足 |

并发与共享互为存在条件。互斥共享要求一个使用者退出后另一使用者进入；同时共享允许同一时期多个使用者访问，具体硬件动作仍可交替完成。增加进程数的收益受切换开销和内存压力限制。

**例 1-1**

下列关于多道程序系统的叙述中，不正确的是（　）。

A. 支持进程的并发执行　　B. 不必支持虚拟存储管理

C. 需要实现对共享资源的管理　　D. 进程数越多 CPU 利用率越高

## 1.3 批处理、分时与实时系统

人工操作使 CPU 等待人工准备；脱机 I/O 减少主机与慢速输入输出的直接等待。单道批处理自动逐个运行作业，当前作业等待 I/O 时 CPU 仍可能空闲；多道批处理在内存保留多个作业，以其他就绪者填补等待间隙，需要中断、存储保护和资源管理。

| 系统 | 组织与目标 | 必要边界 |
| --- | --- | --- |
| 多道批处理 | 成批提交，重吞吐量、资源利用率与周转时间 | 交互性通常较弱；可不使用虚拟内存 |
| 分时 | 时间片轮转，多用户交互 | 重及时响应、独立性、交互性和同时性；时间片无需完成整个任务 |
| 实时 | 按任务期限组织响应或完成 | 硬实时须满足截止时间，软实时容忍一定超期 |

网络操作系统提供联网资源和通信服务；分布式系统组织多机协作并可提供统一资源视图。单用户／多用户、单任务／多任务是独立的分类维度。

## 1.4 用户态、内核态与原语

模式位属于程序状态信息，由硬件按受控事件改变。用户程序通过预定入口请求服务，内核检查参数和权限。内核态可执行特权指令及普通指令；用户态仅可执行允许的非特权指令。

| 操作 | 权限与用途 |
| --- | --- |
| 修改中断屏蔽、关键内存映射、直接控制受保护 I/O | 通常属于特权操作 |
| 算术、逻辑、普通寄存器操作 | 通常可在两种模式执行 |
| 陷入指令 | 用户态可执行；硬件转入规定的内核入口 |

原语完成关键状态更新，要求原子性，如保存现场、修改进程状态、移动队列。单处理器可短暂屏蔽可屏蔽中断来保护合适的内核路径；多处理器还须用原子指令或锁保护共享数据。本 CPU 关中断的作用范围不覆盖其他 CPU。时钟中断支持计时、定时等待和时间片管理。

## 1.5 中断、异常与系统调用

| 事件 | 来源 | 典型恢复行为 |
| --- | --- | --- |
| 外部中断 | 当前指令以外的时钟、设备事件 | 通常在指令边界响应；恢复原进程或按调度切换 |
| 陷阱 trap | 主动执行陷入指令等受控事件 | 通常从规定后继指令继续 |
| 故障 fault | 当前指令执行条件不满足，如合法页面缺页 | 修复条件后重试原指令 |
| 终止 abort | 难以恢复的严重事件 | 通常结束相关执行或进入故障处理 |

中断响应硬件保存断点、程序状态等规定现场，切换权限并按中断向量定位入口；内核入口代码保存必要的通用寄存器，执行处理，恢复现场并用专门指令返回。硬件、软件保存范围以及异常断点由体系结构规定。

**例 1-2**

执行系统调用的过程涉及下列操作，其中由操作系统完成的是（ ）。

I. 保存断点和程序状态字　　II. 保存通用寄存器的内容

III. 执行系统调用服务例程　　IV. 将 CPU 模式改为内核态

A. 仅 I、III　　B. 仅 II、III

C. 仅 II、IV　　D. 仅 II、III、IV

**系统调用路径：**准备调用号及参数 → 陷入 → 内核检查并分派服务 → 返回结果。参数检查包括用户缓冲地址与访问权限。系统调用可立即完成，也可阻塞后再继续。

**库函数**是已有程序代码：纯计算可全在用户态完成，文件 I/O 库函数可处理格式、用户缓冲，再调用内核服务。普通函数调用保存调用现场；系统调用还跨越保护边界。中断、系统调用可以只发生模式切换，进程身份可保持不变。

**合法物理区间**

基址为 $B$、长度为 $L$ 且按字节编址时，可访问的物理字节地址为 $[B, B+L)$；最大合法地址为 $B+L-1$。若题给上下限均可取，则长度为 “上限 − 下限 + 1”。

## 1.6 编译、链接、装入与地址绑定

编译或汇编产生目标模块；链接解析外部符号并形成可装入程序；装入建立地址空间与运行条件。静态链接在运行前完成，装入时动态链接在装入时完成，运行时动态链接推迟到运行需要时。

| 装入方式 | 地址确定或转换时机 | 移动条件 |
|---|---|---|
| 绝对装入 | 编译或汇编时形成绝对地址 | 改位置通常须重生成或修补 |
| 可重定位装入 | 装入时按实际起点修改有关地址 | 装入后搬移仍须修正有关地址 |
| 动态运行时装入／重定位 | 访存时由硬件转换逻辑地址 | 修改基址或映射可改变物理位置，需要硬件支持 |

基址为 $B$、界限保存长度 $L$ 时，逻辑地址 $a$ 须满足 $0 \le a < L$，物理地址为 $B + a$；界限若保存最大合法地址，上界按题设改为包含。

## 1.7 进程地址空间

| 区域 | 常规 C 运行时内容 |
|---|---|
| 代码区 | 机器指令，通常只读、可共享 |
| 初始化数据区 | 已初始化全局、静态存储期对象 |
| 零填充区 | 未显式初始化或零初始化的静态存储期对象 |
| 堆 | 动态申请的对象 |
| 栈 | 函数活动记录、返回信息及相应自动局部对象 |
| 映射区 | 共享库、映射文件等 |

局部指针变量自身通常属于函数活动记录， `malloc` 返回空间属于堆；优化可能将局部变量放入寄存器或消除。PCB 是内核管理结构。虚拟

映像、可执行文件布局与物理页框布局分别判断；地址增长方向和实际排列采用题设。

## 1.8 内核结构

| 结构 | 主要组织原则 | 收益与代价 |
|---|---|---|
| 分层 | 上层调用规定下层服务 | 便于逐层验证，划层困难，调用路径可能加长 |
| 模块化 | 清晰接口，高内聚、低耦合 | 便于替换维护；预先设计接口及跨模块协调有成本 |
| 宏内核 | 较多功能处于同一内核地址空间 | 可直接调用；模块故障可能扩散 |
| 微内核 | 保留基本调度、低级地址空间管理和 IPC 等机制，较多服务在用户态 | 隔离、扩展和移植较便利；IPC 与跨域切换增加开销 |
| 外核 | 分离资源保护复用机制与高层抽象 | 应用或库操作系统自定义抽象，内核继续保护资源 |

模块划分不规定独立保护域，实际系统可混合采用多种组织方式。

**例 1-3**

与宏内核操作系统相比，下列特征中，微内核操作系统具有的是（ ）。

I. 较好的性能 II. 较高的可靠性 III. 较高的安全性 IV. 较强的可扩展性

A. 仅 II、IV B. 仅 I、II、III

C. 仅 I、III、IV D. 仅 II、III、IV

## 1.9 系统引导

处理器复位 → 固件初始化硬件并选择启动设备 → 引导程序分阶段装入内核 → 建立中断入口、内存及设备管理 → 创建初始进程并进入调度。

传统 BIOS ／ MBR 模型中，MBR 可含主引导代码和分区表，卷内引导块可继续装载后续程序；UEFI ／ GPT 使用不同组织。分区、根目录、磁盘 inode 表等持久结构通常在分区或创建文件系统时建立；本次启动须建立运行所需的内存管理与中断结构。

**例 1-4**

下列选项中，需要在操作系统进行初始化过程中创建的是（ ）。

A. 中断向量表 B. 文件系统的根目录

C. 硬盘分区表 D. 文件系统的索引节点表

## 1.10 虚拟机与虚拟机管理程序

VMM 为客户操作系统提供虚拟 CPU、内存和设备，并复用、隔离物理资源。客户内核的高权限受 VMM 和硬件虚拟化机制约束；敏感操作通过受控执行、陷入模拟等完成。

| 维度 | 第一类 VMM | 第二类 VMM |
|---|---|---|
| 依托层次 | 直接运行在硬件上 | 依托宿主操作系统 |
| 设备与存储服务 | 由 VMM 及服务组件管理 | 可利用宿主驱动，虚拟磁盘常为宿主文件 |
| 开销与迁移 | 通常少一层宿主转接 | 还占用宿主资源；镜像便于复制，迁移仍受兼容性约束 |

两类都可使用硬件加速，多虚拟机可在单核上并发、在多核上并行。进程虚拟地址空间隔离访存，系统虚拟机提供完整机器抽象；容器通常共享宿主内核。

# 2

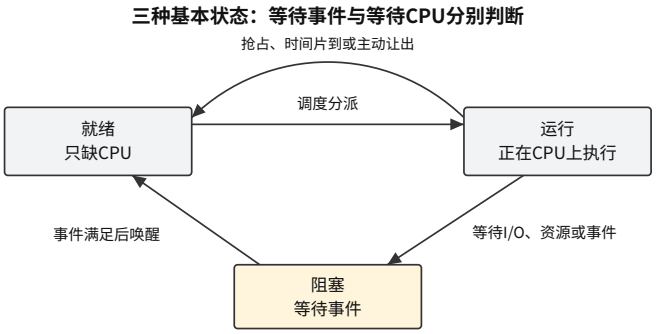



Figure 2.1: 运行、就绪、阻塞的转换条件

## 2.1 进程实体、PCB 与线程

进程是程序的一次执行过程及其资源环境，是资源分配和管理的基本单位。进程实体由程序段、数据段和 PCB 构成；PCB 是进程存在和管理的标志，记录标识、状态、调度信息、资源与处理器现场。进程具有动态性、并发性、独立性、异步性和结构性，动态性是最基本特征。

| 对象 | 共享与独有内容 |
|---|---|
| 不同进程 | 各有 PCB 与地址空间；通信共享须按机制建立 |
| 同进程线程 | 共享代码、数据、堆、打开文件等进程资源 |
| 每条线程 | 独有 TID、PC、寄存器现场、栈、运行状态、优先级等 TCB 信息 |

支持内核级线程时，内核按可调度线程分配处理器；纯用户级线程由线程库调度，内核仍调度其承载实体。同进程线程的栈用于各自调用链，其他线程取得有效地址后仍可能访问它。父子进程各有 PCB、可并发执行；写时复制可暂时共享物理页，逻辑地址空间仍分别管理。

## 2.2 状态转换与进程控制

| 转换 | 条件 |
|---|---|
| 就绪 → 运行 | 被调度并获得 CPU |
| 运行 → 就绪 | 时间片到期、被抢占、主动让出 CPU 且仍具执行条件 |
| 运行 → 阻塞 | 等待 I/O、资源或同步事件 |
| 阻塞 → 就绪 | 对应等待条件满足，执行唤醒 |

就绪仅缺 CPU；阻塞还缺事件。单 CPU 同一时刻至多一个相应实体运行。阻塞者须先就绪再运行；就绪者未执行，不能主动发起新的阻塞操作。

七状态模型在创建、就绪、运行、阻塞、终止上增加就绪挂起和阻塞挂起。经典模型的挂起表示映像暂不驻留内存，PCB 继续保留；阻塞挂起

者事件满足后转为就绪挂起；激活后分别进入就绪或阻塞。运行者也可被挂起，创建者可按策略进入就绪或就绪挂起。

**例 2-1**

下列由当前线程引起的事件或执行的操作中，可能导致该线程由执行态变为就绪态的是（ ）。

A. 键盘输入　　B. 缺页异常　　C. 主动出让 CPU　　D. 执行信号量的 wait() 操作

| 控制原语 | 必须一致更新的内容 |
|---|---|
| 创建 | 申请标识、PCB 与资源，初始化现场，加入就绪等队列 |
| 撤销 | 停止执行、处理通知及等待关系、回收资源与 PCB；子进程政策依系统 |
| 阻塞 | 保存现场、改状态、进入对应事件等待队列并调度 |
| 唤醒 | 移出等待队列、改为就绪、加入就绪队列 |
| 切换 | 保存旧现场、更新状态队列、恢复新现场 |

PCB 可按状态链接成队列，或由索引表定位；阻塞队列可按事件细分。状态字段与队列关系必须原子维护；进程阻塞后的长时间等待允许 CPU 调度其他进程。

## 2.3 用户级、内核级线程与映射

| 模型 | 管理与调度 | 阻塞及并行边界 |
|---|---|---|
| 多对一 | 用户线程库管理多个 ULT，内核仅见一个承载实体 | 承载实体阻塞会挡住全部；这些线程不能多核并行 |
| 一对一 | 每个用户线程对应一个 KLT，由内核分别调度 | 一个阻塞时其他可运行，可多核并行；管理成本较大 |
| 多对多 | 用户线程复用若干 KLT，库与内核分层调度 | 并行度受可用 KLT 与 CPU 核数约束 |

纯用户级切换可不进入内核，开销较小。内核级描述管理方式，应用代码仍可在用户态执行。创建、切换开销随实现而变。

## 2.4 进程通信、管道与信号

| 机制 | 数据或事件的组织 | 使用条件 |
|---|---|---|
| 共享内存 | 多个虚拟映射指向共同物理区，建立后普通访存交换数据 | 共享布局与访问同步由通信方约定 |
| 消息传递 | 直接指定进程，或经信箱／端口间接通信 | 发送、接收是否阻塞由接口规定 |
| 管道 | 有限容量、顺序消费的数据流；经典无名管道为半双工、常用于亲缘进程 | 双向可用两条管道；命名管道可按名称联系 |
| 信号 | 通知事件，可采用默认动作或获准的用户处理程序 | 普通信号可能合并；部分信号不可捕获、忽略或屏蔽 |

共享内存可按规定数据结构组织，也可提供存储区由进程约定布局。管道空时阻塞读、满时阻塞写；有数据即可读，有空间即可写，无须等待整管装满或读空。读出消耗数据，多读者取得不同数据；字节流通常不保留应用记录边界，原子写长度和请求大小按接口。

普通信号位向量模型： `pending` 记录待处理，`blocked` 记录暂缓递送，当前可递送集合

$$D = \text{pending} \ \& \ \sim \text{blocked}.$$

屏蔽信号不使整个进程进入阻塞态。递送可发生在返回用户态等适当时机，用户处理程序一般在用户态执行。信号通知、信号量许可、处理器异常分属不同机制；合法缺页可由内核修复而不递送用户信号。

**RR 首次响应上界**

在时间片边界有 $n$ 个就绪进程（含待考察者）、忽略切换开销，且新到达者不插入其前方时，时间片为 $q$ 的轮转使该进程至多等待 $(n-1)q$ 后首次运行。若观察时刻处于片内，还须计当前片的剩余部分及其前方队列。

## 2.5 调度层次、时机与指标

| 层次 | 选择或管理的对象 |
|---|---|
| 高级／作业调度 | 后备作业进入内存并取得运行资格 |
| 中级／内存调度 | 换出、换入，调节驻留进程数量 |
| 低级／进程调度 | 从就绪执行实体中分配 CPU |

结束、主动阻塞或让出 CPU 时须选择可运行者；新到达、唤醒、时钟到期可触发抢占判断。非抢占策略让当前执行者运行到结束或主动放弃；抢占策略可在规定事件点改选。

调度器选择执行者，分派器恢复现场并交付 CPU。一次中断、一次调度条件检查、一次实际上下文切换须分别计数。内核管理队列的原子临界操作期间限制调度；持有普通资源互斥权的进程仍可能被切换。无普通就绪者时运行最低优先级的 idle 路径，保留中断响应能力。

设到达 $A_i$、首次运行 $S_i$、完成 $C_i$、CPU 服务 $B_i$，则

$$T_i = C_i - A_i, \quad R_i = S_i - A_i, \quad T_i^{\mathrm{w}} = T_i / B_i.$$

仅一个 CPU 段、无 I/O、忽略切换成本时，等待 $W_i = T_i - B_i$；有 I/O 时还须扣去阻塞时间。各指标逐项计算后取算术平均，平均带权周转通常无法由两项平均数相除得到。吞吐量为完成任务数／观察时长，利用率为设备忙时长／观察时长；切换、空闲与有效计算的统计口径按题设。

## 2.6 调度算法与事件时间线

| 算法 | 选择规则 | 条件及限制 |
|---|---|---|
| FCFS | 就绪到达最早者，通常非抢占 | 长任务可使短任务等待较久，产生护航效应 |
| SJF | 当前就绪者中预计服务时间最短，通常非抢占 | 需预测时间；持续短任务可使长任务饥饿 |
| SRTF | 当前就绪者中剩余时间最短，可抢占 | 新到达时比较当前剩余量 |
| HRRN | 最大 $1 + W/B$，通常非抢占 | 等待提高响应比，兼顾长短任务 |
| 优先级 | 当前就绪者中优先级最高 | 数值方向、抢占及同级顺序按题设；老化可缓解饥饿 |

| 算法 | 选择规则 | 条件及限制 |
|---|---|---|
| RR | 就绪队列轮转，至多运行一个时间片 | 用完未结束者回队尾；过大片接近 FCFS，过小增加开销 |
| 多级队列 | 任务通常固定所属队列 | 队间固定优先级或时间比例，队内规则分别规定 |
| 多级反馈 | 根据运行行为迁移队列，常用逐级加长时间片 | 高层可抢占低层；降级、I/O 返回及剩余片规则按题设 |

时间线只在已到达且就绪者中选择。记录每次事件的当前执行者、剩余量、各队列顺序；同刻到达与片耗尽的入队次序须明确。CPU 总服务量须与题设一致。最小平均等待等最优性结论需同时到达、已知服务、忽略开销等相应模型前提。

**例 2-2**

4 个作业到达系统的时刻和运行时间如下表所示。

| 作业 | 到达时刻 | 运行时间 |
|---|---|---|
| J1 | 0 | 3 |
| J2 | 1 | 3 |
| J3 | 1 | 2 |
| J4 | 3 | 1 |

系统在 $t = 2$ 时开始作业调度。若分别采用先来先服务和短作业优先调度算法，则选中的作业分别是（ ）。

A. J2、J3　　B. J1、J4　　C. J2、J4　　D. J1、J3

**例 2-3**

进程进入就绪队列的时刻、优先级（值越大优先权越高）及 CPU 执行时间如下表所示。

| 进程 | 到达 | 优先级 | CPU 时间 |
|---|---|---|---|
| P1 | 0 ms | 1 | 60 ms |
| P2 | 20 ms | 10 | 42 ms |
| P3 | 30 ms | 100 | 13 ms |

采用基于优先权的抢占式 CPU 调度算法，从 0 ms 开始调度，三个进程的平均周转时间为（ ）。

A. 60 ms　　B. 61 ms　　C. 70 ms　　D. 71 ms

**例 2-4**

P0、P1、P2、P3 的到达、优先级（值越小优先权越高）和执行时间如下表所示。采用基于优先权的抢占式进程调度，从 0 ms 开始，到四个进程都运行结束为止，发生进程调度的总次数为（ ）。

| 进程 | 到达 | 优先级 | CPU 时间 |
|---|---|---|---|
| P0 | 0 ms | 15 | 100 ms |
| P1 | 10 ms | 20 | 60 ms |
| P2 | 10 ms | 10 | 20 ms |
| P3 | 15 ms | 6 | 10 ms |

A. 4　　B. 5　　C. 6　　D. 7

**例 2-5**

系统采用二级反馈队列调度。Q1 采用时间片轮转，时间片为 10 ms；Q2 采用短进程优先。系统优先调度 Q1，Q1 为空才调度 Q2；新创建进程先进入 Q1，在 Q1 执行一个时间片后仍未结束则转入 Q2。当前两队列为空，系统依次创建 P1、P2 后开始调度，两者需要的 CPU 时间分别为 30 ms、20 ms。平均等待时间为（ ）。

A. 25 ms　　B. 20 ms　　C. 15 ms　　D. 10 ms

**例 2-6**

系统有 4 个进程，预计运行时间和到达就绪队列的时刻如下，单位均为时间配额。分别采用可抢占式短进程优先和时间片轮转调度（时间片为 2），求各进程的调度次序与平均周转时间。

| 进程 | 到达时刻 | 预计运行时间 |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 1 | 4 |
| P3 | 2 | 9 |
| P4 | 3 | 5 |

**例 2-7**

某进程调度程序采用基于优先数（priority）的调度策略，即选择优先数最小的进程运行。进程创建时由用户指定一个 nice 作为静态优先数。为了动态调整优先数，引入运行时间 cpuTime 和等待时间 waitTime，初值均为 0。进程处于执行态时，cpuTime 定时加 1，且 waitTime 置 0；进程处于就绪态时，cpuTime 置 0，waitTime 定时加 1。请回答下列问题。

（1）若调度程序只将 nice 的值作为进程的优先数，即 priority=nice，则可能会出现饥饿现象，为什么?

（2）使用 nice、cpuTime 和 waitTime 设计一种动态优先数计算方法，以避免产生饥饿现象，并说明 waitTime 的作用。

## 2.7 多处理机与上下文切换

公共就绪队列便于分担负载，访问须互斥；每核私有队列降低争用，需迁移平衡负载。推迁移由系统负载监测程序推动，拉迁移由空闲或较空闲 CPU 的调度程序取任务。软亲和性倾向留在原 CPU，硬亲和性限制允许 CPU 集合；迁移收益须扣除缓存、TLB 局部性和同步成本。SMT 的逻辑处理器共享核内部分执行资源。

上下文包含 PC、PSW、通用寄存器、栈等恢复状态；例如 `x++` 在寄存器已加一、尚未写回时被切换，必须保存该寄存器与下一指令位置。模式切换只改变执行权限，可仍执行原进程。

同进程线程切换需换线程现场，通常保留进程页表与文件资源；跨进程还需建立新地址空间环境。TLB 是否清空取决于地址空间标识等机制，普通数据 Cache 无须因进程切换一律全部清空。

## 2.8 临界区与互斥要求

临界资源在同一时期只许一个进程使用；临界区是访问该资源的代码段，进入区申请、退出区释放。多个进程访问同一资源可有多段相关临

界区。同步规定操作先后，互斥限制访问重叠。高级语言 `x=x+1` 的读、加、写可被交错，单核也可能丢失更新。

| 要求 | 准确含义 |
| --- | --- |
| 空闲让进 | 临界区空闲，申请者有进入机会 |
| 忙则等待 | 已有进入者时，其他相关申请者等待 |
| 有限等待 | 请求后被其他进入者越过次数有有限上界；需执行机会和有限临界区等前提 |
| 让权等待 | 无法进入时阻塞，让出 CPU |

**例 2-8**

进程 A 和进程 B 通过共享缓冲区协作完成数据处理，进程 A 负责产生数据并放入缓冲区，进程 B 从缓冲区读数据并输出。进程 A 和进程 B 之间的制约关系是（ ）。

A. 互斥关系　　B. 同步关系　　C. 互斥和同步关系　　D. 无制约关系

## 2.9 软件互斥、原子指令与锁

双进程软件模型假定共享单次读写原子、顺序一致。`i=0,1`，`j=1-i`，`flag` 初值均为假。

| 方案 | 进入／退出规则 | 失效原因 |
| --- | --- | --- |
| 单标志 | 等 `turn==i`；退出置 `turn=j` | 强制交替，对方不申请时也可能阻挡，违背空闲让进 |
| 双标志先检查 | 等 `!flag[j]` 后置 `flag[i]=true`；退出清自身标志 | 双方可先检查通过再置真，破坏互斥 |
| 双标志后检查 | 先置自身真，再等对方假；退出清自身标志 | 双方都置真后互等，临界区空闲仍无法进入 |

Peterson 同时表达意愿与让步：

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { }
临界区;
flag[i] = false;
```

在上述内存模型与持续调度、有限临界区条件下，Peterson 满足互斥和有限等待；循环检查仍占 CPU。实际机器需原子变量和适当内存序。

```
// TestAndSet 原子返回旧值并置真；lock 初值 false
while (TestAndSet(lock)) { }
临界区；
lock = false;

// Swap 原子交换；key 为进程私有
key = true;
while (key) Swap(lock, key);
临界区；
lock = false;
```

TS ／ Swap 的检查与占用整体原子；普通赋值展开式须额外原子化。这些自旋实现保证互斥，公平和有限等待还需另设机制。持锁很短且持有者可继续执行时可减少睡眠切换开销；较长等待适合阻塞锁。单核自旋须允许持锁者获得执行机会。本 CPU 关中断仅保护合适的单处理器内核路径，普通用户程序无权执行，亦无法阻挡其他 CPU。

## 2.10 信号量的数值语义与前驱同步

本章 `P=wait`、`V=signal`，均采用记录型信号量；队列公平性须额外声明。P、V 的计数更新、判断与入队／唤醒须原子化。

```
P(S): S.value--;
      if (S.value < 0) 入队并阻塞;
      // 唤醒后本次 P 返回，不再减 1
V(S): S.value++;
      if (S.value <= 0) 移出一个等待者并置为就绪;
```

`value>0` 为未分配许可数；`value=0` 为无剩余许可且无该量等待者；`value<0` 的绝对值为等待者数。P 减后为 0 可继续；V 加后为 0 仍须唤醒。V 不因缺资源而阻塞，唤醒后仍等 CPU。整型信号量在 `S<=0` 时忙等、再原子减一，通常无负值，需与记录型数值规则分开。

互斥量初值 1；容量 $n$、初存 $k$ 件时 `empty=n-k, full=k`；未完成通知初值 0。信号量初值来自初态，P 与对应 V 可分属不同进程，通常无互斥锁 API 的持有者限制。

**例 2-9**

用 P、V 操作实现进程同步，信号量的初值为（　）。

A. $-1$　　B. 0　　C. 1　　D. 由用户确定

前驱边 $A \to B$ 可用初值 0 的 `done`：前进程 `A; V(done)`，后进程 `P(done); B`。同线程顺序已保证的边无需重复通知；多路汇合须等待每路完成，分叉须分别发布许可。通知放在前操作完成后、后操作开始前，避免增加题设未要求的先后约束。

**例 2-10**

某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。题 46 图表示上述 6 个操作的执行顺序所必须满足的约束：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执行。请使用信号量的 wait()、signal() 操作描述 T1 和 T2 之间的同步关系，并说明所用信号量的作用及其初值。

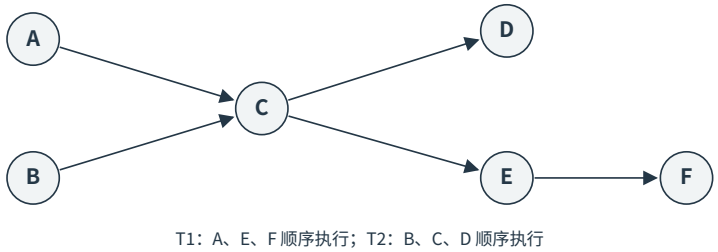



Figure 2.2: 原题前驱图；图下文字标明题干给出的线程归属

## 2.11 生产者与消费者

容量 $n>0$，初始为空，生产和消费可反复进行，每次放取一件；共享缓冲及指针须互斥。局部生产、使用产品放在锁外。

```
semaphore empty=n, full=0, mutex=1;
producer: while (true) {
    x=produce();
    P(empty); P(mutex);
    put(x);
    V(mutex); V(full);
}
consumer: while (true) {
    P(full); P(mutex);
    x=get();
    V(mutex); V(empty);
    consume(x);
}
```

本模板先取得容量／产品许可，再取得缓冲锁。空缓冲时若先持 `mutex` 再等 `full`，生产者无法放入，形成死锁。放入完成后两次 V 可交换，条件是消费者仍须持同一锁访问；这种可交换性依赖具体代码。

许可可能已预留或尚未发布，记录型值还可为负；逐指令执行时 `empty+full=n` 与 “full 等于实存产品数” 均不恒成立。静止且无预留、未发布或等待者时，才能直接对应物理数量。

**例 2-11**

计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

（1）假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？（2 分）

（2）假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

（3）假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

分类产品分别设置可消费许可，共用空位和缓冲锁；连续取 $k$ 件需消费

者批锁覆盖整批，单件缓冲锁只覆盖放取，等待产品时保留生产者补货通路。

**例 2-12**

三个进程 P1、P2、P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce() 生成一个正整数并用 put() 送入缓冲区某一空单元中；P2 每次用 getodd() 从该缓冲区中取出一个奇数并用 countodd() 统计奇数个数；P3 每次用 geteven() 从该缓冲区中取出一个偶数并用 counteven() 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义（要求用伪代码描述）。

**例 2-13**

系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。当缓冲区未满时，生产者进程可以放入其生产的一件产品，否则等待；当缓冲区未空时，消费者进程可以从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号量 P、V（或 wait()、signal()）操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

## 2.12 单盘水果模型

盘子容量 1，初始空；父亲放苹果、母亲放橘子，女儿取苹果、儿子取橘子：

```
semaphore plate=1, apple=0, orange=0;
father:   while (true) { 准备苹果; P(plate); 放苹果; V(apple); }
mother:   while (true) { 准备橘子; P(plate); 放橘子; V(orange);
    }
daughter: while (true) { P(apple); 取苹果; V(plate); 吃苹果; }
son:      while (true) { P(orange); 取橘子; V(plate); 吃橘子; }
```

该容量及交接规则使访问权始终由一个活动持有，同步兼顾盘子互斥。容量增加后，多个生产者可同时取得空位，共享结构还需 `mutex`；在实际放取动作周围加锁。

## 2.13 读者与写者

多个读者可并发读；写者与读者、写者之间互斥。`count` 记录已登记且未退出的读者，首读者为全组申请 `rw`，末读者归还，`cm` 保护计数及首末判断。

```
semaphore rw=1, cm=1;
int count=0;
reader() {
    P(cm);
    if (count==0) P(rw);
    count++;
    V(cm);
    读文件;
    P(cm);
    count--;
    if (count==0) V(rw);
    V(cm);
}
writer() { P(rw); 写文件; V(rw); }
```

检查首末与计数修改必须在同一临界区。首读者持 cm 等 rw 时 count=0，无旧读者退出依赖 cm，写者释放也无需 cm。本方案读者优先，连续读者可使写者饥饿。

加入入口 gate=1：读者在上述登记入口前 P(gate)、登记后 V(gate)；写者 P(gate); P(rw); 写; V(rw); V(gate)，读者退出保持原样。写者取得 gate 后阻止新读者登记，旧读者可退出；FIFO 等公平排队、持续调度及有限读写共同支持无饥饿结论，普通信号量本身无此保证。

## 2.14 哲学家进餐

五人 $P_i$ 需同时持有 $C_i$ 与 $C_{(i+1) \bmod 5}$，各筷子初值 1。全员先取同侧会形成五人各持一根的等待环。限制最多四人进入申请到释放区间：

```
semaphore room=4, C[5]={1,1,1,1,1};
philosopher(i) {
    while (true) {
        思考;
        P(room); P(C[i]); P(C[(i+1)%5]);
        吃饭;
        V(C[(i+1)%5]); V(C[i]); V(room);
    }
}
```

四人是申请人数上限，五根筷子最多支持两人同时进餐。其他无环方案：全局按资源编号从小到大申请；用额外互斥量串行化拿两根的过程，拿齐后释放该量，吃完直接归还筷子。所有方案的无饥饿仍需公平条件；各人独立先左后右无法形成全局递增次序。

## 2.15 管程与条件变量

管程封装共享数据、操作及初始化，同一时刻至多一个进程在其中执行，由语言或运行系统保证入口互斥。条件变量管理等待队列，布尔条件由共享状态表示。执行 x.wait() 即入队阻塞并释放管程；x.signal() 通知一个等待者，无人等待时不积累许可。信号量 V 则会留下计数。

**例 2-14**

若 x 是管程内的条件变量，则当进程执行 x.wait() 时所做的工作是（　）。

A. 实现对变量 x 的互斥访问

B. 唤醒一个在 x 上阻塞的进程

C. 根据 x 的值判断该进程是否进入阻塞态

D. 阻塞该进程，并将之插入 x 的阻塞队列中

Hoare 语义可立即把管程交给被通知者；Mesa 语义允许通知者继续，等待者以后重新竞争。Mesa ／ POSIX 式等待应使用 `while`：

```
put(x) {
    lock(m);
    while (count==n) wait(notFull,m);
    插入x; count++;
    signal(notEmpty); unlock(m);
}
get() {
    lock(m);
    while (count==0) wait(notEmpty,m);
    x=取出; count--;
    signal(notFull); unlock(m);
    return x;
}
```

`wait` 原子释放锁并入队，返回前重新取得锁；重检处理竞争与可能的虚假唤醒。拆开 “解锁、入队” 会丢失中间通知。Java 同步实例方法锁定相应对象，静态同步方法锁定类对象；多个实例的锁通常分别存在。

## 2.16 死锁条件、资源图与安全性

以下讨论可重复使用资源，完成时归还；最大需求可信、可满足后可有限完成。死锁是一组进程各等组内才能产生的事件而无法自行推进；饥饿是个别进程长期得不到服务、其他进程仍可推进；死循环是执行路径持续重复。非重入锁可发生单进程自我死锁。

| 死锁必要条件 | 内容 |
|---|---|
| 互斥 | 实例同一时刻只给一个使用者 |
| 占有并等待 | 保持已有资源，再等新增资源 |
| 不可剥夺 | 使用中的资源不能被任意收回 |
| 循环等待 | 存在首尾相接的资源等待链 |

资源分配图中，进程指向资源为请求边，资源实例指向进程为分配边。无环可排除此模型的死锁；有环且相关资源每类单实例时足以判死锁，多实例时须化简。

安全序列保证每个进程的最大剩余需求可由当时可用量及前者归还量满足。安全表示存在至少一条完整序列；序列可不唯一，也无须强制按其执行。死锁必不安全，不安全可能尚未死锁。

## 2.17 死锁预防、避免与解除

| 策略 | 实施方法 | 条件或代价 |
|---|---|---|
| 预防互斥条件 | 可共享资源改按共享方式组织；SPOOLing 提供逻辑打印服务 | 物理打印仍串行，固有互斥语义须保留 |
| 预防占有并等待 | 一次申请全部，或申请失败先释放已有资源 | 需知需求、可能降低利用率 |
| 允许剥夺 | 保存恢复资源状态并回滚 | 适合可恢复资源，纸上输出难撤回 |
| 预防循环等待 | 统一编号，严格递增申请 | 所有参与者须遵循同一全局次序 |
| 避免 | 每次拟分配后检查安全性 | 需最大需求等未来信息，可能推迟现有量足够的请求 |
| 检测及解除 | 允许进入死锁，检测后终止、剥夺或回滚 | 考虑已做工作、恢复成本、优先级；反复同一牺牲者可饥饿 |

**例 2-15**

若系统 S1 采用死锁避免方法，S2 采用死锁检测方法。下列叙述中，正确的是（ ）。

I. S1 会限制用户申请资源的顺序，而 S2 不会

II. S1 需要进程运行所需的资源总量信息，而 S2 不需要

III. S1 不会给可能导致死锁的进程分配资源，而 S2 会

A. 仅 I、II　　B. 仅 II、III　　C. 仅 I、III　　D. I、II、III

仅降低死锁者优先级无法使其释放资源。预防改变请求规则，避免依据当前状态和最大需求，检测依据当前等待请求。

## 2.18 银行家算法

向量按同类资源逐项比较。`Need=Max-Allocation`，`Available` 为当前未分配量；始终满足总资源守恒。

```
Work=Available; Finish[0..n-1]=false;
while 存在 i 满足 !Finish[i] && Need[i]<=Work:
    Work += Allocation[i];
    Finish[i]=true;
全部标记：安全；仍有未标记且无可选者：不安全。
```

模拟中借出的 Need 随完成归还相抵，净增为原 Allocation。检查新请求 $Q_i$：① $Q_i \le Need_i$；② $Q_i \le Available$；③ 试减 Available、试增 Allocation、试减 Need，完整执行安全检查，安全则提交，否则回滚并等待。

**例 2-16**

某时刻进程的资源使用情况如下表所示，可用资源向量为 $(0,2,1)$。

| 进程 | 已分配 $(R_1,R_2,R_3)$ | 尚需分配 $(R_1,R_2,R_3)$ |
|---|---|---|
| P1 | (2,0,0) | (0,0,1) |
| P2 | (1,2,0) | (1,3,2) |
| P3 | (0,1,1) | (1,3,1) |

| 进程 | 已分配 $(R_1, R_2, R_3)$ | 尚需分配 $(R_1, R_2, R_3)$ |
|---|---|---|
| P4 | (0,0,1) | (2,0,0) |

此时的安全序列是（ ）。

A. P1、P2、P3、P4　　B. P1、P3、P2、P4

C. P1、P4、P3、P2　　D. 不存在

**例 2-17**

系统有三类资源 A、B、C 和五个进程，某时刻分配如下。若系统安全，则可用资源向量 $(x, y, z)$ 可能是哪些?

| 进程 | Allocation | Max |
|---|---|---|
| P0 | (0,0,3) | (0,0,4) |
| P1 | (1,0,0) | (1,7,5) |
| P2 | (1,3,5) | (2,3,5) |
| P3 | (0,0,2) | (0,6,4) |
| P4 | (0,0,1) | (0,6,5) |

I. (1,4,0)　II. (0,6,2)　III. (1,1,1)　IV. (0,4,7)

A. I、II、IV　　B. I、II　　C. 仅 I　　D. I、III

单资源模型总量 100，B、A、T 的 Max 为 70、40、50，Allocation 为 20、10、30，可用 40。给 B 再分 30 后可用 10、Need 为 20、30、20，无人可先完成，应推迟；独立从原态给 A 再分 20 后可用 20、Need 为 50、10、20，可取 T→B→A，Work 为 20→50→70→100。

若 $n$ 个进程每个最多需 $k \geq 1$ 个同类资源，最坏全员各持 $k-1$ 个仍未满足；总量

$$m \geq n(k-1)+1$$

足以保证任意分配顺序下至少一人可拿齐并完成。它属于任意分配的总量保证；银行家可通过推迟不安全请求在更少资源下安全运行。

## 2.19 多实例资源的死锁检测

检测用当前未满足 `Request`，避免用未来最大 `Need`。设 `Work=Available`，通常先将 Allocation 全零者标记完成；反复选择未标记且 `Request[i]<=Work` 者，令 `Work+=Allocation[i]` 并标记。最终持有资源且无法化简者构成死锁核心；无持有量的等待者也可能受其影响，集合范围按题用定义。

设 $R_1$ 共 3、$R_2$ 共 2：P1 持 $(2,0)$，P2 持 $(1,1)$ 并请求 $(1,0)$。若 P1 请求 $(0,1)$，初始 Work=$(0,1)$，可先 P1 后 P2 完成；若另一 $R_2$ 由无请求的 P3 持有，P1 请求 $(0,2)$，P3 归还后 Work 仅 $(0,1)$，P1、P2 仍无法化简。

# 3 内存管理

## 3.1 地址、装入与保护

逻辑／虚拟地址描述进程地址空间，物理地址定位主存。OS 建立并维护映射，MMU 完成通常的运行时转换；异常进入内核处理。按字节编址时，$v$ 位虚拟地址表示 $2^v$ B 范围，$p$ 位物理地址表示 $2^p$ B 可寻址范围，实际可用量还受保留区、硬件有效位和资源约束。

| 机制 | 工作与时机 |
|---|---|
| 编译、汇编 | 生成目标模块 |
| 静态链接 | 运行前连接模块并解析外部符号 |
| 装入时动态链接 | 装入时连接所需模块 |
| 运行时动态链接 | 运行需要时连接相应模块 |
| 绝对装入 | 编译或汇编时确定绝对地址 |
| 可重定位装入 | 装入时按实际起点修正有关地址 |
| 动态运行时装入／重定位 | 访存时硬件按当前映射转换，可支持运行中移动 |

基址 $B$、界限为长度 $L$ 时，先检查 $0 \le a < L$ 再形成 $B + a$；若界限存最大合法地址，则上界包含。上下限保护须比较物理地址，基址加界限方式可先检查逻辑地址，按寄存器定义使用。

代码、数据、堆、栈、共享库等属于虚拟映像，物理位置可离散。局部指针自身与所指堆对象分别定位，编译宏不必形成运行时对象。地址范围、驻留状态、访问权限分别检查。

## 3.2 共享、写时复制与回收

共享页使多个进程映射到同一物理页框，虚拟页号可不同；共享只读代码可禁止写，共享可写数据仍需同步。写时复制先共享只读物理页，写入异常时，若仍须保留其他共享者的原内容，则为写者复制私有页并更新映射；若已无其他共享者，可按实现直接恢复写权限。共享页回收须检查其他使用者，映射或权限改变须使相应 TLB 项一致失效或更新。

**例 3-1**

进程 R 和 S 共享数据 data。若 data 在两进程中的页号分别为 p1、p2，对应页框号分别为 f1、f2，下列叙述中，正确的是（　）。

A. p1、p2 一定相等，f1、f2 一定相等

B. p1、p2 一定相等，f1、f2 不一定相等

C. p1、p2 不一定相等，f1、f2 一定相等

D. p1、p2 不一定相等，f1、f2 不一定相等

## 3.3 连续分配与动态分区

| 方法 | 特征与碎片 |
|---|---|
| 单一连续分配 | 系统区加单个用户区，仅容纳一道用户作业 |
| 固定分区 | 可等大或不等大，每区一道作业；未用区内空间为内部碎片 |
| 动态分区 | 按请求划连续区；离散空洞为外部碎片 |

固定分区说明表保存分区号、起址、大小、状态；作业超过最大分区时，不能直接把几个离散分区拼成连续区。动态分区用空闲表或链记录起址和长度。

| 选择算法 | 查找规则及常见链序 | 主要影响 |
|---|---|---|
| 首次适应 | 地址递增，选首个足够大的洞 | 较可能保留高地址大洞，反复搜索低址碎片 |
| 邻近／循环首次适应 | 地址循环，从上次位置继续 | 分散搜索，可能过早切分大洞 |
| 最佳适应 | 选最小够用洞，常按容量递增 | 易留下难用的小碎片 |
| 最坏适应 | 选最大洞，常按容量递减 | 大洞可能较快耗尽 |

每次按更新后的空闲状态重新选择；空闲总量足够仍可能无足够连续洞。分配剩余区则更新起址与长度，恰好用完则删表项；回收按物理地址邻接合并：两侧均无空闲则新增，仅一侧空闲则扩展，两侧空闲则三段合一。按容量相邻的表项未必在物理地址相邻。紧凑通过搬移已分配区聚合空洞，须重定位支持并支付搬移成本。

**例 3-2**

某基于动态分区存储管理的计算机，主存容量为 55 MB（初始为空闲），采用最佳适配算法。分配和释放顺序为：分配 15 MB，分配 30 MB，释放 15 MB，分配 8 MB，分配 6 MB。此时最大空闲分区为（　）。

A. 7 MB　　B. 9 MB

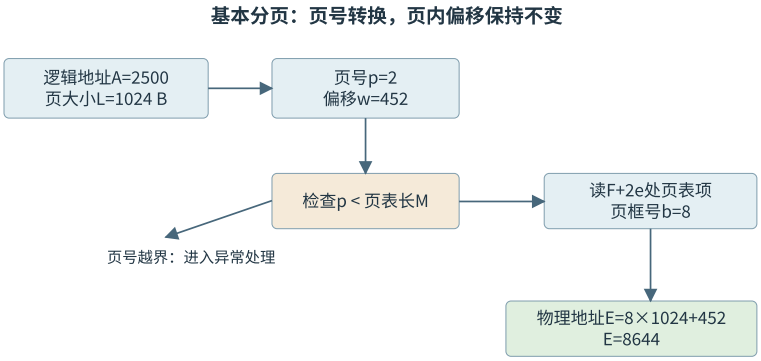
C. 10 MB　　D. 15 MB

## 3.4 覆盖、交换与虚拟存储

| 机制 | 单位与条件 |
|---|---|
| 覆盖 | 常驻模块与覆盖区；同区模块不能同时需要有效内容，通常需程序员给覆盖结构 |
| 交换 | 经典模型按进程映像换出、换入，挂起表示不驻留；须考虑重定位与正在进行的 I/O |
| 请求调页 | 按页在实际引用时调入，利用局部性，由 OS 管理 |

覆盖安排：常驻 A 为 8 KiB，B/C 共用最大 10 KiB 区，D/E/F 共用最大 12 KiB 区，共 30 KiB；需满足给定调用和返回关系。设备正在直接访问的内存通常须固定，不能随意换出。

虚拟内存的多次性表示分批装入,对换性表示运行中可换出再换入,虚拟性表示逻辑使用空间可超过当前物理内存。容量受地址表示范围、后备能力及系统限制。仅在题给理想化模型中, 可用 min(地址范围, 可用内存 + 可用后备) 估算。




**未对齐对象占页数**

长度 $L>0$ B 的连续虚拟对象，起址页内偏移为 $w$，页大小为 $P$ B，占用页数为 $\lceil (w+L)/P \rceil$；只有页对齐即 $w=0$ 时，才可直接用 $\lceil L/P \rceil$。

Figure 3.1: 页号查表与页内偏移拼接

## 3.5 分页地址变换与页表空间

页与页框等大，页间可映射到离散页框，页内连续；消除连续分配外部碎片，末页可有内部碎片，页表另占空间。默认字节编址，页号、页框号从 0 开始；容量单位按题设，本章 KiB 为 $2^{10}$ B。

页大小 $P$，地址 $a$：

$$p=\lfloor a/P \rfloor, \qquad w=a \bmod P, \qquad PA=fP+w.$$

当 $P=2^d$ 时，低 $d$ 位为偏移，转换只替换页号。单级连续紧密页表基址 $F$、表长 $N$ 项、每项 $e$ B：先检查 $0 \le p < N$，表项地址 $F+pe$，取页框号、驻留与权限，再访问目标；表项地址和数据地址分别计算。

虚拟地址 $v$ 位、物理地址 $b$ 位、页 $2^d$ B 时：潜在页表项 $2^{v-d}$ 项；页框号至少 $b-d$ 位，另加状态与权限。单级全空间页表为 $2^{v-d}e$ B，不能用物理页框数代替虚拟页数。若每页留对齐空隙、表项不得跨页，项地址还须按实际布局计空隙。

## 3.6 多级页表与 TLB

多级页表只为使用区间分配下级表，适合稀疏虚拟空间；全展开会增加目录开销。下级表可分散，单个最终表项宽度仍由页框和状态决定。

每表限定一页、表项 $e$ B 时，扇出 $q=\lfloor P/e \rfloor$；若 $q=2^k$，每级最多索引 $k$ 位页号，至少需 $\lceil (v-d)/k \rceil$ 级。顶级可使用较少位。逐级用索引乘表项长度定位下一表，最终页框才用于数据地址。

**例 3-3**

某计算机采用二级页表，按字节编址，页大小为 $2^{10}$ B，页表项大小为 2 B，逻辑地址依次由页目录号、页号、页内偏移量组成。逻辑地址空间大小为 $2^{16}$ 页，表示整个逻辑地址空间的页目录表至少包含多少表项?

A. 64　　B. 128

C. 256　　D. 512

**例 3-4**

某计算机主存按字节编址，二级分页地址结构依次为页目录号 10 位、页号 10 位、页内偏移 12 位。虚拟地址 2050 1225H 对应的页目录号、页号分别是（　）。

A. 081H、101H　　B. 081H、401H

C. 201H、101H　　D. 201H、401H

TLB 缓存转换及权限，数据 Cache 缓存数据。TLB 未命中可从内存页表取得驻留映射；合法页不驻留才进入缺页分支。页表与 TLB 的访问、修改状态须按题给策略保持一致。

无缺页、页表各级均在内存、无其他缓存、各查询串行，TLB 查询 $t$、主存一次 $m$、命中率 $h$、页表 $r$ 级：

$$E[T] = h(t+m) + (1-h)[t + (r+1)m].$$

忽略 TLB 查询时间，平均主存次数为 $1 + (1-h)r$。并行查询须重列时间线；单级、$t = 1\ \mu s, m = 100\ \mu s, h = 0.9$ 时，串行 111 μs；TLB 与主存页表同起，按题给命中路径 101、未命中路径 200 μs，则平均 110.9 μs。

## 3.7 分段与段页式

分段按代码、数据等逻辑单位，段长可变；地址为段号、段内偏移。检查段号、段长、驻留与权限，合法后 $PA =$ 段基址 $+$ 偏移。偏移等于段长即越界。

| 机制 | 地址与表路径 | 物理布局、碎片 |
|---|---|---|
| 基本分页 | 页号、页内偏移；页表 → 目标 | 页内连续、页间离散，内部碎片 |
| 基本分段 | 段号、段内偏移；段表 → 目标 | 每段连续、段间离散，外部碎片 |
| 段页式 | 段号、段内页号、页内偏移；段表 → 本段页表 → 目标 | 段内页可离散，各段末页可能有内部碎片 |

上述单级各表驻留、无 TLB 及其他缓存、无异常时分别 2、2、3 次主存访问。段页式逻辑坐标仍可视为段号、段内地址二维；编码可把段内地址拆为两个字段。先查段范围权限，再查段内页范围；题给字节段长时还须检查完整段内偏移。

## 3.8 请求分页与缺页处理

时间局部性来自重复使用，空间局部性来自邻近访问。请求分页页表记录存在位、访问位 A、修改位 M、后备位置及保护信息。M 表示已写，写权限表示能否写，二者独立。

合法地址查询 $\rightarrow$ 页未驻留 $\rightarrow$ 保存重启状态并定位后备页 $\rightarrow$ 取得空框或选择牺牲页 $\rightarrow$ 必要时写回脏页 $\rightarrow$ 调入 $\rightarrow$ 更新页表、失效旧 TLB 并建立新转换 $\rightarrow$ 重试原指令。权限不足或非法地址按保护错误处理。

有空框的缺页无需置换；干净且有有效后备副本的牺牲页可直接丢弃；需保留的脏内容在复用页框前回写。一条复杂指令可引用多页、产生多次缺页，最少页框数须满足指令及寻址重启要求。锁定内核页和正在供设备直接访问的页通常不参与置换。

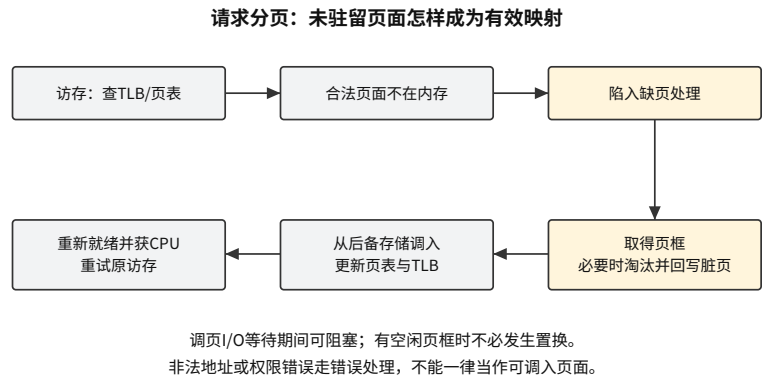



Figure 3.2: 缺页处理与原指令重试

## 3.9 页框分配与页面后备

驻留集是当前在物理内存的页面集合。固定分配保持进程页框数，可变分配按运行调整；局部置换仅选本进程页面，全局置换可影响其他进程的页框占用。经典组合为固定分配局部置换、可变分配局部置换、可变分配全局置换。页框可平均、按大小或按优先级分配，仍须满足最少页框要求。

请求调页在引用时调入，预调页提前装入预计会使用的页；预调有无用 I/O 风险。后备位置模型：对换区充分时可先复制到对换区，运行中在内存和对换区交换；对换区不足时只读文件页从文件读、淘汰丢弃，修改内容写入对换区；教材 UNIX 模型首次从文件调入、已换出者从对换区调回。匿名页、私有映射、共享文件映射具体后备关系按题设。

## 3.10 页面置换算法

| 算法 | 淘汰依据 | 状态更新与边界 |
|---|---|---|
| OPT | 未来最久不用或以后不用 | 理论比较下界，需未来引用串 |
| FIFO | 最早装入 | 命中不改变装入队列；可有 Belady 异常 |
| LRU | 过去最久未使用 | 每次命中都更新最近次序，实现需额外记录 |
| Clock | 循环扫描 A=0 者；A=1 清零并跳过 | 命中置 A=1；装入、指针规则采用题设 |
| 改进 Clock | 同时考虑 A、M，优先近期未访问且干净者 | 扫描只清 A，脏内容回写后才可清 M |

LRU、OPT 在相同引用串和一致初态下具有栈性质，增框不会增加缺页；LRU 对任意串优于 FIFO 的结论不成立。缺页总数分为使用空框与淘汰旧页两类，置换次数须扣除空框初装。

**例 3-5**

某系统采用 LRU 页置换和局部置换。为进程预分配 4 个页框，访问页号依次为 0、1、2、7、0、5、3、5、0、2、7、6。产生页面置换的总次数为（　）。

A. 3　　B. 4

C. 5　　D. 6

教材改进 Clock 四轮：① 自起点找首个 $(0,0)$，不改 A；② 找首个 $(0,1)$，

经过但未选中的页 A 清零；③ 若仍失败，重新从起点找 $(0,0)$，不改 A；④ 再找 $(0,1)$。每轮最多一圈；第 2 轮改变了后续候选分类，须动态更新。起点与指针推进规则决定同类中选择哪个页。

**例 3-6**

某请求分页系统页大小为 4 KB，按字节编址。为进程 P 分配 2 个固定页框，采用改进型 Clock，页表部分内容如下。

| 页号 | 页框号 | 存在位 | 访问位 |
|---|---|---|---|
| 2 | 20H | 0 | 0 |
| 3 | 60H | 1 | 1 |
| 4 | 80H | 1 | 1 |

| 页号 | 修改位 |
|---|---|
| 2 | 0 |
| 3 | 0 |
| 4 | 1 |

P 访问虚拟地址 02A01H，经地址变换得到的物理地址是（　）。

A. 00A01H　　B. 20A01H
C. 60A01H　　D. 80A01H

## 3.11 工作集、抖动与数组局部性

工作集是最近窗口内访问过的不同页集合；按最近 $\Delta$ 次引用定义时，先取引用再去重，集合大小可小于窗口。驻留集反映物理状态，工作集反映近期需求；工作集未必总为驻留集子集。

**例 3-7**

某进程访问页面的序列如下，t 位于所示竖线处，尚未访问竖线右侧的 0。

$$\ldots, 1, 3, 4, 5, 6, 0, 3, 2, 3, 2 \,\big|_t\, 0, 4, 0, 3, 2, 9, 2, 1, \ldots$$

工作集窗口大小为 6，则 t 时刻工作集为（　）。

A. {6,0,3,2}　　B. {2,3,0,4}
C. {0,4,3,2,9}　　D. {4,5,6,0,3,2}

总工作集需求超过可用页框可引发抖动，大量时间用于换页、计算进展少。可减少活动进程、增加相应页框或用工作集／缺页率调节；扩充交换区容量无法直接补足驻留页框。

行优先数组元素 `a[i][j]` 字节偏移为 $(i \cdot \text{列数} + j) \cdot \text{元素字节数}$。先加起始地址、再分解页号，形成引用串；缺页数还依初态、对齐、页框和置换。4 B 整数、页对齐 `a[128][128]`、页 512 B、仅 1 个数据页框且初始未装入时：行遍历每行一缺页共 128 次，列遍历每次跨行共 16384 次。

## 3.12 有效访问时间与综合地址计算

无缺页平均耗时 $T_0$、缺页率 $p$、完整缺页分支耗时 $T_f$：

$$E[T] = (1-p)T_0 + pT_f.$$

若服务时间 $D$ 定义为正常访存外的附加量，可用 $E[T] = T_0 + pD$。逐项确认失败查询、调入、脏页回写、表更新与重试是否已含；统一 ns、μs、ms。TLB 命中率与缺页率按条件路径组合。

**例 3-8**

请求分页管理系统中，假设某进程的页表内容如下表所示。

| 页号 | 页框号 | 有效位（存在位） |
|---|---|---|
| 0 | 101H | 1 |
| 1 | — | 0 |
| 2 | 254H | 1 |

页面大小为 4 KB，一次内存的访问时间为 100 ns，一次快表（TLB）的访问时间为 10 ns，处理一次缺页的平均时间为 $10^8$ ns（已含更新 TLB 和页表的时间）。进程的驻留集大小固定为 2，采用最近最少使用置换算法（LRU）和局部淘汰策略。假设：①TLB 初始为空；② 地址转换时先访问 TLB，若 TLB 未命中，再访问页表（忽略访问页表之后的 TLB 更新时间）；③ 有效位为 0 表示页面不在内存中，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：

（1）依次访问上述三个虚地址，各需多少时间？给出计算过程。

（2）基于上述访问序列，虚地址 1565H 的物理地址是多少？请说明理由。

**例 3-9**

某 32 位系统采用基于二级页表的请求分页存储管理方式，按字节编址，页目录项和页表项长度均为 4 字节，虚拟地址结构如下所示。

| 页目录号（10 位） | 页号（10 位） | 页内偏移量（12 位） |
|---|---|---|

某 C 程序中数组 `a[1024][1024]` 的起始虚拟地址为 1080 0000H，数组元素占 4 字节，该程序运行时，其进程的页目录起始物理地址为 0020 1000H。请回答下列问题。

（1）数组元素 `a[1][2]` 的虚拟地址是什么？对应的页目录号和页号分别是什么？对应的页目录项的物理地址是什么？若该目录项中存放的页框号为 00301H，则 `a[1][2]` 所在页对应的页表项的物理地址是什么？

（2）数组 a 在虚拟地址空间中所占的区域是否必须连续？在物理地址空间中所占区域是否必须连续？

（3）已知数组 a 按行优先方式存放，若对数组 a 分别按行遍历和按列遍历，则哪种遍历方式的局部性更好？

## 3.13 内存映射文件

文件范围关联进程虚拟范围，应用用普通访存访问，系统按需驻留和写回。共享映射可指向相同文件页，修改共享数据须同步；私有映射可用写时复制。建立映射、页面驻留和脏页持久化分别发生。统计设备 I/O 时检查缓存、缺页、写回及后备位置。

# 4 文件管理

## 4.1 文件、目录项与索引节点

文件是具有名称和属性的持久数据集合。FCB 保存类型、大小、所有者、权限、时间、存储位置等管理信息；采用 inode 时，文件名与其余属性分开，目录项建立名字到 inode 号的关联。inode 号仅在所属文件系统内解释。

| 对象 | 内容与作用 |
|---|---|
| 目录项 | 文件名、inode 号；目录本身也是文件 |
| 磁盘 inode | 文件属性、硬链接计数、块定位等持久信息 |
| 内存 inode | 已载入属性及运行时锁、引用状态等 |
| 打开实例 | 一次打开的访问方式、当前偏移及 inode 引用 |
| 进程描述符表 | 在本进程内把 fd 关联到打开实例 |

知道名字、知道 inode 号、inode 已缓存、数据已缓存分别消除不同访问步骤。教材若把打开实例和内存 inode 合称系统打开表，应先确定字段归属再判断共享关系。

## 4.2 建立、打开、读写、关闭与删除

| 操作 | 主要变化 |
|---|---|
| 建立 | 分配控制信息，父目录新增名字；空文件可暂不占数据块 |
| 打开 | 路径查找、权限检查、建立或引用打开状态，返回 fd |
| 读取 | 按当前偏移定位块，必要时 I/O，复制实际字节并更新位置 |
| 写入 | 可分配新块、改变长度、标记缓存为脏，按规则写回 |
| 关闭 | 释放本次打开引用，必要时更新元数据 |
| 删除名字 | 取消目录关联，减少硬链接数，满足回收条件后释放内容 |

打开无需读入整个文件，写返回成功不必表示已持久化。短读、文件结束、缓存命中及错误按接口和题设。

**例 4-1**

若文件 F 仅被进程 P 打开并访问，则当进程 P 关闭 F 时，文件系统需要完成的是（ ）。

A. 删除目录文件中 F 的目录项

B. 释放 F 的索引节点所占的内存空间

C. 释放 F 的索引节点所占的外存空间

D. 将文件磁盘索引节点中的链接计数减 1

两个进程分别 `open` 同一文件通常各有打开实例与独立偏移，指向共同 inode；`dup` 或继承共享同一打开实例时可共享偏移。把各进程偏移放在进程打开表、共享属性放在系统表的教材模型，按其字段使用。打

开计数与硬链接数分别统计运行引用和名字。单次写的原子性无须推出“读—改—写”事务整体原子，应用仍可能需要锁。

## 4.3 文件保护

访问矩阵按主体为行、对象为列，单元格为允许操作集合；按对象列保存为 ACL，按主体行组织为能力表。检查须同时确定身份、所属组、对象和相应操作。

| 方法 | 直接作用及边界 |
|---|---|
| 口令 | 验证访问凭证；保存验证信息，注意凭证泄露与共享 |
| 加密 | 用密钥控制内容可读性，算法与密钥管理决定效果 |
| 权限表 | 规定主体可执行的读、写、执行等操作，访问时检查 |

每类主体每个独立权限需一位，$u$ 类、$r$ 种独立权限需 $ur$ 位。权限间逻辑蕴含关系须由题设给出。目录查找、列出、修改权限与文件数据权限分开检查。Windows 访问控制中，拒绝条目可影响有效权限；实际合并与继承按系统规则。

**例 4-2**

某文件系统针对每个文件，把用户分成安全管理员、文件主、文件主的伙伴、其他用户 4 类；访问权限有完全控制、执行、修改、读取、写入 5 种。若 FCB 中用二进制位串表示不同类别用户对一个文件的访问权限，至少需要多少位?

A. 5　　B. 9　　C. 12　　D. 20

## 4.4 逻辑结构与记录索引

无结构文件向应用提供字节序列，应用解释记录边界；有结构文件由定长或变长记录组成。

| 记录组织 | 定位方式及限制 |
|---|---|
| 串结构顺序文件 | 按存入先后排列，关键字一般顺序查找 |
| 顺序结构顺序文件 | 按关键字有序；适当连续布局下可用有序检索 |
| 索引文件 | 索引项保存关键字、记录逻辑地址等，可定位变长记录 |
| 索引顺序文件 | 每组一索引，先定位组再组内查找，平衡空间与查找 |

定长、逻辑连续记录，首址 $F$、记录长 $L$，第 $i$ 条从 0 编号时起址 $F+iL$；变长须累加长度或另建索引。记录号直接定位、关键字查找分别处理；链接组织仍可能要追链。

$N$ 条记录分 $g$ 组，两层均线性扫描且成功位置近似均匀时，平均比较约 $g/2+N/(2g)$，$g\approx\sqrt{N}$ 时两项平衡；精确成功扫描平均数用各层 $(n+1)/2$。改用折半或树索引须重算。

用户记录索引映射“关键字 $\to$ 文件内逻辑位置”，系统块分配映射“逻辑块号 $\to$ 物理块号”。每记录 $L$ B、块 $B$ B 且 $B/L$ 为整数、记录不跨块时，第 $i$ 条的逻辑块为 $\lfloor iL/B\rfloor$，块内偏移 $iL \bmod B$。逻辑记录结构可搭配不同物理分配方式。

## 4.5 连续、链接与索引分配

下表从文件控制信息已在内存、相关数据和索引尚未缓存出发，访问从 0 编号的逻辑块 $k$。

| 方式 | 定位与访盘数 | 增长及开销 |
|---|---|---|
| 连续 | 起始块号加 $k$，读数据 1 次 | 顺序、随机方便；扩展受邻接空间限制，有外部碎片 |
| 隐式链接 | 从首块沿数据块后继追链，共 $k+1$ 次读 | 离散追加，指针占数据块空间，不便随机访问 |
| FAT 显式链接 | 内存 FAT 追链后读目标 1 次 | 全卷表占内存；定位仍可有多次内存追链 |
| 索引 | 按块号逐级查索引，经 $h$ 级间接则 $h+1$ 次读 | 离散增长，索引块另占空间 |

链接、索引消除整体连续要求，最后一块仍可有内部未用空间。FAT 驻留条件只免除表的访盘。多个索引块可链接，也可用多层索引；链接索引块时查后块仍需追前面的索引块。

**二级区实际索引开销**

扇出为 $q$，文件在二级间接区实际使用 $u$ 个数据块，$1 \le u \le q^2$，且索引按需分配时，该区须有 1 个上层索引块和 $\lceil u/q \rceil$ 个下层索引块；数据容量与索引空间分别计数。

## 4.6 混合索引容量与访问路径

块 $B$ B，块号 $e$ B，每索引块扇出 $q = \lfloor B/e \rfloor$。inode 有 $d$ 个直接项和一级、二级、三级间接项各一个，则理论用户数据长度

$$L_{\max} = (d + q + q^2 + q^3)B.$$

须同时满足地址字段、文件大小字段及卷容量等限制；用户长度不含索引块。直接区逻辑块为 $[0, d)$，一级为 $[d, d+q)$，二级随后 $q^2$ 块，三级再后 $q^3$ 块。

对二级区的逻辑块 $k$，先减 $d+q$ 得区内序号 $u$，两层索引依次为 $\lfloor u/q \rfloor$、$u \bmod q$。读数据前须计未缓存索引块；仅求块号时不加入最终数据块读盘。

**例 4-3**

文件索引节点中有 10 个直接索引指针，一级和二级索引指针各 1 个。磁盘块为 1 KB，每个索引指针 4 B。某文件索引节点已在内存，把该文件偏移量为 1234 和 307400（按字节编址）处所在的磁盘块读入内存，需要访问的磁盘块个数分别是（　）。

A. 1、2　　B. 1、3　　C. 2、3　　D. 2、4

**例 4-4**

一个记录文件采用隐式链接分配，定长逻辑记录为 100 B，采用记录成组分解，盘块为 512 B。目录项已读入内存，对第 22 个逻辑记录完成修改，共启动磁盘多少次?

A. 3　　B. 4　　C. 5　　D. 6

## 4.7 目录结构与路径解析

单级目录命名冲突多；两级按用户分目录；树形支持多层分类；无环图允许多个目录关联同一对象，通常限制目录硬链接以避免环。共享目录

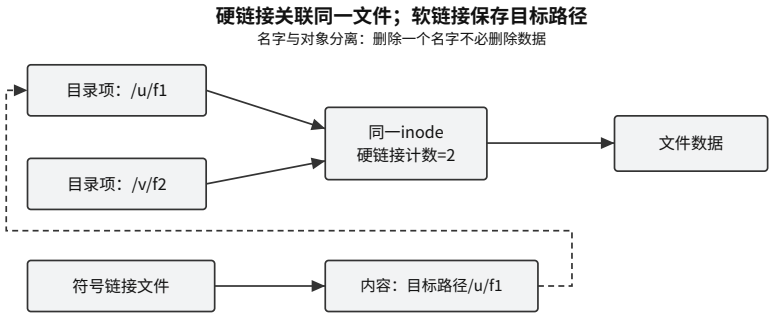



Figure 4.1: 名字、inode 与目标路径的不同关系

边与独立复制分别管理生命周期。

绝对路径从根解析，相对路径从当前目录解析；`.` 为本目录，`..` 为父目录。每个分量通过目录项找到下级 inode，再查下级目录；实际 I/O 由各层目录、inode 与数据缓存决定。当前目录可省去共同前缀查找，权限检查继续执行。

分离文件名与 inode 可缩小目录项。块 $B$ B、目录项 $e$ B、项不得跨块时，每块容纳 $\lfloor B/e \rfloor$ 项。若共 $b$ 个满目录块、目标均匀分布且逐块成功查找，平均读 $(b+1)/2$ 个目录块；再按缓存补计 inode 读取。哈希、树索引或末块不满时按实际概率计算。

## 4.8 硬链接、符号链接与回收

| 项目 | 硬链接 | 软／符号链接 |
|---|---|---|
| 保存关系 | 目录项直接指向同一 inode | 独立链接文件保存目标路径 |
| 目标计数 | 增删名字改变目标硬链接数 | 不因此增加目标硬链接数 |
| 跨文件系统 | 通常受限 | 可跨文件系统 |
| 目录与失效 | 通常限制任意目录硬链接；删除一个名其余可用 | 可指目录；保存路径消失后可悬空 |

目标硬链接数归零且无有效使用引用等条件满足后才回收数据和 inode；已打开但无名字的文件可继续使用至相应引用释放。符号链接相对目标通常相对于链接所在目录解释。两个硬链接共享文件身份，独立打开仍可有独立位置。

**例 4-5**

若文件 f1 的硬链接为 f2，两个进程分别打开 f1、f2，获得文件描述符 fd1、fd2，则正确的是（ ）。

I. f1 和 f2 的读写指针位置保持相同

II. f1 和 f2 共享同一个内存索引节点

III. fd1 和 fd2 分别指向各自的用户打开文件表中的一项

A. 仅 III　　B. 仅 II、III　　C. 仅 I、II　　D. I、II、III

## 4.9 文件系统分层与持久结构

| 职责 | 典型工作 |
|---|---|
| 用户接口与目录系统 | 接收系统调用、路径查找、管理打开关系 |
| 存取控制 | 检查主体权限 |
| 逻辑文件系统 | 记录号转换为文件内逻辑位置 |
| 物理文件系统 | 逻辑块到设备块位置 |
| 辅助分配 | 空间分配回收 |
| 设备管理 | 缓冲、I/O 调度、启动及完成处理 |

层名可按教材模型合并，责任对应关系保持清楚。删除末尾记录时，仍含保留数据的末块须保留，只更新长度等相关信息。

磁盘引导信息服务启动，卷控制块／超级块记录块长、规模、空闲管理等；目录、inode 区记录文件及名字，数据区保存内容。MBR 属整盘引导和分区管理，卷内引导块属于相应卷，布局采用题设。

内存维护挂载信息、目录与 inode 缓存、打开对象和进程描述符表。扩展文件可同时改变位图、inode、数据与索引，部分写入后崩溃可能不一致；一致性检查按引用与分配关系核对，日志按提交和重放规则恢复，保证范围由记录对象与写回规则决定。

## 4.10 空闲空间管理与位图

| 方法 | 组织与分配回收 |
|---|---|
| 空闲表 | 连续空闲区起点、长度；选区后切分，回收按物理邻接合并 |
| 空闲盘块链 | 每块为节点，申请多个块须逐次追链，适合离散分配 |
| 空闲盘区链 | 每节点是一段连续区，记录长度与后继，可批量分出 |
| 位图 | 每受管块一位，0／1 空闲约定按题设，适合连续或离散分配 |
| 成组链接 | 当前组的空闲块号栈及后续组入口，按组装入与保存 |

管理 $N$ 块，位图需 $N$ 位，向上取整为 $\lceil N/8 \rceil$ B；大小由管理范围决定。块号与字、位均 0 基，字长 $n$ 位时：

$$b = ni + j, \quad i = \lfloor b/n \rfloor, \quad j = b \bmod n.$$

三者均 1 基时：$b = n(i-1)+j, i = \lfloor (b-1)/n \rfloor + 1, j = (b-1) \bmod n + 1$。位图磁盘起始块在算出位图内相对位置后加入。

**例 4-6**

文件系统用位图表示磁盘空间分配，位图存于磁盘 32—127 号块，每块 1024 B，盘块和块内字节都从 0 编号。释放盘块号 409612 时，要修改的位所在盘块号和块内字节序号分别为（　）。

A. 81、1　　B. 81、2　　C. 82、1　　D. 82、2

成组链接在当前组最后的承载块内保存下一组：分配到它时，先读出下一组再交付承载块；回收时组未满则压栈，组已满则把整组写入新回收

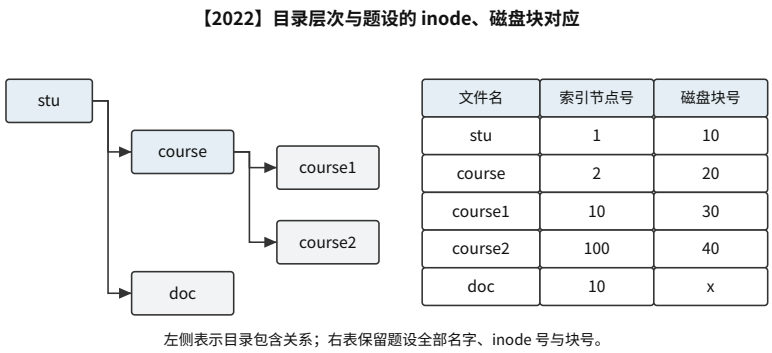



Figure 4.2: stu、course 的目录层次，以及各文件的索引节点号和数据块号

块，让内存组改为该块入口。终止标记不计作可分配块号，内存与磁盘分配元数据须按规定同步。

## 4.11 VFS 与挂载

VFS 提供统一操作抽象，经操作函数入口分派到具体文件系统；磁盘格式可分别设计。vnode 是内存统一虚拟节点，保存通用信息与操作函数指针；Linux 式抽象进一步区分超级块、inode、dentry、file 对象，分别描述已挂载卷、文件本体、名字关系和一次打开。

挂载将新文件系统根接到现有挂载点；原有下层内容通常被当前视图遮蔽，继续保留。卸载须处理使用引用与待写回信息。具体文件系统须向 VFS 注册所需信息和操作入口。

## 4.12 目录、索引与容量综合

名称查找、inode 读取、索引定位、数据读取分别计未缓存成本。文件数量同时受 inode 数、每文件数据块数、间接索引开销及题给其他资源限制，取最紧上界。

**例 4-7**

某文件系统的磁盘块大小为 4 KB，目录项由文件名和索引节点号构成，每个索引节点占 256 字节，其中包含直接地址项 10 个，一级、二级和三级间接地址项各 1 个，每个地址项占 4 字节。该文件系统中子目录 stu 的结构如下图所示，stu 包含子目录 course 和文件 doc，course 子目录包含文件 course1 和 course2。各文件的文件名、索引节点号、占用磁盘块的块号如图中表所示。请回答下列问题。

（1）目录文件 stu 中每个目录项的内容是什么?

（2）文件 doc 占用的磁盘块的块号 x 的值是多少?

（3）若目录文件 course 的内容已在内存，则打开文件 course1 并将其读入内存，需要读几个磁盘块？说明理由。

（4）若文件 course2 的大小增长到 6 MB，则为了存取 course2 需要使用该文件索引节点的哪几级间接地址项？说明理由。

**例 4-8**

某文件系统采用索引节点存放文件的属性和地址信息，簇大小为 4 KB。每个文件索引节点占 64 B，有 11 个地址项，其中直接地址项 8 个，一级、二级和三级间接地址项各 1 个，每个地址项长度为 4 B。请回答下列问题。

（1）该文件系统能支持的最大文件长度是多少？（给出计算表达式即可。）

（2）文件系统用 1M（$1\text{M} = 2^{20}$）个簇存放文件索引节点，用 512M 个簇存放文件数据。若一个图像文件的大小为 5600 B，则该文件系统最多能存放多少个这样的图像文件？

（3）若文件 F1 的大小为 6 KB，文件 F2 的大小为 40 KB，则该文件系统获取 F1 和 F2 最后一个簇的簇号需要的时间是否相同？为什么？

# 5 输入输出管理

## 5.1 设备、控制器与 I/O 地址

设备执行实际输入、输出或存储；控制器管理设备并向主机提供接口。控制器由 CPU—控制器接口、I/O 控制逻辑、控制器—设备接口构成，可连接多个设备。

| 寄存器 | 作用 |
|---|---|
| 控制 | 命令、方向及参数 |
| 状态 | 就绪、忙、错误等，状态编码由设备定义 |
| 数据 | 暂存待传输或已收到的数据 |

I/O 端口表示接口寄存器位置。独立编址由专门 I/O 指令或规定机制访问；内存映射 I/O 复用地址空间及访存指令，缓存属性和副作用须按设备规则。

| 分类维度 | 类别及边界 |
|---|---|
| 信息组织 | 字符设备通常为字符流，块设备支持块号寻址，网络设备按分组收发 |
| 共享性质 | 独占、共享、虚拟；共享可在同一时期交替服务 |
| 使用特性 | 人机交互、外存储、网络通信 |
| 传输速率 | 低、中、高速，范围按具体设备 |

字符单位传送不自动提供任意字节地址的随机访问。设备类型、共享方式和接口阻塞语义分别判断。

## 5.2 轮询、中断、DMA 与通道

| 控制方式 | 数据搬运与教材单位 | CPU 介入 |
|---|---|---|
| 程序直接控制／轮询 | CPU 逐字读写，设备 →CPU 寄存器 → 内存 | 启动、反复查状态、每字搬运 |
| 中断驱动 | CPU 在中断服务中读写，基本模型逐字传输 | 启动后可做其他工作，每次中断服务 |
| DMA | 控制硬件直接在设备和主存间搬一块或连续多块 | 设置地址、方向和长度，完成及异常处理 |
| 通道 | 执行主存中的通道程序，组织一组块和 I/O 操作 | 提供程序地址与对象，处理整组完成或异常 |

DMA 仍会竞争总线及内存带宽，完成可用中断通知。基本 DMA 模型传连续设备块和连续内存区；散布／聚集硬件可扩展，按题给能力使用。

DMA 控制器：DR 暂存数据，MAR 指示主存位置，DC 记录剩余计数，CR 保存命令／状态，传送中按设备规则更新。驱动须提供合法受保护的传输区，可用固定页、内核缓冲、IOMMU 等；CPU 无须逐字搬运仍需初始化与内核资源管理。

## 5.3 软件层次与请求完成

| 层次 | 主要职责 |
|---|---|
| 用户层 I/O | 格式转换、用户缓冲、部分假脱机服务，调用系统接口 |
| 设备独立性软件 | 统一命名映射、权限、分配、缓冲、请求调度、共同差错接口 |
| 设备驱动 | 具体设备地址转换、寄存器、命令与状态，排队并启动 |
| 中断处理 | 响应完成或异常，确认状态、推进请求并唤醒等待者 |
| 硬件 | 设备动作、数据传送与中断响应等规定行为 |

**双缓冲的稳态收益**

在本节模型且 $T, M, C \geq 0$ 时，双缓冲相对单缓冲每块节省 $\min\{M, \max(0, T-C)\}$。当 $C \geq T$，单缓冲输入已被分析覆盖，增加第二缓冲不会缩短稳态间隔。

缓存未命中的阻塞读：应用请求 → 内核定位、申请缓冲 → 驱动设置 DMA 等并启动 → 调用者阻塞 → 完成中断 → 记录数据可用并唤醒 → 调度后复制、返回。若数据已缓存可省去设备分支。驱动运行位置及是否独立调度由系统架构规定。

**例 5-1**

用户程序发出磁盘 I/O 请求后，系统处理流程为：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。其中，计算数据所在磁盘柱面号、磁头号和扇区号的程序是（　）。

A. 用户程序　　B. 系统调用处理程序　　C. 设备驱动程序　　D. 中断处理程序

## 5.4 阻塞、非阻塞、异步与设备接口

| 接口语义 | 行为 |
|---|---|
| 阻塞 | 暂不能完成时等待；已有数据时仍可立即返回 |
| 非阻塞 | 暂不能完成即返回相应状态；反复重试可形成忙等 |
| 异步 | 提交与完成分离，随后经通知或完成队列取得结果 |

用户线程是否等待与硬件搬运方式独立，可使用阻塞接口配合 DMA。字符接口可为 get ／ put，块接口可为 read ／ write ／ seek，网络接口还需通信地址与协议。设备独立性抽象共同操作，具体操作集合、对齐与返回语义仍有差别。

套接字由 `socket` 创建，`bind` 关联本地地址和端口，`connect` 建立远端关联，读写经协议栈与网卡驱动完成。接收事件经中断、驱动及协议层处理后满足套接字等待条件，进程先就绪再等 CPU。

## 5.5 单缓冲与双缓冲时间

缓冲缓和速度差、适配单位并支持阶段重叠；按机制积累多项再通知可减少中断，缓存还利用重用、预读及延迟写减少部分实际 I/O。缓冲数量有限，持续输入快于消费时仍会等待。

设每块输入系统缓冲耗时 $T$、复制至用户工作区 $M$、CPU 分析 $C$。条件：输入可与 CPU 工作重叠；复制与分析共用 CPU、不能重叠；仅一

个用户工作区；每缓冲一块，忽略其他开销，初态全空，$n \ge 1$。

**单缓冲：**输入下一块须等当前缓冲复制完成；复制下一块须等工作区分析结束，稳态间隔 $\max(T, C) + M$，完整批次

$$T_{\text{single}} = T + M + (n-1)[\max(T, C) + M] + C.$$

**双缓冲：**设备可填另一系统缓冲，CPU 复制并分析前块，两阶段节拍 $\max(T, M + C)$，完整批次

$$T_{\text{double}} = T + (n-1)\max(T, M + C) + M + C.$$

$n = 1$ 时均为 $T + M + C$。稳态每块间隔与从空系统开始处理整批的时间分别使用；复制资源、工作区数量或半双工限制改变时须重画时间线。

**例 5-2**

某文件占 10 个磁盘块，逐块读入主存缓冲，再送用户区分析。一个缓冲区与一个磁盘块大小相同，读一块到缓冲区为 100 μs，复制到用户区为 50 μs，CPU 分析一块为 50 μs。在单缓冲、双缓冲结构下，读入并分析完该文件的时间分别是（　）。

A. 1500 μs、1000 μs　　B. 1550 μs、1100 μs

C. 1550 μs、1550 μs　　D. 2000 μs、2000 μs

## 5.6 循环缓冲与缓冲池

循环缓冲按环复用，维护空、满、正在用等状态及下一填入、取出位置。缓冲池按空闲、输入、输出三队列组织，角色随操作变化：

| 活动 | 取得角色 | 完成后 |
|---|---|---|
| 设备收容输入 | 空队列取块作 hin | 满后进输入队列 |
| 计算进程取输入 | 输入队列取块作 sin | 空后回空队列 |
| 计算进程产输出 | 空队列取块作 hout | 满后进输出队列 |
| 设备提取输出 | 输出队列取块作 sout | 空后回空队列 |

队列为空按策略等待，队列与角色转移须同步。分别设置收、发缓冲且链路控制支持时可组织双向并行；共用一个独占缓冲或半双工链路仍受相应限制。

## 5.7 设备分配与控制表

静态分配在作业运行前取得设备，减少占有并等待但可能闲置；动态分配使用时申请，更灵活，需处理死锁风险。设备独占／共享描述使用性质，静态／动态描述分配时机。

| 表 | 内容与关联 |
|---|---|
| SDT 系统设备表 | 设备目录，设备类型、标识、DCT 及驱动入口 |
| DCT 设备控制表 | 每设备一表，状态、控制器指针、重试及等待队列 |
| COCT 控制器控制表 | 每控制器一表，状态、通道指针及等待队列 |

| 表 | 内容与关联 |
|---|---|
| CHCT 通道控制表 | 每通道一表，状态、关联控制器及等待队列 |
| LUT 逻辑设备表 | 逻辑名 → 物理设备及驱动入口 |

按物理名查 SDT→DCT→COCT→CHCT，逐级检查并取得设备及传输路径，全部可用才启动。释放前确认上一请求结束，再改状态、处理等待队列。逻辑名方案由系统选同类设备并登记 LUT；全系统一表时名字须全局唯一，每用户一表时可重复。驱动必须符合相应 OS 的接口、同步、内存与完成规则。

设备安全分配模型规定取得一个设备后立即因本次 I/O 阻塞、完成再继续，从而避免持设备再申请另一设备；同进程计算与该 I/O 串行。不安全分配允许取得后继续执行并再申请，可能占有并等待，需安全检查或检测解除；该名称不表示每次实际分配均已死锁。

## 5.8 SPOOLing

磁盘输入井、输出井及相应进程形成假脱机：输入进程先把设备数据写入输入井，作业后续读取；输出先在井中保存数据并建立请求表，输出进程按队列经缓冲驱动实际设备。

多个作业可分别提交打印数据后继续计算，后台逐份使用物理打印机，逻辑打印服务可共享。井通常在磁盘，内存输入／输出缓冲负责传送暂存，多道程序支持服务与应用并发。

## 5.9 磁盘结构、地址与访问时间

同一半径各盘面磁道组成柱面，同一时刻磁头随磁臂一起移动；盘面内再按扇区定位。设柱面 $C$ 个、每柱面磁道 $H$ 条、每道扇区 $S$ 个、每扇区 $B$ B，容量 $CHSB$。

从 0 编号的线性扇区 $b$：柱面 $\lfloor b/(HS) \rfloor$，磁头 $\lfloor (b \bmod HS)/S \rfloor$，扇区 $b \bmod S$。簇号先乘每簇扇区数，再按几何参数转换；跨磁道等边界按题设。

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

转速 $r$ 转／秒、随机均匀角位置时平均旋转延迟 $1/(2r)$；转速 $R$ 转／分时为 $30/R$ 秒。每道有效容量 $B_t$ B、忽略额外开销时传 $b$ B 的时间可用 $b/(rB_t)$，其他命令、排队和缓存耗时另按题设。

**例 5-3**

磁盘有 100 个柱面，每柱面 8 个磁道，每磁道 8 个扇区。文件 A 有 6400 个逻辑记录，每记录与扇区大小相同，顺序存放；第 0 个记录在 (0 号柱面、0 号盘面、0 号扇区)。磁盘地址 (78、6、6) 存放文件的第几个逻辑记录?

A. 5045　　B. 5046　　C. 5047　　D. 5048

## 5.10 格式化、引导与坏块

物理／低级格式化划分扇区、建立扇区控制信息及校验等；分区划出逻辑区域；逻辑格式化创建超级块、空闲管理、inode ／目录等文件系统结构。文件系统根目录及磁盘 inode 表通常在逻辑格式化建立。

**例 5-4**

下列选项中，磁盘逻辑格式化程序所做的工作是（ ）。

I. 对磁盘进行分区　　II. 建立文件系统根目录

III. 确定磁盘扇区校验码所占位数　　IV. 初始化保存空闲磁盘块信息的数据结构

A. 仅 II　　B. 仅 II、IV　　C. 仅 III、IV　　D. 仅 I、II、IV

固件初始引导从固定入口开始，装入磁盘后续引导程序及内核；整盘 MBR 和卷内引导块分别定位。简单设备可由 OS 记录坏块并避开；控制器也可用备用扇区重映射。不能将坏块当空闲块反复分配，实际管理者按设备模型。

## 5.11 磁盘调度与布局

| 算法 | 服务顺序与折返点 | 边界 |
|---|---|---|
| FCFS | 到达顺序 | 后到者不插队，可能长距离往返 |
| SSTF | 每步选距当前磁头最近者 | 需每步重算；持续近处新请求可使远者饥饿 |
| SCAN | 沿一方向服务，到规定物理端点再反向 | 反向也服务 |
| LOOK | 本方向最后请求处反向 | 省去无请求的端段 |
| C-SCAN | 单向服务，到物理端点回另一端 | 回跳不服务，移动可仍计时 |
| C-LOOK | 单向至最后请求，再回另一侧首个请求 | 回跳不服务 |

部分教材算例将 SCAN、C-SCAN 按 LOOK、C-LOOK 折返点计算。具体题目须明确物理端点、初始方向、回跳计费及最后停止规则；服务次序与移动距离按该约定计算。距离为相邻位置差绝对值之和，平均寻道长度为总移动／请求数。固定有限请求且每次处理有限时 SSTF 可全部完成；无固定等待上界或可能饥饿的结论需结合持续到达模型。

**例 5-5**

某硬盘有 200 个磁道，最外侧磁道号为 0，请求序列为 130、42、180、15、199。当前磁头位于 58 号磁道，从外侧向内侧移动。按 SCAN 处理完这些请求，磁头移动的磁道数为（ ）。

A. 208　　B. 287　　C. 325　　D. 382

交替编号为连续逻辑扇区间留控制器处理时间；跨盘面／柱面错位为磁头切换留时间，减少错过目标后的整圈等待。柱面、盘面、扇区次序可使连续地址先遍历同一柱面的各盘面，减少机械寻道；收益依布局与控制器。

## 5.12 SSD、FTL 与磨损均衡

SSD 无机械寻道和旋转，由芯片、擦除块、页组成；通常按页读和编程、按更大块擦除。FTL 映射主机逻辑块到闪存实际位置；文件系统先

把文件偏移映射到逻辑块，两层分别处理。

已写页修改通常异地写并更新映射，旧页失效。垃圾回收迁移仍有效页后整块擦除，产生额外写入和写放大；备用空间、队列与回收策略影响性能。TRIM 通知逻辑数据已无需保留，删除文件与物理擦除可分时发生。

动态磨损均衡在新写时选择可用低磨损块；静态磨损均衡还迁移长期冷数据，使低磨损块参加后续写入。目标是平衡有限擦写次数，单次延迟仍依实际过程。无寻道后，请求公平、合并、并行深度及写管理继续有意义。

## 5.13 I/O 状态与地址综合

完成中断使等待条件满足，阻塞者先就绪后运行；执行中断处理的 CPU 通常在内核态，当前执行身份可属于其他进程。CHS 题按设备参数由驱动转换，现代 LBA 接口可把几何转换隐藏在设备内部。

**例 5-6**

进程 P 通过执行系统调用从键盘接收一个字符的输入，已知此过程中与进程 P 相关的操作包括：① 将进程 P 插入就绪队列；② 将进程 P 插入阻塞队列；③ 将字符从键盘控制器读入系统缓冲区；④ 启动键盘中断处理程序；⑤ 进程 P 从系统调用返回；⑥ 用户在键盘上输入一个字符。以上编号 ①—⑥ 仅用于标记操作，与操作的先后顺序无关。请回答下列问题。

（1）按照正确的操作顺序，操作 ① 的前一个和后一个操作分别是上述操作中的哪一个？操作 ⑥ 的后一个操作是上述操作中的哪一个?

（2）在上述哪个操作之后 CPU 一定从进程 P 切换到其他进程？在上述哪个操作之后 CPU 调度程序才能选中进程 P 执行?

（3）完成上述哪个操作的代码属于键盘驱动程序?

（4）键盘中断处理程序执行时，进程 P 处于什么状态？CPU 处于内核态还是用户态?

**例 5-7**

某计算机系统中的磁盘有 300 个柱面，每个柱面有 10 个磁道，每个磁道有 200 个扇区，扇区大小为 512 B。文件系统的每个簇包含 2 个扇区。请回答下列问题。

（1）磁盘的容量是多少?

（2）假设磁头在 85 号柱面上，此时有 4 个磁盘访问请求，簇号分别为 100260、60005、101660 和 110560。若采用最短寻道时间优先（SSTF）调度算法，则系统访问簇的先后次序是什么?

（3）第 100530 簇在磁盘上的物理地址是什么？将簇号转换成磁盘物理地址的过程由 I/O 系统的什么程序完成?

# 1 计算机网络体系结构

## 1.1 网络组成、分类与拓扑

计算机网络将具有独立功能的计算设备通过链路和网络设备连接，按共同协议实现数据通信、资源共享和分布式处理。

| 分类依据 | 划分及含义 |
|---|---|
| 工作方式 | 边缘：端系统与应用；核心：互联网络和转发设备 |
| 功能 | 资源子网：计算、存储与共享资源；通信子网：数据传送 |
| 覆盖范围 | PAN、LAN、MAN、WAN；名称没有统一公里数界限 |
| 传输方式 | 广播式：多站共享介质；点到点式：链路连接两端 |
| 其他维度 | 公用/专用、有线/无线，可独立组合 |

internet 泛指网络互连；Internet 指采用 TCP/IP 协议族的全球互联网。ISP 提供互联网接入等服务。网络组成包括硬件、软件和协议。服务复制可提高可用性，还须配合故障检测和一致性机制。

总线、星形、环形、网状描述连接关系。集线器组成物理星形、逻辑共享介质；交换机星形各端口可并行。路径由多段链路组成：H1—R1—R2—H2 有 3 段链路、2 个中间节点。

## 1.2 速率、吞吐量与时延

速率单位 bit/s，数据单位换算为 1B $=$ 8bit；k, M, G 分别为 $10^3, 10^6, 10^9$，二进制量使用 KiB、MiB 或题设约定。

| 指标 | 公式或定义 | 使用条件 |
|---|---|---|
| 发送时延 | $t_{tx} = L/R$ | 从首比特进入链路至末比特进入；$L$ 用 bit |
| 传播时延 | $t_{prop} = d/v$ | 同一比特走完链路；距离与速度单位统一 |
| 单跳完整接收时间 | $L/R + d/v$ | 无排队、处理开销 |
| 总时延 | 各跳发送、传播、处理、排队按事件累加 | 流水重叠部分只计一次 |
| 时延带宽积 | $Rt_{prop}$ | 单向传播中可容纳的比特数 |
| 往返在途量 | $R \cdot RTT$ | 为覆盖反馈等待估计所需窗口 |

带宽在性能题中常指额定速率；吞吐量是观察期间实际通过的数据率；有效吞吐量只计有效载荷。路径单流长期吞吐量不超过最慢链路速率，竞争和协议开销会继续降低它。

RTT 须读清计时边界。题给往返传播时延时，数据与确认的发送时间另计；给出实际请求响应时间时，按题意避免重复累加。信道利用率按题设确定分子，可指忙时比例，也可特指发送有效数据所占比例。

理想排队模型 $D = D_0/(1-U)$ 仅用于给定模型：$D_0$ 为空闲时延，$U$ 为归一化利用率。独立比特误码概率为 $p$、帧长为 $L$ bit 时，帧错概率 $1-(1-p)^L$；$pL \ll 1$ 时近似 $pL$，突发错误不能直接沿用此模型。

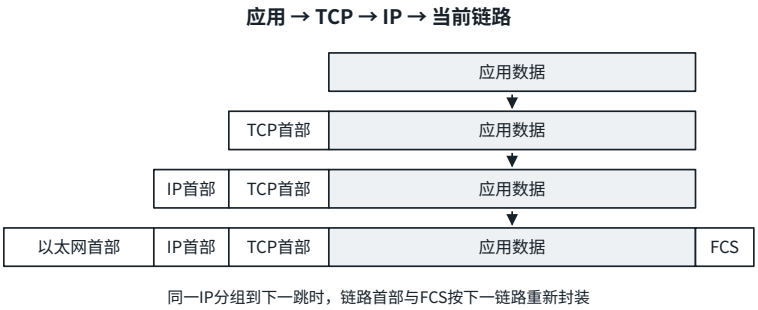



Figure 1.1: 逐层封装与链路层当前一跳

## 1.3 协议、服务、接口与封装

| 概念 | 约束关系与内容 |
|---|---|
| 协议 | 对等实体的水平规则；语法规定格式，语义规定含义和动作，时序规定事件次序 |
| 服务 | 本层与下层协作后向上层提供的能力 |
| 接口、SAP | 同机相邻层使用服务的方式、服务访问位置 |
| 体系结构 | 规定分层、功能及相互关系；内部算法和数据结构由实现选择 |

服务原语描述同机层间事件：请求、指示，以及需应答时的响应、证实。一次原语可引起多次协议报文、超时或重试。

$n$ 层 PDU 由控制信息 PCI 和服务数据 SDU 封装：$n$-PDU $=$ $n$-PCI $+$ $n$-SDU。上层 PDU 成为下层 SDU；PCI 可位于首部或尾部。接收端逐层解封装，对等通信依靠本机下层实际传送。

| 标识 | 所属层及长度 | 普通跨路由转发 |
|---|---|---|
| MAC | 链路层，以太网 48 bit | 每跳重新封装，目的为当前下一跳接口 |
| IP | 网络层，IPv4 为 32 bit | 无 NAT 时源、目的通常保持 |
| 端口 | 传输层，16 bit | 无端口转换时保持，用于进程端点分用 |

面向连接服务通常先建立、再传送、后释放；无连接服务独立提交数据单元。可靠交付、是否确认、是否预留资源分别由协议规定。以太网 CRC 只检错；802.11 单播 ACK 只确认当前无线跳；端到端可靠性由端系统协议保障。

## 1.4 OSI 与 TCP/IP 功能对应

| OSI 层次 | 对象与主要功能 | TCP/IP 对应 |
|---|---|---|
| 应用 | Web、邮件、名称服务等应用功能 | 应用层 |
| 表示 | 表示转换、压缩、加密 | 应用层 |
| 会话 | 对话管理、同步与恢复 | 应用层 |
| 传输 | 进程端到端复用分用；按协议提供可靠性、流控 | TCP、UDP |

| OSI 层次 | 对象与主要功能 | TCP/IP 对应 |
|---|---|---|
| 网络 | 跨网络分组寻址、路由和转发 | 网际层 IP，尽最大努力数据报 |
| 数据链路 | 相邻节点成帧、检错、介质访问；可含流控和重传 | 网络接口层 |
| 物理 | 原始比特、电平、编码和接口 | 网络接口层 |

五层教学模型把网络接口层展开为数据链路层和物理层。流控所属层由控制范围决定：相邻节点用链路层，应用进程端到端用传输层。具体协议是否实现某项功能，须查该协议提供的服务。

**例 1-1**

在 OSI 参考模型中，自下而上第一个提供端到端服务的层次是（ ）。

A. 数据链路层　B. 传输层　C. 会话层　D. 应用层

**例 1-2**

在 ISO/OSI 参考模型中，实现两个相邻结点间流量控制功能的是（ ）。

A. 物理层　B. 数据链路层　C. 网络层　D. 传输层

**等长分组经过异速链路**

固定路径、等长分组、链路独立且无竞争/处理开销时，令 $s_i = L/R_i$，则 $T = \sum_i (s_i + t_{prop,i}) + (n-1)\max_i s_i$。首组走完全程，后续受最慢链路间隔约束。

## 1.5 交换方式与存储转发计算

| 方式 | 转发条件 | 主要时间和资源代价 |
|---|---|---|
| 电路交换 | 先建立电路，再连续传送比特流 | 建立时间、源端发送、全程传播；空闲时可仍占资源 |
| 报文交换 | 中间节点收全整份报文后转发 | 长报文逐跳发送与缓存 |
| 分组交换 | 存储转发时收全一组再转发 | 分组首部、处理、排队；可跨链路流水 |

固定路径含 $h$ 段等速链路，$n$ 个等长分组，每组总长 $L$ bit、速率 $R$，无竞争和处理开销时：

$$T = (n + h - 1)\frac{L}{R} + \sum_{i=1}^{h} t_{prop,i}.$$

第一组完成需 $hL/R + \sum t_{prop,i}$，后续每组间隔 $L/R$。报文长 $F$ bit，整报文存储转发发送项为 $hF/R$；已建立理想电路发送项为 $F/R$，再加传播及题给开销。

异速、变长、含处理时延时用事件递推。令 $F_{i,j}$ 为第 $j$ 组在链路 $i$ 发送完的时刻，$p_{i-1,j}$ 为上一节点收到后处理时间：

$$F_{i,j} = \max\{F_{i,j-1},\ F_{i-1,j} + t_{prop,i-1} + p_{i-1,j}\} + L_j/R_i.$$

源端准备时刻作边界条件；目的收齐时间为 $F_{h,n} + t_{prop,h}$。若处理器串行且会排队，另记录处理器空闲时刻。

**例 1-3**

在下图所示的分组交换网络中，主机 H1 和 H2 通过路由器互连，2 段链路的带宽均为 100 Mb/s，时延带宽积（即单向传播时延 × 带宽）均为 1000 bit。若 H1 向 H2 发送 1 个大小为 1 MB 的文件，分组长度为 1000

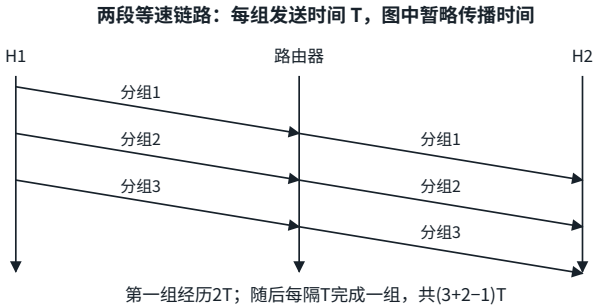



Figure 1.2: 等速链路上的分组流水

Figure 1.3: 原题的两段链路，均为 100 Mb/s 且时延带宽积 1000 bit

B，则从 H1 开始发送时刻起到 H2 收到文件全部数据时刻止，所需的时间至少是（注：$M = 10^6$）（　）。

A. 80.02 ms　　B. 80.08 ms　　C. 80.09 ms　　D. 80.10 ms

## 1.6 数据报与虚电路

| 比较项 | 数据报 | 虚电路 |
| --- | --- | --- |
| 准备与路由 | 无预建逐跳连接；各分组独立选路 | 建立时选路并保存状态 |
| 转发依据 | 分组目的地址 | 输入链路和局部 VCID 映射 |
| 顺序 | 路径可不同，可乱序 | 通常沿既定路径，保持顺序 |
| 链路资源 | 分组动态共享 | 仍可动态共享，保障由具体服务规定 |

VCID 可逐跳变化；永久虚电路可长期保留。虚电路的逻辑连接与电路交换的专用资源分配分别判断。

**例 1-4**

下列关于虚电路网络的叙述中，错误的是（　）。

A. 可以确保数据分组传输顺序

B. 需要为每条虚电路预分配带宽

C. 建立虚电路时需要进行路由选择

D. 依据虚电路号（VCID）进行数据分组转发

# 2 物理层

## 2.1 数据、信号与码元速率

数据承载信息，信号是数据在介质中的表示。数字或模拟数据均可通过相应变换形成数字或模拟信号。信源产生数据，信宿接收数据；编码器/解码器处理编码，调制器/解调器处理载波映射。

码元是在固定时间间隔内选择的一种波形或状态。有 $M$ 种可区分状态，理想等长二进制编码每码元携带 $\log_2 M$ bit：

$$R_b = R_s \log_2 M.$$

$R_s$ 用 Baud（码元/s），$R_b$ 用 bit/s；每秒已给的码元数直接代入。频率带宽 $B$ 为频率上下限之差，用 Hz；链路数据率意义的带宽用 bit/s。

单工只单向；半双工双向交替；全双工双向同时。一条物理线路可按频率、时间或编码承载多个逻辑信道。基带传输直接传基带信号，频带传输通过调制搬移到载波频带。

**功率比和分贝增量**

信号功率保持不变，噪声功率减半，信噪比提高 $10\log_{10}2 \approx 3.01$ dB；功率比乘 10 才增加 10 dB。

## 2.2 奈奎斯特极限与香农容量

| 已知条件 | 关系 | 前提 |
|---|---|---|
| 无噪声低通信道、带宽 $B$ | $R_s \le 2B$；$R_b \le 2B\log_2 M$ | 无码间串扰，$M$ 为可区分状态数 |
| 带限高斯噪声信道 | $C = B\log_2(1+S/N)$ | $S/N$ 为无量纲功率比；可靠传输理论极限 |
| 信噪比 $\gamma$ dB | $S/N = 10^{\gamma/10}$ | 30 dB 对应 1000 |

同时给出固定调制状态数和信噪比时，按教材信道模型分别计算两项上界，取有效限制的较小值。仅提高信噪比，固定状态数仍可能限制速率。采样次数超过 $2B$ 不能增加带限信道的独立码元数。

**例 2-1**

在一条带宽为 200 kHz 的无噪声信道上，若采用 4 个幅值的 ASK 调制，则该信道的最大数据传输速率是（　）。

A. 200 kb/s　　B. 400 kb/s　　C. 800 kb/s　　D. 1600 kb/s

**例 2-2**

某信道带宽为 10 MHz，信噪比为 30 dB，采用 QAM-32 调制。若带宽提高到 20 MHz，信噪比提高到 40 dB，则信道的极限数据传输速率大约提高到原来的（　）倍。

A. 2　　B. 2.2　　C. 2.4　　D. 2.6

## 2.3 线路编码与调制

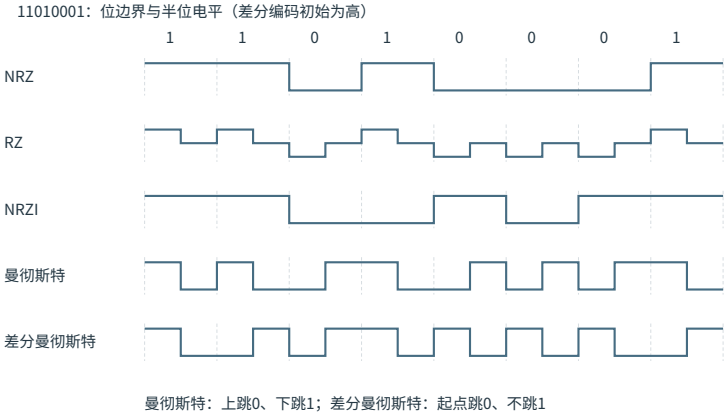



Figure 2.1: 比特串 11010001 的五种编码

| 编码 | 逐位规则 | 同步与代价 |
|---|---|---|
| NRZ 不归零 | 每位保持相应电平 | 长串相同位可缺少跳变 |
| RZ 归零 | 前半位按数据取电平，后半位回零 | 每位含回零，占更多变化能力 |
| NRZI | 位起点是否跳变表意，位中保持 | 0/1 约定按题意；某种长连码可缺少跳变 |
| 曼彻斯特 | 每位中点必跳，方向表意 | 中点同步，每位占两个半位区间 |
| 差分曼彻斯特 | 中点必跳，起点是否跳变表意 | 首位须给前一电平；依赖相对关系 |

本图约定 NRZ 低 0 高 1、NRZI 起点跳变为 0、曼彻斯特中点上跳为 0、差分曼彻斯特起点跳变为 0，差分初始电平为高。题目约定优先。

每秒最多承载 $2R$ 个半位信号区间时，曼彻斯特比特率至多 $R$；若题给最终比特率，直接使用。4B/5B 效率为 $4/5$，传同量数据需线路比特数乘 $5/4$。

ASK 改变振幅，FSK 改变频率，PSK 改变相位，QAM 组合振幅与相位。4 种振幅 $\times 4$ 种相位在波形互不重合时为 16 态，每码元 4 bit；状态数按实际可区分的星座点计算。

## 2.4 PCM 采样、量化与编码

PCM 流程：采样取得离散时刻幅值，量化到有限级别，编码成比特。理想带限模拟信号最高频率为 $f_{max}$：

$$f_s \geq 2f_{max}, \qquad R_b = f_s q,$$

其中 $q$ 为每样本编码位数，量化级数最多 $2^q$，上式未计额外信道编码。8 kHz 采样、每样本 8 bit 得到 64 kb/s。采样定理约束信号恢复，奈氏码元极限约束传输，两者按各自场景使用。

## 2.5 传输介质与物理制式

| 介质 | 性质与适用条件 |
|---|---|
| 双绞线 | 绞合减小电磁干扰；屏蔽型再加屏蔽结构 |
| 同轴电缆 | 内导体、绝缘层、外导体屏蔽层；速率仍取决于标准、长度和设备 |
| 多模光纤 | 多种空间模式，模间色散展宽脉冲，限制距离和速率 |
| 单模光纤 | 单一空间模式，减轻模间色散，适合较长距离及高容量；可另做波分复用 |
| 无线电、微波、红外 | 传播条件依频率而定；地面微波常要求视距，高轨卫星传播时延大 |

光纤电绝缘、抗电磁干扰、衰减低；容量与传播速度分别计算。频率、波长满足 $v = f\lambda$，真空中 $v = c$。介质位于物理层之下，物理层规定其信号收发方式。

10BASE5：10 Mb/s、基带、传统粗同轴单段 500 m；10BASE2 最大段常取 185 m；10BASE-T 使用双绞线。100BASE-T 主要使用双绞线。曼彻斯特适用于传统 10 Mb/s 以太网，更高速制式按相应物理标准取编码。

## 2.6 物理层接口四类特性

| 特性 | 规定内容 |
|---|---|
| 机械 | 连接器形状、尺寸、接点排列 |
| 电气 | 电压、电平、阻抗、传输速率 |
| 功能 | 各信号线承担的作用 |
| 规程 | 信号出现与双方动作的顺序 |

**例 2-3**

下列选项中，不属于物理层接口规范定义范畴的是（　）。

A. 接口形状　B. 引脚功能　C. 物理地址　D. 信号电平

## 2.7 中继器与集线器

中继器判决、整形并再生比特信号；已判错的比特无法靠再生恢复。它不检查 MAC、不学习转发表。连接不同介质须满足速率、编码及协议等兼容条件。

集线器是多端口中继器，一个端口收到比特便向其他端口转送，所有站共享带宽和冲突域。$n$ 站共享 $R$ 的理想均分为 $R/n$，实际竞争和协议开销另计。

最远站传播时延先累计途经各段距离，再除传播速度；冲突反馈需要往返，设备时延也计入。传统 10BASE5 的 5-4-3 规则：最多 5 段、4 个中继设备，其中 3 段可挂接终端；其他标准按各自距离、延迟和连接规范。

# 3 数据链路层

## 3.1 链路功能与成帧

数据链路层在物理链路上组织帧，完成帧定界、透明传输、差错检测；可靠性、流量控制和介质访问由具体协议提供。流量控制保护接收能力，差错控制处理损坏、丢失、重复与次序；链路确认的范围为相邻节点。

| 成帧方法 | 边界依据 | 透明传输与限制 |
|---|---|---|
| 字符计数 | 首部长度字段 | 字段损坏可使后续边界错位 |
| 字节填充 | 定界字节 | 数据中的定界与转义字节均须转义 |
| 零比特填充 | 标志 `01111110` | 被扫描区每连续 5 个 1 后插 0，连续计数跨字节；标志自身不填充 |
| 违规编码 | 保留的数据编码禁用组合 | 取决于具体物理编码 |

去填充只删第五个连续 1 后的填入 0，原数据中的 0 须保留。例如 `111110` 变成 `1111100`。

**例 3-1**

HDLC 协议对 `01111100 01111110` 组帧后对应的比特串为（　）。

A. `01111100 00111110 10`

B. `01111100 01111101 01111110`

C. `01111100 01111101 0`

D. `01111100 01111110 01111101`

## 3.2 CRC、奇偶校验与码距

奇偶校验保证检测奇数位翻转，偶数位错可能漏检，一位校验不能定位单错。

生成多项式 $G(x)$ 次数为 $r$，二进制形式有 $r+1$ 位。CRC 发送端计算 $D(x)x^r$ 模 2 除以 $G(x)$ 的余数 $R(x)$，发送 $D$ $R$；余数固定写成 $r$ bit。模 2 减法逐位异或。接收端对完整接收串直接除以 $G$，余数非零则检出错误，余数 0 表示检验通过，存在漏检。

| CRC 条件 | 保证 |
|---|---|
| 生成式最高、最低系数为 1，次数 $r$ | 所有长度不超过 $r$ 的突发错误可检出 |
| 含因子 $x+1$ | 所有奇数位错误可检出 |
| 双错位间距为 $k$ | 当 $G(x)$ 不整除 $x^k+1$ 时可检出；须结合码长判断全部双错 |
| 特定码长单错余数均非 0 且互异 | 可建综合征表定位单错；实际链路通常用 CRC 检错重传 |

较长突发错误的 $2^{-r}$ 漏检概率须有相应随机模型，不能用于某个确定错误模式。

**例 3-2**

若甲向乙发送数据时采用 CRC 校验，生成多项式为 $G(X)=X^4+X+1$（即 $G=10011$），则乙接收到下列比特串时，可以断定其在传输过程中未发生错误的是（ ）。

A. `1 0111 0000`　B. `1 0111 0100`

C. `1 0111 1000`　D. `1 0111 1100`

最小海明距离 $d_{min}$ 为任意两合法等长码字不同位数的最小值：保证检出 $e$ 位错需 $d_{min}\geq e+1$；保证纠正 $t$ 位错需 $d_{min}\geq 2t+1$。

**例 3-3**

若某差错编码的编码集为 `{1001 1010, 0101 1100, 1111 0000, 0000 1111}`，则该差错编码的检错和纠错能力是（ ）。

A. 不超过 2 位错的 100% 检错、不超过 1 位错的纠错

B. 不超过 2 位错的 100% 检错、不超过 2 位错的纠错

C. 不超过 3 位错的 100% 检错、不超过 1 位错的纠错

D. 不超过 3 位错的 100% 检错、不超过 2 位错的纠错

## 3.3 海明码校验分组与综合征

$m$ 个信息位、$r$ 个单错校验位须满足 $2^r\geq m+r+1$。校验位置为 1、2、4、8……，数据填其余位置；位号二进制中相应位为 1 的所有位置归该校验组。7 位码分组为：1 组检查 1、3、5、7；2 组检查 2、3、6、7；4 组检查 4、5、6、7。

偶校验综合征 $s_4s_2s_1$ 在单错假设下给出错位编号。先固定位置编号、数据位映射和字符串方向。例如数据放 3、5、6、7 且依次为 1、0、1、0，校验位为 1、0、1，按位号递增写为 `1011010`。

普通海明码通常 $d_{min}=3$，单错纠正；增加总体偶校验可实现 SECDED，在最多两错的模型下：

| 原海明综合征 | 总体校验 | 判决 |
|---|---|---|
| 0 | 通过 | 无错 |
| 非 0 | 失败 | 原海明部分单错，按综合征纠正 |
| 0 | 失败 | 总体校验位单错 |
| 非 0 | 通过 | 双错，只检出 |

## 3.4 ARQ 协议、窗口与利用率

检错发现位错，ACK 反馈接收进度，计时器恢复无反馈状态，序号识别新数据与重复。ACK 丢失也会引起重传，接收方须防止重复交付；NAK 本身也可能丢失，仍需超时机制。

本章一般 `ACK i` 表示已收到 $i$ 号帧，题干指定“下一期待帧”时按题意。$k$ 位帧序号按模 $2^k$ 循环，窗口以帧数计。

| 项目 | 停止—等待 | GBN 后退 N 帧 | SR 选择重传 |
|---|---|---|---|
| 发送/接收窗口 | 1/1，0、1 交替序号 | $W_s\leq 2^k-1$，$W_r=1$ | $W_s+W_r\leq 2^k$；对称时各 $\leq 2^{k-1}$ |

| 项目 | 停止—等待 | GBN 后退 N 帧 | SR 选择重传 |
|---|---|---|---|
| 确认 | 确认当前帧 | 累计确认连续收到的帧 | 各帧分别确认 |
| 失序正确帧 | 下一新帧尚未发送 | 通常丢弃并重复旧 ACK | 窗口内缓存，补缺后连续交付 |
| 超时 | 重传当前帧 | 从最早未确认至最后已发帧全重传 | 只重传相应超时帧 |
| 状态成本 | 一个计时器 | 通常计时最早未确认帧 | 各帧计时，保存失序帧 |

SR 教材常用配置还满足 $W_r \le W_s$；不对称时先用两窗和条件。若窗口左沿缺口未确认或未交付，即使高号帧已收到也不能越过左沿。序号空间过小会使旧重传与下一轮新帧混淆。

无错、立即确认、全双工、确认链路不构成额外瓶颈，数据发送时间 $T_d = L_d/R$、ACK 发送时间 $T_a = L_a/R$、单向传播 $\tau$：

$$U_{stop} = \frac{T_d}{T_d + 2\tau + T_a}, \qquad U_W = \min\left(1, \frac{WT_d}{T_d + 2\tau + T_a}\right).$$

填满链路要求 $W \ge \lceil (T_d + 2\tau + T_a)/T_d \rceil$，再按协议窗口上限求序号位数。捎带 ACK 用承载确认的数据帧发送时间，题给 RTT 按其边界代替相应传播项。

**例 3-4**

假设主机甲采用停—等协议向主机乙发送数据帧，数据帧长与确认帧长均为 1000 B，数据传输速率是 10 kb/s，单向传播延时是 200 ms。则甲的最大信道利用率为（　）。

A. 80%　B. 66.7%　C. 44.4%　D. 40%

**例 3-5**

数据链路层采用后退帧（GBN）协议，发送方已经发送了编号为 0～7 的帧。当计时器超时时，若发送方只收到 0、2、3 号帧的确认，则发送方需要重发的帧数是（　）。

A. 2　B. 3　C. 4　D. 5

**例 3-6**

假设两台主机之间采用后退 N 帧协议传输数据，数据传输速率为 16 kb/s，单向传播时延为 250 ms，数据帧长度为 128 B，确认帧长度也是 128 B。为使信道利用率达到最高，帧序号的比特数至少为（　）。

A. 2　B. 3　C. 4　D. 5

**例 3-12**

甲乙双方均采用后退 N 帧协议（GBN）持续进行双向数据传输，始终采用捎带确认，帧长均为 1000 B。$S_{x,y}$、$R_{x,y}$ 分别表示甲方、乙方发送的数据帧，$x$ 为发送序号，$y$ 为确认序号，表示希望接收对方的下一帧序号。两个序号字段均为 3 bit，信道传输速率为 100 Mb/s，RTT=0.96 ms。下图为甲方发送和接收数据帧的两种场景；$t_0$ 时甲方的发送序号和确认序号均为 0，$t_1$ 时甲方有足够多的数据待发送。

（1）对于图 (a)，$t_0$ 到 $t_1$ 期间，甲方可以断定乙方已正确接收多少个数据帧？是哪几个帧？请用 $S_{x,y}$ 表示。

（2）对于图 (a)，从 $t_1$ 起，在不出现超时且未收到乙方新的数据帧之前，甲方最多还可以发送多少个数据帧？第一个和最后一个分别是什么？请用 $S_{x,y}$ 表示。

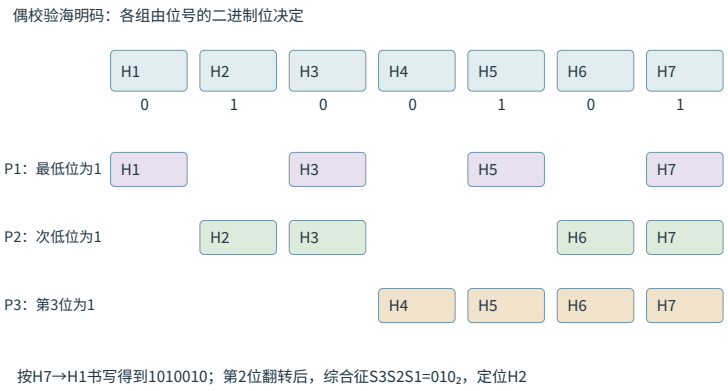



Figure 3.1: 海明码位号和校验分组

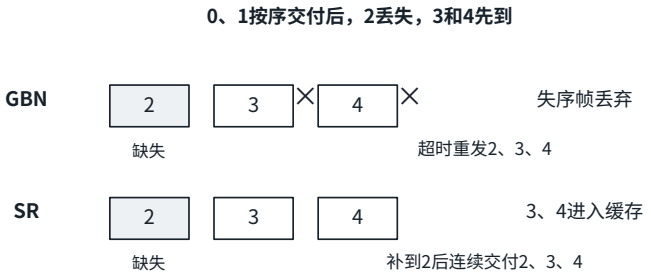



Figure 3.2: GBN 丢弃失序帧，SR 缓存后补缺

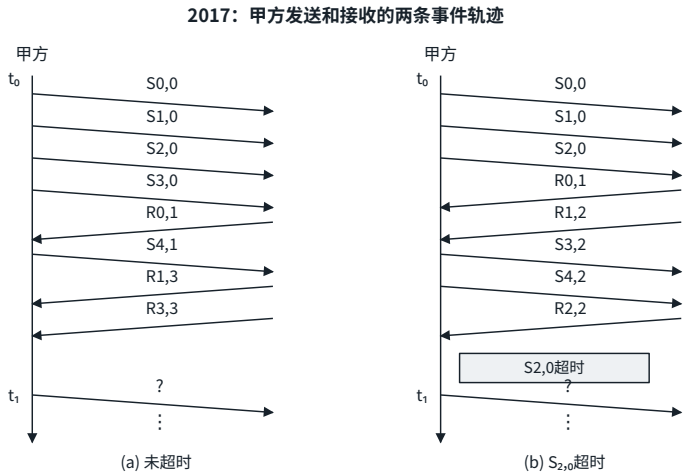



Figure 3.3: 箭头从甲方发出为 S、指向甲方为 R；两列分别对应未超时和 S2,0 超时的原题场景

（3）对于图 (b)，从 $t_1$ 起，在不出现新的超时且未收到乙方新的数据帧之前，需要重发多少个数据帧？重发的第一个帧是什么？请用 $S_{x,y}$ 表示。

（4）甲方可以达到的最大信道利用率是多少？

## 3.5 复用与受控介质访问

| 方法 | 分配维度或操作 |
|---|---|
| FDM | 频带并行，设保护频带；数字数据调制后也可使用 |
| 同步 TDM | 固定周期时隙，用户无数据时其时隙可闲置 |
| 统计 TDM | 按需求分配，需标识归属与排队规则 |
| WDM | 光波长复用 |
| CDMA | 码片序列分离用户 |
| 轮询 | 控制站依次给站点发送机会，承担控制开销和单点失效风险 |
| 令牌 | 持有唯一有效令牌者发送，限制持有时间或额度；需处理丢失、重复与失效 |

理想同步、等功率、正交码模型中，比特 1 发送 $c$，0 发送 $-c$，未发为 0；接收叠加信号 $s$ 对目标码长 $m$ 的码 $c$ 计算 $(s \cdot c)/m$，结果 +1、−1、0 分别判 1、0、未发。

**例 3-7**

站点 A、B、C 通过 CDMA 共享链路，A、B、C 的码片序列分别是 `(1,1,1,1)`、`(1,-1,1,-1)` 和 `(1,1,-1,-1)`。若 C 从链路上收到的序列是 `(2,0,2,0,0,-2,0,-2,0,2,0,2)`，则 C 收到 A 发送的数据是（　）。

A. 000　　B. 101　　C. 110　　D. 111

令牌环目的站复制数据并设置状态，帧回源站后由源站移除，再按协议释放令牌。无数据及时转交，有数据也须遵守额度；单有效令牌、正常运行模型可避免数据争用冲突。

## 3.6 ALOHA、CSMA 与 CSMA/CD

等长帧发送时间 $T$、经典泊松负载 $G$ 模型：纯 ALOHA 易冲突区间 $2T$，吞吐 $Ge^{-2G}$，最大 $1/(2e)$；时隙 ALOHA 只在时隙边界开始，区间 $T$，吞吐 $Ge^{-G}$，最大 $1/e$。

| CSMA 策略 | 遇忙 | 遇空闲 |
|---|---|---|
| 1 坚持 | 持续监听 | 立即发送 |
| 非坚持 | 随机等待再监听 | 发送 |
| $p$ 坚持（时隙） | 继续按协议监听 | 概率 $p$ 发送，$1-p$ 延后一时隙 |

传播期间远站尚未听到载波，仍可能同时发送。共享半双工以太网用 CSMA/CD：边发边检，冲突后停止正常发送、发强化信号、随机退避。最坏单向传播 $\tau$ 时：

$$L_{min}/R \geq 2\tau, \qquad L_{min} \geq 2R\tau.$$

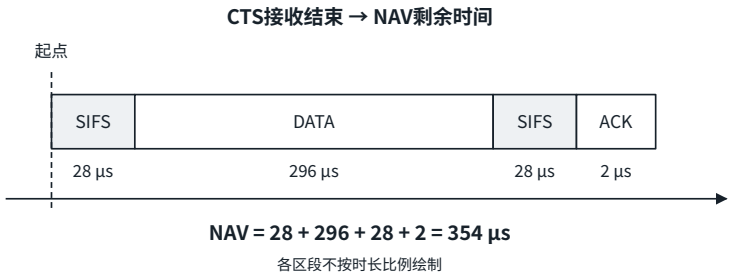



Figure 3.4: CTS 结束后的剩余预约

计入中继器等最坏往返延迟；提高速率而固定帧长会缩小允许跨度。全双工点到点以太网不使用 CSMA/CD。

第 $n$ 次冲突，$k=\min(n,10)$，随机整数 $r\in[0,2^k-1]$，退避 $r$ 个争用时间片；传统 10 Mb/s 时间片 51.2 µs。达到重试上限则报告失败。

**例 3-8**

在一个采用 CSMA/CD 协议的网络中，传输介质是一根完整的电缆，传输速率为 1 Gb/s，电缆中的信号传播速度为 200000 km/s。若最小数据帧长度减少 800 bit，则最远的两个站点之间的距离至少需要（ ）。

A. 增加 160 m　　B. 增加 80 m　　C. 减少 160 m　　D. 减少 80 m

## 3.7 802.11 访问、NAV 与无线地址

CSMA/CA 结合物理/虚拟载波监听、帧间间隔和随机退避；倒计时遇忙冻结，空闲并满足间隔后继续。SIFS 短于 DIFS，使当前交换的 CTS、ACK 等响应优先于新数据竞争。普通单播正确接收后发 ACK，缺 ACK 按协议重试；广播、组播不套用该逐帧确认流程。

可选 RTS/CTS 流程：RTS—SIFS—CTS—SIFS—DATA—SIFS—ACK。隐藏站听到 CTS 后按持续期设置 NAV，表示本帧结束后的剩余预约时间；RTS/CTS 有开销，也不能消除所有冲突。基本 RTS 为 20 B，CTS 和 ACK 各 14 B；三者属于控制帧，探测与关联属于管理帧。

**例 3-9**

在采用 CSMA/CA 协议的 802.11 无线局域网中，DIFS=128 µs，SIFS=28 µs，RTS 帧、CTS 帧和 ACK 帧的传输时延分别是 3 µs、2 µs 和 2 µs，忽略信号传播时延。若主机 A 要向 AP 发送一个总长度为 1998 B 的数据帧，无线链路带宽为 54 Mb/s，则隐藏站 B 收到 AP 发送的 CTS 帧时，设置的 NAV 值是（ ）。

A. 326 µs　　B. 354 µs　　C. 385 µs　　D. 513 µs

BSS 为 AP 与关联站点，BSA 为覆盖范围，SSID 为服务集名称，BSSID 通常为 AP 无线接口 MAC；多个 BSS 经分布式系统 DS 组成 ESS，门户连接其他局域网。AP 负责接入和桥接，跨子网路由须有三层功能。

| ToDS/FromDS | 地址 1 | 地址 2 | 地址 3 |
|---|---|---|---|
| 1/0，站点 →AP | AP 接收接口 | 当前站点发送接口 | 最终目的 |
| 0/1，AP→ 站点 | 当前接收站点 | AP 发送接口 | 原始源 |
| 0/0，自组织 | 目的 | 源 | BSSID |

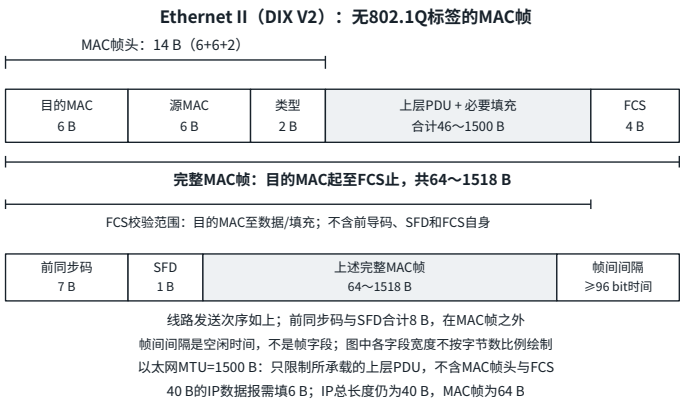



Figure 3.5: 无标签 Ethernet II 字段和计量边界

1/1 的无线分布式转发可用第四地址。无线链路 ACK 只反馈当前无线跳，TCP ACK 反馈端到端字节流。

## 3.8 Ethernet II、VLAN 与交换机

IEEE 802 把链路层分为 LLC 和 MAC；局域网技术通常覆盖物理层与链路层，网卡同时处理信号和帧。

无标签 Ethernet II：目的 6 B、源 6 B、类型 2 B、数据及填充 46～1500 B、FCS4 B；帧头 14 B，完整 MAC 帧 64～1518 B，MTU1500 B 约束完整上层 PDU。类型 `0800` 为 IPv4、`0806` 为 ARP；802.3 在对应位置采用长度字段。

7 B 前同步码和 1 B SFD 在 MAC 帧外；帧间间隔 96 bit 时间也在帧外。FCS 检验从目的 MAC 至数据/填充，不含前导部分和间隔；错误帧丢弃，基本以太网无 MAC 逐帧确认重传。MAC 通常 48 bit，目的全 1 为广播，首字节最低位区分单播/组播。

40 B IP 分组装入无标签帧须填 6 B，IP 总长度保持 40 B。最短 MAC 帧若计前导和间隔，一个周期为 $(8+64)\times 8+96=672$ bit 时间，吞吐量分母按题设边界选取。

VLAN 划分逻辑广播域，不同 VLAN 通信需要三层转发。接入口可收发无标签帧，干道携多个 VLAN；802.1Q 在源 MAC 之后加 4 B 标签，VID12 bit，普通 ID 为 1～4094；单标签常见最大帧 1522 B，FCS 随内容更新。教材最短 64 B 模型下，含标签首尾共 22 B、数据下限 42 B。

**例 3-10**

一个长度为 40 B 的 IP 数据报需要封装成 802.1Q 帧进行传输，则此 802.1Q 帧的数据载荷部分需要填充的字节数是（ ）。

A. 2　B. 4　C. 6　D. 8

交换机按源 MAC 和入端口学习，按目的 MAC 转发：已知且异端口定向转发；同入端口过滤；未知、广播在当前 VLAN 除入端口外泛洪。表项会老化，环形拓扑需生成树等防循环机制。

交换机各端口独立冲突域，广播域由 VLAN 或三层边界划分。不同输出端口可并行，同一低速出口仍共享瓶颈。存储转发收全并可检验 FCS；直通至少收到目的 MAC 后决策，帧尾尚未到达，不能预先排除尾部差错。吞吐量需区分单向发送与收发合计。

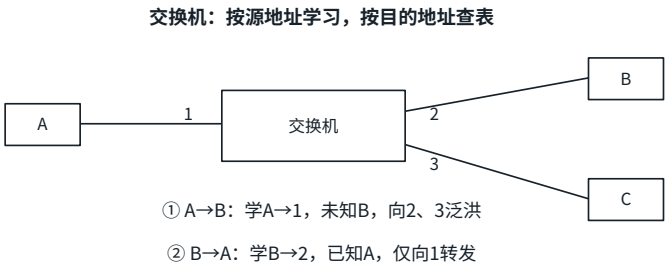



Figure 3.6: 空表下 A 发 B 和 B 回复 A 的学习过程

**例 3-11**

对于 100 Mb/s 的以太网交换机，当输出端口无排队，以直通交换方式转发一个以太网帧（不包括前导码）时，引入的转发时延至少是（ ）。

A. 0 μs　B. 0.48 μs　C. 5.12 μs　D. 121.44 μs

## 3.9 PPP 封装与链路建立

PPP 用于点到点链路，无多站竞争和争用时间片所需的 64 B 最短帧限制。基本帧：`7E | FF | 03 | 协议 | 信息 | FCS | 7E`；协议字段分用 IPv4、LCP、NCP 等，固定地址不作以太网式多站寻址。

建立过程：物理连接 →LCP 配置和测试 → 可选鉴别 → 相应 NCP 配置 → 网络层传送；关闭时撤销网络层配置、LCP 和物理连接。LCP 负责链路，NCP 是一族网络层配置协议。

异步 PPP：`7E→7D 5E`，`7D→7D 5D`；同步教材模型使用零比特填充。PPP 基本协议用 FCS 检错，不靠数据帧编号与逐帧 ACK 实现可靠交付。PPPoE 从外到内为以太网、PPPoE/PPP、被封装的网络层分组。

# 网络层 4

## 4.1 路由、转发、SDN 与拥塞

IP 在不同链路技术之上提供统一的尽最大努力数据报服务。路由选择依据可达性、度量和策略形成路径信息；转发对当前分组查表选输出接口和下一跳。普通路由器处理到网络层，下一跳用新的链路格式封装；协议转换须有明确的转换功能。

| 路由方式 | 性质 |
|---|---|
| 静态 | 人工配置，开销低，故障适应依赖维护 |
| 动态 | 交换信息自动更新，有报文、计算和收敛成本 |
| 层次 | 区域或 AS 内部掌握细节，对外汇总；减少信息量，可能牺牲全局最短路径 |

SDN 逻辑分离控制平面与数据平面：控制器计算并下发规则，交换设备匹配流表执行转发、丢弃、改写或上送。逻辑集中可由多个物理控制器实现；已有匹配规则无需逐包询问控制器。北向接口连接应用与控制器，南向连接控制器与设备，OpenFlow 是南向协议之一。流表可按入端口、MAC、IP、协议、端口匹配，含优先级、计数器和动作。

拥塞源于多个输入竞争共享资源，持续到达速率超过服务速率会积压、丢包；加缓存只能吸收部分突发，可能增加时延。流量控制保护接收端，拥塞控制保护网络承载。可采用准入、整形、调度、主动丢弃/标记和反馈减速。全双工每方向分别合计汇聚流量，与该方向容量比较。

## 4.2 距离向量与链路状态算法

距离向量由邻居通告到各目的的距离：

$$D_x(d) = \min_{v \in N(x)} \{c(x,v) + D_v(d)\}.$$

先加本机到邻居代价，再比较候选；取最小者作为下一跳，同值按题意处理。坏消息可形成环路并计数到无穷，水平分割、毒性逆转和触发更新只能按各自条件缓解。

**例 4-1**

某网络中的所有路由器均采用距离向量路由算法。路由器 E 与邻居 A、B、C、D 的直接链路距离分别是 8、10、12、6，收到的距离向量如下。E 更新后到 Net1～Net4 的距离分别是（　）。

| 目的网络 | A、B 的通告 | C、D 的通告 |
|---|---|---|
| Net1 | 1，23 | 20，22 |
| Net2 | 12，35 | 30，28 |
| Net3 | 24，18 | 16，36 |
| Net4 | 36，30 | 8，24 |

A. 9，10，12，6　　B. 9，10，28，20

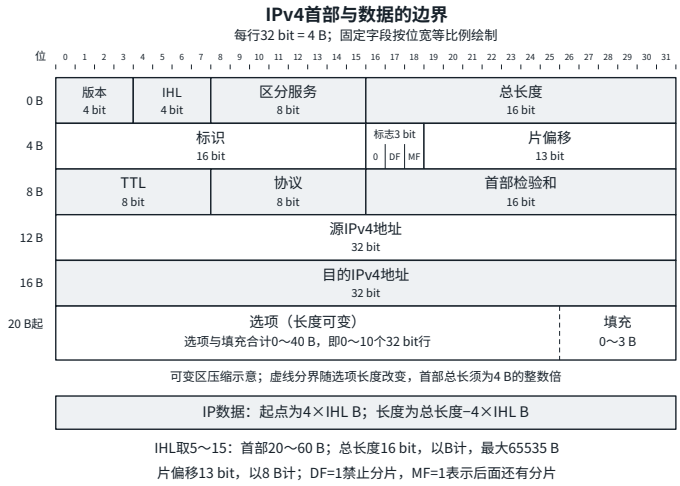



Figure 4.1: IPv4 固定首部 20 B，可选部分另计

C. 9，20，12，20　　D. 9，20，28，20

链路状态流程：发现邻居和测量代价 → 在规定范围洪泛本地链路状态 → 建立拓扑数据库 → 各自以本机为根运行最短路径算法。通告用序号和老化区分新旧。非负权 Dijkstra 每轮选暂定距离最小的未确定节点，确定后松弛相邻边，保存前驱并推出下一跳。

## 4.3 IPv4 首部与分片

| 字段 | 位数 | 单位、含义与条件 |
|---|---|---|
| 版本、IHL | 各 4 | 版本 4；首长 $H = 4IHL$ B，IHL=5～15，首部 20～60 B |
| 区分服务 | 8 | 服务分类与拥塞标记 |
| 总长度 | 16 | 首部与数据共计，B，最大 65535 |
| 标识 | 16 | 同一原数据报分片保持；重组还结合源、目的、协议 |
| 标志 | 3 | 保留 0；DF=1 禁分片；MF=1 原数据后面还有片 |
| 片偏移 | 13 | 8 B 单位，相对原始 IP 数据区起点 |
| TTL | 8 | 转发每跳减 1，耗尽丢弃，条件允许时报告超时 |
| 协议 | 8 | ICMP=1、TCP=6、UDP=17，用于网络层分用 |
| 首部检验和 | 16 | 只覆盖 IPv4 首部，TTL 或地址变化须更新 |
| 源、目的地址 | 各 32 | 标识源、目的 IP 接口 |

MTU 约束完整 IP 分组。超 MTU 且 DF=0 可分片，每片有独立 IP 首部，目的主机重组；DF=1 则丢弃并在允许时通知源端。非末片数据长度须为 8 B 整数倍，偏移不计任何 IP 首部。未分片为 MF=0 且偏移 0，末片 MF=0 但偏移可非 0。

首部固定 $H$、出口 MTU 为 $M$ 时，非末片最大数据量：

$$D_{max} = 8 \left\lfloor \frac{M - H}{8} \right\rfloor .$$

先算原数据长 “总长减原首长”，列各片原数据区间，再填总长、偏移和 MF。再次分片仍用原数据坐标，原片 MF=1 时，其最后子片也为 MF=1。分片只切 IP 数据，不给每片再添 TCP/UDP 首部。

**例 4-2**

若路由器向 MTU=800 B 的链路转发一个总长度为 1580 B 的 IP 数据报（首部长度为 20 B）时进行了分片，且每个分片尽可能大，则第 2 个分片的总长度字段和 MF 标志位的值分别是（ ）。

A. 796，0　B. 796，1　C. 800，0　D. 800，1

## 4.4 CIDR、特殊地址与子网划分

IPv4 按 32 位处理。地址 $A$、掩码 $M$、前缀 $p$：网络地址 $N = A \,\&\, M$；地址块大小 $2^{32-p}$；广播地址 $N + 2^{32-p} - 1$。普通多址子网主机位 $h$，可分配 $2^h - 2$，网关接口也占地址。点对点 `/31`、主机路由`/32` 按特殊用途处理。

| 地址或类别 | 用途 |
|---|---|
| `0.0.0.0` | 未获地址时可作本机源地址 |
| `255.255.255.255` | 本地受限广播，路由器不转发 |
| `127.0.0.0/8` | 回环，不作外部链路普通地址 |
| 主机位全 1 | 子网定向广播，跨路由转发受配置限制 |
| `10/8`、`172.16/12`、`192.168/16` | 私有地址，可在不同私网重复使用 |
| 历史 A/B/C 类 | 首字节普通范围 1～126、128～191、192～223，默认前缀/8、/16、/24 |
| `224/4` | D 类组播；E 类为保留类别 |

CIDR 按显式前缀判断，独立于历史分类。两主机是否同网须分别与发送方掩码按位与。异常不同掩码时，两个方向的本地判断可能不一致。

**例 4-3**

主机 `168.16.84.24/20` 所在子网的最小可分配 IP 地址和最大可分配 IP 地址分别是（ ）。

A. 168.16.80.1，168.16.84.254

B. 168.16.80.1，168.16.95.254

C. 168.16.84.1，168.16.84.254

D. 168.16.84.1，168.16.95.254

定长划分借 $s$ 位得 $2^s$ 个等长子网，采用允许全 0、全 1 子网号的口径。VLSM 按主机需求从大到小，先计网关等接口，选满足容量的最小地址块，再按块边界对齐分配，保证不重叠。

若一个原块被**完整**二分为 $n$ 个子网，总拆分 $n-1$ 次，最深分支最多延长 $n-1$ 位；此结论依赖完整覆盖条件。子网增加不会增加总地址数，普通子网的网络/广播地址开销会增加。

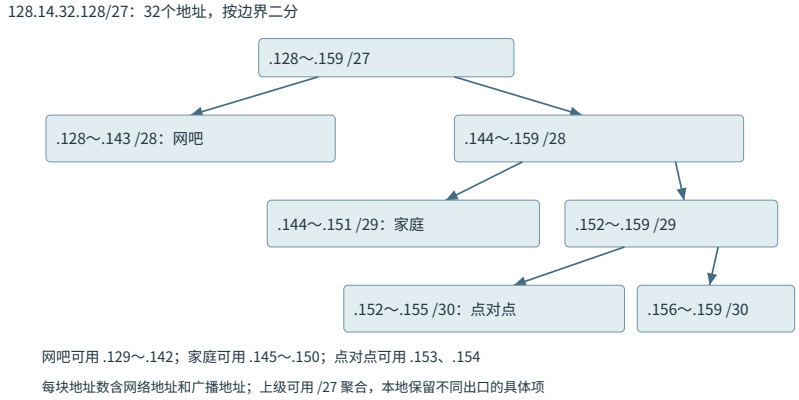



Figure 4.2: 地址块逐级二分与 VLSM 对齐

**例 4-4**

若将 `101.200.16.0/20` 划分为 5 个子网，则可能的最小子网的可分配 IP 地址数是（　）。

A. 126　　B. 254　　C. 510　　D. 1022

## 4.5 路由聚合与最长前缀匹配

精确聚合须连续、对齐、同出口，汇总块完全覆盖且不额外包含其他地址；等长 $2^s$ 块聚合时前缀缩短 $s$ 位。允许覆盖空隙时，须保留需要的更具体路由，并检查未分配范围的丢弃或回程环路。

**例 4-5**

某路由表中有转发接口相同的 4 条路由表项，其目的网络地址分别为 `35.230.32.0/21`、`35.230.40.0/21`、`35.230.48.0/21` 和 `35.230.56.0/21`，将该 4 条路由聚合后的目的网络地址为（　）。

A. 35.230.0.0/19　　B. 35.230.0.0/20

C. 35.230.32.0/19　　D. 35.230.32.0/20

查表先选匹配位数最多的前缀，再读取输出接口及下一跳；不同前缀长度间不直接比较度量。`/32` 仅匹配一主机，`/0` 为无更具体匹配时的默认路由，仍遵守广播、TTL 等限制。本地出口与下一跳 IP 是两个字段。

**例 4-11**

Internet 的两个自治系统构成的网络如下图所示。AS1 由 R1 连接两个子网构成；AS2 由 R2、R3 互联并连接三个子网构成。各子网地址、R2 的接口名以及 R1、R3 的部分接口 IP 地址均在图中标出。

（1）路由表结构为“目的网络、下一跳、接口”。利用路由聚合技术给出 R2 的路由表，要求包括到达图中所有子网的路由，且路由项尽可能少。

（2）若 R2 收到目的 IP 地址为 194.17.20.200 的 IP 分组，会通过哪个接口转发?

（3）R1 与 R2 之间利用哪个路由协议交换路由信息？该协议报文封装到哪个协议的数据单元中传输?

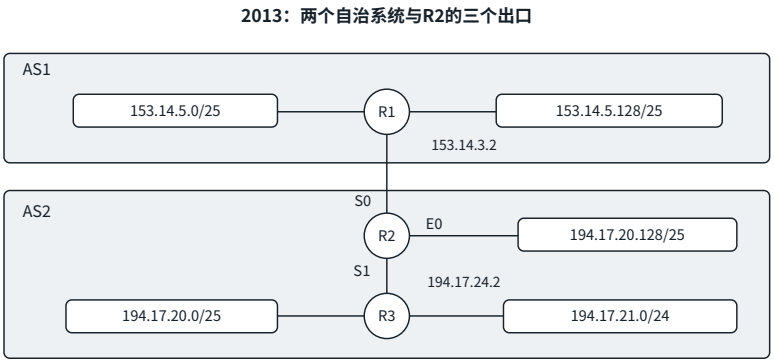



Figure 4.3: R2 经 S0 到 R1、经 S1 到 R3，E0 直连 194.17.20.128/25

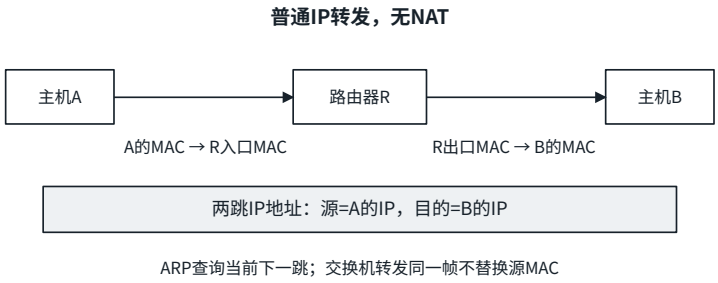



Figure 4.4: 无 NAT 时 IP 两端保持，MAC 逐跳重封装

## 4.6 ARP、逐跳封装与 NAT

发送步骤：按掩码和路由判定下一跳 → 查下一跳 IP 的 ARP 缓存 → 缺项时在本广播域广播 ARP 请求 → 目标通常单播回复 → 以本机出口 MAC 为源、下一跳 MAC 为目的封装。

目的在本地，ARP 查询目的 IP；目的在外地，通常查询默认网关或路由指定邻居 IP。普通路由器不转发 ARP 广播。二层交换机转发同一帧时保留源 MAC，路由器下一跳重新封装。ARP 缓存会老化，代理 ARP 须有明确条件。

**例 4-6**

某以太网中，甲 IP 为 211.71.136.23，子网掩码 255.255.240.0，网关 211.71.136.1。若甲向乙 211.71.130.25 发送 IP 分组，则（　）。

A. 直接发给乙，帧目的 MAC 为网关 MAC

B. 直接发给乙，帧目的 MAC 为乙 MAC

C. 交网关转发，帧目的 MAC 为网关 MAC

D. 交网关转发，帧目的 MAC 为乙 MAC

NAT 在边界维护内部/外部地址映射；NAPT 再用端口复用公网 IP。普通源 NAT 出向改源 IP，NAPT 还可改源端口，回包按映射恢复；远端目的通常保持。IP 首部校验随地址改写更新，TCP/UDP 伪首部也含 IP 地址，启用的传输层校验须相应更新。IPv4 UDP 全 0 未启用的情况单独处理。外部主动访问通常需要静态映射、端口转发等机制。

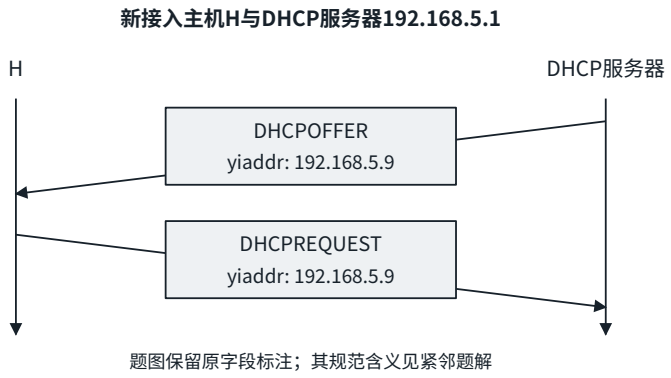



Figure 4.5: 原题的两个报文及字段标记；时间向下

## 4.7 DHCP 与 ICMP

DHCP 提供 IP、掩码、网关、DNS 及租期，UDP 服务器 67、客户端 68。初次获取：Discover 寻找 →Offer 提议 →Request 选择 →ACK 确认使用；初始 Request 源 `0.0.0.0`、目的受限广播，申请地址通过 Requested IP Address 选项携带。续租状态可单播；跨网段可用 DHCP 中继。

**例 4-7**

一台新接入网络的主机 H 通过 DHCP 服务器动态请求 IP 地址过程中,与 DHCP 服务器交换 DHCP 报文的部分过程如下图。封装 DHCPREQUEST 报文的 IP 数据报的目的 IP 地址和源 IP 地址分别是（　）。

A. 192.168.5.1，0.0.0.0

B. 192.168.5.1，192.168.5.9

C. 255.255.255.255，0.0.0.0

D. 255.255.255.255，192.168.5.9

ICMP 由 IP 封装，报告目的不可达、时间超过、参数问题等，回送请求/应答用于 Ping；错误报告通常含原 IP 首部及一部分数据，返回原源端，不负责重传。ICMP 差错自身、非首片、广播/组播目的和某些特殊源地址通常不再触发 ICMP 差错。IP 首部检验错误通常直接丢弃。

Traceroute 逐增 TTL，通过沿途超时回复定位，探测可用 UDP、ICMP 或 TCP 变体。历史 “源点抑制” 类型已由 RFC 6633 废弃；旧题按年代识别，现行拥塞处理使用适用反馈机制。

**例 4-8**

若路由器 R 因为拥塞丢弃 IP 分组，则此时 R 可向发出该 IP 分组的源主机发送的 ICMP 报文类型是（　）。

A. 路由重定向　B. 目的不可达　C. 源点抑制　D. 超时

## 4.8 IPv6 首部、地址与过渡

IPv6 地址 128 bit，8 组 16 位十六进制，单组可省前导 0，`::`最多一次，恢复至 8 组。基本首部固定 40 B：版本 4 bit、流量类别 8 bit、流标签 20 bit、有效载荷长度 16 bit、下一首部 8 bit、跳数限制 8 bit、源/目的各 128 bit。

| 项目 | IPv6 规则 |
|---|---|
| 长度 | 有效载荷长度计基本首部后的扩展首部和上层数据 |
| 扩展 | 下一首部可指扩展首部或上层协议，基本首部无首部校验和 |
| 转发与分片 | 路由器不分片，过大时发 Packet Too Big；源端按路径 MTU 调整或加分片扩展首部 |
| 邻居与广播 | 无广播；用 ICMPv6 邻居发现取代 ARP |
| 地址 | 未指定`::`，回环`::1`，链路本地 `fe80::/10`，组播 `ff00::/8` |
| 任播 | 多接口共地址，按路由交付其中一个 |
| 配置 | 支持无状态自动配置和 DHCPv6，接口标识生成不限于 MAC |

IPv6 地址空间/IPv4 地址空间为 $2^{96}$。双栈同时运行两协议；隧道把一类 IP 分组封装到另一类 IP 分组中，内外端点分别解释。流量类别、流标签仅提供分类信息，服务保证还需要资源和机制。

**例 4-9**

下列关于 IPv4 和 IPv6 的叙述中，正确的是（ ）。

I. IPv6 地址空间是 IPv4 地址空间的 96 倍

II. IPv4 首部和 IPv6 基本首部的长度均可变

III. IPv4 向 IPv6 过渡可以采用双协议栈和隧道技术

IV. IPv6 首部的 Hop Limit 字段等价于 IPv4 首部的 TTL 字段

A. 仅 I、II　B. 仅 I、IV　C. 仅 II、III　D. 仅 III、IV

## 4.9 RIP、OSPF 与 BGP

自治系统 AS 为统一管理和路由策略下的一组网络与路由器。IGP 负责 AS 内部，域间协议交换 AS 间可达性；AS 可含多子网、多出口。

| 项目 | RIP | OSPF | BGP |
|---|---|---|---|
| 方法 | 距离向量，跳数 | 链路状态，SPF | 路径向量，策略 |
| 范围 | AS 内 | AS 内、区域 | AS 间，并在 AS 内传播 BGP 路由 |
| 核心信息 | 到目的的距离 | 本地链路状态，范围内洪泛 | 前缀、AS_PATH、NEXT_HOP 等属性 |
| 承载 | UDP520 | IP 协议 89 | TCP179 |
| 更新 | 通常 30 s 周期及触发；坏消息慢 | 变化更新、刷新、老化和可靠洪泛 | 会话建立后增量发布/撤销，保活另行 |

RIP 最大可达 15，16 为不可达；教材直连网距离计 1，邻居报告加 1 并改下一跳为邻居。未登记则加入；候选更优则替换；原下一跳就是该邻居时，采用其新距离，即使变差；其他邻居较差通常保留原项。失效判断课件示例 180 s。

OSPF 区域内同步拓扑，跨区经骨干区域，边界路由器参与多个区域。LSDB 为数据库，LSA 为通告，五种分组如下。

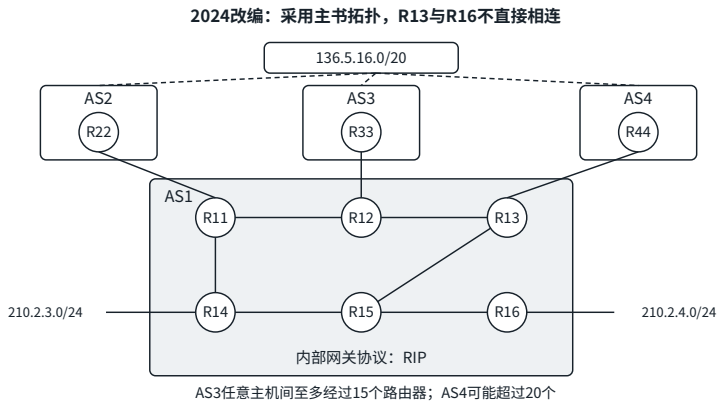



Figure 4.6: AS1 内仅有 R11—R12、R12—R13、R11—R14、R14—R15、R15—R16、R15—R13 六条路由器间链路

| 类型 | 分组 | 作用 |
|---|---|---|
| 1 | Hello | 发现、维持邻居 |
| 2 | DD | 数据库摘要，LSA 头 |
| 3 | LSR | 请求缺失或更新 LSA |
| 4 | LSU | 发送 LSA 正文并洪泛 |
| 5 | LSAck | 确认收到 LSA |

BGP 的 AS_PATH 用于 AS 环检测和策略，含本 AS 号通常拒绝；NEXT_HOP 还需 IGP 递归求本机出口。eBGP 两端属不同 AS，iBGP 同 AS 且不必物理直连；完整网状 iBGP 可由路由反射等机制扩展。OPEN 交换会话参数，UPDATE 发布/撤销，KEEPALIVE 保活，NOTIFICATION 报告错误并结束。按题给本地偏好、AS 路径等规则选路；属性相同的教材题才进一步比较到出口域内代价。

**例 4-12**

网络空间是继陆海空天之后的“第五疆域”，路由算法与协议是网络核心技术之一。现有互联网中 4 个自治系统的互连拓扑如下图。AS1 运行 RIP；AS3 内任意两台主机通信经过的路由器不超过 15 个，AS4 内可能超过 20 个。本题采用下图拓扑：**R13 与 R16 不直接相连**。

（1）若仅有 RIP、OSPF 可选，AS4 应选择哪个内部网关协议？

（2）AS3 中的某主机向本 AS 另一主机发送一个 IP 分组，为保证正常接收，初始 TTL 至少应为多少？

（3）AS1 的路由器同时启动，立即构建并交换初始距离向量，此后每隔 30 s 交换一次最新距离向量。从交换初始向量的时刻算起，R11～R16 全部获得到达 210.2.3.0/24 的正确路由至少需多久？全部获得到达 210.2.4.0/24 的正确路由至少需多久？

（4）R44 向 R13 通告 136.5.16.0/20 的路由，由哪类 BGP 会话完成、通过哪种 BGP 报文通告？R13 又通过哪类会话将该可达性信息通告给 R14、R15？

（5）R14、R15 均收到下表可达性信息。在无策略约束的情况下，更新后两者到 136.5.16.0/20 的下一跳各是谁？

| 目的网络 | AS 路径 | 下一跳 |
|---|---|---|
| 136.5.16.0/20 | AS2、AS8、AS19 | R11 |

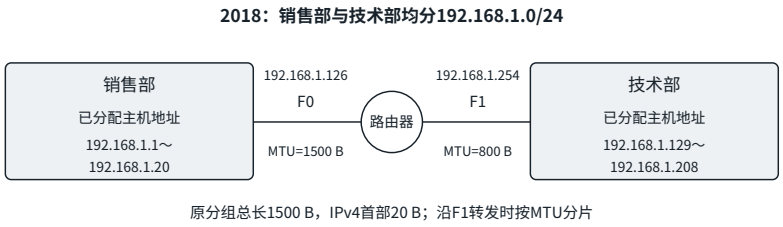



Figure 4.7: 销售部已分配 192.168.1.1～20, 技术部已分配 192.168.1.129～208；F0 为.126，F1 为.254

| 目的网络 | AS 路径 | 下一跳 |
|---|---|---|
| 136.5.16.0/20 | AS3、AS7、AS11、AS19 | R12 |
| 136.5.16.0/20 | AS4、AS10、AS19 | R13 |

## 4.10 组播与移动 IP

IPv4 组播 `224.0.0.0/4` 表示接收者组，公共路径只传一份，在分叉接口复制。以太网组播 MAC 由 `01:00:5E` 和 IP 低 23 位映射，32 个 IP 组播地址可映射同一 MAC，上层仍须检查组成员。IGMP 管理主机与相邻路由器的本地成员关系，跨网转发树由组播路由协议建立。

移动 IPv4 保留归属地址，用转交地址表示当前位置，向归属代理注册绑定。传统三角路由：对端 → 归属网络 → 归属代理隧道送转交地址 → 解封装交移动节点；内层目的为归属地址，外层目的为转交地址。基础模型中移动节点可直接发给对端，源地址过滤等条件下可用反向隧道。

## 4.11 路由器结构与综合计算

输入端口接收、链路检查、查表；交换结构把分组送到输出端；输出端排队、封装、发送；路由处理器维护控制协议和表。交换结构可用内存、总线或互联网络，容量和并行性各异。多个输入争同一输出会排队；输入队首等待忙出口可阻挡后续去空闲出口的分组，称队首阻塞。

普通 IPv4 转发顺序：接收帧并检查 → 取出 IP→ 若目的本机按协议上交 → 否则检查 TTL、最长前缀选出口和下一跳 → 按 MTU/DF 处理 → 解析下一跳地址 → 出口排队并重封装。直通等特殊实现的启动时间按题设。

**例 4-10**

某公司的网络如下图所示。IP 地址空间 192.168.1.0/24 被均分给销售部和技术部两个子网，并已分别为部分主机和路由器接口分配了 IP 地址。销售部子网的 MTU=1500 B，技术部子网的 MTU=800 B。

(1) 销售部子网的广播地址是什么？技术部子网的子网地址是什么？若每个主机仅分配一个 IP 地址，则技术部子网还可以连接多少台主机?

(2) 主机 192.168.1.1 向主机 192.168.1.208 发送一个总长度为 1500 B、首部长度为 20 B 的 IP 分组。路由器通过 F1 转发时进行分片，且尽可能分为最大片。一个最大 IP 分片封装数据的字节数是多少？至少需要几个分片？每个分片的片偏移量是多少?

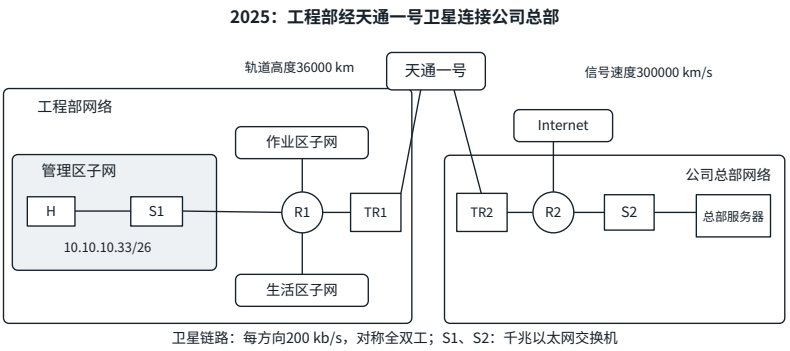



Figure 4.8: H 配置 10.10.10.33/26 并位于管理区；工程部还有作业区、生活区两个子网

**例 4-13**

某公司工程部长期驻扎偏远山区，租用天通一号卫星通信链路连接工程部 R1 和总部 R2，网络如下图。S1、S2 为千兆以太网交换机；TR1、TR2 是卫星地面收发设备，实现全双工调制解调。卫星轨道高度 36000 km，电磁波传播速度 300000 km/s；卫星链路为 R1、R2 提供对称全双工信道，每方向数据传输速率 200 kb/s。

（1）忽略卫星中继及 TR1、TR2 调制解调开销，卫星链路单向传播时延是多少？H 向总部服务器传输数据可达到的最大吞吐量是多少？忽略各层首部开销与以太网内传播时延，上传 4000 B 工程进度报告文件至少需要多久？

（2）用 GBN 为卫星链路设计单向可靠的数据链路层协议 SLP，支持 R1 向 R2 发送数据。SLP 数据帧长 1500 B，忽略 ACK 帧长度；要求单向信道利用率不低于 80%，发送窗口至少为多少？帧序号字段至少需要几位？

（3）总部分配 10.10.10.0/24 给工程部，需划给三个子网：生活区可分配地址不少于 120 个，作业区、管理区分别不少于 60 个，H 的地址配置正确。作业区、管理区、生活区的子网地址分别是什么？请用 CIDR 表示。

# 5

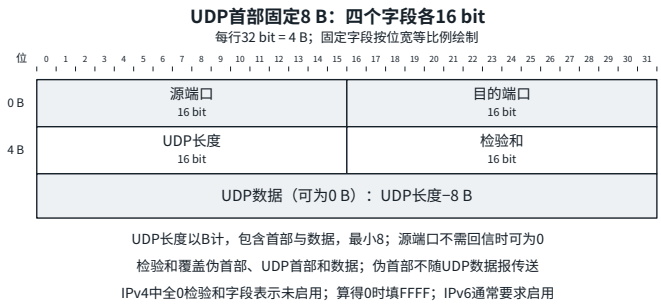



Figure 5.1: UDP 四字段各 16 bit

## 5.1 端口、连接与数据边界

传输层为不同主机的进程端点提供逻辑通信，完成复用与分用。端口 16 bit，TCP 与 UDP 端口空间独立；熟知端口 0～1023、登记 1024～49151、动态/私有 49152～65535。端口标识应用端点，具体进程可管理多个套接字。

TCP 连接由源 IP、源端口、目的 IP、目的端口四元组区分，服务器同一服务端口可对应多个客户。UDP 基本分用按目的端口等本地端点信息，实际绑定或连接方式可再限定远端。

**例 5-1**

UDP 协议实现分用时所依据的头部字段是（　）。

A. 源端口号　B. 目的端口号　C. 长度　D. 校验和

| 比较项 | UDP | TCP |
|---|---|---|
| 服务 | 无连接、尽最大努力；可按下层能力单播/组播/广播 | 面向连接、一对一、全双工可靠有序字节流 |
| 应用边界 | 保留每个报文边界 | 不保留写入边界，应用自行定界 |
| 首部 | 固定 8 B | 20～60 B |
| 可靠性与控制 | 无自身确认重传、顺序恢复或拥塞窗口 | 序号、累计 ACK、重传、接收窗口与拥塞控制 |

TCP 发送缓冲保留未确认字节以便重传；接收缓冲可存失序字节，按序交付。应用读取后释放接收空间。MSS 限制单段应用数据上限，分段可跨越应用写入边界；普通 MTU1500、IP/TCP 各 20 B 时简单模型 MSS1460 B，选项或路径改变须重算。

## 5.2 UDP 长度与反码检验和

UDP 源端口供回复，不需回复时可为 0；目的端口分用；长度计首部加数据，最小 8 B；校验和检错。普通 IPv4 首部 20 B 时 UDP 应用数据最

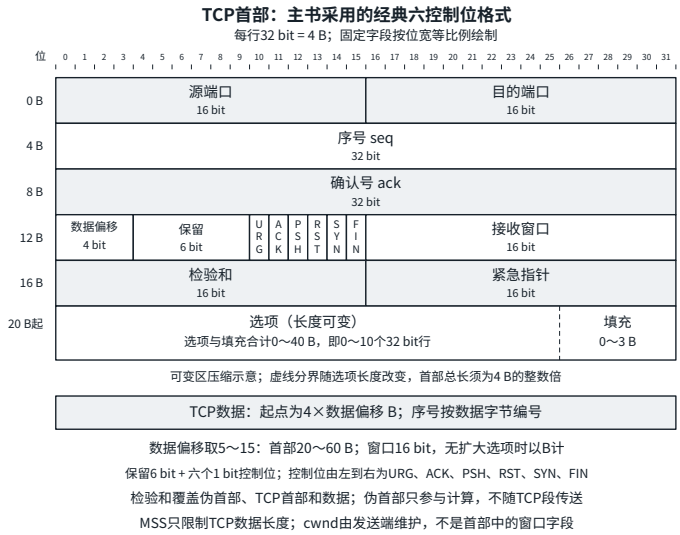



Figure 5.2: 教材 TCP 固定首部及六控制位

大 $65535-20-8=65507$ B。UDP 不自行拆成多个 UDP 报文，承载 IP 可分片，目的 IP 重组后交 UDP。

发送端校验和置 0，按 16 bit 分组求反码和：进位回卷到最低位，直到无进位，再按位取反；奇数字节末尾补 0 仅参与计算。接收端连同校验和求反码和应为 `FFFF`，仍存在抵消漏检。

| 检验 | 覆盖内容 | 零字段规则 |
| --- | --- | --- |
| IPv4 首部 | IP 首部、选项、填充 | 无关闭校验语义 |
| UDP | 伪首部、UDP 首部与数据 | IPv4 可用全 0 表示未启用；算得 0 传全 1；普通 IPv6 UDP 通常必须启用 |
| TCP | 伪首部、TCP 首部与数据 | 无以全 0 关闭校验约定 |

IPv4 伪首部 12 B：源 IP、目的 IP、0、协议号、传输层长度；UDP 协议 17，TCP 协议 6。伪首部及求和补 0 不在线上传输，不计入应用长度。NAT 改 IP 会影响伪首部，启用的传输校验需更新；普通 TTL 变化只影响 IPv4 首部检验和。

**例 5-2**

若 UDP 协议在计算校验和过程中，计算得到中间结果为 `1011 1001 1011 0110` 时，还需要加上最后一个 16 位数 `0110 0101 1100 0101`，则最终计算得到的校验和是（ ）。

A. `0001 1111 0111 1011`

B. `0001 1111 0111 1100`

C. `1110 0000 1000 0011`

D. `1110 0000 1000 0100`

## 5.3 TCP 首部、序号与累计确认

源/目的端口各 16 bit，seq/ack 各 32 bit；数据偏移 4 bit、单位 4 B，表示 TCP 首长，取 5～15；接收窗口 16 bit，未扩大时以 B 计、最大 65535；

检验和和紧急指针各 16 bit，选项填充至 4 B 对齐。题图采用教材六控制位口径，扩展位按题设解释。

未分片或已重组的普通 IPv4 仅承载一个 TCP 段时：

$$L_{TCP\text{数据}} = IP\text{总长} - 4IHL - 4 \times TCP\text{数据偏移}.$$

| 控制位 | 含义 |
|---|---|
| SYN | 同步初始序号，占 1 个序号 |
| ACK | 为 1 时确认号有效；纯 ACK 不占序号 |
| FIN | 本方向不再发送，占 1 个序号 |
| RST | 复位、拒绝不适当连接或段 |
| PSH | 请求及时向应用交付 |
| URG | 紧急指针有效；指针相对 seq 给出紧急数据边界，按教材前置紧急数据模型理解 |

TCP 按字节编号，序号模 $2^{32}$；无回绕时，seq=$s$、数据长 $L$ 覆盖 $s$～$s+L-1$，下一号 $s+L$。ack=$a$ 累计确认连续收到至 $a-1$，期望 $a$。两方向序号空间独立，捎带段中的 seq 和 ack 分别记账。SYN 序号 $x$ 后首数据 $x+1$；FIN 后确认号也加 1，这些消耗不计为应用字节。

**例 5-3**

主机甲与主机乙之间已建立一个 TCP 连接，甲向乙发送了两个连续 TCP 段，分别包含 300 B 和 500 B 有效载荷，第一个段的序列号为 200，乙正确接收到两个段后，发送给甲的确认序列号是（　）。

A. 500　　B. 700　　C. 800　　D. 1000

## 5.4 三次握手、半关闭与 TIME-WAIT

| 事件 | 客户状态/动作 | 服务器状态/动作 |
|---|---|---|
| 客户发 SYN，seq=$x$ | SYN-SENT | LISTEN 等待 |
| 服务器收 SYN 并回 SYN+ACK，seq=$y$、ack=$x+1$ | 等待回复 | SYN-RECEIVED |
| 客户收到并回 ACK，seq=$x+1$、ack=$y+1$ | ESTABLISHED，可带数据 | 收到此 ACK 后 ESTABLISHED |

第三次握手确认服务器初始序号，避免服务器仅凭迟到旧 SYN 就长期保留无效连接。

主动端发 FIN 进入 FIN-WAIT-1；收其 ACK 到 FIN-WAIT-2；收对方 FIN 并回 ACK 到 TIME-WAIT；2MSL 后 CLOSED。被动端收 FIN 回 ACK 到 CLOSE-WAIT，本方向仍可发数据；应用关闭后发 FIN 到 LAST-ACK，收确认后 CLOSED。ACK 与 FIN 可合并；同时关闭可经 CLOSING，按事件次序判断。

TIME-WAIT 的 2MSL 使最后 ACK 丢失时仍可响应重传 FIN，并让旧连接报文消散。两端关闭时间分别按本端状态进入事件计算。

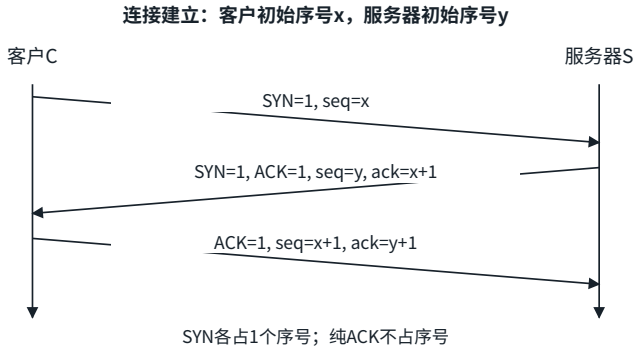



Figure 5.3: 两个方向各同步一个初始序号

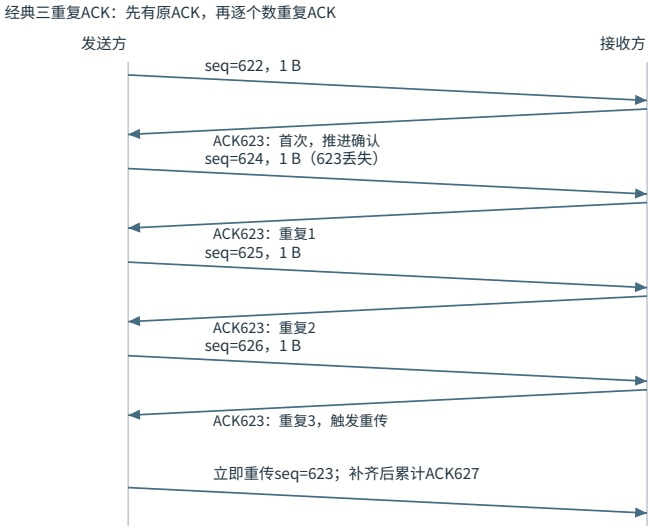



Figure 5.4: 正常 ACK 与三次重复 ACK 分别计数

**例 5-4**

假设客户 C 和服务器 S 已建立 TCP 连接，RTT=50 ms，最长报文段寿命 MSL=800 ms，数据传输结束后 C 主动请求断开连接。若从 C 主动向 S 发出 FIN 段时刻算起，则 C 和 S 进入 CLOSED 状态所需的时间至少分别是（　）。

A. 850 ms，50 ms　　B. 1650 ms，50 ms

C. 850 ms，75 ms　　D. 1650 ms，75 ms

## 5.5 重传、重复 ACK 与 RTT 估计

失序到达时累计 ACK 保持缺口首字节；接收方可缓存，SACK 另报已收分块而不改变累计 ACK。发送端保留未确认字节，超时重传相关未确认数据。经典快重传在 3 个重复 ACK 时触发，不计最初推进确认号的正常 ACK。

例如 622 到达返回 ACK623，623 丢失；624、625、626 依次到达分别产生重复 ACK1、2、3，第三份才触发快重传。若已缓存 624～626，补齐 623 后 ACK 可推进至 627。

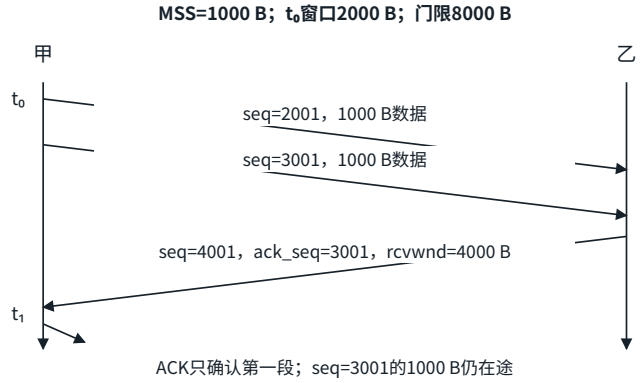



Figure 5.5: 原题数据：第二个数据段仍未被 ack_seq=3001 确认

**固定窗口下的吞吐上界**

长连接、窗口固定、确认及时且无损失的理想模型中，窗口约束的数据率不超过 $W/RTT$；$W$ 用 bit。还须与路径瓶颈速率取小。

$$SRTT \leftarrow (1-\alpha)SRTT + \alpha RTT_{sample}.$$

RTO 结合平滑 RTT 及偏差估计，题给 $RTO = \beta SRTT$ 时按其简化式；超时后 RTO 退避。Karn 原则排除有重传歧义的 RTT 样本。旧 SRTT100 ms、$\alpha = 1/8$、有效样本 140 ms，则新值 105 ms。

## 5.6 接收窗口、拥塞窗口与可新发额度

rwnd 由接收方依据当前接收能力通告，cwnd 由发送方依据网络反馈维护。ack=$a$、rwnd=$w$ 允许范围 $a$～$a+w-1$；首部窗口字段反映本段发送者的接收能力。扩大选项在握手协商，未给则不自行放大。

当最新 ACK 与窗口左沿一致，已发未确认字节量 $F$：

$$W = \min(cwnd, rwnd), \qquad A = \max(0, W - F).$$

$W$ 为总发送额度，$A$ 为还可新发；已在途量继续占额度，窗口缩小不要求撤回已发数据。复杂窗口更新按实际字节区间和有效右边界记录。

零窗口暂停普通数据发送，持续计时器触发零窗口探测，防止窗口重开通知丢失造成永久等待。它与重传、保活、TIME-WAIT 计时器分别处理不同事件。

接收缓存 8 B 且应用不读，从 seq600 开始，依次收到 600～602、603～604、605～606、607：ACK/rwnd 为 603/5、605/3、607/1、608/0；应用读走 8 B 后为 608/8，确认号保持。

**例 5-6**

主机甲通过 TCP 向主机乙发送数据的部分过程如下图。seq 为序号，ack_seq 为确认序号，rcvwnd 为接收窗口。甲在 $t_0$ 的拥塞窗口和发送窗口均为 2000 B，拥塞控制阈值为 8000 B，MSS=1000 B，甲始终以 MSS 发送 TCP 段。若甲在 $t_1$ 收到图示确认段，则未收到新的确认之前，还可以继续发送的 TCP 段数是（　）。

A. 2　　B. 3　　C. 4　　D. 5

## 5.7 慢开始、拥塞避免与 Reno 恢复

以下为教材逐段确认、持续有数据、无其他限制的简化模型，初始窗口按题设，常用 1 MSS。慢开始每个新数据 ACK 增加约 1 MSS，完整一轮约翻倍；达到 ssthresh 后拥塞避免，每 RTT 约增 1 MSS。延迟 ACK、应用受限、rwnd 受限会改变轨迹，轮次须注明发送前或确认后记录。

| 事件 | ssthresh | cwnd 及后续 |
|---|---|---|
| 超时 | 教材常取丢包时 cwnd 一半 | 回 1 MSS，再慢开始 |
| Reno 三重复 ACK | 教材常取原窗口一半 | 快重传，快恢复临时为门限 +3 MSS；新 ACK 确认缺失数据后回门限，转拥塞避免 |
| Tahoe 三重复 ACK | 减半门限 | 回 1 MSS 并慢开始 |

课件稳定轮次图通常记录快恢复完成后的 cwnd=ssthresh，恢复中临时 +3 另算。规范实现设门限还参考在途量，题目明确模型优先。

初始 1、门限 8，未受其他限制时轮次 1、2、4、8、9、10……；若在 12 超时，门限 6，后续 1、2、4、6、7……。

**例 5-5**

若主机甲与主机乙已建立 TCP 连接，MSS 为 1 KB，RTT 为 2 ms，则在不出现拥塞的前提下，拥塞窗口从 8 KB 增长到 32 KB 所需的最长时间是（ ）。

A. 4 ms　B. 8 ms　C. 24 ms　D. 48 ms

**例 5-7**

主机甲和乙已建立 TCP 连接，甲始终以 MSS=1 KB 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 $t$ 时刻发生超时时拥塞窗口为 8 KB，则从 $t$ 起不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是（ ）。

A. 10 KB　B. 12 KB　C. 14 KB　D. 15 KB

## 5.8 TCP 字段与接收缓存综合计算

按“IP 长度和首长 →TCP 起点与数据偏移 → 控制位 → 两方向 seq/ack→ 窗口”解码。以太网填充比较整个 IP 长度与最小数据区，填充不占 IP 总长或 TCP 序号。TTL 差计路由器跳数，交换机不减 TTL；有 NAT 时结合 IP 标识和 TCP 字段识别对应分组。

**例 5-8**

主机 H 通过快速以太网连接 Internet，IP 地址为 192.168.0.8；服务器 S 的 IP 地址为 211.68.71.80。H 与 S 使用 TCP 通信，在 H 上捕获的 5 个 IP 分组及其中一个分组在 S 发出时的前 40 字节如下图，均为十六进制、按网络字节序排列。**第 2 包首字节采用 45，第 3 包 TCP 数据偏移与控制字节采用 50 10**。

字段布局参照第 4 章 IPv4 首部格式和本章TCP 首部图：IPv4 源、目的地址分别位于第 13～16、第 17～20 字节，TTL 位于第 9 字节；IHL 与 TCP 数据偏移字段均以 4 B 为单位。

（1）5 个 IP 分组中，哪几个由 H 发送？哪几个完成 TCP 连接建立过程？哪几个在通过快速以太网传输时进行了填充？

**2012改编：在H捕获的5个IP分组及S发出的对应分组**

每项仅列前40 B；每行20 B，均为网络字节序的十六进制

| | |
|---|---|
| 1 | 45 00 00 30 01 9b 40 00 80 06 1d e8 c0 a8 00 08 d3 44 47 50<br>0b d9 13 88 84 6b 41 c5 00 00 00 00 70 02 43 80 5d b0 00 00 |
| 2 | 45 00 00 30 00 00 40 00 31 06 6e 83 d3 44 47 50 c0 a8 00 08<br>13 88 0b d9 e0 59 9f ef 84 6b 41 c6 70 12 16 d0 37 e1 00 00 |
| 3 | 45 00 00 28 01 9c 40 00 80 06 1d ef c0 a8 00 08 d3 44 47 50<br>0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 10 43 80 2b 32 00 00 |
| 4 | 45 00 00 38 01 9d 40 00 80 06 1d de c0 a8 00 08 d3 44 47 50<br>0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 18 43 80 e6 55 00 00 |
| 5 | 45 00 00 28 68 11 40 00 31 06 06 7a d3 44 47 50 c0 a8 00 08<br>13 88 0b d9 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 57 d2 00 00 |
| S | 45 00 00 28 68 11 40 00 40 06 ec ad d3 44 47 50 ca 76 01 06<br>13 88 a1 08 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 b7 d6 00 00 |

第2包首字节45；第3包TCP控制字节50 10；其余字节按主书表保留

Figure 5.6: 1～5 为 H 处捕获的分组，S 行为在服务器发出时捕获的对应分组；每行 20 B

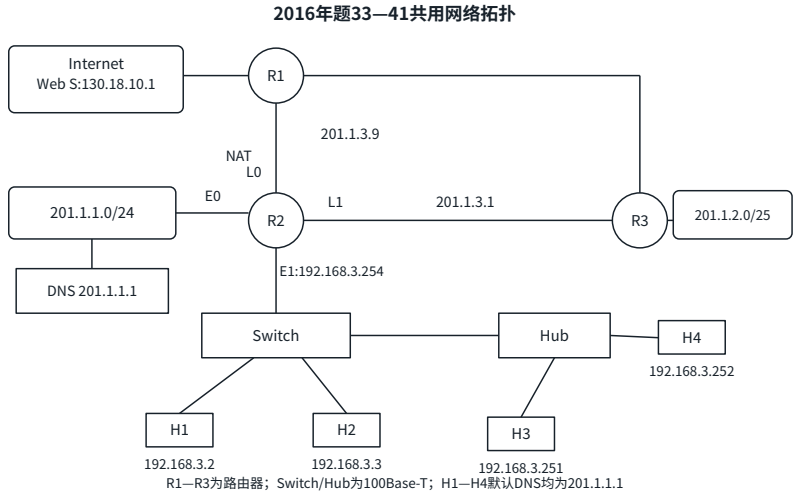



Figure 5.7: H3 为 192.168.3.251，经 Hub 和 R2 访问 130.18.10.1；复用 2016 年共用题图

（2）根据这 5 个 IP 分组，分析 S 已经收到的应用层数据有多少字节。

（3）若某个 IP 分组在 S 发出时的前 40 字节为图中 S 行，该 IP 分组到达 H 时经过了多少个路由器?

缓存只进不出时，累计接收量决定 rwnd；每轮发送量同时受 cwnd 和剩余缓存限制。求零窗口时间须计至零窗口通告返回发送端；平均应用速率用总应用字节/题定时间，KB 和 kb/s 分别按题设换算。

**例 5-9**

下图中 H3 访问 Web 服务器 S。S 为新建 TCP 连接分配 20 KB（K=1024）接收缓存，MSS=1 KB，平均 RTT=200 ms。H3 建立连接时初始序号为 100，持续按 MSS 大小发送数据，拥塞窗口初始阈值 32 KB；S 对收到的每个段确认并通告新接收窗口。连接建立后，S 的 TCP 接收缓存只有数据存入、没有数据取出。图中 R2 的 E0 侧采用 201.1.1.0/24。

（1）连接建立时，H3 收到的第二次握手 TCP 段的 SYN、ACK 标志位分

别是多少？确认号是多少？

（2）H3 收到的第 8 个确认段所通告的接收窗口是多少？此时拥塞窗口变为多少？发送窗口变为多少？

（3）发送窗口等于 0 时，下一个待发送的数据段序号是多少？从 H3 发送第 1 个数据段到发送窗口等于 0，平均数据传输速率是多少？忽略段的传输时延。

（4）H3 与 S 通信已结束，H3 在 $t$ 时刻请求断开连接。从 $t$ 起，S 释放该连接的最短时间是多少？

# 6 应用层

## 6.1 C/S、P2P 与协议承载

C/S 由客户主动请求、长期运行的服务器提供服务，可集中管理但可能形成瓶颈；副本与负载均衡可扩展。P2P 对等方同时获取和提供资源，节点动态变化，需解决定位、访问控制和内容一致性；集中索引可以只定位资源，数据仍由对等方传送。

| 协议 | 作用 | 典型承载与端口 |
| --- | --- | --- |
| DNS | 名称解析、资源记录查询 | UDP/TCP53 |
| FTP | 文件传送，控制/数据连接分离 | TCP，控制 21，主动数据传统源 20 |
| SMTP | 提交、服务器间邮件推送 | TCP，服务器间 25 |
| POP3 | 用户代理读取邮件 | TCP110 |
| HTTP/1.x | 请求响应获取或处理资源 | TCP，HTTP 通常 80 |

应用按可靠性、时延、吞吐和通信模式选择下层服务。一条应用消息可跨多个 TCP 段，一个 TCP 段也可含多个应用写入片段。

## 6.2 DNS 层次、资源记录与查询

域名由右至左层级降低，顶级域包括国家/地区类、通用类和 arpa 等基础设施类。普通主机名标签通常用字母、数字、连字符，大小写不敏感；单标签最长 63 B，完整 DNS 线格式最长 255 B（含长度字节与根标签），普通不带末尾根点的文本常限 253 B。

域是名称空间子树；区是权威服务器管理的范围，子域委派后域与区可不同。根服务器提供顶级域委派，权威服务器负责本区记录；本地 DNS 为递归解析器，不要求与客户处同一 LAN。

| 记录 | 值及作用 |
| --- | --- |
| A、AAAA | 名称对应 IPv4、IPv6 地址 |
| NS | 权威服务器名称 |
| CNAME | 别名指向规范名称 |
| MX | 邮件交换服务器名称和优先级 |
| TTL | 资源记录可缓存的有效期 |

NS/MX 中的名称可能继续需要地址解析。名称、IP 与主机无一一对应关系，一个名称可有多个地址，多个名称可共享地址。

递归：被询问者负责继续解析并返回答案/失败。迭代：被询问者可返回下一站，当前查询者继续询问。常见流程为主机向本地 DNS 递归，本地 DNS 向根、顶级、权威迭代；按实际委派层数展开。

有效最终记录命中可 0 次远程查询；有中间委派缓存则从对应级继续；无相关缓存从根开始。查询次数固定计数主体，并按题意排除或包含别名链、失败重试。DNS 常用 UDP，也支持 TCP 区域传送、较大响应或回退等场景。

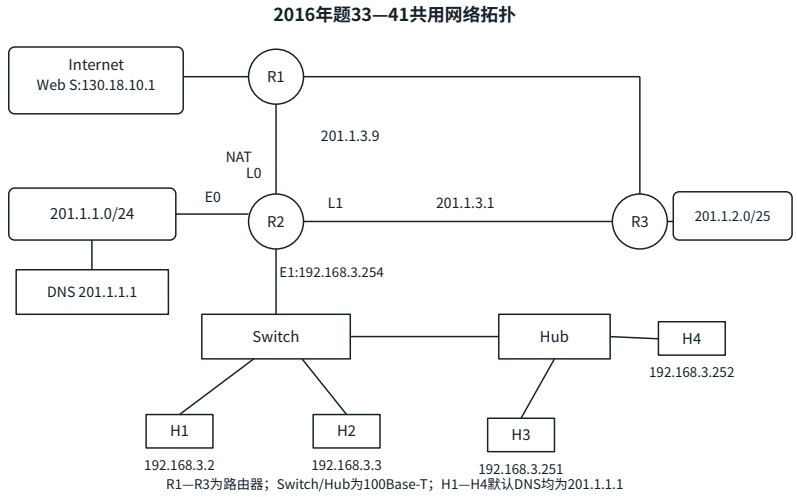



Figure 6.1: 2016 年共用拓扑；本题只使用 H4 及其默认 DNS 服务器 201.1.1.1

**Base64 的整块长度**

$n$ 字节数据的 Base64 字符数为 $4\lceil n/3 \rceil$，包含末块的等号补齐，不含换行、MIME 首部和边界。1 B 或 2 B 数据也均需 4 字符。

**例 6-1**

假设所有域名服务器均采用迭代查询方式进行域名解析。当 H4 访问规范域名为 `www.abc.xyz.com` 的网站时，域名服务器 201.1.1.1 完成该域名解析过程中，可能发出 DNS 查询的最少和最多次数分别是（ ）。共用网络拓扑如下。

A. 0，3　　B. 1，3　　C. 0，4　　D. 1，4

## 6.3 FTP 控制、数据连接与传输模式

控制连接用 TCP 连接服务器 21 端口，承载登录、目录、传送命令及状态，通常持续整个会话；数据连接按需要建立，承载文件与目录列表。`LIST` 命令走控制，列表内容走数据。

主动模式：客户告知接收地址/端口，服务器主动建立数据连接，传统服务器数据源端口 20。被动模式：服务器通告监听端点，客户主动连接。实现可用进程、线程或事件模型维护各会话，连接与 OS 进程不要求一一对应。

**例 6-2**

FTP 客户和服务器间传递 FTP 命令时，使用的连接是（ ）。

A. 建立在 TCP 之上的控制连接

B. 建立在 TCP 之上的数据连接

C. 建立在 UDP 之上的控制连接

D. 建立在 UDP 之上的数据连接

## 6.4 邮件系统、SMTP、POP3 与 MIME

邮件系统由用户代理、邮件服务器和传送/读取协议组成。路径：用户代理 SMTP 提交 → 发送服务器按 MX 等确定收件服务器 → 服务器间 SMTP 转送 → 收件用户 POP3、IMAP 或 Web 读取。

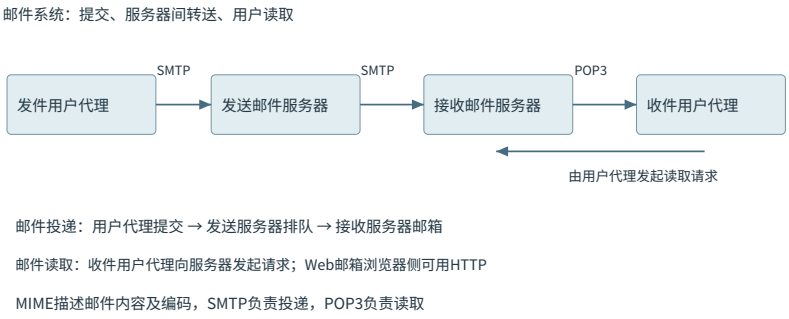



Figure 6.2: 提交、转送、读取分别使用相应协议

邮件由首部、空行、正文组成，`From:`、`To:`、`Subject:`属于可见邮件内容；SMTP 的 `MAIL FROM`、`RCPT TO` 属于投递信封，实际投递地址可与显示地址不同。基本交互为 TCP 建立 → 问候 → 信封发件人 → 一个或多个收件人 →DATA 内容 → 结束；正文终止标志按透明传输规则处理。

基础 SMTP 按 7 bit ASCII 背景理解。MIME 定义内容类型、字符集、传输编码和多部分边界，支持非 ASCII 文本及附件；Base64 把 3 B 变 4 个 ASCII 字符，净编码约膨胀 4/3，另有格式开销。公开编码可逆，不提供保密性；扩展 SMTP 按协商能力解释。

POP3 经历授权、事务、更新；同一 TCP 连接可读取多封邮件，删除通常在更新阶段落实，也可下载并保留。Web 邮箱浏览器使用 HTTP 类交互，后端仍可运行邮件协议。提交被接受与用户已读取是不同事件。

**例 6-3**

下列关于 POP3 协议的叙述中，正确的是（　）。

I. 支持用户代理从邮件服务器读取邮件

II. 支持用户代理向邮件服务器发送邮件

III. 支持邮件服务器之间发送与接收邮件

IV. 支持通过一条 TCP 连接收取多封邮件

A. 仅 I、IV　　B. 仅 II、III　　C. 仅 I、II、III　　D. 仅 I、III、IV

## 6.5 URL 与 HTTP 报文语义

URL 包含方案、主机、可选端口、路径、查询和片段。`http://www.abc.com:80/index.html?q=408#part1` 的请求目标为`/index.html?q=408`，片段 `part1` 由浏览器文档内定位，不进入普通 HTTP 请求目标。HTML 可引用多个对象、不同域名，可能需要额外 DNS 和连接。

HTTP/1.x 请求含请求行、首部、空行、可选消息体；响应含状态行、首部、空行、可选消息体。

```
GET /index.html?q=408 HTTP/1.1
Host: www.abc.com
If-None-Match: "v2"
```

| 方法或状态 | 语义 |
|---|---|
| GET、HEAD | 获取资源、获取相应首部；按规定语义属安全方法 |
| POST | 提交内容供处理，可改变状态 |
| 200 | 成功 |
| 301/302 | 重定向 |
| 304 | 条件请求中资源未改动，复用缓存实体；验证仍有往返 |
| 404、5xx | 未找到、服务器错误类别 |

有效的新鲜缓存可直接使用，无需访问源站；过期或需验证时，可用 `If-Modified-Since` 或 `If-None-Match` 条件请求。代理缓存可缩短访问路径、降低上游流量，是否发起验证由新鲜度及缓存策略决定。

HTTP 无状态表示处理请求不自动要求保留上次会话；`Set-Cookie` 设置客户端标识，后续 `Cookie` 带回以关联服务器状态。持久连接减少 TCP 建立开销，Cookie 和连接复用分别判断，认证安全依实际机制。

HTTP/1.x 题按其 TCP 与流水模型计算；HTTPS 还需题给 TLS 等开销；HTTP/2 多路复用和 HTTP/3 QUIC 按相应题设处理。

## 6.6 HTTP 连接复用与访问时延

条件：DNS 已完成、TCP 未建，先收 HTML 才发现其中 $m \geq 1$ 个同站小对象；不并行建连接，忽略发送、处理、窗口与其他开销，往返均 RTT。

| 模型 | 依赖过程 | 总时延 |
|---|---|---|
| 非持久串行 | 每对象重新 TCP 建立并请求响应，共 $m+1$ 对象 | $2(m+1)RTT$ |
| 持久非流水 | 首次建立，HTML 及 $m$ 对象逐个请求响应 | $(m+2)RTT$ |
| 持久流水 | 首次建立、HTML、已知对象并发请求三个阶段 | $3RTT$ |

若 TCP 已建立，减首次握手 1RTT；允许并行连接时，仅在资源发现后按并行数分批。较大对象还受发送时间、拥塞/接收窗口限制。DNS 所需各次查询按依赖累加，有效缓存或已知权威入口可跳过对应阶段。

**例 6-4**

若浏览器不支持并行 TCP 连接，使用非持久的 HTTP/1.0 协议请求浏览 1 个 Web 页，该页中引用同一网站上的 7 个小图像文件，则从浏览器为传输 Web 页请求建立 TCP 连接开始，到接收完所有内容为止，所需要的 RTT 数至少是（ ）。

A. 4　B. 9　C. 14　D. 16

**例 6-5**

假设主机通过 HTTP/1.1 流水线方式请求 Web 服务器上的 rfc.html，它引用同目录下 3 个 JPEG 小图像，只有收到 rfc.html 后才能发送对应图像请求。一次请求响应为 RTT，忽略其他时延，不考虑拥塞与流量控制。从发出 HTTP 请求报文开始，到收到全部内容，所耗时间是（ ）。

A. 2RTT　B. 2.5RTT　C. 4RTT　D. 4.5RTT

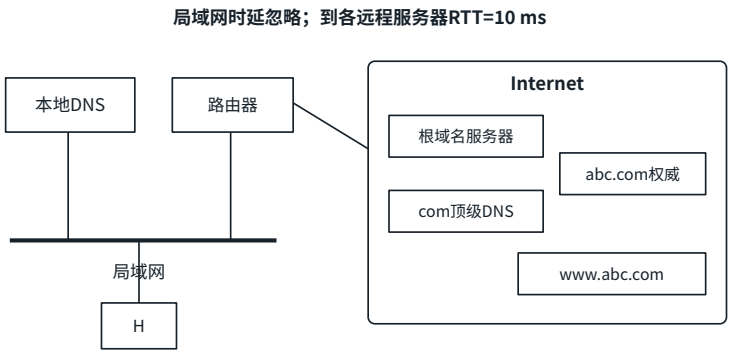



Figure 6.3: 主机、本地 DNS 与路由器共享局域网，远程服务器分别承担名称解析和 Web 响应

**例 6-6**

假设下图网络中的本地域名服务器只提供递归查询服务，其他域名服务器均只提供迭代查询服务；局域网内主机访问 Internet 上各服务器的 RTT 均为 10 ms，忽略其他各种时延。若主机 H 通过超链接 `http://www.abc.com/index.html` 请求浏览纯文本 Web 页 index.html，从单击超链接到浏览器收到页面，所需的最短与最长时间分别是（ ）。

A. 10 ms，40 ms　　B. 10 ms，50 ms

C. 20 ms，40 ms　　D. 20 ms，50 ms

**例 6-7**

假设主机 H 通过 HTTP/1.1 请求 Web 服务器 S 上的 news408.html，它引用同目录下 1 幅图像。HTML 文件大小为 1 MSS，图像 3 MSS，RTT=10 ms，忽略 HTTP 响应首部开销和 TCP 段传输时延。若 H 已完成域名解析，则从 H 请求与 S 建立 TCP 连接时刻起，到收到全部内容，所需的时间至少是（ ）。

A. 30 ms　　B. 40 ms　　C. 50 ms　　D. 60 ms

## 6.7 名称解析、连接与逐跳封装综合

访问依赖：DNS 或缓存取得服务器 IP→ 路由决定下一跳 →ARP 取得下一跳 MAC→TCP 握手 →HTTP 请求 HTML→ 解析引用对象 → 按复用、并行与窗口条件传送。ARP 缓存、本地 DNS 缓存、交换机转发表分别记录不同信息。

发送 DNS 查询时，若 DNS 在本子网，ARP 解析 DNS；服务器在外网，HTTP/TCP 帧目的为网关 MAC。交换机仅从实际帧的源 MAC 与入端口学习，未发送帧的站不会仅因收到广播就被登记。NAT 按映射改变边界处地址/端口，端到端字节序号仍按连接方向追踪。

**例 6-8**

某网络拓扑如下图。H 登录 FTP 服务器后，向服务器上传 18000 B 文件 F。H 为传输 F 建立数据连接时选择初始序号 100，MSS=1000 B，拥塞控制初始阈值 4 MSS，RTT=10 ms，忽略 TCP 段的传输时延。在 F 的传输过程中，H 均按 MSS 大小发送数据，未发生差错、丢包和乱序。

（1）FTP 控制连接是持久的还是非持久的？数据连接是持久的还是非持久的？H 登录服务器时建立的 TCP 连接是控制连接还是数据连接？

Figure 6.4: H 经交换机、路由器访问 Internet 中的 FTP 服务器，控制连接与数据连接均由 TCP 承载

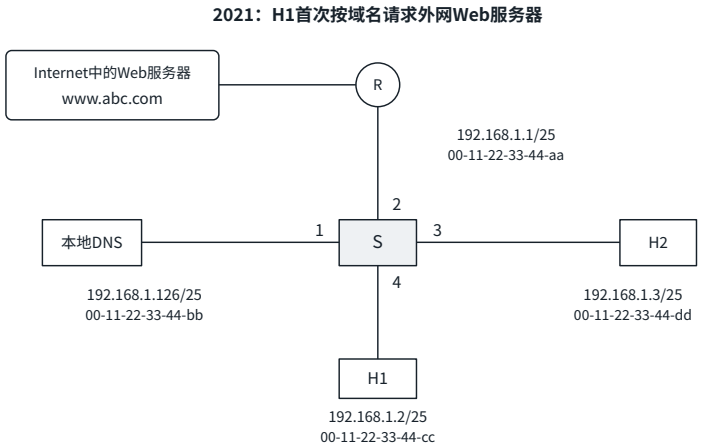



Figure 6.5: 本地 DNS 接 S 的 1 口，R 接 2 口，H2 接 3 口，H1 接 4 口；四者同属 192.168.1.0/25

（2）H 通过数据连接发送 F 时，F 第 1 个字节的序号是多少？断开数据连接过程中，FTP 服务器发送的第二次挥手 ACK 段的确认号是多少？

（3）当 H 收到确认号 2101 的确认段时，拥塞窗口调整为多少？收到确认号 7101 的确认段时，拥塞窗口调整为多少？

（4）H 从请求建立数据连接开始，到确认 F 已被服务器全部接收，至少需要多久？期间应用层数据的平均发送速率是多少？

**例 6-9**

某网络拓扑如下图，以太网交换机 S 通过路由器 R 与 Internet 互联。路由器接口、本地 DNS、H1、H2 的 IP 地址与 MAC 地址如图。$t_0$ 时 H1 的 ARP 表和 S 的交换表均为空；H1 此时用浏览器通过域名 www.abc.com 请求访问 Web 服务器。$t_1 > t_0$ 时，S 第一次收到封装 HTTP 请求报文的以太网帧。其间没有与此次 Web 访问无关的网络通信。

（1）从 $t_0$ 到 $t_1$，H1 除 HTTP 外还运行哪个应用层协议？从应用层到数据链路层，该协议报文经过哪些协议逐层封装？

（2）S 交换表结构为 “MAC 地址、端口”，$t_1$ 时交换表内容是什么？

（3）其间 H2 至少接收多少个与此次 Web 访问有关的帧？是什么帧？目的 MAC 地址是什么？